

SystemVerilog 验证平台

[TOC]

验证导论

SystemVerilog 硬件验证语言(Hard Verification Language, HVL)

具有以下性质;

1. 受约束的随机激励生成
2. 功能覆盖率
3. 更高层次的结构,尤其是面向对象的编程
4. 多线程及线程间通信
5. 支持HDL数据类型,例如verilog的四状态数值
6. 集成了事件仿真器,便于对设计施加控制

验证流程并行于设计流程:对于每个设计模块,设计者首先阅读硬件规范,解析其中的自然语言表述,然后使用RTL代码之类的机器语言创建相应的逻辑

测试平台用于确定待测设计的正确性;

1. 产生激励
2. 把激励施加到DUT上
3. 捕捉响应
4. 检验正确性
5. 对照整个验证目标测算进展情况

分层的测试平台

下面是一个不分层的测试平台的例子:

一个用于驱动APB引脚的任务

```
task write(reg[15:0] addr,reg[31:0] data);
    //驱动控制总线
    @(posedge clk)
    PAddr <= addr;
    PWDData <= data;
    PWrite<=1'b1;
    PSel<= 1'b1;

    //使PEnable翻转
    @(posedge clk)
    PEnable<=1'b0;
    @(posedge clk)
    PEnable<=1'b1;

endtask
```

```
module test(PAddr,PWrite,PSel,PWData,PEnable,Rst,clk);
initial begin
    //初始化
    reset();
    write(16'h50,32'h50);

    if(top.mem.memory[16'h50] != 32'h50)
        $display("Test failed!");
    else
        $display("Test passed!");
end

endmodule
```

上面代码中: 在 Verilog 的 initial 块中使用 @(posedge clk), 其含义是让当前的执行流程暂停, 并等待指定信号 (如 clk) 的下一个上升沿到来后, 再继续执行后续的语句。

信号和命令层

在底部的信号层, 包含待测设计和把待测设计连接到测试平台的信号

再往上一层是命令层: 执行总线读或写命令的驱动器驱动了待测设计的输入

待测设计的输出与监视器, 连监视器负责检测信号的变化, 并把这些变化按照命令分组

断言也穿过命令层和信号层, 负责监视独立的信号以寻找穿越整个命令的信号变化

功能层

功能层向下面对命令层

代理(VMM中的事务处理器)接收来自上层的事务, 例如DMA读或写, 把它们分解成独立的命令, 这些命令也被送往用于预测事务结果的计分板

检验器负责比较来自监视器和计分板的命令

场景层

场景层被位于场景层中的发生器驱动, 比如mp3播放器, 下载存储播放, 调整播放内容这些操作的每一个都可以称作场景, 场景层负责组织协调这些步骤, 操作的参数如音轨大小和寄存器位置等都采用受约束的随机值

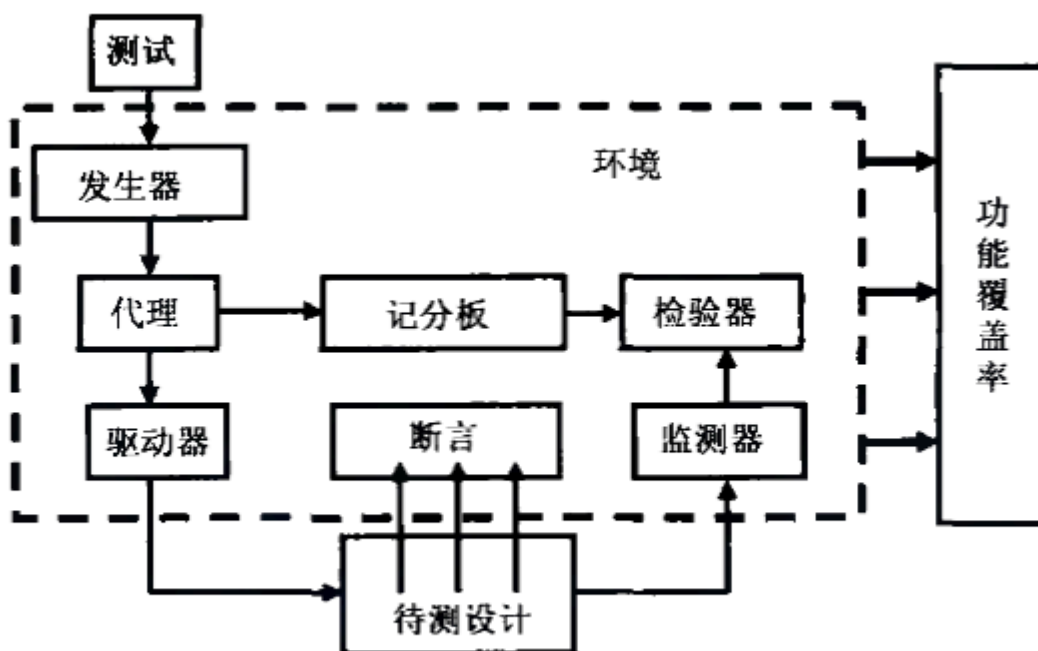


图 1.12 带着所有层次的完整测试平台

顶层的测试包含了用于创建激励的约束

功能覆盖率可以衡量所有测试在满足验证计划要求方面的进展,随着各项测试标准的完成,功能覆盖率代码在整个项目过程中会经常变化

在随机序列中间插入定向测试的代码,或者把两部分代码并列 定向代码执行你期望的任务,而随机的背景噪声可能使漏洞暴露出来,而且漏洞还有可能是在未期望的模块中

建立一个分层的测试平台

驱动器可能会注入错误或者增加时延,然后再把命令分解为一些信号的变化,例如总线请求或握手 事务处理器,核心部分是一个循环:有关事务处理器的师范代码如下:

```

task run();
  done = 0;
  while(!done) begin
    //获取下一个事务
    //进行变换
    //发送事务
  end
endtask

```

仿真环境阶段

使项目中的所有代码能够一起工作 三个基本阶段是建立,运行和收尾,每个阶段都可以再细分为更小的步骤;

建立阶段可以分为如下步骤:

1. 生成配置:把待测设计的配置和周围的环境随机化

2. 建立环境:基于配置来分配和连接测试平台构件,测试平台构件指的是存在于测试平台中的部分,与RTL代码描述的设计物理构件区分开:例如如果配置选择了三个总线驱动器,那么测试平台应该在这个阶段对它们进行分配和初始化
3. 对待测设计进行复位
4. 配置待测设计; 基于第一步中生成的配置, 载入待测设计的命令寄存器

运行阶段指测试实际运行的阶段,可分为以下步骤:

1. 启动环境: 运行各种测试平台构件,例如各种BFM和激励发生器
2. 运行测试:启动测试然后等待测试完成,定向测试的完成很容易判断,但随机测试却比较困难,可以使用测试平台的层作为引导,从顶层启动,等待一个层接收完来自上一层的所有输入,接着等待当前层空闲下来,然后再等待下一层

收尾阶段:

1. 清空:在最下层完成以后,等待设计清空最后的事务
2. 报告:一旦待测设计空闲下来,清空遗留在测试平台中的数据,有时保存在积分板利的数据从来就没有送出来过, 根据信息创建最终报告,说明测试通过或者失败,如果测试失败, 将相应的结果功能覆盖率删除

数据类型

SystemVerilog 引进了一些新的数据类型,具有以下优点:

1. 双状态数据类型:更好的性能,更低的内存消耗
2. 队列,动态和关联数组:减小内存消耗,自带搜索和分类功能
3. 类和结构: 支持抽象数据结构
4. 联合和合并结构: 允许对同一数据有多种视图
5. 字符串:支持内建的字符序列
6. 枚举类型: 方便代码编写,增加可读性

逻辑logic类型

Verilog中,初学者对经典的reg数据类型进行了改进,使得它除了作为一个变量以外,还可以被连续赋值,门单元和模块所驱动

为了与寄存器类型相区别,这种改进的数据类型被称为logic,任何使用线网的地方均可以使用logic,但要求logic不能有多个结构型的驱动 例如双向总线建模的时候,需要使用线网类型

logic 类型的使用

```
module logic_data_type(input logic rst_h);
    parameter CYCLE = 20;
    logic q,q_l,d,clk,rst_l;
    initial begin
        clk = 0;    //过程赋值
        forever # (CYCLE/2) clk = ~clk;    //连续赋值
    end

    assign rst_l = ~rst_h; // 连续赋值
    not n1(q_l,q); // q_l被门驱动
```

```
my_dff d1(q,d,clk,rst_1); //被模块驱动
endmodule
```

由于logic类型只能有一个驱动,所以其可被适用于查找网单中的漏登 如果存在多个驱动时,编译就会出现错误

双状态数据类型

相比四状态数据类型: 0, 1, X(未知), Z(高阻态)

双状态数据类型有利于提高仿真性能并减少内存的使用量,最简单的双状态数据类型是bit,它是无符号的,另外四种带符号的双状态数据类型是byte,shortint,int 和longint

```
bit b;                //双状态,单比特
bit [31:0] b32;       //双状态,32比特无符号整数
int unsigned ui;      //双状态,32比特无符号整数
int i;                //双状态,32比特有符号整数
byte b8;              //双状态,8比特有符号整数
shortint s;           //双状态,16比特有符号整数
longint l;            //双状态,64比特有符号整数
integer i4;           //四状态,32比特有符号整数
time t;               //四状态,64比特无符号整数
real r;               //双状态,双精度浮点数
```

对于双状态变量,被测设计试图产生X或Z, 这些值会被转换成双状态值,而测试代码永远察觉不了

对四状态值的检查:

```
if($isunknown(iport) == 1)
    $display("@%0t: 4-state value detected on iport %b", $time, iport);
```

定宽数组

定宽数组的声明和初始化 Verilog要求在声明中必须给出数组的上下界,因为几乎所有数组都使用0作为索引下界 所以SystemVerilog允许只给出数组宽度的便捷声明方式,跟C语言类似

定宽数组的声明

```
int lo_hi[0:15];      //16个整数[0]...[15]
int c_style[16];      //等价声明方式
```

可以通过在变量后面指定维度的方式来创建多维定宽数组

下面创建了几个二维数组

```
int arr[3][4];           //3行4列的整数数组
int arr2[0:2][0:3];     //等价声明方式
arr[1][2] = 1;          //设置元素值
```

很多SystemVerilog仿真器在存放数组元素时使用32比特的字边界,所以byte,shortint和int都是存放在一个字中,而longint则存放到两个字中

在非合并数组中,字的低位用来存放数据,高位则不使用

```
bit [7:0] arr[3];        //3个字节的数组
```

常量数组

```
int ascend[4] = '{1,2,3,4}; //等价声明方式
int descend[4];
descend = '{4,3,2,1};

descend[0:2] = '{3,2,1}; //等价声明方式
ascend = '{4{8}}; //四个值全部为0
descend = '{9,8,default:1};
```

基本的数组操作:for和foreach

在数组操作中使用for和foreach循环

```
initial begin
    bit [31:0] src[5],dst[5];
    for(int i = 0;i<$size(src);i++)
        src[i] = i;
    foreach(dst[j])
        dst[j] = src[j]*2; //dst的值是src的两倍
end
```

初始化并遍历多维数组

```
int md[2][3] = '{ '{1,2,3}, '{4,5,6}}; //二维数组的初始化
initial begin
    $display("Initial value:");
    foreach(md[i,j]) //这是正确的语法格式
        $display("md[%0d][%0d] = %0d", i, j, md[i][j]);
    $display("New value:");
    md = '{ '{7,8,9}, '{3{32'd5}}; //修改数组元素
    foreach(md[i,j])
```

```

        $display("md[%0d][%0d] = %0d", i, j, md[i][j]);
    end

```

```

initial begin
    byte twoD[4][6];
    foreach(twoD[i,j])
        twoD[i][j] = i*10+j;

    foreach(twoD[i]) begin //遍历第一个维度
        $write("%2d: ", i);
        foreach(twoD[,j]) //遍历第二个维度
            $write("%0d ", twoD[i][j]);
        $display("");
    end
end

```

基本的数组操作 ----复制和比较

数组的复制和比较操作

```

initial begin
    bit [31:0] src[5] = '{1,2,3,4,5},
                dst[5] = '{5,4,3,2,1};

    //两个数组的聚合比较
    if(src == dst)
        $display("src == dst");
    else
        $display("src!= dst");
    dst = src; //复制src到dst
    // 只改变一个元素的值
    src[0] = 5;
    //所有元素的值是否相等(否!)
    $display("src == dst: %b", (src == dst)? "==" : "!=");

    //使用数组片段对第1-4个元素进行比较
    $display("src[1:4] %s dst[1:4]",(src[1:4] == dst[1:4])? "==" : "!=");
end

```

同时使用位下标和数组下标

```

initial begin
    bit [31:0] arr[5] = '{5{5}};
    $display(src[0],,src[0][0],,src[0][2:1]);
    //使用位下标访问数组元素
end

```

```

bit [3:0][7:0] bytes; //4个字节组装成32比特
bytes = 32'hCafe_Data;
$displayh(bytes,, //显示所有32比特
          bytes[3],, //最高字节“CA” 两个h为一个字节
          bytes[3][7]); //最高比特位“1”

```

合并/非合并混合数组的声明

```

bit [3:0][7:0] barray [3]; //合并:3x32比特
bit [31:0] lw = 32'h0123_4567; //字
bit [7:0][3:0] nibbles; //合并数组
barray[0] = lw;
barray[0][3] = 8'h01;
barray[0][1][6] = 1'b1;
nibbles = barray[0]; //非合并数组

```

合并数组和非合并数组的选择

动态数组在声明时使用空的下标[], 即数组的宽度不在编译时给出而在程序运行时再指定

数组最开始时是空的,所以你必须调用new[]操作符来分配空间,同时在方括号中传递数组宽度

```

int dyn[],d2[]; //声明动态数组

initial begin
    dyn = new[5]; //分配5个元素
    foreach(dyn[j]) dyn[j] = j; //对元素进行初始化
    d2 = dyn; //复制一个动态数组
    d2[0] = 5; //修改复制值
    $display("dyn[0] = %0d, d2[0] = %0d", dyn[0], d2[0]);
    dyn = new[20](dyn); //分配20个整数值并进行复制
    dyn = new [100]; //分配100个元素的数组,旧值将不存在
    dyn.delete(); //删除所有元素
end

```

队列

队列结合了链表和数组的优点 队列和链表相似,可以在一个队列中任何地方增加或者删除元素,这类操作在性能上的损失比动态数组小得多,因为动态数组需要分配新的数组并复制所有元素的值,队列与元素相似,可以通过索引实现对任意元素的访问,而不需要像链表那样去遍历目标元素之前的所有元素

队列的操作

```

int j = 1,
    q2[$] = {3, 4}, // 队列的常量不需要使用“ ’ ”
    q[$] = {0, 2, 5}; // {0, 2, 5}

```



```

initial begin
    q.insert(1, j);          // {0, 1, 2, 5}    在 2 之前插入 1
    q.insert(3, q2);         // {0, 1, 2, 3, 4, 5} 在 q 中插入一个队列①
    q.delete(1);             // {0, 2, 3, 4, 5} 删除第 1 个元素
end

// 下面的操作执行速度很快
q.push_front(6);           // {6, 0, 2, 3, 4, 5} 在队列前面插入
j = q.pop_back;            // {6, 0, 2, 3, 4}    j = 5
q.push_back(8);            // {6, 0, 2, 3, 4, 8} 在队列末尾插入
j = q.pop_front;          // {0, 2, 3, 4, 8}    j = 6

foreach (q[i])
    $display(q[i]);        //                打印整个队列

q.delete();                // {}                删除整个队列

```

```

// 例 2.20 队列操作
int j = 1,
    q2[$] = {3, 4},        // 队列的常量不需要使用“ ’ ”
    q[$] = {0, 2, 5};      // {0, 2, 5}

initial begin
    // 结果
    q = {q[0], j, q[1:$]}; // {0, 1, 2, 5} 在 2 之前插入 1
    q = {q[0:2], q2, q[3:$]}; // {0, 1, 2, 3, 4, 5} 在 q 中插入一个队列
    q = {q[0], q[2:$]};     // {0, 2, 3, 4, 5} 删除第 1 个元素

    // 下面的操作执行速度很快
    q = {6, q};             // {6, 0, 2, 3, 4, 5} 在队列前面插入

    // ① 并不是所有的 SystemVerilog 仿真器都支持使用 insert() 对队列插入新值。

    j = q[$];               // 等同于 j = 5
    q = q[0:$ - 1];         // {6, 0, 2, 3, 4} 从队列末尾取出数据
    q = {q, 8};             // {6, 0, 2, 3, 4, 8} 在队列末尾插入
    j = q[0];               // 等同于 j = 6
    q = q[1:$];             // {0, 2, 3, 4, 8} 从队列前面取出数据

    q = {};                 // {} 删除整个队列
end

```

关联数组

关联数组采用在方括号中放置数据类型的形式来进行声明,例如[int]或[packet]

关联数组的声明,初始化和使用

```

initial begin
    bit[63:0] assoc[bit[63:0]],idx = 1;

    //对稀疏分布的元素进行初始化
    repeat (64) begin
        assoc[idx] = idx;
        idx = idx << 1;
    end

    //使用foreach遍历数组
    foreach(assoc[i])
        $display("assoc[%0d] = %0d", i, assoc[i]);

    //使用函数遍历数组
    // 使用 foreach 遍历数组
    foreach (assoc[i])
        $display("assoc[%h]=%h", i, assoc[i]);

    // 使用函数遍历数组
    if (assoc.first(idx))
        begin
            // 得到第一个索引
            do
                $display("assoc[%h]=%h", idx, assoc[idx]);
                while (assoc.next(idx)); // 得到下一个索引
            end

            // 找到并删除第一个元素
            assoc.first(idx);
            assoc.delete(idx);
            $display("The array now has %0d elements", assoc.num);
        end
end

```

使用带字符索引的关联数组

```

int switch[string],min_address,max_address;
initial begin
    int i,r,file;
    string s;
    file = $fopen("switch.txt","r");
    while(!$feof(file)) begin
        r = $fscanf(file,"%s %d %d",s,i,r);
        switch[s] = i;
    end
    $fclose(file);

    // 获取最小地址值
    min_address = switch["min_address"];

```

```
// 获取最大地址值,缺省为1000
if(switch.exists("max_address"))
  max_address = switch["max_address"];
else
  max_address = 1000;

//打印数组的所有元素
foreach(switch[s])
  $display("switch[%s] = %d", s, switch[s]);
end
```

数组方法

```
bit on[10]; //单比特数组
int total;

initial begin
  foreach(on[i])
    on[i] = i;

  //打印出单比特和
  $display("on.sum = %0d", on.sum);

  //打印出单比特和
  $display("on.sum = %0d", on.sum + 32'd0);

  // 由于 total 是 32 比特 变量,所以数组和也是32 比特
  total = on.sum;
  $display("total = %0d", total);

  // 将数组和与一个32比特进行比较
  if(on.sum >= 32'd5)
    $display("sum has 5 or more bits");

  // 使用带32bit有符号运算的with表达式
  $display("on.sum with 32-bit signed addition = %0d", on.sum.with(32'sd0));
end
```

如果想从一个关联数组中随机选取一个元素,需要逐个访问它之前的元素 原因是没有办法直接访问到第N个元素

从一个关联数组中随机选取一个元素

```
int aa[int],rand_idx,element_count;

element = $urandom_range(aa.size() - 1);
```

```
foreach(aa[i])
  if(count++ == element) begin
    rand_idx = i;
    break;
  end
  $display("0d element aa[%0d] = %0d", rand_idx, aa[rand_idx]);
```

数组定位方法

```
int f[6] = '{1,6,2,6,8,6};

tq = q.min();
tq = d.max();
tq = f.unique();

tq = d.find with(item > 3);
tq.delete();
foreach(d[i])
  if(d[i] > 3)
    tq.push_back(d[i]);

tq = d.find_index with(item > 3);
tq = d.find_first with(item > 99);
tq = d.find_first_index with(item == 8);
tq = d.find_last with(item == 4);
tq = d.find_last_index with(item == 4);
```

数组定位方法

```
int count,total,d[] = '{9 , 1 , 8 , 3 ,4 ,4};

count = d.sum_with(item > 7);// 2 : {9, 8}

total = d.sum with((item > 7) * item); // 17 = 9 + 8

count = d.sum with(item < 8);
```

数组的排序

```
int d [] = '{9 , 1 , 8 , 3 ,4 ,4};
d.reverse();
d.sort();
d.rsort();
d.shuffle();
```

```

struct packed {byte red,green,blue;} c[];

initial begin
    c = new[100];
    foreach(c[i])
        c[i] =$urandom;
    c.sort with(item.red);

    c.sort with({x.green,x.blue});

end

```

使用数组定位方法建立计分板

```

typedef struct packed
{
    bit[7:0]addr;
    bit[7:0] pr;
    bit[15:0] data;
} Packet;

Packet scb[$];

function void check_addr(bit [7:0] addr);
    int into[$];

    intq = scb.find_index() with(item.addr == addr);
    case(intq.size())
    0: $display("Addr %h not found in scoreboard",addr);
    1: scb.delete(intq[0]);
    default:
        $display("Error : Multiple hits for addr %h",addr);

    endcase
endfunction : check_addr

```

使用typedef 创建新的类型

Verilog 中用户自定义的类型宏

```

`define OPSIZE 8
`define OPREG reg[`OPSIZE-1:0]

`OPREG op_a,op_b;

```

```
parameter OPSIZE = 8;
typedef reg[OPSIZE-1:0] opreg_t;

opreg_t op_a,op_b;
```

uint的定义

```
typedef bit [31:0] uint;
typedef int unsigned uint; //等效的定义
```

用户自定义数组类型

```
typedef int fixed_array5[5];
fixed_array5 f5;

initial begin
    foreach(f5[i])
        f5[i] = i;
end
```

使用struct 创建新类型

```
struct {bit[7:0] r,g,b;} pixel;

typedef struct {bit[7:0] r,g,b;} pixel_s;
pixel_s my_pixel;
```

对结构体进行初始化

对struct 类型进行初始化

```
initial begin
    typedef struct {
        int a;
        byte b;
        shortint c;
        int d;
    }my_struct_s;

    my_struct_s st = '{32'haaaa_aaaad,8'hbb,16'hcccc,32'hdddd_dddd};
    $display("st.a = %0d, st.b = %0d, st.c = %0d, st.d = %0d", st.a, st.b, st.c,
```

```
st.d);
end
```

创建可容纳不同类型的联合

```
typedef union {int i ; real f;} num_u;
num_u un;
un.f = 0.0; //把数值设置为浮点形式
```

合并结构

pixel使用三个数值,所以它占用了三个长字的存储空间,即使它实际只需要三个字节

```
typedef struct packed {
    bit[7:0] r;
    bit[7:0] g;
    bit[7:0] b;
} pixel_p_s;

pixel_p_s p;
```

类型转换

静态转换

在整型和实型之间进行静态转换

```
int i;
real r;

i = int '(10.0 - 0.1); //转换为非强制的
r = real '(42);
```

动态转换

\$cast动态转换函数允许对越界的数值进行检查

流操作符

```
initial begin
    int h;
    bit [7:0] b, g[4], j[4] = '{8'ha, 8'hb, 8'hc, 8'hd};
    bit [7:0] q, r, s, t;
```

```

h = {>>{j}};          // 0a0b0c0d- 把数组打包成整型
h = {<<{j}};          // b030d050 位倒序
h = {<<byte{j}};      // 0d0c0b0a 字节倒序
g = {<<byte{j}};      // 0d,0c,0b,0a 拆分成数组
b = {<<{8'b0011_0101}}; // 1010_1100 位倒序
b = {<<4{8'b0011_0101}}; // 0101_0011 半字节倒序
{>>{q,r,s,t}} = j;    // 把 j 分散到四个字节变量里
h = {>>{t,s,r,q}};    // 把字节集中到 h 里
end

```

也可以使用很多连接符来完成同样的操作

数组元素会根据需要自动分配

```

initial begin
  bit [15:0] wq[$] = '{16'h1234, 16'h5678, 16'h9abc,16'ddef0};
  bit [7:0] bq[$];

  // 把数组转换成字节数组
  bq = {>>{wq}}; // 12 34 56 78

  // 把字节数组转换成数组
  bq = {8'h98, 8'h76, 8'h54, 8'h32};
  wq = {>>{bq}}; // 98 76 54 32
end

```

流操作符也可用来将结构打包或者拆分到字节数组中

```

initial begin
  typedef struct {
    int a;
    byte b;
    shortint c;
    int d;
  }my_struct_s;

  my_struct_s st = '{32'haaaa_aaaad,8'hbb,16'hcccc,32'hdddd_dddd};

  byte b[];

  //将结构转换成字节数组
  b = {>> {st}}; // aa aa aa aa bb cc cc dd dd dd dd

  //将字节数组转换成结构
  my_struct_s st2;
  st2 = {>> {b}}; // st2.a = 32'haaaa_aaaad, st2.b = 8'hbb, st2.c = 16'hcccc,
  st2.d = 32'hdddd_dddd
end

```


枚举类型

```
enum {RED,BLUE,GREEN} color;
```

```
// 创建代表 0,1,2 的数据类型
typedef enum {INIT, DECODE, IDLE} fsmstate_e;
fsmstate_e pstate, nstate;          // 声明自定义类型变量

initial begin
    case (pstate)
        IDLE: nstate = INIT;        // 数据赋值
        INIT: nstate = DECODE;
        default: nstate = IDLE;
    endcase
    $display("Next state is %s",
            nstate.name());          // 显示状态的符号名
end
```

1. first() 返回第一个枚举常量
2. last() 返回最后一个枚举常量
3. next() 返回下一个枚举常量
4. next(N) 返回以后第N个枚举常量
5. prev() 返回前一个枚举 变量
6. prev(N) 返回以前第N个枚举常量

常量

```
// 常量声明
initial begin
    const byte colon = ":";
end
```

字符串方法

```
string s ;
initial begin
    s = "IEEE";
    $display(s.getc(0));
    $display(s.tolower());

    s.putc(s.len()-1, "-");
    s = s(s,"P1800");

    $display(s.substr(2,5));

    my_log($psprintf("s = %s %5d",s,42));
end
```

```
task my_log(string message);
    $display("@%0t: %s", $time, message);
endtask
```

表达式的位宽

```
bit [7:0] b8l;
bit one = 1'b1;

$display(one + one); // A: 1+ 1 =0

b8 = one + one; // B: 1+ 1 = 2

$display(b8); // C: 2

display(b8l); // D: 2

$display(b8l); // E: 2
```

过程语句和子程序

过程语句

```
initial
begin:example
    integer array[10],sum,j;

    // 在for语句中声明 i
    for(int i = 0; i < 10; i++)
        array[i] = i;

    // 把数组里的元素相加
    sum = array[9];
    j = 8;
    do // do 。。。 while循环
        sum += array[j]; //累加
    while(j --); // 判断j = 0是否成立
    $display("sum = %0d", sum); // 指定宽度
end : example //结束标识符
```

任务,函数 void函数

如果有一个不消耗时间的SystemVerilog任务,应该把它定义成void函数,这种函数没有返回值,便于被任何其他任务或函数所调用

```
function void print_state(...);
    $display("@%0t:state = %s", $time, cur_state.name());
endfunction
```

忽略函数的返回值

```
void' ($fscanf(file, "%d",i));
```

任务和函数概述

不带begin...end的简单任务

```
task multiple_lines;
    $display("Line 1");
    $display("Line 2");
endtask
```

子程序参数

C语言风格的子程序参数

```
task mytask2;
    output [31:0] x; //方向声明
    reg [31:0] x; //类型声明
    input y;
    ....
endtask
```

而在SystemVerilog,可以采用简明C语言风格

```
task mytask3(output logic [31:0] x,input logic y);
    ....
endtask
```

Verilog对参数的处理方式很简单,在子程序开头把input和inout的值复制给本地变量,在子程序退出时则复制output和inout的值,除了标量以外,没有把任何存储器传递给Verilog子程序

而SystemVerilog中,可以指定为引用而不是复制 这种ref参数类型比input,output或inout更好用

```
function void print_checksum(const ref bit [31:0] a[]);
    bit [31:0] checksum = 0;
    for(int i = 0 ;i < a.size(); i++)
        checksum ^= a[i];
    $display("Checksum = %0d", checksum);
endfunction
```

ref还有好处: 在任务里可以修改变量而且修改结果对调用它的函数随时可见

```
task bus_read(input logic [31:0] addr,
              ref logic [31:0] data);

    // 请求总线并驱动地址
    bus.request = 1'b1;
    @(posedge bus.grant) bus.addr = addr;

    // 等待来自存储器的数据
    @(posedge bus.enable) data = bus.data;

    // 释放总线并等待许可
    bus.request = 1'b0;
    @(negedge bus.grant);
endtask

logic [31:0] addr,data;

initial
    fork
        bus_read(addr,data);
        thread2:
            begin
                @data //在数据变化时触发
                $display("Data = %0d", data);
            end
    join
```

在SystemVerilog中,可以为参数指定一个缺省值,如果在调用时不指明参数(和cpp一样)

```
task sticky (ref int array[50],int a,b);
```

此时int a,b的方向为ref与初始设计不符

子程序的返回

```
task load_array(int len, ref int array[]);
    if(len < 0) begin
```

```

        $display("Bad len");
        return;
    end

    //任务中其余代码
    ...
endtask

// 也可以用于函数返回
function bit transmit(...);
    // 发送处理
    return ~ifc.cb.error; //返回状态: 0 = error
endfunction

```

从函数中返回一个数组

```

typedef int fixed_array5[5];
fixed_array5 f5;

function fixed_array5 init(int start);
    foreach(init[i])
        init[i] = start + i;
endfunction

initial begin
    f5 = init(5);
    foreach(f5[i])
        $display("f5[%0d] = %0d", i, f5[i]);
end

```

函数的返回值类型是 `fixed_array5`（一维数组）。在 SystemVerilog 中，当函数需要返回数组时，会隐式创建一个临时数组变量来承载返回值，这个临时变量的名字就是函数名 `init`。

另一种方式是通过引用传递数组给函数

```

function void init(ref int f[5],input int start);
    foreach(f[i])
        f[i] = start + i;
endfunction

```

局部数据存储

在 SystemVerilog 中，模块 (module) 和 program 块中的子程序缺省情况下仍然使用静态存储（局部变量和子程序参数被存放在共享的静态存储区，多线程之间会窜用局部变量），如果使用自动存储，则必须在程序语句中加入 `automatic` 关键字

```

program automatic test;
    task wait_for_mem(input [31:0] addr,expect_data,output success);
        while(bus.addr != addr)
            @(bus.addr);
        success = (bus.data == expect_data);
    endtask
endprogram

```

如果没有修饰符automatic,由于第一次调用的任务处于等待状态,所以对wait_for_mem的第二次调用会覆盖它的两个参数

修复静态初始化漏洞

```

program automatic initialization; //漏洞被修复
...
endprogram

```

修复静态初始化的漏洞,把声明和初始化拆开

```

logic [7:0] local_addr;
local_addr = addr << 2;

```

时间值

时间单位和精度

SystemVerilog允许使用数值和单位来明确指定一个时间值, 代码里可以使用类似0.1ns和20ps的时延,只要记得使用timeunit和timeprecision或者timescale即可

你还可以通过使用经典的verilog时间函数\$timeformat, \$time和\$realtime 来使代码在时间标度上更清楚

```

module timing;
    timeunit 1ns;
    timeprecision 1ps;
    initial begin
        $timeformat(-9,3,"ns",8);
        #1 $display("%t",$realtime);// 1.000ns
        #2ns $display("%t",$realtime);// 3.000ns
        #0.1ns $display("%t",$realtime);// 3.100ns
        #41ps $display("%t",$realtime);// 3.141ns
    end
endmodule

```

使用实型变量real保存精确的数值，它们只在用作时延量的时候才被舍入

时间变量及舍入

```
`timescale 1ps/1ps
module ps;
    initial begin
        real rdelay=800fs;           // 以 0.800 存储
        time tdelay=800fs;           // 舍入后得到 1
        $timeformat(-15,0,"fs",5);
        #rdelay;                      // 时延舍入后得到 1ps
        $display("%t",rdelay);        // "800fs"

        #tdelay;                      // 再次延时 1ps
        $display("%t",tdelay);        // "1000fs"
    end
endmodule
```

系统任务\$time的返回值是一个根据所在模块的时间精度要求进行舍入的整数,不带小数的部分,而\$realtime的返回值则是一个带小数部分的完整实数

连接设计和测试平台

由于verilog的端口描述繁琐,代码会长达数页

使用端口的仲裁器模型

```
module arb_port(output logic [1:0] grant,
                input logic [1:0] request,
                input logic rst,
                input logic clk);
    always @(posedge clk or posedge rst) begin
        if(rst)
            grant <= 2'b00;
        else
            ...
    end
endmodule
```

测试平台定义在另一个模块中,与设计所在的 模块相互独立

```
module test (input logic [1:0] grant,
             output logic [1:0] request,
             output logic rst,
             input logic clk);

    initial begin
        @(posedge clk) request <= 2'b01;
```

```

        $display("@%0t: Drove req=01", $time);
        repeat (2) @(posedge clk);
        if (grant != 2'b01)
            $display("@%0t: a1: grant!=2'b01", $time);
        ...
        $finish;
    end

endmodule

```

顶层网单连接了测试平台和DUT 没有接口的顶层网单

```

module top;
    logic [1:0] grant,request;
    bit clk,rst;
    always #5 clk = ~clk;

    arb_port a1(grant,request,rst,clk);
    test t1(grant,request,rst,clk);
endmodule

```

接口

对仲裁器的第一个改进就是将连线捆绑成一个接口

```

interface arb_if(input bit clk);
    logic [1:0] grant,request;
    logic rst;
endinterface

```

使用了简单接口的仲裁器

```

module arb_port(arb_if arb);
    always@(posedge arbif.clk or posedge arbif.rst)
        begin
            if(arbif.rst)
                arbif.grant <= 2'b00;
            else
                arbif.grant <= arbif.request;
        end
endmodule

```

使用简单仲裁器接口的测试平台


```

module test(arb_if arbif)
    initial begin
        @(posedge arbif.clk) arbif.request <= 2'b01;
        $display("@%0t: Drove req=01", $time);
        if(arbif.grant != 2'b01)
            $display("@%0t: a1: grant!=2'b01", $time);
        $finish;
    end
endmodule test

```

使用简单仲裁器接口的top模块

```

module top;
    bit clk;
    always #5 clk = ~clk;
    arb_if arbif(clk);
    arb a1(arbif);
    test t1(arbif);
endmodule top

```

接口引入的 错误示范

```

// 错误示例 (bad_test.sv)
module bad_test(arb_if arbif); // 1. 这里引用了 arb_if
    'include "MyTest.sv"        // 2. 合法的 include
    'include "arb_if.sv"        // 3. 致命错误!
endmodule

```

'include 的本质：'include 并不是导入库，而是简单的 文本复制粘贴。编译器在处理时，会把 arb_if.sv 里的内容直接粘贴到 'include 这一行所在的位置。作用域冲突：当你把 arb_if.sv 放在 module bad_test ... endmodule 内部时，实际上你是在告诉编译器：“请在 bad_test 这个模块 里面 定义一个叫 arb_if 的接口。”这违反了语法规则。接口不能嵌套定义在模块内部。后果：编译器会报错，因为它在模块内部看到了非法的 interface 关键字。即使某些宽松的编译器不报错，这个接口也会变成 bad_test 的局部变量，外部的其他模块根本无法通过端口列表连接到它。

正确做法:

```

// 正确做法 (good_test.sv)
// 正确写法
'include "arb_if.sv"          // 1. 先定义接口（此时处于全局作用域）

module good_test(arb_if arbif); // 2. 现在可以使用 arb_if 作为端口类型了
    'include "MyTest.sv"
endmodule

```

```
// ... 逻辑代码
endmodule
```

连接接口和端口

如果不能符合对Verilog-2001的旧代码进行修改,将其中的端口改为接口 可以将接口的信号直接连接到每个端口上 连接接口到使用端口 的模块

```
module top;
    bit clk;
    always #5 clk = ~clk;
    arb_if arbif(clk);
    arb_port a1(.grant(arbif.grant),.request(arbif.request),.rst(arbif.rst));

    test t1(arbif);
endmodule top
```

使用modimport将接口中的信号分组

在接口中使用modport结构能够将信号分组并指定方向

```
interface arb_if(input bit clk);

    logic [1:0] grant,request;
    logic rst;

    modport TEST(output request,rst,input grant,clk);
    modport DUT(input request,rst,clk,output grant);
    modport MONITOR(input request,grant,rst,clk);
endinterface

module arb (arb_if.DUT arbif);
    ...
endmodule

module test (arb_if.TEST arbif);
    ...
endmodule
```

通过指定 .DUT, 模块内部的 request 信号被强制视为 input, 而 grant 被强制视为 output。这确保了 DUT 只能读取请求, 不能误操作请求信号。

创建接口监视模块

```
module monitor (arb_if.MONITOR arbif);

    always @(posedge arbif.request[0]) begin
```

```

        $display("@%0t: request[0] asserted", $time);
        @(posedge arbif.grant[0]);
        $display("@%0t: grant[0] asserted", $time);
    end

    always @(posedge arbif.request[1]) begin
        $display("@%0t: request[1] asserted", $time);
        @(posedge arbif.grant[1]);
        $display("@%0t: grant[1] asserted", $time);
    end

endmodule

```

激励时序

带时钟的接口

```

interface arb_if(input bit clk);
    logic [1:0] grant,request;
    logic rst;

    clocking cb @(posedge clk);//声明cb
        output request;
        input grant;
    endclocking

    modport TEST(clocking cb,output rst);
    modport DUT(input request,rst,output grant);
endinterface

//这是一个简单的测试平台
module test(arb_if.TEST arbif);
    initial begin
        arbif.cb.request <= 0;
        @arbif.cb;
        $display("@%0t: request=0", $time,arbif.cb.grant);
    end
endmodule

```

这行代码定义了一个名为 cb (Clocking Block) 的块，它绑定在 clk 信号的上升沿上。这意味着该块内的所有操作都将以 clk 为基准进行同步。

虽然VMM中有一条规则指明将接口信号定义为wire,但是本书建议在接口中将信号定义为logic

```

interface asynch_if();
    logic l;
    wire w;

```

```

endinterface

module test(asynch_if ifc);
    logic local_wire;
    assign ifc.w = local_wire;

    initial begin
        ifc.l <= 0;           // 直接驱动异步 logic 信号 ...

        local_wire <= 1;     // 但是只能用 assign 驱动 wire 信号
        ...
    end
endmodule

```

Verilog时序问题

对设计输出信号的采样存在着相同的问题 你可能知道下一个时钟将会出现在100ns的时候

```

module memory(input wire start,write,
               input wire [7:0] addr,
               inout wire [7:0] data);
    logic [7:0] mem[256];
    always @(posedge start) begin
        if(write)
            mem[addr] <= data;
        end
    endmodule

module test(output logic start,write,
             output logic [7:0] addr,data);
    initial begin
        start = 0; //信号初始化
        write = 0;
        #10; //短暂的延时
        addr = 8'h42; //发起第一个指令
        data = 8'h5a;
        start = 1;
        write = 1;
    end
endmodule

```

竞争状态出现在测试平台先产生start信号,然后再产生其他信号的时候 内存被start信号唤醒的时候,write,addr,data信号仍然保留着原来的值 可以使用非阻塞赋值将所有这些信号都做一个细微延迟

对设计输出信号的采样存在着同样的问题,希望在时钟有效沿到来的之前的最后时刻捕获这些信号的值,你可能知道下一个时钟沿会出现在100ns的时候 你不能在100ns出现时钟边沿的时候采样,因为设计的输出值可能已经改变了,应当在时钟到达之前的testup时间上采样

程序块(Program Block) 和 时序区域(Timing Region)

如果存在一种可以在时间轴上分开这些事件的方法

根据定义,这些值是前一个时间片的最后值

SystemVerilog在一个程序块中,跟模块非常相似:模块含有代码和变量,可以在 其他模块中实例化,但是,程序不能有任何层次级别,例如模块的实例,接口或程序

```
flowchart TD
    Start([From previous time slot]) --> Active
    subgraph CurrentTimeSlot [Current Time Slot]
        direction TB
        Active[Active<br>design] --> Observed
        Observed[Observed<br>assertions] --> Reactive
        Reactive[Reactive<br>testbench] --> Postponed
    end
    Postponed --> End([To next time slot])

    %% 循环逻辑
    Reactive --> |Loop back if more events| Active
    Observed --> |Loop back if more events| Active

    style Active fill:#f9f9f9,stroke:#333,stroke-width:2px
    style Observed fill:#f9f9f9,stroke:#333,stroke-width:2px
    style Reactive fill:#f9f9f9,stroke:#333,stroke-width:2px
    style Postponed fill:#f9f9f9,stroke:#333,stroke-width:2px
```

- 1. 首先第一个时间片执行Active,在这个区域中运行设计事件,包括RTL门级代码和时钟发生器
- 2. Observed区域,执行断言
- 3. Reactive区域和Observed区域事件可以触发本周期内Active区域进一步设计事件
- 4. Postponed区域将在时间片的最后,所有设计活动都结束后的只读时间段采样信号

区域名	行为
Active	仿真模块中的设计代码
Observed	执行 SystemVerilog 断言
Reactive	执行程序中的测试平台部分
Postponed	为测试平台的输入采样信号

```
program automatic test(arb_if.TEST arbif);

    initial begin
        arbif.cb.request <= 2'b01;
        $display("@%0t: Drove req=01", $time);
    end
endprogram
```

```

repeat(2) @arbif.cb;
if(arbif.cb.grant != 2'b01)
    $display("@%0t: a1: grant!=2'b01", $time);
end
endprogram

```

测试代码应当包含在一个单个的程序块中,应当使用OOP通过对象而非模块来创建一个动态,分层的测试平台

如果使用了其他人的代码或者是把多个测试代码结合在一起,那么一次仿真就可能有多多个程序块

在Verilog中,仿真在调度事件存在的时候会继续执行,直到遇到\$finish, SystemVerilog增加了一种结束仿真的方法, SystemVerilog把任何一个程序块都视作含有一个测试

如果仅有一个程序块,那么当完成所有initial块中的最后一个语句时,仿真就结束了,因为编译器认为这就是测试的结尾

即使还有模块或者程序库的线程在运行,仿真也会结束

所以当测试结束的时候无需关闭所有的监视器或驱动器

指定设计和测试平台之间的延迟

1. 核心概念：默认时序

文中提到：“时钟块的默认时序是在 ``systemverilog

```

module timing; timeunit 1ns; timeprecision 1ps; initial begin $timeformat(-9,3,"ns",8); #1
$display("%t", $realtime); // 1.000ns #2ns $display("%t", $realtime); // 3.000ns #0.1ns $display("%t", $realtime); //
3.100ns #41ps $display("%t", $realtime); // 3.141ns end endmodule

```

使用实型变量real保存精确的数值，它们只在用作时延量的时候才被舍入

时间变量及舍入

```

``systemverilog
`timescale 1ps/1ps
module ps;
    initial begin
        real rdelay=800fs;           // 以 0.800 存储
        time tdelay=800fs;           // 舍入后得到 1
        $timeformat(-15,0,"fs",5);
        #rdelay;                     // 时延舍入后得到 1ps
        $display("%t", rdelay);       // "800fs"

        #tdelay;                     // 再次延时 1ps
        $display("%t", tdelay);       // "1000fs"
    end
endmodule

```

系统任务\$time的返回值是一个根据所在模块的时间精度要求进行舍入的整数,不带小数的部分,而\$realtime的返回值则是一个带小数部分的完整实数

连接设计和测试平台

由于verilog的端口描述繁琐,代码会长达数页

使用端口的仲裁器模型

```
module arb_port(output logic [1:0] grant,
               input logic [1:0] request,
               input logic rst,
               input logic clk);
    always @(posedge clk or posedge rst) begin
        if(rst)
            grant <= 2'b00;
        else
            ...
        end
    endmodule
```

测试平台定义在另一个模块中,与设计所在的 模块相互独立

```
module test (input logic [1:0] grant,
            output logic [1:0] request,
            output logic rst,
            input logic clk);

    initial begin
        @(posedge clk) request <= 2'b01;
        $display("@%0t: Drove req=01", $time);
        repeat (2) @(posedge clk);
        if (grant != 2'b01)
            $display("@%0t: a1: grant!=2'b01", $time);
        ...
        $finish;
    end

endmodule
```

顶层网单连接了测试平台和DUT 没有接口的顶层网单

```
module top;
    logic [1:0] grant,request;
    bit clk,rst;
    always #5 clk = ~clk;

    arb_port a1(grant,request,rst,clk);
```

```
test t1(grant,request,rst,clk);
endmodule
```

接口

对仲裁器的第一个改进就是将连线捆绑成一个接口

```
interface arb_if(input bit clk);
    logic [1:0] grant,request;
    logic rst;
endinterface
```

使用了简单接口的仲裁器

```
module arb_port(arb_if arb);
    always@(posedge arbif.clk or posedge arbif.rst)
        begin
            if(arbif.rst)
                arbif.grant <= 2'b00;
            else
                arbif.grant <= arbif.request;
            end
        end
endmodule
```

使用简单仲裁器接口的测试平台

```
module test(arb_if arbif)
    initial begin
        @(posedge arbif.clk) arbif.request <= 2'b01;
        $display("@%0t: Drove req=01", $time);
        if(arbif.grant != 2'b01)
            $display("@%0t: a1: grant!=2'b01", $time);
        $finish;
    end
endmodule test
```

使用简单仲裁器接口的top模块

```
module top;
    bit clk;
    always #5 clk = ~clk;
    arb_if arbif(clk);
    arb a1(arbif);
endmodule
```



```
test t1(arbif);
endmodule top
```

接口引入的 错误示范

```
// 错误示例 (bad_test.sv)
module bad_test(arb_if arbif); // 1. 这里引用了 arb_if
    'include "MyTest.sv"        // 2. 合法的 include
    'include "arb_if.sv"        // 3. 致命错误!
endmodule
```

'include 的本质：'include 并不是导入库，而是简单的 文本复制粘贴。编译器在处理时，会把 arb_if.sv 里的内容直接粘贴到 'include 这一行所在的位置。作用域冲突：当你把 arb_if.sv 放在 module bad_test ... endmodule 内部时，实际上你是在告诉编译器：“请在 bad_test 这个模块 里面 定义一个叫 arb_if 的接口。”这违反了语法规则。接口不能嵌套定义在模块内部。后果：编译器会报错，因为它在模块内部看到了非法的 interface 关键字。即使某些宽松的编译器不报错，这个接口也会变成 bad_test 的局部变量，外部的其他模块根本无法通过端口列表连接到它。

正确做法:

```
// 正确做法 (good_test.sv)
// 正确写法
'include "arb_if.sv"          // 1. 先定义接口（此时处于全局作用域）

module good_test(arb_if arbif); // 2. 现在可以使用 arb_if 作为端口类型了
    'include "MyTest.sv"
    // ... 逻辑代码
endmodule
```

连接接口和端口

如果不能符合对Verilog-2001的旧代码进行修改,将其中的端口改为接口 可以将接口的信号直接连接到每个端口上 连接接口到使用端口 的模块

```
module top;
    bit clk;
    always #5 clk = ~clk;
    arb_if arbif(clk);
    arb_port a1(.grant(arbif.grant),.request(arbif.request),.rst(arbif.rst));

    test t1(arbif);
endmodule top
```

使用modimport将接口中的信号分组

在接口中使用modport结构能够将信号分组并指定方向

```
interface arb_if(input bit clk);

    logic [1:0] grant,request;
    logic rst;

    modport TEST(output request,rst,input grant,clk);
    modport DUT(input request,rst,clk,output grant);
    modport MONITOR(input request,grant,rst,clk);
endinterface

module arb (arb_if.DUT arbif);
    ...
endmodule

module test (arb_if.TEST arbif);
    ...
endmodule
```

通过指定 .DUT，模块内部的 request 信号被强制视为 input，而 grant 被强制视为 output。这确保了 DUT 只能读取请求，不能误操作请求信号。

创建接口监视模块

```
module monitor (arb_if.MONITOR arbif);

    always @(posedge arbif.request[0]) begin
        $display("@%0t: request[0] asserted", $time);
        @(posedge arbif.grant[0]);
        $display("@%0t: grant[0] asserted", $time);
    end

    always @(posedge arbif.request[1]) begin
        $display("@%0t: request[1] asserted", $time);
        @(posedge arbif.grant[1]);
        $display("@%0t: grant[1] asserted", $time);
    end

endmodule
```

激励时序

带时钟的接口

```
interface arb_if(input bit clk);
    logic [1:0] grant,request;
```

```

    logic rst;

    clocking cb @(posedge clk); // 声明 cb
        output request;
        input grant;
    endclocking

    modport TEST(clocking cb, output rst);
    modport DUT(input request, rst, output grant);
endinterface

// 这是一个简单的测试平台
module test(arb_if.TEST arbif);
    initial begin
        arbif.cb.request <= 0;
        @arbif.cb;
        $display("@%0t: request=0", $time, arbif.cb.grant);
    end
endmodule

```

这行代码定义了一个名为 cb (Clocking Block) 的块，它绑定在 clk 信号的上升沿上。这意味着该块内的所有操作都将以 clk 为基准进行同步。

虽然VMM中有一条规则指明将接口信号定义为wire,但是本书建议在接口中将信号定义为logic

```

interface asynch_if();
    logic l;
    wire w;
endinterface

module test(asynch_if ifc);
    logic local_wire;
    assign ifc.w = local_wire;

    initial begin
        ifc.l <= 0;           // 直接驱动异步 logic 信号 ...

        local_wire <= 1;     // 但是只能用 assign 驱动 wire 信号
        ...
    end
endmodule

```

Verilog时序问题

对设计输出信号的采样存在者相同的问题 你可能知道下一个时钟将会出现在100ns的时候

```

module memory(input wire start, write,
              input wire [7:0] addr,

```

```

        inout wire [7:0] data);
    logic [7:0] mem[256];
    always @(posedge start) begin
        if(write)
            mem[addr] <= data;
        end
    endmodule

    module test(output logic start,write,
                output logic [7:0] addr,data);
        initial begin
            start = 0; //信号初始化
            write = 0;
            #10; //短暂的延时
            addr = 8'h42; //发起第一个指令
            data = 8'h5a;
            start = 1;
            write = 1;
        end
    endmodule

```

竞争状态出现在测试平台先产生start信号,然后再产生其他信号的时候 内存被start信号唤醒的时候,write,addr,data信号仍然保留着原来的值 可以使用非阻塞赋值将所有这些信号都做一个细微延迟

对设计输出信号的采样存在着同样的问题,希望在时钟有效沿到来的之前的最后时刻捕获这些信号的值,你可能知道下一个时钟沿会出现在100ns的时候 你不能在100ns出现时钟边沿的时候采样,因为设计的输出值可能已经改变了,应当在时钟到达之前的testup时间上采样

程序块(Program Block) 和 时序区域(Timing Region)

如果存在一种可以在时间轴上分开这些事件的方法

根据定义,这些值是前一个时间片的最后值

SystemVerilog在一个程序块中,跟模块非常相似:模块含有代码和变量,可以在 其他模块中实例化,但是,程序不能有任何层次级别,例如模块的实例,接口或程序

```

flowchart TD
    Start([From previous time slot]) --> Active
    subgraph CurrentTimeSlot [Current Time Slot]
        direction TB
        Active[Active<br>design] --> Observed
        Observed[Observed<br>assertions] --> Reactive
        Reactive[Reactive<br>testbench] --> Postponed
    end
    %% 循环逻辑
    Reactive -.-> |Loop back if more events| Active
    Observed -.-> |Loop back if more events| Active

```

```
Postponed[Postponed<br>sample] --> End([To next time slot])
```

```
style Active fill:#f9f9f9,stroke:#333,stroke-width:2px
style Observed fill:#f9f9f9,stroke:#333,stroke-width:2px
style Reactive fill:#f9f9f9,stroke:#333,stroke-width:2px
style Postponed fill:#f9f9f9,stroke:#333,stroke-width:2px
```

1. 首先第一个时间片执行Active,在这个区域中运行设计事件,包括RTL门级代码和时钟发生器
2. Observed区域,执行断言
3. Reactive区域和Observed区域事件可以触发本周期内Active区域进一步设计事件
4. Postponed区域将在时间片的最后,所有设计活动都结束后的只读时间段采样信号

区域名	行为
Active	仿真模块中的设计代码
Observed	执行 SystemVerilog 断言
Reactive	执行程序中的测试平台部分
Postponed	为测试平台的输入采样信号

```
program automatic test(arb_if.TEST arbif);

    initial begin
        arbif.cb.request <= 2'b01;
        $display("@%0t: Drove req=01", $time);
        repeat(2) @arbif.cb;
        if(arbif.cb.grant != 2'b01)
            $display("@%0t: a1: grant!=2'b01", $time);
        end
    endprogram
```

测试代码应当包含在一个单个的程序块中,应当使用OOP通过对象而非模块来创建一个动态,分层的测试平台

如果使用了其他人的代码或者是把多个测试代码结合在一起,那么一次仿真就可能有多多个程序块

在Verilog中,仿真在调度事件存在的时候会继续执行,直到遇到\$finish, SystemVerilog增加了一种结束仿真的方法, SystemVerilog把任何一个程序块都视作含有一个测试

如果仅有一个程序块,那么当完成所有initial块中的最后一个语句时,仿真就结束了,因为编译器认为这就是测试的结尾

即使还有模块或者程序库的线程在运行,仿真也会结束

所以当测试结束的时候无需关闭所有的监视器或驱动器

指定设计和测试平台之间的延迟

1. 核心概念：默认时序

文中提到：“时钟块的默认时序是在 `#1step` 延时之后采样输入信号，在 `#0` 延时之后驱动输出信号。”

这意味着在仿真器处理一个时钟边沿（例如上升沿）时，时钟块会自动插入微小的时间偏移来保证顺序：

- **输入采样 (Input Sampling):**
 - **时机:** `#1step` (即前一个时间步长的最后时刻)。
 - **对应区域:** Postponed Region (推迟区)。
 - **作用:** 在设计 (DUT) 根据当前时钟沿发生任何状态改变之前，先把接口上的信号值“拍个照”存下来。这样测试代码读取到的就是上一周期的稳定值，避免了读到 DUT 刚刚翻转产生的毛刺或中间态。
- **输出驱动 (Output Driving):**
 - **时机:** `#0` (即时钟沿发生的同一时刻)。
 - **对应区域:** Active Region (活动区)。
 - **作用:** 测试平台想要发送的新数据，会在这个时刻同步地送到接口上，供 DUT 在同一个时钟沿进行采样。

2. 图 4.5 解析：同步器的角色

图中的结构展示了数据流向：

- **Testbench (左侧):** 通过 `test out` 发送数据。
- **Design Under Test (中间):** 接收 `in`，产生 `out`。
- **Clocking Block (右侧方框):** 这是一个逻辑上的“同步器”。
 - 它从 DUT 的输出端 (`q`) 采样数据，提供给 Testbench 读取 (`test in`)。
 - 它接收 Testbench 的驱动数据 (`test out`)，并在正确的时间点发送给 DUT 的输入端 (`d`)。

3. 为什么这样做？（解决竞争冒险）

如果不使用时钟块，直接在 `always @(posedge clk)` 中读写信号，很容易出现 **竞争冒险 (Race Condition)**：

- **场景:** DUT 在时钟上升沿更新输出，而 Testbench 也在同一时刻去读取这个输出。
- **风险:** 由于仿真器执行顺序的不确定性，Testbench 可能读到旧值，也可能读到新值，导致验证结果不稳定。

时钟块的解决方案：

- 通过强制将 **采样动作** 提前到 **Postponed** 区域（上一个时间片末尾），确保读到的是绝对稳定的旧值。
- 通过强制将 **驱动动作** 放在 **Active** 区域（当前时间片开始），确保 DUT 能按时收到新激励。

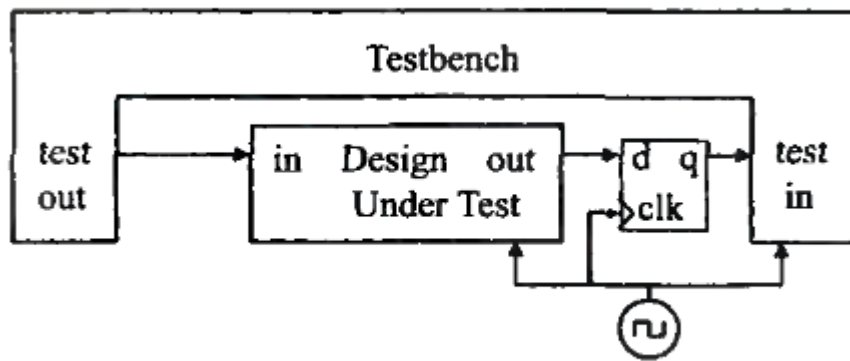


图 4.5 时钟块同步了 DUT 和测试平台

接口的驱动和采样

使用Verilog的@ 和 wait 来同步测试平台中的信号

```
program automatic test(bus_if.TB bus);
  initial begin
    @bus.cb; //在时钟块的有效时钟沿继续
    repeat (3) @bus.cb; //等待3个时钟有效沿
    @bus.cb.grant; //在任何边沿继续
    @(posedge bus.cb.grant); //上升沿继续
    @(negedge bus.cb.grant); //下降沿继续
    wait(bus.cb.grant == 1); //等待表达式被执行,如果已经为真,不作任何延时
    @(posedge bus.cb.grant or negedge bus.rst); // 等待几个信号
  end
endprogram
```

接口信号采样

从时钟模块中读取一个信号的时候,在时钟沿之前得到采样值,例如在Postponed区域,下面的代码给出从一个DUT中读取grant同步信号的程序块

arb在一个时钟周期的中间产生grant信号的值1和2,然后在时钟沿产生值3

```
`timescale 1ns/1ns
program test(arb_if.TEST arbif);
  initial begin
    $monitor("@%0t: grant=%0d", $time, arbif.cb.grant);

    #50ns $display("End of TEST");
  end
endprogram

module arb(arb_if.DUT arbif);
  initial begin
    # 7 arbif.grant = 1; // @ 7ns
```

```

# 10 arbif.grant = 2; // @ 17ns
# 8 arbif.grant = 3; // @ 25ns
end
endmodule

```

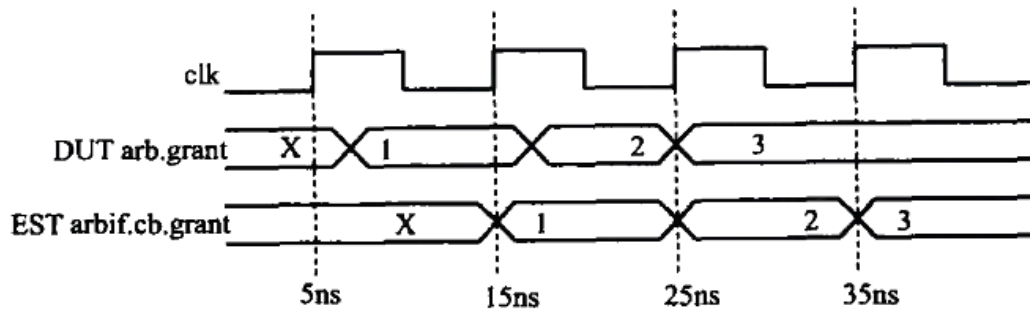


图 4.6 同步接口的采样

arbif.cb.grant在时钟边沿到来之前获得数值,当接口的输入信号恰好在时钟边沿25ns变化的时候,信号的新值并不是在下一个时钟周期传递给测试平台

接口信号驱动

```

program automatic test(arb_if.TEST arbif);
  arbif.cb.request <= 2'b01;
  $display("@%0t: Drove req=01", $time);
  repeat(2) @arbif.cb;
  if(arbif.cb.grant != 2'b01)
    $display("@%0t: a1: grant!=2'b01", $time);
end
endprogram

```

挡在时钟模块中使用modport时候,任何同步接口信号都必须加上接口(arbif)和时钟快名(cb)的前缀

通过时钟块驱动接口信号

接口中的双向信号

程序和接口中的双向信号

```

interface master_if (input bit clk);
  wire [7:0] data; //双向信号
  clocking cb @(posedge clk);
  inout data;
endclocking
modport TEST(clocking cb);
endinterface

program test(master_if mif);
  initial begin
    mif.cb.data<='z;

```



```

    @mif.cb;
    $display(mif.cb.data); //从总线读取
    @mif.cb;
    mif.cb.data <= 7'h5a; // 驱动总线
    @mif.cb;
    mif.cb.data <= 'z; // 读取总线
end
endprogram

```

SystemVerilog程序比Verilog更接近C程序 拥有一个或更多程序入口

一个always块可能从仿真的开始就会在每个时钟上升沿触发 一个测试平台的执行过程经过初始化,驱动和响应设计行为等步骤后结束仿真

当program 中最后一个initial块结束的时候,仿真实际上也就默认结束了

如果确实需要always块,可以使用initial forever来完成

时钟发生器 不应该将时钟发生器放在程序块里

以下为一个正确的时钟发生器

```

module clock_generator(output bit clk);
    initial
        always #5 clk = ~clk; //在时间0之后生成时钟沿
endmodule

```

所有时钟边沿都使用阻塞赋值生成,它们将在active区域触发事件的发生 如果确实需要在0时刻产生一个时钟边沿,可以使用非阻塞赋值语句设置初始值,这样一来所有的时钟敏感逻辑电路比如always块都会在时钟变化之前执行

将这些模块连接起来

```

module top;
    bit clk;
    always #4 clk = ~clk; // 4ns周期的时钟

    arb_if arbif(.*);
    arb a1(.*);
    test t1(.*);
endmodule top

```

快捷符号.*(隐式端口连接),能自动在当前级别自动连接模块实例的端口到具体信号,只要端口和信号的名字和数据类型相同

端口列表中的接口必须连接

SystemVerilog编译器不会让你成功编译任何一个端口列表中含有接口的模块或程序块

```
module uses_a_port(inout bit not_connected);
...
endmodule
```

端口列表中的接口必须连接

SystemVerilog编译器不会让你成功编译任何一个在端口列表中含有接口的模块或者程序块

1. 普通信号 vs 接口对象的区别

文中首先做了一个对比，帮助理解为什么接口比较特殊。

- **普通信号**（如 ``systemverilog program automatic test(bus_if.TB bus); initial begin @bus.cb; //在时钟块的有效时钟沿继续 repeat (3) @bus.cb; //等待3个时钟有效沿 @bus.cb.grant; //在任何边沿继续 @(posedge bus.cb.grant); //上升沿继续 @(negedge bus.cb.grant); //下降沿继续 wait(bus.cb.grant == 1); //等待表达式被执行,如果已经为真,不作任何延时 @(posedge bus.cb.grant or negedge bus.rst); // 等待几个信号 end endprogram

接口信号采样

从时钟模块中读取一个信号的时候,在时钟沿之前得到采样值,例如在Postponed区域,下面的代码给出从一个DUT中读取grant同步信号的程序块

arb在一个时钟周期的中间产生grant信号的值1和2,然后在时钟沿产生值3

```
``systemverilog
`timescale 1ns/1ns
program test(arb_if.TEST arbif);
    initial begin
        $monitor("@%0t: grant=%0d", $time, arbif.cb.grant);

        #50ns $display("End of TEST");
    end
endprogram

module arb(arb_if.DUT arbif);
    initial begin
        # 7 arbif.grant = 1; // @ 7ns
        # 10 arbif.grant = 2; // @ 17ns
        # 8 arbif.grant = 3; // @ 25ns
    end
endmodule
```

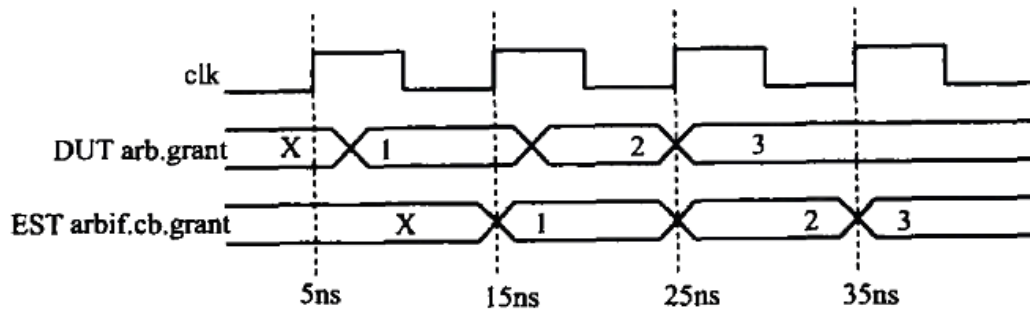


图 4.6 同步接口的采样

arbif.cb.grant在时钟边沿到来之前获得数值,当接口的输入信号恰好在时钟边沿25ns变化的时候,信号的新值并不是在下一个时钟周期传递给测试平台

接口信号驱动

```
program automatic test(arb_if.TEST arbif);
    arbif.cb.request <= 2'b01;
    $display("@%0t: Drove req=01", $time);
    repeat(2) @arbif.cb;
    if(arbif.cb.grant != 2'b01)
        $display("@%0t: a1: grant!=2'b01", $time);
    end
endprogram
```

挡在时钟模块中使用modport时候,任何同步接口信号都必须加上接口(arbif)和时钟快名(cb)的前缀

通过时钟块驱动接口信号

接口中的双向信号

程序和接口中的双向信号

```
interface master_if (input bit clk);
    wire [7:0] data; //双向信号
    clocking cb @(posedge clk);
        inout data;
    endclocking
    modport TEST(clocking cb);
endinterface

program test(master_if mif);
    initial begin
        mif.cb.data<='z;
        @mif.cb;
        $display(mif.cb.data); //从总线读取
        @mif.cb;
        mif.cb.data <= 7'h5a; // 驱动总线
        @mif.cb;
        mif.cb.data <='z; // 读取总线
    end
endprogram
```

```

    end
endprogram

```

SystemVerilog程序比Verilog更接近C程序 拥有一个或更多程序入口

一个always块可能从仿真的开始就会在每个时钟上升沿触发 一个测试平台的执行过程经过初始化,驱动和响应设计行为等步骤后结束仿真

当program 中最后一个initial块结束的时候,仿真实际上也就默认结束了

如果确实需要always块,可以使用initial forever来完成

时钟发生器 不应该将时钟发生器放在程序块里

以下为一个正确的时钟发生器

```

module clock_generator(output bit clk);
    initial
        always #5 clk = ~clk; //在时间0之后生成时钟沿
endmodule

```

所有时钟边沿都使用阻塞赋值生成,它们将在active区域触发事件的发生 如果确实需要在0时刻产生一个时钟边沿,可以使用非阻塞赋值语句设置初始值,这样一来所有的时钟敏感逻辑电路比如always块都会在时钟变化之前执行

将这些模块连接起来

```

module top;
    bit clk;
    always #4 clk = ~clk; // 4ns周期的时钟

    arb_if arbif(.*);
    arb a1(.*);
    test t1(.*);
endmodule top

```

快捷符号.*(隐式端口连接),能自动在当前级别自动连接模块实例的端口到具体信号,只要端口和信号的名字和数据类型相同

端口列表中的接口必须连接

SystemVerilog编译器不会让你成功编译任何一个端口列表中含有接口的模块或程序块

```

module uses_a_port(inout bit not_connected);
    ...

```

```
endmodule
```

端口列表中的接口必须连接

SystemVerilog编译器不会让你成功编译任何一个在端口列表中含有接口的模块或者程序块

1. 普通信号 vs 接口对象的区别

文中首先做了一个对比，帮助理解为什么接口比较特殊。

- **普通信号（如 `bit`, `wire`）：**
 - 如果你定义了一个模块 `module uses_a_port(inout bit not_connected);`，即使你在上层模块例化它时不连接任何信号（悬空），或者只是声明了模块本身，编译器通常也能通过。因为对于普通信号，编译器可以自动推断并创建一个空的连线（Net）。
- **接口（Interface）：**
 - 接口不仅仅是一根线，它是一个包含了信号、任务、函数甚至时钟块（Clocking Block）的复杂对象。
 - 如果模块定义为 `module uses_an_interface(arb_ifc.DUT ifc);`，这里的 `ifc` 是一个指向接口实例的句柄（Handle）。
 - **关键点：**编译器无法凭空“捏造”一个接口实例。它必须知道这个接口具体连到了哪里，里面包含了什么具体的信号和时序逻辑。因此，**必须在例化时显式地连接一个已经存在的接口实例。**

2. 代码示例解析

例 4.29（错误示范）

```
// 这是一个错误的写法，或者说不完整的写法
module uses_an_interface(arb_ifc.DUT ifc);
    initial ifc.grant = 0;
endmodule
```

- 这里定义了模块 ```systemverilog` // 这是一个错误的写法，或者说不完整的写法 `module uses_an_interface(arb_ifc.DUT ifc); initial ifc.grant = 0; endmodule`

- 这里定义了模块 ``uses_an_interface``，它需要一个类型为 ``arb_ifc.DUT`` 的接口 ``ifc``。
- 如果在顶层模块中只是声明了这个模块，而没有给它传具体的接口进去，编译器就会报错：“缺少必要的 `modports`”或“未连接的接口端口”。因为它不知道 ``ifc`` 到底是谁，也就无法解析 ``ifc.grant`` 是什么。

例 4.30（正确做法）

这是标准的 SystemVerilog 验证环境搭建方式：

1. ****顶层模块（Top Module）****：作为所有组件的容器。
2. ****实例化接口****：``arb_ifc ifc(clk);``
 - 首先在顶层创建了一个真实的接口实例 ``ifc``，并把时钟 ``clk`` 传给它（用于接口内部的时

钟块同步)。

3. ****实例化被测模块/组件****: ``uses_an_interface u1(ifc);``

- 在例化 ``u1`` 时, 将上面创建好的接口实例 ``ifc`` 传递进去。
- 这样, 模块 ``u1`` 内部的 ``ifc`` 就真正指向了顶层的那个物理接口, 所有的信号交互才能正常进行。

```
```systemverilog
module top;
 bit clk;
 always #10 clk = !clk;

 arb_ifc ifc(clk); //带有时钟块的接口
 uses_an_interface u1(ifc); //必须这样定义才能被编译
endmodule
```

## 顶层作用域

在仿真过程中创建程序或者模块之外的对象,以便参与仿真的所有对象都可以访问它们

在Verilog中只有宏定义可以跨越模块的边界, 而且经常被用创建全局变量

SystemVerilog引入了编译单元(compilation unit),它是一起编译的源文件的组合 任何 module,macromodule,interface,program,package或者primitive边界之外的作用域都被称为编译单元作用域,也称为\$unit

这个作用域内的任何成员,比如parameter都类似于全局成员,因为它可以被所有低一级的块访问,但是它们又不同于真正的全局成员

本书将块外作用域称为"顶层作用域", 在这个作用域内你可以定义变量,参数,数据类型甚至方法

```
//root.sv
`timescale 1ns/1ns
parameter int TIMEOUT = 1_000_000;
const string time_out_msg = "ERROR: Timeout";

module top;
 test t1();
endmodule

program automatic test;
 initial begin
 $display("%s",time_out_msg);
 $finish;
 end
endprogram
```

使用\$root的跨模块的引用

```

`timescale 1ns/1ns
parameter TIMEOUT = 1_000_000;
top t1(); //顶层模块的显式例化

module top;
 bit clk;
 test t1(.*);
endmodule

`define TOP $root.top
program automatic test;
 initial begin
 //绝对引用
 $display("clk = %b", $root.top.clk);
 $display("clk = %b", `top.clk); //使用宏
 //相对引用
 $display("clk = %b", top.clk);
 endprogram

```

top t1(); 这是整段代码中最关键的一行。它在顶层作用域直接实例化了 module top，并将其命名为 t1。为什么需要这一行？如果没有这一行，设计中就没有名为 t1 的实例存在。后续的 test 程序试图去访问 top.clk 或 \$root.top.clk 时，会因为找不到路径而报错。这一行建立了从“根”到“设计模块”的物理连接。

## 程序--模块交互

程序块可以读写模块中的所有信号,可以调用模块中的所有例程,但是模块却看不到程序块,因为测试平台需要访问和控制设计,但是设计却独立于测试平台

程序可以调用模块中的例程来执行不同的动作,这个例程可以改变内部信号的值,这也称为后门

在测试平台中使用函数从DUT获取信息是一个好办法 在大多数情况下读取信号是可行的,但是如果设计代码变化,测试平台就可能错误的解释数值

## System Verilog断言

可以使用SystemVerilog断言(SVA) 在你的设计中创建时序断言 断言的例化跟其他设计块的例化相似,而且在整个仿真过程中都是有效的

立即断言 测试平台的过程代码可以检查待测设计的信号值和测试平台的信号值 并且在存在问题的时候采取相应的行动,例如产生了总线请求,就期望在两个时钟周期后产生应答

```

bus.cb.request<=1;
repeat (2) @bus.cb;
if(bus.cb.grant != 2'b01)
 $display("Error,grant!= 1");
//测试平台剩余部分

```

```
bus.cb.request <= 1;
repeat (2) @bus.cb;
a1: assert (bus.cb.grant == 2'b01);
```

```
bus.cb.request <= 1;
repeat(2) @bus.cb;
a1 : assert(bus.cb.grant == 2'b01);
```

一个立即断言有可选的then和else分句,如果你想改变默认的消息,可以添加自己的输出信息

```
a1:assert(bus.cb.grant == 2'b01)
else $error("Grant not asserted");
```

## 并发断言

可以认为它是一个连续运行的模块,它为整个仿真过程检查信号的值 需要在断言内指定一个采样时钟

```
interface arb_if(input bit clk);
 logic [1:0] grant,request;
 logic rst;

 property request_2state;
 @(posedge clk) disable iff(rst);
 $isunknown(request) == 0;
 endproperty
 assert_request_2state : assert property(request_2state);
endinterface
```

@(posedge clk): 指定这是一个并发断言。它会在时钟 clk 的每一个上升沿进行采样和评估。disable iff(rst): 这是一个异常处理机制。当复位信号 rst 为高电平（有效）时，该断言会被暂时挂起/失效。因为在复位期间，信号处于 X 或 Z 状态是正常现象，不需要报错。\$isunknown(request) == 0: 使用了系统函数 \$isunknown。如果 request 信号的任意一位是 X 或 Z，该函数返回 1；要求它等于 0，即强制规定 request 必须是一个确定的二值逻辑（0 或 1）。

## 四端口ATM路由器

```
module atm_router(
 // 4 x Level 1 Utopia ATM layer Rx Interfaces (接收接口)
 Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3,
 Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3,
 Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3,
 Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3,
 Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3,
```



```

// 4 x Level 1 Utopia ATM layer Tx Interfaces (发送接口)
Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3,
Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3,
Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3,
Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3,
Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3,

// 其他控制信号
rst, clk
);

// 此处通常会有端口方向定义 (input/output) 和内部逻辑
// 由于图片仅展示了端口列表头部, 后续内容未显示。

// 4 x Level 1 Utopia Rx Interfaces
output Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3;
input [7:0] Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3;
input Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3;
output Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3;
input Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3;

// 4 x Level 1 Utopia Tx Interfaces
output Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3;
output [7:0] Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3;
output Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3;
output Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3;
input Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3;

// 其他控制信号
input rst, clk;
...①
endmodule

```

```

// 例 4.1 未使用接口的顶层简单
module top;
 bit clk;

 // 产生周期为 10ns (5+5) 的时钟信号
 always #5 clk = !clk;

 // 声明所有用于连接的线网信号
 wire Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3,
 Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3,
 Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3,
 Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3,
 Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3,
 Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3,
 Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3,
 Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3, rst;

```

```

wire [7:0] Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3,
 Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3;

// 实例化 ATM 路由器设计模块 (DUT)
atm_router al(
 Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3,
 Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3,
 Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3,
 Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3,
 Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3,
 Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3,
 Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3,
 Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3,
 Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3,
 Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3,
 rst, clk
);

// 实例化测试程序模块
test t1(
 Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3,
 Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3,
 Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3,
 Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3,
 Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3,
 Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3,
 Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3,
 Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3,
 Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3,
 Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3,
 rst, clk
);

endmodule

```

```

// 例 4.2 使用端口的测试平台 (Verilog-1995)
module test(
 // 4 x Level 1 Utopia ATM layer Rx Interfaces
 Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3,
 Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3,
 Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3,
 Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3,
 Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3,

 // 4 x Level 1 Utopia ATM layer Tx Interfaces
 Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3,
 Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3,
 Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3,
 Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3,
 Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3,

```

```

// 其他控制信号
rst, clk
);

// -----
// 4 x Level 1 Utopia Rx Interfaces (方向声明)
// -----
input Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3;
output [7:0] Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3;
reg [7:0] Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3; // reg类型以支持赋值

output Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3;
reg Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3;
input Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3;
output Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3;
reg Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3;

// -----
// 4 x Level 1 Utopia Tx Interfaces (方向声明)
// -----
input Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3;
input [7:0] Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3;
input Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3;
input Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3;
output Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3;
reg Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3;

// -----
// 其他控制信号 (方向声明)
// -----
output rst;
reg rst;
input clk;

initial begin
 // 复位设备
 rst = 1;
 Rx_data_0 <= 0;
 ...
end

endmodule

```

## ATM接口简化

```

interface Rx_if(input logic clk);
 logic [7:0] data;
 logic soc,en,clav,rclk;

 clocking cb @(posedge clk);
 output data,soc,clav; // 方向是相对测试平台的
 input en;

```

```

 endclocking : cb

 modport DUT(output en,rclk,
 input data,soc,clav);

 modport TB(clocking cb);
endinterface Rx_if

```

## Tx接口

```

interface Tx_if(input logic clk);
 logic [7:0] data;
 logic soc,en,clav,tclk;

 clocking cb @(posedge clk);
 input data,soc,en;
 output clav;
 endclocking : cb

 modport DUT(output soc,tclk,en,
 input data,clav);
 modport TB (clocking cb);
endinterface Tx_if

```

使用module atm\_router(Rx\_if.DUT Rx0,Rx1,Rx2,Rx3, Tx\_if.DUT Tx0,Tx1,Tx2,Tx3, input logic clk,rst); endmodule

## 使用接口的ATM顶层网单

```

module top;
 bit clk, rst;

 // 产生周期为 10ns (5+5) 的时钟信号
 always #5 clk = !clk;

 // 实例化 4 个接收端接口 (Rx Interface)
 Rx_if Rx0 (clk), Rx1 (clk), Rx2 (clk), Rx3 (clk);

 // 实例化 4 个发送端接口 (Tx Interface)
 Tx_if Tx0 (clk), Tx1 (clk), Tx2 (clk), Tx3 (clk);

 // 实例化 ATM 路由器设计模块 (DUT)
 // 注意: 这里直接传递接口实例, 而不是单独的信号线
 atm_router a1 (
 Rx0, Rx1, Rx2, Rx3, // 或者仅使用 (.*) 进行自动端口匹配
 Tx0, Tx1, Tx2, Tx3,
 clk, rst
);

 // 实例化测试平台模块 (Testbench)
 test t1 (

```

```

 Rx0, Rx1, Rx2, Rx3, // 或者仅使用 (.*) 进行自动端口匹配
 Tx0, Tx1, Tx2, Tx3,
 clk, rst
);

endmodule top

```

## 使用接口的ATM测试平台

接口中的名字都使用了固定名字,所以要把同样的代码为4x4 ATM路由器复制四次

```

program test(Rx_if.TB Rx0,Rx1,Rx2,Rx3,
 Tx_if.TB Tx0,Tx1,Tx2,Tx3,
 input logic clk,output logic rst);
 bit [7:0] bytes[ATM_CELL_SIZE];
 initial begin
 //复位设备
 rst <= 1;
 Rx0.cb.data <= 0;
 receive_cell0();
 end

 task receive_cell0();
 @(Tx0.cb);
 Tx0.cb.clav <=1;
 wait(Tx0.cb.soc == 1);
 for(int i = 0; i < ATM_CELL_SIZE; i++) begin
 wait(Tx0.cb.en == 0);
 @(Tx0.cb);
 bytes[i] = Tx0.cb.data;

 @(Tx0.cb);
 Tx0.cb.clav <= 0;
 end
 endtask : receive_cell0
endprogram

```

## ref端口的方向

SystemVerilog引入了一种新的端口方向:ref 你应该很熟悉input,output和inout端口方向了,其中inout用于建模双向连接

如果使用多个inout端口驱动一个信号,SystemVerilog将会根据所有的驱动器的值,驱动强度来计算最终的值

ref端口的行为完全不同,它其实是对变量的引用,它的值是该变量最后一次赋值 如果将一个变量连接到多个ref端口,就可能产生竞争,因为多个模块的端口都可能更新同一个变量

## 仿真的结束

在最后一个initial块完成时,隐性调用\$exit来标识程序的结束 当所有程序块都退出了,\$finish函数的隐形调用就结束了

可以在需要的时候直接调用\$finish来结束仿真

模块或者程序可以定义一个或者多个final块来执行仿真器退出前的代码

```
program test;
 int errors,warnings;

 initial begin
 //程序块主要行为
 end

 final
 $display("Test done with %d errors and %d warnings",errors,warnings);
endprogram
```

LC3取指模块的定向测试(directed test)

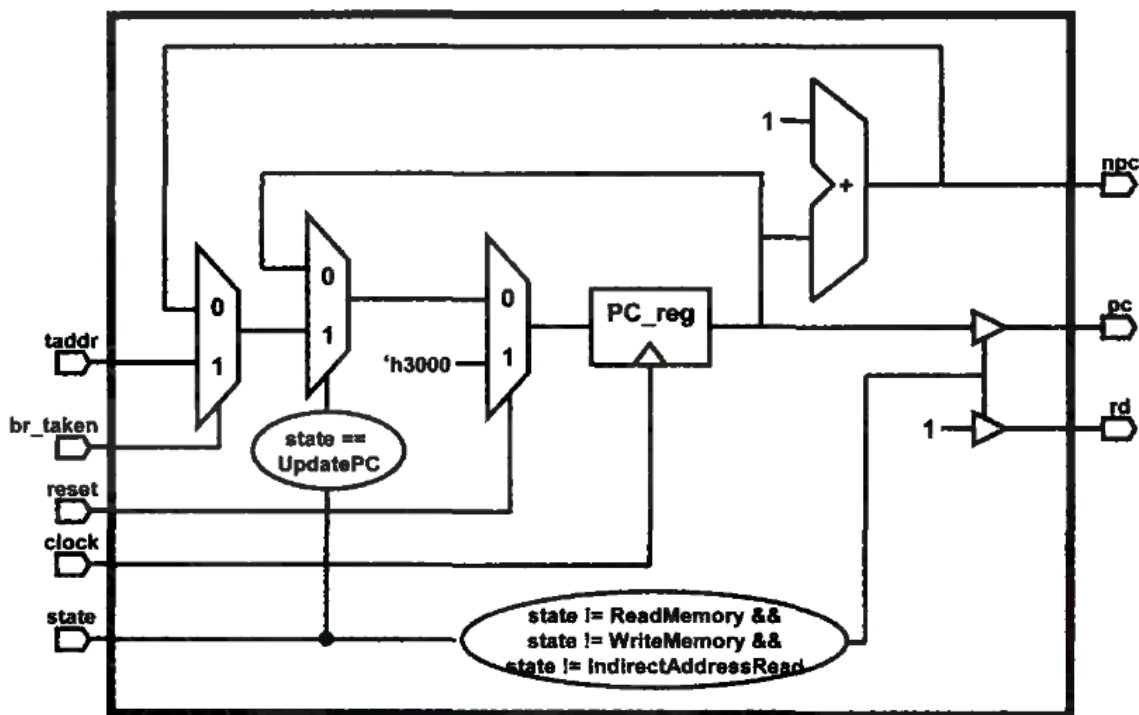


图 4.10 LC3 微控制器取指模块

图 4.10 中的 fetch 模块计算从内存中取数的地址，它有下列输入：

- (1) clock, reset: 1 位。
- (2) br\_taken: 1 位。通知 fetch 块遇到控制信号，所以 npc 的值需要从 pc+1 改变为由指令计算出的地址值 taddr (目标地址)。
- (3) taddr: 16 位。为分支或者跳转指令计算得到的目标地址。

(4) state: 4 位。controller 块当前状态，比如 fetch, decode 等等。

fetch 块有如下输出：

(1) rd: 1 位。通知内存执行读操作。因为 memAccess 块在 ReadMemory、WriteMemory 和 IndirectAddressRead 状态时会驱动共享总线，所以在这些状态时该信号处于高阻态 (Z)。在所有其他状态，rd 的值为高阻。

(2) pc: 16 位。程序计数器寄存器的当前值，即 PC\_reg，在 rd 为高阻时其值为高阻。

#### 4.12 LC3 取指模块的定向测试 (directed test) 99

(3) npc: 16 位，其值始终为 PC\_reg + 1。

在 clock 的上升沿，当 br\_taken 为真时，taddr 送入 PC\_reg；而当 br\_taken 为假时，npc 送入 PC\_reg。PC\_reg 复位时为 16'h3000。所有信号都在同一个周期内更新。

取指模块的 verilog 代码具有输入和输出端口。

```
module fetch(clock,reset,state,pc,npc,rd,taddr,br_taken);
 input clock,reset,br_taken;
 input [15:0] taddr;
 input [3:0] state;
 output [15:0] pc,npc; //当前和下一个PC
 output rd;

 //略去受保护的代码
endmodule
```

```
interface fetch_ifc(input bit clock);
 logic reset,br_taken,rd;
 logic [15:0] taddr;
 cntrl_e state;

 logic [15:0] pc,npc;

 clocking cb @(posedge clock);
 input pc,npc,rd;
 output taddr,state,br_taken,reset;
 endclocking //cb

 modport TEST(clocking cb,output reset);

 modport DUT(input clock,reset,br_taken,taddr,state,
 output pc,npc,rd);

 // 用于监控DUT信号
 clocking cbm@(posedge clock);
 input pc,npc,rd,taddr,state,br_taken;
```

```

 endclocking //cbm
 modport MONITOR(clocking cbm);
endinterface fetch_ifc

```

## 取指模块的定向测试

```

program automatic test(fetch_ifc.TEST if_t,fetch_ifc.MONITOR if_m);
 initial begin
 cntrl_e cntrl;

 $timeformat(-9,0,"ns",5);
 $monitor("time = %t, pc = %h, npc = %h, rd = %h, taddr = %h, state = %h,
br_taken = %h, reset = %h", $time(), if_m.cbm.pc, if_m.cbm.npc, if_m.cbm.rd,
if_m.cbm.taddr, if_m.cbm.state, if_m.cbm.br_taken, if_m.cbm.reset);
 $display("%t:Reset all signals", $realtime);
 if_t.reset<=1;
 if_t.cb.taddr<= 16'hFFFC;
 if_t.cb.br_taken <= 0;
 if_t.cb.state <= CNTGRL_UPDATE_PC;

 repeat(2) @if_t.cb;
 pc_post_reset: assert(if_t.cb.pc == 16'h3000);

 // --- 第一部分: 复位与回绕测试 ---
 #1 if_t.cb.reset <= 0; // 同步地释放复位信号

 @(if_t.cb);
 $display("\n\t: Test loading of target address", $realtime);
 if_t.cb.state <= CNTRL_UPDATE_PC;
 if_t.cb.br_taken <= 1;

 @(if_t.cb);
 @(if_t.cb);
 pc_br_taken: assert (if_t.cb.pc == 16'hFFFC);

 $display("%t: Did the PC rollover as expected?", $realtime);

 // --- 第二部分: 遍历控制器状态测试 ---
 if_t.cb.br_taken <= 0;
 if_t.cb.state <= CNTRL_UPDATE_PC;
 repeat (5) @(if_t.cb);
 pc_rollover: assert (if_t.cb.pc == 16'h0000);

 $display("\n\t: Step through all the controller states", $realtime);
 for (int i = CNTRL_FETCH; i <= CNTRL_COMPUTE_MEM; i++) begin
 $cast(cntrl, i);
 if (cntrl == CNTRL_UPDATE_PC) continue; // 跳过 UPDATE_PC, 因为上面已经测过
了

 $display("%t: Try with controller state=%0d %s", $realtime, cntrl,
cntrl.name);
 $realtime; // 注意: 原图此处可能有误, 通常是打印时间或作为延时, 这里按原文保留结

```



构

```

 if_t.cb.br_taken <= 0;
 if_t.cb.state <= cntrl;
 repeat (2) @(if_t.cb);
 pc_no_load: assert (if_t.cb.pc == 16'h0001);
 end // for i

 // --- 第三部分: 高阻态 (Tristate) 测试 ---
 $display("\n\t: Tristate on PC output", $realtime);

 if_t.cb.state <= CNTRL_READ_MEM;
 @(if_t.cb);
 pc_z_read_mem: assert (if_t.cb.pc === 16'hz);

 if_t.cb.state <= CNTRL_IND_ADDR_RD;
 @(if_t.cb);
 pc_z_ind_addr_rd: assert (if_t.cb.pc === 16'hz);

 if_t.cb.state <= CNTRL_WRITE_MEM;
 @(if_t.cb);
 pc_z_write_mem: assert (if_t.cb.pc === 16'hz);

endprogram // test

```

```

`timescale 1ns / 1ns

typedef enum(CNTRL_UPDATE_PC = 0,
 CNTRL_FETCH = 1,
 CNTRL_DECODE = 2,
 CNTRL_EXECUTE = 3,
 CNTRL_UPDATE_REGF = 4,
 CNTRL_COMPUTE_PC = 5,
 CNTRL_COMPUTE_MEM = 6,
 CNTRL_READ_MEM = 7,
 CNTRL_IND_ADDR_RD = 8,
 CNTRL_WRITE_MEM = 9) cntrl_e;

module top;
 bit clock;
 always #10 clock = ~clock;
 fetch_ifc fif(clock);
 test t1(fif,fif);
 fetch
f1(clock,fif.reset,fif.state,fif.pc,fif.npc,fif.rd,fif.taddr,fif.br_taken);
endmodule //top

```

## 面向对象编程基础

### 编写一个类

类封装了数据和操作这些数据的子程序，这个数据包包含了地址，CRC和一个存储数值的数组,在Transaction类中有两个子程序:一个输出数据包地址的函数和一个计算循环冗余校验(CRC: cyclic redundancy check) 函数

```
class Transaction;
 bit [31:0] addr,crc,data[8];

 function void display;
 $display("Transaction: %h",addr);
 endfunction

 function void calc_crc;
 crc = addr ^ data.xor;
 endfunction

endclass
```

## OOP术语

- (1) **类 (class)**: 包含变量和子程序的基本构建块。Verilog 中与之对应的是模块 (module)。
- (2) **对象 (object)**: 类的一个实例。在 Verilog 中，你需要实例化一个模块才能使用它。
- (3) **句柄 (handle)**: 指向对象的指针。在 Verilog 中，你通过实例名在模块外部引用信号和方法。一个 OOP 句柄就像一个对象的地址，但是它保存在一个只能指向单一数据类型的指针中。
- (4) **属性 (property)**: 存储数据的变量。在 Verilog 中，就是寄存器 (reg) 或者线网 (wire) 类型的信号。
- (5) **方法 (method)**: 任务或者函数中操作变量的程序性代码。Verilog 模块除了 initial 和 always 块以外，还含有任务和函数。
- (6) **原型 (prototype)**: 程序的头，包括程序名、返回类型和参数列表。程序体则包含了执行代码。

## 创建新对象

Transaction tr; //声明一个句柄 tr = new(); //为一个Transaction 对象分配空间

```
class Transaction;
 logic [31:0] addr,crc,data[8];

 function new;
 addr = 3;
 foreach (data[i])
 data[i] = 5;
 endfunction

endclass
```

```

class Transaction;
 logic [31:0] addr,crc,data[8];
 function new(logic [31:0] a = 3,d = 5);
 addr = a;
 foreach(data[i])
 data[i] = d;
 endfunction
endclass

initial begin
 Transaction tr;
 tr = new(10);
end

```

```

class Transaction;

endclass

class Driver;
 Transaction tr;
 funtion new();
 tr = new();
 endfunction
endclass Driver

```

为对象创建一个句柄

```

Transaction t1,t2; //声明两个句柄
t1 = new(); // 为第一个Transaction 对象分配地址
t2 = t1; // t1和t2都指向该对象
t1 = new(); // 为第二个Transaction对象分配地址

```

对象的解除分配(deallocation)

当最后一个句柄不再引用某个对象了 SystemVerilog就释放该对象的空间

```

Transaction t; //创建一个句柄
t = new(); //分配一个新的Transaction
t = new(); //分配第二个,并且释放第一个t
t = null; //解除分配第二个

```

Verilog会自动回收垃圾

使用对象

```

Transaction t; //声明一个Transaction句柄
t = new(); //创建一个Transaction对象
t.addr = 32'h42; //设置变量的值
t.display(); //调用一个子程序

```

## 静态变量和全局变量

含有一个静态变量的类

```

class Transaction;
 static int count = 0;
 int id;
 function new()
 id = count++;
 endfunction
endclass

Transaction t1,t2;
initial begin
 t1 = new();
 t2 = new();
 $display("second id = %0d",t2.id);
end

```

通过类名访问静态变量

```

class Transaction;
 static int count = 0;
endclass

initial begin
 run_test();
 $display("%d transaction were created",Transaction::count);
end

```

// 例 5.12 显示静态变量的静态方法

```

class Transaction;
 static Config cfg;
 static int count = 0;
 int id;

 // 显示静态变量的静态方法
 static function void display_statics();
 $display("Transaction cfg.mode=%s, count=%0d",
 cfg.mode.name(), count);
 endfunction
endclass

```

```

 endfunction
 endclass

 Config cfg;

 initial begin
 cfg = new(MODE_ON);
 Transaction::cfg = cfg;
 Transaction::display_statics(); // 调用静态方法
 end

```

## 类的方法

```

// 例 5.13 类中的方法

class Transaction;
 bit [31:0] addr, crc, data[8];

 function void display();
 $display("@%0t: TR addr=%h,crc=%h", $time, addr, crc);
 $write("\tdata[0-7]=");
 foreach (data[i]) $write(data[i]);
 $display();
 endfunction
endclass

class PCI_Tran;
 bit [31:0] addr, data; // 使用真实的名字
 function void display();
 $display("@%0t: PCI: addr=%h,data=%h", $time, addr, data);
 endfunction
endclass

Transaction t;
PCI_Tran pc;

initial begin
 t = new(); // 创建一个 Transaction 对象
 t.display(); // 调用 Transaction 的方法
 pc = new(); // 创建一个 PCI 事务
 pc.display(); // 调用 PCI 事务的方法
end

```

## 在类之外定义方法

```

// 例 5.14 块外方法声明

class Transaction;
 bit [31:0] addr, crc, data[8];

```

```

 extern function void display();
endclass

function void Transaction::display();
 $display("@%0t: Transaction addr=%h,crc=%h",
 $time, addr, crc);
 $write("\tdata[0-7]=");
 foreach (data[i]) $write(data[i]);
 $display();
endfunction

class PCI-Tran;
 bit [31:0] addr, data; // 使用实名
 extern function void display();
endclass

function void PCI-Tran::display();
 $display("@%0t: PCI: addr=%h,data=%h",
 $time, addr, data);
endfunction

```

## 作用域规则

### 名字作用域

```

int limit; // $root.limit
program automatic p;
 int limit; // p.limit
 class Foo;
 int limit,array[]; ;//$root.p.limit

 // $root.p.Foo.print.limit
 function void print(int limit);
 for(int i = 0; i < limit; i++)
 $display("%m : array[%0d] = %0d", i, array[i]);
 endfunction
 endclass

initial begin
 int limit = $root.limit;
 Foo bar;
 bar = new;
 bar.array = new[limit];
 bar.print(limit);
end
endprogram

```

将类移入package来查找程序错误

```

package Mistake;
class Bad;
 logic [31:0] data[];

 //未定义,不会编译
 function void display;
 for(i = 0; i< data.size(); i++)
 $display("data[%0d] = %0d", i, data[i]);
 endfunction
endclass
endpackage

program test;
 int i;
 import Mistake::* ;
endprogram

```

// 例 5.20 Statistics 类的声明

```

class Statistics;
 time startT, stopT; // 事务的时间
 static int ntrans = 0; // 事务的数目
 static time total_elapsed_time = 0;

 function time how_long;
 how_long = stopT - startT;
 ntrans++;
 total_elapsed_time += how_long;
 endfunction

 function void start;
 startT = $time;
 endfunction
endclass

```

在另外一个类中使用这个类

// 例 5.21 封装 Statistics 类

```

class Transaction;
 bit [31:0] addr, crc, data[8];
 Statistics stats; // Statistics 句柄

 function new();
 stats = new(); // 创建 stats 实例
 endfunction

```

```

 task create_packet();
 // 填充包数据
 stats.start();
 // 传送数据包
 endtask
endclass

```

## 编译顺序问题

```

// 例 5.22 顺序编译
typedef class Statistics; //定义低级别级

class Transaction;
 Statistics stats; //使用Statistics类
endclass

class Statistics; //定义Statistics类

endclass

```

## 理解动态对象

传递对象, SystemVerilog传递该标量的地址, 所以方法可以修改标量变量的值, 如果你不使用ref关键字, SystemVerilog将该变量的值复制到参数变量中

```

task transmit(Transaction t);
 CBbus.rx_data <= t.data;
 t.stats.startT = $time;
 ...
endtask

Transaction t;
initial begin
 t = new();
 t.addr = 42; //初始化数值
 transmit(t); //将对象传递给任务
end

```

## 在任务中修改句柄

```

// 例 5.21 封装 Statistics 类

class Transaction;
 bit [31:0] addr, crc, data[8];
 Statistics stats; // Statistics 句柄

```



```

function new();
 stats = new(); // 创建 stats 实例
endfunction

task create_packet();
 // 填充包数据
 stats.start();
 // 传送数据包
endtask
endclass

```

## 正确的产生器,创建多个对象

```

// 例 5.23 产生器
task generator_good(int n)
 Transaction t;
 repeat (n) begin
 t = new(); //创建一个新对象
 t.addr = $random; //随机赋值
 $display("Transaction addr=%h", t.addr);
 transmit(t);
 end
endtask

```

句柄数组 在写测试平台的时候,可能需要保存并且引用许多对象,可以创建句柄数组,数组的每个元素指向一个对象

```

// 例 5.24 句柄数组
task generator();
 transmit tarray[10];
 foreach (tarray[i]) begin
 begin
 tarray[i] = new();
 tarray[i].addr = $random;
 $display("Transaction addr=%h", tarray[i].addr);
 transmit(tarray[i]);
 end
 end
endtask

```

## 对象的复制

使用new操作符复制一个对象

```

class Transaction;
 bit [31:0] addr,crc,data[8];
endclass

```

```
Transaction src,dst;
initial begin
 src = new;
 dst = new src; //使用new操作符进行复制
end
```

使用new操作符复制一个复杂类

// 例 5.30 使用 new 操作符复制一个复杂类

class Transaction; bit [31:0] addr, crc, data[8]; static int count = 0; int id; Statistics stats; // 指向 Statistics 对象的句柄

```
function new;
 stats = new(); // 构造一个新的 Statistics 对象
 id = count++;
endfunction
```

endclass

Transaction src, dst;

```
initial begin src = new(); // 创建一个 Transaction 对象 src.stats.startT = 42; // 结果见图 5.5 dst = new src; // 用
new 操作符将 src 拷贝到 dst 中, 结果见图 5.6 dst.stats.startT = 96; // 改变 dst 和 src 的 stats
$display(src.stats.startT); // "96", 见图 5.7 end
```

这两个Transaction对象都指向同一个Statistics对象，  
所以使用src句柄修改startT会影响dst句柄可以看到的值

```
```systemverilog
// 例 5.31 含有 copy 函数的简单类
class Transaction;
    bit [31:0] addr, crc, data[8]; // 没有 Statistic 句柄

    function Transaction copy;
        copy = new(); // 创建目标对象
        copy.addr = addr; // 填入数值
        copy.crc = crc;
        copy.data = data; // 复制数组
    endfunction
endclass

// 例 5.32 使用 copy 函数
Transaction src, dst;

initial begin
    src = new(); // 创建第一个对象
```

```
    dst = src.copy();          // 复制对象
end
```

编写自己的深层复制函数

```
// 例 5.33 深层复制函数
class Transaction;
    bit [31:0] addr, crc, data[8];
    Statistics stats; // 指向 Statistics 对象的句柄
    static int count = 0;
    int id;

    function new();
        stats = new();
        id = count++;
    endfunction

    function Transaction copy();
        copy = new();          // 创建目标对象
        copy.addr = addr;      // 填入数值
        copy.crc = crc;
        copy.data = data;
        copy.stats = stats.copy(); // 调用 Statistics::copy 函数
        id = count++;
    endfunction
endclass
```

使用pack和unpack函数

```
// 例 5.34 使用 pack 和 unpack 函数
Transaction tr,tr2;
byte b[40];

initial begin
    tr = new();
    tr.addr = 32'h0a0a0a0a;
    tr.crc = '1;
    foreach(tr.data[i])
        tr.data[i] = i;
    tr.pack(b); //打包对象到字节数组
    $write("Pack results:");
    foreach(b[i])
        $write("%0h",b[i]);
    $display;

    tr2 = new();
    tr2.unpack(b);
    tr2.display();
end
```

建立一个测试平台

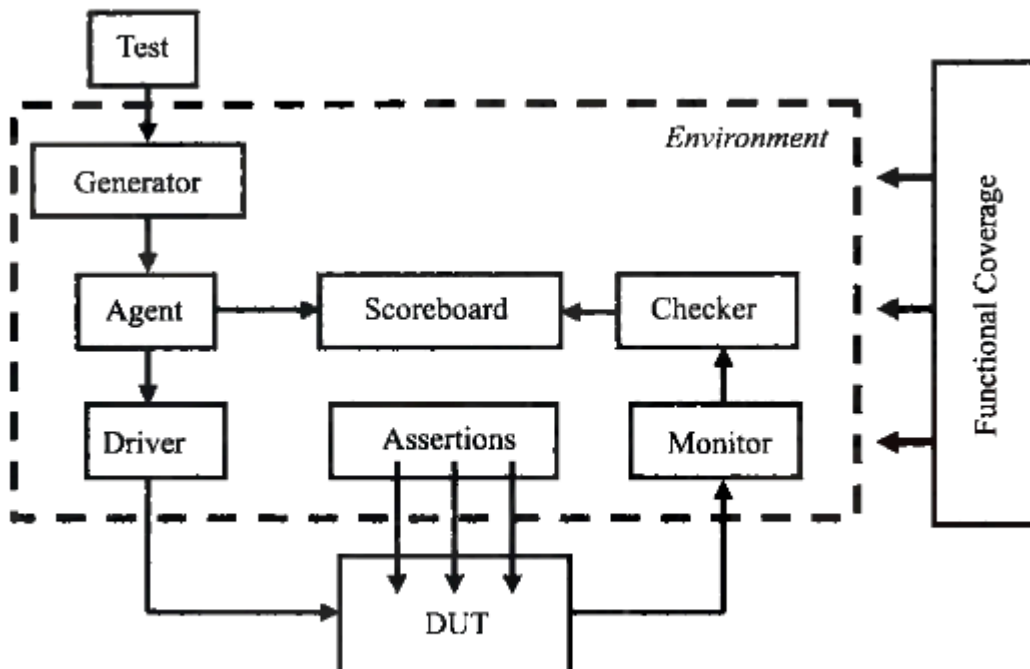


图 5.9 分层的测试平台

图中的Generator,Agent,Driver,Monitor,Checker和Scoreboard都是类 建模成事务处理器,它们在Environment类内部例化

Test处在最高层,即处在例化Environment类的程序中,功能覆盖定义可以放在Environment类的内部或者外部

```
// 基本的事务类
class Transator; //通用类
    Transaction tr;
    task run;
        forever begin
            //从前一个块中获取事务
            ...
            //处理事务
            ...
            //将事务发送到下游模块
            ...
        end
    endtask
endclass
```

随机化

带有随机变量的简单类

```
class Packet;
    //随机变量
    randbit [31:0] src,dst,data[8];
    randc bit [7:0] kind;
    //.src 的约束
    constraint c{
        src > 0;
        src < 15;
    }

endclass

Packet p;
    initial begin
        p = new(); //产生一个包
        assert(p.randomize());
        else $fatal(0,"Packet::randomize failed");
        transmit(p);
    end
```

约束

有用的激励并不仅仅是随机值——各个变量之间有着相互关系,否则仿真器可能需要很长时间才能产生需要的激励值,或者激励向量里会包含无效的值,需要用包含一个或多个约束表达式的约束块定义这些相关关系

```
// 例 6.3 受约束的随机类
class Stim;
    const bit [31:0] CONGEST_ADDR = 42;
    typedef enum {READ, WRITE, CONTROL} stim_e;
```

```

randc stim_e kind;      // 枚举变量
rand bit [31:0] len, src, dst;
bit congestion_test;

constraint c_stim {
    len < 1000;
    len > 0;
    if (congestion_test) {
        dst inside {[CONGEST_ADDR-100 : CONGEST_ADDR+100]};
        src == CONGEST_ADDR;
    }
    else
        src inside {0, [2:10], [100:107]};
}
endclass

```

固定顺序的约束

```

// 例 6.4 固定顺序的约束
class order;
    rand bit [15:0] lo,med,hi;
    constraint good{lo < med; med < hi;}
endclass

```

```

// 例 6.8 动态改变权重
// 总线操作:字节、字或长字
class BusOp;
    // 操作数长度
    typedef enum {BYTE, WORD, LWRD} length_e;
    rand length_e len;

    // dist 约束的权重
    bit [31:0] w_byte=1, w_word=3, w_lwr=5;

    constraint c_len {
        len dist {BYTE := w_byte,    // 使用可变的权重
                  WORD := w_word,    // 来选择随机的操作数长度
                  LWRD := w_lwr};
    }
endclass

```

集合成员和inside运算符

```

rand int c;
int lo,hi;
constraint c_lohi {

```

```
c inside {[lo:hi]};
}
```

```
// 例 6.10 使用"$"指定最大和最小值
rand bit [6:0] b; // 0 <= b <= 127
rand bit [5:0] e; // 0 <= e <= 63

constraint c_range {
    b inside {[$:4], [20:$]}; // 0 <= b <= 4 || 20 <= b <= 127
    e inside {[$:4], [20:$]}; // 0 <= e <= 4 || 20 <= e <= 63
}
```

```
// 例 6.11 (推测) 在 inside 约束中有重复的值
class Weighted;
    rand int val;
    // 定义权重数组, 数值出现的次数代表其权重
    int array[] = '{1, 1, 2, 3, 5, 8, 8, 8, 8, 8};

    constraint c {val inside array;}
endclass

Weighted w;

initial begin
    int count[9], maxx[$]; // count用于统计, maxx用于存储最大值
    w = new();

    repeat (2000) begin
        assert(w.randomize());
        count[w.val]++; // 统计值的个数
    end

    maxx = count.max(); // 获取最大统计值 (用于归一化绘图)

    // 输出值的分布
    foreach(count[i])
        if (count[i]) begin
            $write("count[%0d]=%5d ", i, count[i]);
            // 根据统计数量打印对应长度的星号条
            repeat (count[i] * 40 / maxx[0]) $write("* ");
            $display;
        end
    end
end
```

集合里的每一个值取出来的概率都是相同的,即使值在数组中出现多次,可以把inside约束看成foreach约束

```
// 例 6.16 从数组中取出随机值的类
class Days;
  // 定义枚举类型：一周七天
  typedef enum {SUN, MON, TUE, WED, THU, FRI, SAT} days_e;

  // 定义一个动态数组，用于存储可选的枚举值
  days_e choices[$];

  // 声明随机变量
  rand days_e choice;

  // 约束：choice 的值必须在 choices 数组包含的范围内
  constraint cday {choice inside choices;}
endclass

// 例 6.17 从数组中取出随机值
initial begin
  Days days;
  days = new();

  // 场景 1：只选择周末
  // 将数组赋值为 SUN 和 SAT
  days.choices = {Days::SUN, Days::SAT};
  assert (days.randomize());
  $display("Random weekend day %s\n", days.choice.name);

  // 场景 2：只选择工作日
  // 重新赋值数组为周一到周五
  days.choices = {Days::MON, Days::TUE, Days::WED,
                  Days::THU, Days::FRI};
  assert (days.randomize());
  $display("Random week day %s", days.choice.name);
end
```

使用randc随机选取数组的值

```
// 例 6.18 使用 randc 随机选取数组的值
// 例 6.18 使用 randc 随机地选取数组的值
class RandcInside;
  int array[];          // 待选取的值
  randc bit [15:0] index; // 指向数组的指针

  function new(input int a[]); // 构造、初始化
    array = a;
  endfunction

  function int pick;      // 返回刚选取出的值
    return array[index];
  endfunction
```



```

    constraint c_size {index < array.size;}
endclass

initial begin
    RandcInside ri;

    ri = new('{1,3,5,7,9,11,13});
    repeat (ri.array.size) begin
        assert(ri.randomize());
        $display("Picked %2d [%0d]", ri.pick(), ri.index);
    end
end
end

```

条件约束 -> 操作符可以产生和case操作符效果类似的语句块,它可以用于枚举类型的表达式, 带有->操作符的约束块

```

// 例 6.20 条件约束
class BusOp;
    constraint c_io{
        (io_space_mode)->(addr[31]==1'b1);
    }
endclass

```

带有if else操作符的约束块

```

class BusOp;`n    constraint c_len_rw{
        if(op == READ)
            len inside {[BYTE:LWRD]};
        else
            len == LWRD;
    }
endclass

```

以下是图片内容的 Markdown 代码:

```

## 例 6.21 双向约束

```systemverilog
rand logic [15:0] r, s, t;

constraint c_bidir {
 r < t;
 s == r;
 t < 30;
 s > 25;
}
```

```

SystemVerilog 同时计算四个约束表达式。 r 必须小于 t ，而 t 必须小于 30。 r 等于 s ，而 s 必须大于 25。尽管没有直接约束 t 的下限，但对于 s 的约束隐含着对 t 的下限的限制。表 6.1 列出了这三个变量的各种可能值。

表 6.1 双向约束的求解

| 解 | r | s | t | 解 | r | s | t |
|---|----|----|----|---|----|----|----|
| A | 26 | 26 | 27 | D | 27 | 27 | 28 |
| B | 26 | 26 | 28 | E | 27 | 27 | 29 |
| C | 26 | 26 | 29 | F | 28 | 28 | 29 |

即使 ``systemverilog module top; bit clk; always #10 clk = !clk;

```
arb_ifc ifc(clk); //带有时钟块的接口
uses_an_interface u1(ifc); //必须这样定义才能被编译
```

endmodule

顶层作用域

在仿真过程中创建程序或者模块之外的对象,以便参与仿真的所有对象都可以访问它们

在Verilog中只有宏定义可以跨越模块的边界,而且经常被用创建全局变量

SystemVerilog引入了编译单元(compilation unit),它是一起编译的源文件的组合
任何module,macromodule,interface,program,package或者primitive边界之外的作用域都被称为编译单元作用域,也称为\$unit

这个作用域内的任何成员,比如parameter都类似于全局成员,因为它可以被所有低一级的块访问,但是它们又不同于真正的全局成员

本书将块外作用域称为"顶层作用域",在这个作用域内你可以定义变量,参数,数据类型甚至方法

```
``systemverilog
//root.sv
`timescale 1ns/1ns
parameter int TIMEOUT = 1_000_000;
const string time_out_msg = "ERROR: Timeout";

module top;
    test t1();
endmodule

program automatic test;
    initial begin
        $display("%s",time_out_msg);
        $finish;
```

```

    end
endprogram

```

使用\$root的跨模块的引用

```

`timescale 1ns/1ns
parameter TIMEOUT = 1_000_000;
top t1(); //顶层模块的显式例化

module top;
    bit clk;
    test t1(.*);
endmodule

`define TOP $root.top
program automatic test;
    initial begin
        //绝对引用
        $display("clk = %b",$root.top.clk);
        $display("clk = %b",`top.clk); //使用宏
        //相对引用
        $display("clk = %b",top.clk);
    endprogram

```

top t1(); 这是整段代码中最关键的一行。它在顶层作用域直接实例化了 module top，并将其命名为 t1。为什么需要这一行？如果没有这一行，设计中就没有名为 t1 的实例存在。后续的 test 程序试图去访问 top.clk 或 \$root.top.clk 时，会因为找不到路径而报错。这一行建立了从“根”到“设计模块”的物理连接。

程序--模块交互

程序块可以读写模块中的所有信号,可以调用模块中的所有例程,但是模块却看不到程序块,因为测试平台需要访问和控制设计,但是设计却独立于测试平台

程序可以调用模块中的例程来执行不同的动作,这个例程可以改变内部信号的值,这也称为后门

在测试平台中使用函数从DUT获取信息是一个好办法 在大多数情况下读取信号是可行的,但是如果设计代码变化,测试平台就可能错误的解释数值

System Verilog断言

可以使用SystemVerilog断言(SVA) 在你的设计中创建时序断言 断言的例化跟其他设计块的例化相似,而且在整个仿真过程中都是有效的

立即断言 测试平台的过程代码可以检查待测设计的信号值和测试平台的信号值 并且在存在问题的时候采取相应的行动,例如产生了总线请求,就期望在两个时钟周期后产生应答

```

bus.cb.request<=1;
repeat (2) @bus.cb;
if(bus.cb.grant != 2'b01)

```

```
$display("Error,grant!= 1");
//测试平台剩余部分
```

```
bus.cb.request <= 1;
repeat (2) @bus.cb;
a1: assert (bus.cb.grant == 2'b01);
```

```
bus.cb.request <= 1;
repeat(2) @bus.cb;
a1 : assert(bus.cb.grant == 2'b01);
```

一个立即断言有可选的then和else分句,如果你想改变默认的消息,可以添加自己的输出信息

```
a1:assert(bus.cb.grant == 2'b01)
else $error("Grant not asserted");
```

并发断言

可以认为它是一个连续运行的模块,它为整个仿真过程检查信号的值 需要在断言内指定一个采样时钟

```
interface arb_if(input bit clk);
    logic [1:0] grant,request;
    logic rst;

    property request_2state;
        @(posedge clk) disable iff(rst);
        $isunknown(request) == 0;
    endproperty
    assert_request_2state : assert property(request_2state);
endinterface
```

@(posedge clk): 指定这是一个并发断言。它会在时钟 clk 的每一个上升沿进行采样和评估。disable iff(rst): 这是一个异常处理机制。当复位信号 rst 为高电平（有效）时，该断言会被暂时挂起/失效。因为在复位期间，信号处于 X 或 Z 状态是正常现象，不需要报错。\$isunknown(request) == 0: 使用了系统函数 \$isunknown。如果 request 信号的任意一位是 X 或 Z，该函数返回 1；要求它等于 0，即强制规定 request 必须是一个确定的二值逻辑（0 或 1）。

四端口ATM路由器

```
module atm_router(
    // 4 x Level 1 Utopia ATM layer Rx Interfaces (接收接口)
    Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3,
```

```

Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3,
Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3,
Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3,
Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3,

// 4 x Level 1 Utopia ATM layer Tx Interfaces (发送接口)
Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3,
Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3,
Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3,
Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3,
Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3,

// 其他控制信号
rst, clk
);

// 此处通常会有端口方向定义 (input/output) 和内部逻辑
// 由于图片仅展示了端口列表头部, 后续内容未显示。

// 4 x Level 1 Utopia Rx Interfaces
output      Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3;
input [7:0]  Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3;
input       Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3;
output      Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3;
input       Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3;

// 4 x Level 1 Utopia Tx Interfaces
output      Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3;
output [7:0] Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3;
output      Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3;
output      Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3;
input       Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3;

// 其他控制信号
input rst, clk;
...①
endmodule

```

// 例 4.1 未使用接口的顶层简单

```

module top;
    bit clk;

    // 产生周期为 10ns (5+5) 的时钟信号
    always #5 clk = !clk;

    // 声明所有用于连接的线网信号
    wire Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3,
          Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3,
          Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3,
          Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3,
          Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3,

```

```

        Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3,
        Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3,
        Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3, rst;

wire [7:0] Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3,
        Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3;

// 实例化 ATM 路由器设计模块 (DUT)
atm_router al(
    Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3,
    Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3,
    Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3,
    Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3,
    Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3,
    Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3,
    Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3,
    Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3,
    Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3,
    Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3,
    rst, clk
);

// 实例化测试程序模块
test t1(
    Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3,
    Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3,
    Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3,
    Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3,
    Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3,
    Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3,
    Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3,
    Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3,
    Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3,
    Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3,
    rst, clk
);

endmodule

```

```

// 例 4.2 使用端口的测试平台 (Verilog-1995)
module test(
    // 4 x Level 1 Utopia ATM layer Rx Interfaces
    Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3,
    Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3,
    Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3,
    Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3,
    Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3,

    // 4 x Level 1 Utopia ATM layer Tx Interfaces
    Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3,
    Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3,

```

```

Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3,
Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3,
Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3,

// 其他控制信号
rst, clk
);

// -----
// 4 x Level 1 Utopia Rx Interfaces (方向声明)
// -----
input          Rx_clk_0, Rx_clk_1, Rx_clk_2, Rx_clk_3;
output [7:0]    Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3;
reg [7:0]       Rx_data_0, Rx_data_1, Rx_data_2, Rx_data_3; // reg类型以支持赋值
output         Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3;
reg            Rx_soc_0, Rx_soc_1, Rx_soc_2, Rx_soc_3;
input          Rx_en_0, Rx_en_1, Rx_en_2, Rx_en_3;
output         Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3;
reg            Rx_clav_0, Rx_clav_1, Rx_clav_2, Rx_clav_3;

// -----
// 4 x Level 1 Utopia Tx Interfaces (方向声明)
// -----
input          Tx_clk_0, Tx_clk_1, Tx_clk_2, Tx_clk_3;
input [7:0]     Tx_data_0, Tx_data_1, Tx_data_2, Tx_data_3;
input          Tx_soc_0, Tx_soc_1, Tx_soc_2, Tx_soc_3;
input          Tx_en_0, Tx_en_1, Tx_en_2, Tx_en_3;
output         Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3;
reg            Tx_clav_0, Tx_clav_1, Tx_clav_2, Tx_clav_3;

// -----
// 其他控制信号 (方向声明)
// -----
output rst;
reg rst;
input clk;

initial begin
    // 复位设备
    rst = 1;
    Rx_data_0 <= 0;
    ...
end

endmodule

```

ATM接口简化

```

interface Rx_if(input logic clk);
    logic [7:0] data;
    logic soc,en,clav,rclk;

```

```

    clocking cb @(posedge clk);
        output data,soc,clav; // 方向是相对测试平台的
        input en;
    endclocking : cb

    modport DUT(output en,rclk,
                input data,soc,clav);

    modport TB(clocking cb);
endinterface Rx_if

```

Tx接口

```

interface Tx_if(input logic clk);
    logic [7:0] data;
    logic soc,en,clav,tclk;

    clocking cb @(posedge clk);
        input data,soc,en;
        output clav;
    endclocking : cb

    modport DUT(output soc,tclk,en,
                input data,clav);
    modport TB (clocking cb);
endinterface Tx_if

```

使用module atm_router(Rx_if.DUT Rx0,Rx1,Rx2,Rx3, Tx_if.DUT Tx0,Tx1,Tx2,Tx3, input logic clk,rst); endmodule

使用接口的ATM顶层网单

```

module top;
    bit clk, rst;

    // 产生周期为 10ns (5+5) 的时钟信号
    always #5 clk = !clk;

    // 实例化 4 个接收端接口 (Rx Interface)
    Rx_if Rx0 (clk), Rx1 (clk), Rx2 (clk), Rx3 (clk);

    // 实例化 4 个发送端接口 (Tx Interface)
    Tx_if Tx0 (clk), Tx1 (clk), Tx2 (clk), Tx3 (clk);

    // 实例化 ATM 路由器设计模块 (DUT)
    // 注意: 这里直接传递接口实例, 而不是单独的信号线
    atm_router a1 (
        Rx0, Rx1, Rx2, Rx3,    // 或者仅使用 (.* ) 进行自动端口匹配
        Tx0, Tx1, Tx2, Tx3,
        clk, rst
    );
endmodule

```



```

);

// 实例化测试平台模块 (Testbench)
test t1 (
    Rx0, Rx1, Rx2, Rx3,    // 或者仅使用 (.* ) 进行自动端口匹配
    Tx0, Tx1, Tx2, Tx3,
    clk, rst
);

endmodule top

```

使用接口的ATM测试平台

接口中的名字都使用了固定名字,所以要把同样的代码为4x4 ATM路由器复制四次

```

program test(Rx_if.TB Rx0,Rx1,Rx2,Rx3,
             Tx_if.TB Tx0,Tx1,Tx2,Tx3,
             input logic clk,output logic rst);
    bit [ 7:0] bytes[ATM_CELL_SIZE];
    initial begin
        //复位设备
        rst <= 1;
        Rx0.cb.data <= 0;
        receive_cell0();
    end

    task receive_cell0();
        @(Tx0.cb);
        Tx0.cb.clav <=1;
        wait(Tx0.cb.soc == 1);
        for(int i = 0; i < ATM_CELL_SIZE; i++) begin
            wait(Tx0.cb.en == 0);
            @(Tx0.cb);
            bytes[i] = Tx0.cb.data;

            @(Tx0.cb);
            Tx0.cb.clav <= 0;
        end
    endtask : receive_cell0
endprogram

```

ref端口的方向

SystemVerilog引入了一种新的端口方向:ref 你应该很熟悉input,output和inout端口方向了,其中inout用于建模双向连接

如果使用多个inout端口驱动一个信号,SystemVerilog将会根据所有的驱动器的值,驱动强度来计算最终的值

ref端口的行为完全不同,它其实是对变量的引用,它的值是该变量最后一次赋值 如果将一个变量连接到多个ref端口,就可能产生竞争,因为多个模块的端口都可能更新同一个变量

仿真的结束

在最后一个initial块完成时,隐性调用\$exit来标识程序的结束 当所有程序块都退出了,\$finish函数的隐形调用就结束了

可以在需要的时候直接调用\$finish来结束仿真

模块或者程序可以定义一个或者多个final块来执行仿真器退出前的代码

```
program test;
  int errors,warnings;

  initial begin
    .....//程序块主要行为
  end

  final
    $display("Test done with %d errors and %d warnings",errors,warnings);
endprogram
```

LC3取指模块的定向测试(directed test)

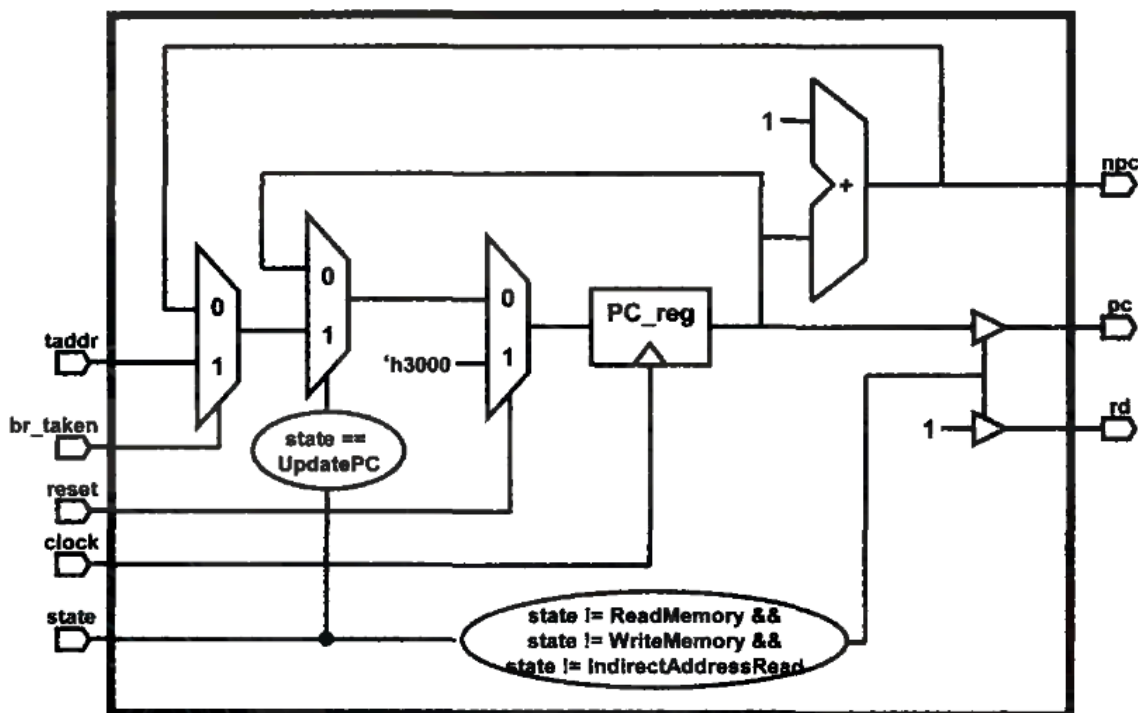


图 4.10 LC3 微控制器取指模块

图 4.10 中的 fetch 模块计算从内存中取数的地址，它有下列输入：

- (1) clock, reset: 1 位。
- (2) br_taken: 1 位。通知 fetch 块遇到控制信号，所以 npc 的值需要从 pc+1 改变为由指令计算出的地址值 taddr (目标地址)。

(3) taddr: 16 位。为分支或者跳转指令计算得到的目标地址。

(4) state: 4 位。controller 块当前状态，比如 fetch, decode 等等。

fetch 块有如下输出：

(1) rd: 1 位。通知内存执行读操作。因为 memAccess 块在 ReadMemory、WriteMemory 和 IndirectAddressRead 状态时会驱动共享总线，所以在这些状态时该信号处于高阻态 (Z)。在所有其他状态，rd 的值为高阻。

(2) pc: 16 位。程序计数器寄存器的当前值，即 PC_reg，在 rd 为高阻时其值为高阻。

4.12 LC3 取指模块的定向测试 (directed test) 99

(3) npc: 16 位，其值始终为 PC_reg + 1。

在 clock 的上升沿，当 br_taken 为真时，taddr 送入 PC_reg；而当 br_taken 为假时，npc 送入 PC_reg。PC_reg 复位时为 16'h3000。所有信号都在同一个周期内更新。

取指模块的 verilog 代码具有输入和输出端口。

```
module fetch(clock,reset,state,pc,npc,rd,taddr,br_taken);
    input clock,reset,br_taken;
    input [15:0] taddr;
    input [3:0] state;
    output [15:0] pc,npc; //当前和下一个PC
    output rd;

    //略去受保护的代码
endmodule
```

```
interface fetch_ifc(input bit clock);
    logic reset,br_taken,rd;
    logic [15:0] taddr;
    cntrl_e state;

    logic [15:0] pc,npc;

    clocking cb @(posedge clock);
        input pc,npc,rd;
        output taddr,state,br_taken,reset;
    endclocking //cb

    modport TEST(clocking cb,output reset);

    modport DUT(input clock,reset,br_taken,taddr,state,
                output pc,npc,rd);

    // 用于监控DUT信号
```

```

    clocking cbm@(posedge clock);
    input pc,npc,rd,taddr,state,br_taken;
endclocking //cbm
modport MONITOR(clocking cbm);
endinterface fetch_ifc

```

取指模块的定向测试

```

program automatic test(fetch_ifc.TEST if_t,fetch_ifc.MONITOR if_m);
    initial begin
        cntrl_e cntrl;

        $timeformat(-9,0,"ns",5);
        $monitor("time = %t, pc = %h, npc = %h, rd = %h, taddr = %h, state = %h,
br_taken = %h, reset = %h", $time(), if_m.cbm.pc, if_m.cbm.npc, if_m.cbm.rd,
if_m.cbm.taddr, if_m.cbm.state, if_m.cbm.br_taken, if_m.cbm.reset);
        $display("%t:Reset all signals", $realtime);
        if_t.reset<=1;
        if_t.cb.taddr<= 16'hFFFC;
        if_t.cb.br_taken <= 0;
        if_t.cb.state <= CNTGRL_UPDATE_PC;

        repeat(2) @(if_t.cb);
        pc_post_reset: assert(if_t.cb.pc == 16'h3000);

        // --- 第一部分: 复位与回绕测试 ---
        #1 if_t.cb.reset <= 0; // 同步地释放复位信号

        @(if_t.cb);
        $display("\n\t: Test loading of target address", $realtime);
        if_t.cb.state <= CNTRL_UPDATE_PC;
        if_t.cb.br_taken <= 1;

        @(if_t.cb);
        @(if_t.cb);
        pc_br_taken: assert (if_t.cb.pc == 16'hFFFC);

        $display("%t: Did the PC rollover as expected?", $realtime);

        // --- 第二部分: 遍历控制器状态测试 ---
        if_t.cb.br_taken <= 0;
        if_t.cb.state <= CNTRL_UPDATE_PC;
        repeat (5) @(if_t.cb);
        pc_rollover: assert (if_t.cb.pc == 16'h0000);

        $display("\n\t: Step through all the controller states", $realtime);
        for (int i = CNTRL_FETCH; i <= CNTRL_COMPUTE_MEM; i++) begin
            $cast(cntrl, i);
            if (cntrl == CNTRL_UPDATE_PC) continue; // 跳过 UPDATE_PC, 因为上面已经测过
了

            $display("%t: Try with controller state=%0d %s", $realtime, cntrl,

```

```

cntrl.name);
    $realtime; // 注意：原图此处可能有误，通常是打印时间或作为延时，这里按原文保留结
构
    if_t.cb.br_taken <= 0;
    if_t.cb.state <= cntrl;
    repeat (2) @(if_t.cb);
    pc_no_load: assert (if_t.cb.pc == 16'h0001);
end // for i

// --- 第三部分：高阻态（Tristate）测试 ---
$display("\n\t: Tristate on PC output", $realtime);

if_t.cb.state <= CNTRL_READ_MEM;
@(if_t.cb);
pc_z_read_mem: assert (if_t.cb.pc === 16'hz);

if_t.cb.state <= CNTRL_IND_ADDR_RD;
@(if_t.cb);
pc_z_ind_addr_rd: assert (if_t.cb.pc === 16'hz);

if_t.cb.state <= CNTRL_WRITE_MEM;
@(if_t.cb);
pc_z_write_mem: assert (if_t.cb.pc === 16'hz);

endprogram // test

```

```

`timescale 1ns / 1ns

typedef enum(CNTRL_UPDATE_PC = 0,
    CNTRL_FETCH = 1,
    CNTRL_DECODE = 2,
    CNTRL_EXECUTE = 3,
    CNTRL_UPDATE_REGF = 4,
    CNTRL_COMPUTE_PC = 5,
    CNTRL_COMPUTE_MEM = 6,
    CNTRL_READ_MEM = 7,
    CNTRL_IND_ADDR_RD = 8,
    CNTRL_WRITE_MEM = 9) cntrl_e;

module top;
    bit clock;
    always #10 clock = ~clock;
    fetch_ifc fif(clock);
    test t1(fif,fif);
    fetch
f1(clock,fif.reset,fif.state,fif.pc,fif.npc,fif.rd,fif.taddr,fif.br_taken);
endmodule //top

```

面向对象编程基础

编写一个类

类封装了数据和操作这些数据的子程序，这个数据包包含了地址，CRC和一个存储数值的数组,在Transaction类中有两个子程序:一个输出数据包地址的函数和一个计算循环冗余校验(CRC: cyclic redundancy check) 函数

```
class Transaction;
    bit [31:0] addr,crc,data[8];

    function void display;
        $display("Transaction: %h",addr);
    endfunction

    function void calc_crc;
        crc = addr ^ data.xor;
    endfunction

endclass
```

OOP术语

- (1) **类 (class)**: 包含变量和子程序的基本构建块。Verilog 中与之对应的是模块 (module)。
- (2) **对象 (object)**: 类的一个实例。在 Verilog 中，你需要实例化一个模块才能使用它。
- (3) **句柄 (handle)**: 指向对象的指针。在 Verilog 中，你通过实例名在模块外部引用信号和方法。一个 OOP 句柄就像一个对象的地址，但是它保存在一个只能指向单一数据类型的指针中。
- (4) **属性 (property)**: 存储数据的变量。在 Verilog 中，就是寄存器 (reg) 或者线网 (wire) 类型的信号。
- (5) **方法 (method)**: 任务或者函数中操作变量的程序性代码。Verilog 模块除了 initial 和 always 块以外，还含有任务和函数。
- (6) **原型 (prototype)**: 程序的头，包括程序名、返回类型和参数列表。程序体则包含了执行代码。

创建新对象

Transaction tr; //声明一个句柄 tr = new(); //为一个Transaction 对象分配空间

```
class Transaction;
    logic [31:0] addr,crc,data[8];

    function new;
        addr = 3;
        foreach (data[i])
            data[i] = 5;
    endfunction
endclass
```

```

class Transaction;
    logic [31:0] addr,crc,data[8];
    function new(logic [31:0] a = 3,d = 5);
        addr = a;
        foreach(data[i])
            data[i] = d;
    endfunction
endclass

initial begin
    Transaction tr;
    tr = new(10);
end

```

```

class Transaction;

endclass

class Driver;
    Transaction tr;
    funtion new();
        tr = new();
    endfunction
endclass Driver

```

为对象创建一个句柄

```

Transaction t1,t2; //声明两个句柄
t1 = new(); // 为第一个Transaction 对象分配地址
t2 = t1; // t1和t2都指向该对象
t1 = new(); // 为第二个Transaction对象分配地址

```

对象的解除分配(deallocation)

当最后一个句柄不再引用某个对象了 SystemVerilog就释放该对象的空间

```

Transaction t; //创建一个句柄
t = new(); //分配一个新的Transaction
t = new(); //分配第二个,并且释放第一个t
t = null; //解除分配第二个

```

Verilog会自动回收垃圾

使用对象

```
Transaction t; //声明一个Transaction句柄
t = new(); //创建一个Transaction对象
t.addr = 32'h42; //设置变量的值
t.display(); //调用一个子程序
```

静态变量和全局变量

含有一个静态变量的类

```
class Transaction;
    static int count = 0;
    int id;
    function new()
        id = count++;
    endfunction
endclass

Transaction t1,t2;
initial begin
    t1 = new();
    t2 = new();
    $display("second id = %0d",t2.id);
end
```

通过类名访问静态变量

```
class Transaction;
    static int count = 0;
endclass

initial begin
    run_test();
    $display("%d transaction were created",Transaction::count);
end
```

// 例 5.12 显示静态变量的静态方法

```
class Transaction;
    static Config cfg;
    static int count = 0;
    int id;

    // 显示静态变量的静态方法
    static function void display_statics();
        $display("Transaction cfg.mode=%s, count=%0d",
            cfg.mode.name(), count);
    endfunction
endclass
```



```

        endfunction
    endclass

    Config cfg;

    initial begin
        cfg = new(MODE_ON);
        Transaction::cfg = cfg;
        Transaction::display_statics(); // 调用静态方法
    end

```

类的方法

```

// 例 5.13 类中的方法

class Transaction;
    bit [31:0] addr, crc, data[8];

    function void display();
        $display("@%0t: TR addr=%h,crc=%h", $time, addr, crc);
        $write("\tdata[0-7]=");
        foreach (data[i]) $write(data[i]);
        $display();
    endfunction
endclass

class PCI_Tran;
    bit [31:0] addr, data; // 使用真实的名字
    function void display();
        $display("@%0t: PCI: addr=%h,data=%h", $time, addr, data);
    endfunction
endclass

Transaction t;
PCI_Tran pc;

initial begin
    t = new(); // 创建一个 Transaction 对象
    t.display(); // 调用 Transaction 的方法
    pc = new(); // 创建一个 PCI 事务
    pc.display(); // 调用 PCI 事务的方法
end

```

在类之外定义方法

```

// 例 5.14 块外方法声明

class Transaction;
    bit [31:0] addr, crc, data[8];

```

```

    extern function void display();
endclass

function void Transaction::display();
    $display("@%0t: Transaction addr=%h,crc=%h",
        $time, addr, crc);
    $write("\tdata[0-7]=");
    foreach (data[i]) $write(data[i]);
    $display();
endfunction

class PCI_Tran;
    bit [31:0] addr, data; // 使用实名
    extern function void display();
endclass

function void PCI_Tran::display();
    $display("@%0t: PCI: addr=%h,data=%h",
        $time, addr, data);
endfunction

```

作用域规则

名字作用域

```

int limit; // $root.limit
program automatic p;
    int limit; // p.limit
    class Foo;
        int limit,array[]; ;//$root.p.limit

        // $root.p.Foo.print.limit
        function void print(int limit);
            for(int i = 0; i < limit; i++)
                $display("%m : array[%0d] = %0d", i, array[i]);
        endfunction
    endclass

initial begin
    int limit = $root.limit;
    Foo bar;
    bar = new;
    bar.array = new[limit];
    bar.print(limit);
end
endprogram

```

将类移入package来查找程序错误

```

package Mistake;
class Bad;
    logic [31:0] data[];

    //未定义,不会编译
    function void display;
        for( i = 0; i< data.size(); i++)
            $display("data[%0d] = %0d", i, data[i]);
    endfunction
endclass
endpackage

program test;
    int i;
    import Mistake::* ;
endprogram

```

// 例 5.20 Statistics 类的声明

```

class Statistics;
    time startT, stopT; // 事务的时间
    static int ntrans = 0; // 事务的数目
    static time total_elapsed_time = 0;

    function time how_long;
        how_long = stopT - startT;
        ntrans++;
        total_elapsed_time += how_long;
    endfunction

    function void start;
        startT = $time;
    endfunction
endclass

```

在另外一个类中使用这个类

// 例 5.21 封装 Statistics 类

```

class Transaction;
    bit [31:0] addr, crc, data[8];
    Statistics stats; // Statistics 句柄

    function new();
        stats = new(); // 创建 stats 实例
    endfunction

```

```

    task create_packet();
        // 填充包数据
        stats.start();
        // 传送数据包
    endtask
endclass

```

编译顺序问题

```

// 例 5.22 顺序编译
typedef class Statistics; //定义低级别级

class Transaction;
    Statistics stats; //使用Statistics类
endclass

class Statistics; //定义Statistics类

endclass

```

理解动态对象

传递对象, SystemVerilog 传递该标量的地址, 所以方法可以修改标量变量的值, 如果你不使用 ref 关键字, SystemVerilog 将该变量的值复制到参数变量中

```

task transmit(Transaction t);
    CBbus.rx_data <= t.data;
    t.stats.startT = $time;
    ...
endtask

Transaction t;
initial begin
    t = new();
    t.addr = 42; //初始化数值
    transmit(t); //将对象传递给任务
end

```

在任务中修改句柄

```

// 例 5.21 封装 Statistics 类

class Transaction;
    bit [31:0] addr, crc, data[8];
    Statistics stats; // Statistics 句柄

```

```

function new();
    stats = new(); // 创建 stats 实例
endfunction

task create_packet();
    // 填充包数据
    stats.start();
    // 传送数据包
endtask
endclass

```

正确的产生器,创建多个对象

```

// 例 5.23 产生器
task generator_good(int n)
    Transaction t;
    repeat (n) begin
        t = new(); //创建一个新对象
        t.addr = $random; //随机赋值
        $display("Transaction addr=%h", t.addr);
        transmit(t);
    end
endtask

```

句柄数组 在写测试平台的时候,可能需要保存并且引用许多对象,可以创建句柄数组,数组的每个元素指向一个对象

```

// 例 5.24 句柄数组
task generator();
    transmit tarray[10];
    foreach (tarray[i]) begin
        begin
            tarray[i] = new();
            tarray[i].addr = $random;
            $display("Transaction addr=%h", tarray[i].addr);
            transmit(tarray[i]);
        end
    end
endtask

```

对象的复制

使用new操作符复制一个对象

```

class Transaction;
    bit [31:0] addr,crc,data[8];
endclass

```

```
Transaction src,dst;
initial begin
    src = new;
    dst = new src; //使用new操作符进行复制
end
```

使用new操作符复制一个复杂类

// 例 5.30 使用 new 操作符复制一个复杂类

class Transaction; bit [31:0] addr, crc, data[8]; static int count = 0; int id; Statistics stats; // 指向 Statistics 对象的句柄

```
function new;
    stats = new();    // 构造一个新的 Statistics 对象
    id = count++;
endfunction
```

endclass

Transaction src, dst;

```
initial begin src = new(); // 创建一个 Transaction 对象 src.stats.startT = 42; // 结果见图 5.5 dst = new src; // 用
new 操作符将 src 拷贝到 dst 中, 结果见图 5.6 dst.stats.startT = 96; // 改变 dst 和 src 的 stats
$display(src.stats.startT); // "96", 见图 5.7 end
```

这两个Transaction对象都指向同一个Statistics对象，
所以使用src句柄修改startT会影响dst句柄可以看到的值

```
```systemverilog
// 例 5.31 含有 copy 函数的简单类
class Transaction;
 bit [31:0] addr, crc, data[8]; // 没有 Statistic 句柄

 function Transaction copy;
 copy = new(); // 创建目标对象
 copy.addr = addr; // 填入数值
 copy.crc = crc;
 copy.data = data; // 复制数组
 endfunction
endclass

// 例 5.32 使用 copy 函数
Transaction src, dst;

initial begin
 src = new(); // 创建第一个对象
```

```
 dst = src.copy(); // 复制对象
end
```

## 编写自己的深层复制函数

```
// 例 5.33 深层复制函数
class Transaction;
 bit [31:0] addr, crc, data[8];
 Statistics stats; // 指向 Statistics 对象的句柄
 static int count = 0;
 int id;

 function new();
 stats = new();
 id = count++;
 endfunction

 function Transaction copy();
 copy = new(); // 创建目标对象
 copy.addr = addr; // 填入数值
 copy.crc = crc;
 copy.data = data;
 copy.stats = stats.copy(); // 调用 Statistics::copy 函数
 id = count++;
 endfunction
endclass
```

## 使用pack和unpack函数

```
// 例 5.34 使用 pack 和 unpack 函数
Transaction tr,tr2;
byte b[40];

initial begin
 tr = new();
 tr.addr = 32'h0a0a0a0a;
 tr.crc = '1;
 foreach(tr.data[i])
 tr.data[i] = i;
 tr.pack(b); //打包对象到字节数组
 $write("Pack results:");
 foreach(b[i])
 $write("%0h",b[i]);
 $display;

 tr2 = new();
 tr2.unpack(b);
 tr2.display();
end
```

## 建立一个测试平台

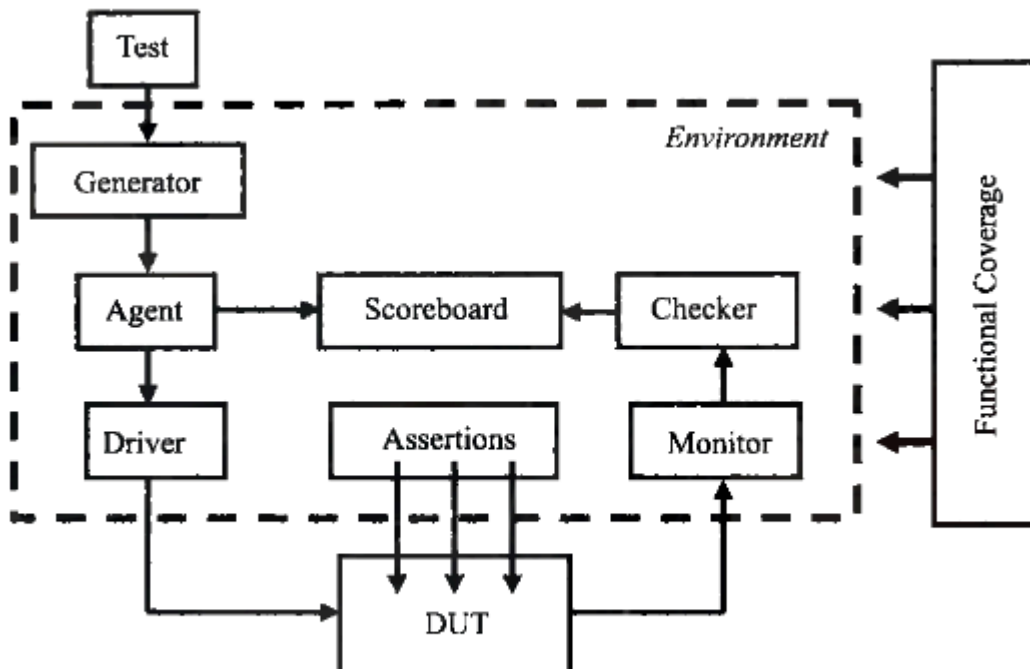


图 5.9 分层的测试平台

图中的Generator,Agent,Driver,Monitor,Checker和Scoreboard都是类 建模成事务处理器,它们在Environment类内部例化

Test处在最高层,即处在例化Environment类的程序中,功能覆盖定义可以放在Environment类的内部或者外部



```
// 基本的事务类
class Transator; //通用类
 Transaction tr;
 task run;
 forever begin
 //从前一个块中获取事务
 ...
 //处理事务
 ...
 //将事务发送到下游模块
 ...
 end
 endtask
endclass
```

## 随机化

带有随机变量的简单类

```
class Packet;
 //随机变量
 randbit [31:0] src,dst,data[8];
 randc bit [7:0] kind;
 //src 的约束
 constraint c{
 src > 0;
 src < 15;
 }

endclass

Packet p;
 initial begin
 p = new(); //产生一个包
 assert(p.randomize());
 else $fatal(0,"Packet::randomize failed");
 transmit(p);
 end
```

## 约束

有用的激励并不仅仅是随机值——各个变量之间有着相互关系,否则仿真器可能需要很长时间才能产生需要的激励值,或者激励向量里会包含无效的值,需要用包含一个或多个约束表达式的约束块定义这些相关关系

```
// 例 6.3 受约束的随机类
class Stim;
 const bit [31:0] CONGEST_ADDR = 42;
 typedef enum {READ, WRITE, CONTROL} stim_e;
```

```

randc stim_e kind; // 枚举变量
rand bit [31:0] len, src, dst;
bit congestion_test;

constraint c_stim {
 len < 1000;
 len > 0;
 if (congestion_test) {
 dst inside {[CONGEST_ADDR-100 : CONGEST_ADDR+100]};
 src == CONGEST_ADDR;
 }
 else
 src inside {0, [2:10], [100:107]};
}
endclass

```

### 固定顺序的约束

```

// 例 6.4 固定顺序的约束
class order;
 rand bit [15:0] lo,med,hi;
 constraint good{lo < med; med < hi;}
endclass

```

```

// 例 6.8 动态改变权重
// 总线操作:字节、字或长字
class BusOp;
 // 操作数长度
 typedef enum {BYTE, WORD, LWRD} length_e;
 rand length_e len;

 // dist 约束的权重
 bit [31:0] w_byte=1, w_word=3, w_lwr=5;

 constraint c_len {
 len dist {BYTE := w_byte, // 使用可变的权重
 WORD := w_word, // 来选择随机的操作数长度
 LWRD := w_lwr};
 }
endclass

```

### 集合成员和inside运算符

```

rand int c;
int lo,hi;
constraint c_lohi {

```

```
c inside {[lo:hi]};
}
```

```
// 例 6.10 使用"$"指定最大和最小值
rand bit [6:0] b; // 0 <= b <= 127
rand bit [5:0] e; // 0 <= e <= 63

constraint c_range {
 b inside {[$:4], [20:$]}; // 0 <= b <= 4 || 20 <= b <= 127
 e inside {[$:4], [20:$]}; // 0 <= e <= 4 || 20 <= e <= 63
}
```

```
// 例 6.11 (推测) 在 inside 约束中有重复的值
class Weighted;
 rand int val;
 // 定义权重数组, 数值出现的次数代表其权重
 int array[] = '{1, 1, 2, 3, 5, 8, 8, 8, 8, 8}';

 constraint c {val inside array;}
endclass

Weighted w;

initial begin
 int count[9], maxx[$]; // count用于统计, maxx用于存储最大值
 w = new();

 repeat (2000) begin
 assert(w.randomize());
 count[w.val]++; // 统计值的个数
 end

 maxx = count.max(); // 获取最大统计值 (用于归一化绘图)

 // 输出值的分布
 foreach(count[i])
 if (count[i]) begin
 $write("count[%0d]=%5d ", i, count[i]);
 // 根据统计数量打印对应长度的星号条
 repeat (count[i] * 40 / maxx[0]) $write("* ");
 $display;
 end
 end
end
```

集合里的每一个值取出来的概率都是相同的,即使值在数组中出现多次,可以把inside约束看成foreach约束

```
// 例 6.16 从数组中取出随机值的类
class Days;
 // 定义枚举类型：一周七天
 typedef enum {SUN, MON, TUE, WED, THU, FRI, SAT} days_e;

 // 定义一个动态数组，用于存储可选的枚举值
 days_e choices[$];

 // 声明随机变量
 rand days_e choice;

 // 约束：choice 的值必须在 choices 数组包含的范围内
 constraint cday {choice inside choices;}
endclass

// 例 6.17 从数组中取出随机值
initial begin
 Days days;
 days = new();

 // 场景 1：只选择周末
 // 将数组赋值为 SUN 和 SAT
 days.choices = {Days::SUN, Days::SAT};
 assert (days.randomize());
 $display("Random weekend day %s\n", days.choice.name);

 // 场景 2：只选择工作日
 // 重新赋值数组为周一到周五
 days.choices = {Days::MON, Days::TUE, Days::WED,
 Days::THU, Days::FRI};
 assert (days.randomize());
 $display("Random week day %s", days.choice.name);
end
```

### 使用randc随机选取数组的值

```
// 例 6.18 使用 randc 随机选取数组的值
// 例 6.18 使用 randc 随机地选取数组的值
class RandcInside;
 int array[]; // 待选取的值
 randc bit [15:0] index; // 指向数组的指针

 function new(input int a[]); // 构造、初始化
 array = a;
 endfunction

 function int pick; // 返回刚选取出的值
 return array[index];
 endfunction
```

```

 constraint c_size {index < array.size;}
endclass

initial begin
 RandcInside ri;

 ri = new('{1,3,5,7,9,11,13});
 repeat (ri.array.size) begin
 assert(ri.randomize());
 $display("Picked %2d [%0d]", ri.pick(), ri.index);
 end
end
end

```

条件约束 -> 操作符可以产生和case操作符效果类似的语句块,它可以用于枚举类型的表达式, 带有->操作符的约束块

```

// 例 6.20 条件约束
class BusOp;
 constraint c_io{
 (io_space_mode)->(addr[31]==1'b1);
 }
endclass

```

带有if else操作符的约束块

```

class BusOp;`n constraint c_len_rw{
 if(op == READ)
 len inside {[BYTE:LWRD]};
 else
 len == LWRD;
 }
endclass

```

以下是图片内容的 Markdown 代码:

```

例 6.21 双向约束

```systemverilog
rand logic [15:0] r, s, t;

constraint c_bidir {
    r < t;
    s == r;
    t < 30;
    s > 25;
}
```

```

SystemVerilog 同时计算四个约束表达式。 $r$  必须小于  $t$ ，而  $t$  必须小于 30。 $r$  等于  $s$ ，而  $s$  必须大于 25。尽管没有直接约束  $t$  的下限，但对于  $s$  的约束隐含着对  $t$  的下限的限制。表 6.1 列出了这三个变量的各种可能值。

表 6.1 双向约束的求解

| 解 | r  | s  | t  | 解 | r  | s  | t  |
|---|----|----|----|---|----|----|----|
| A | 26 | 26 | 27 | D | 27 | 27 | 28 |
| B | 26 | 26 | 28 | E | 27 | 27 | 29 |
| C | 26 | 26 | 29 | F | 28 | 28 | 29 |

即使 `->` 和 `if-else` 这些看起来像 if-else 程序性语句的条件约束，也是双向的。例如，约束 `{(a==1) -> (b==0)}` 和 `{(! (a==1)) || b==0;}` 是等价的。约束求解器并不是先检查 `a==1`，然后再令 `b==0`。事实上，如果增加一个约束 `{b==1;}`，约束求解器将把 `a` 置为 0。

解的概率

关系操作

约束块中的关系操作决定了y的值依赖于x的值

带有关系操作的类

```
class Impl;
 rand bit x;
 rand bit [1:0] y;

 constraint c_xy{
 (x == 0)-> (y == 0);
 }
endclass
```

y的值依赖于x的值

关系操作和双向约束

```
class Imp2;
 rand bit x; //0或1
 rand bit [1:0] y; //0,1,2或3
 constraint c_xy{
 y>0;
 (x == 0)-> y == 0;
 }
endclass
```

使用solve...before 约束引导概率分布

```
class SolveBefore;
 rand bit x;
 rand bit [1:0] y;

 constraint c_xy{
 (x == 0) -> (y == 0);
 solve x before y;
 }
endclass
```

| 解 | x | y | 概率  |
|---|---|---|-----|
| A | 0 | 0 | 1/2 |
| B | 0 | 1 | 0   |
| C | 0 | 2 | 0   |
| D | 0 | 3 | 0   |
| E | 1 | 0 | 1/8 |
| F | 1 | 1 | 1/8 |
| G | 1 | 2 | 1/8 |
| H | 1 | 3 | 1/8 |

solve y before x 约束的解

| 解 | x | y | 概率  |
|---|---|---|-----|
| A | 0 | 0 | 1/8 |
| B | 0 | 1 | 0   |
| C | 0 | 2 | 0   |
| D | 0 | 3 | 0   |
| E | 1 | 0 | 1/8 |
| F | 1 | 1 | 1/4 |
| G | 1 | 2 | 1/4 |
| H | 1 | 3 | 1/4 |

控制多个约束块

```
// 例 6.28 使用 constraint_mode() 函数
class Packet;
 rand int length;
```

```

 constraint c_short {length inside {[1:32]}}; }
 constraint c_long {length inside {[1000:1023]}}; }
endclass

Packet p;
initial begin
 p = new();

 // 通过禁止 c_short 约束产生长包
 p.c_short.constraint_mode(0);
 assert (p.randomize());
 transmit(p);

 // 通过禁止所有的约束，然后使能短包约束来产生短包
 p.constraint_mode(0);
 p.c_short.constraint_mode(1);
 assert (p.randomize());
 transmit(p);
end

```

这段代码演示了如何使用 ``systemverilog class Impl; rand bit x; rand bit [1:0] y;

```

constraint c_xy{
 (x == 0)-> (y == 0);
}

```

endclass

y的值依赖于x的值

关系操作和双向约束

``systemverilog

```

class Imp2;
 rand bit x; //0或1
 rand bit [1:0] y; //0,1,2或3
 constraint c_xy{
 y>0;
 (x == 0)-> y == 0;
 }
endclass

```

使用solve...before 约束引导概率分布

```

class SolveBefore;

```



```

 rand bit x;
 rand bit [1:0] y;

 constraint c_xy{
 (x == 0) -> (y == 0);
 solve x before y;
 }
endclass
```

| 解 | x | y | 概率  |
|---|---|---|-----|
| A | 0 | 0 | 1/2 |
| B | 0 | 1 | 0   |
| C | 0 | 2 | 0   |
| D | 0 | 3 | 0   |
| E | 1 | 0 | 1/8 |
| F | 1 | 1 | 1/8 |
| G | 1 | 2 | 1/8 |
| H | 1 | 3 | 1/8 |

solve y before x 约束的解

| 解 | x | y | 概率  |
|---|---|---|-----|
| A | 0 | 0 | 1/8 |
| B | 0 | 1 | 0   |
| C | 0 | 2 | 0   |
| D | 0 | 3 | 0   |
| E | 1 | 0 | 1/8 |
| F | 1 | 1 | 1/4 |
| G | 1 | 2 | 1/4 |
| H | 1 | 3 | 1/4 |

控制多个约束块

```

// 例 6.28 使用 constraint_mode() 函数
class Packet;
 rand int length;
 constraint c_short {length inside {[1:32]}}; }
 constraint c_long {length inside {[1000:1023]}}; }
endclass
```

```

Packet p;
initial begin
 p = new();

 // 通过禁止 c_short 约束产生长包
 p.c_short.constraint_mode(0);
 assert (p.randomize());
 transmit(p);

 // 通过禁止所有的约束，然后使能短包约束来产生短包
 p.constraint_mode(0);
 p.c_short.constraint_mode(1);
 assert (p.randomize());
 transmit(p);
end

```

这段代码演示了如何使用 `constraint_mode()` 方法在运行时动态控制约束的开启与关闭。

- **class Packet**: 定义了一个包含随机变量 `length` 和两个互斥约束 (`c_short` 和 `c_long`) 的类。
- **p.c\_short.constraint\_mode(0)**: 显式地禁用了 `c_short` 约束。此时只有 `c_long` 生效，因此生成的 `length` 会在 1000 到 1023 之间（长包）。
- **p.constraint\_mode(0)**: 禁用了该对象上的 **所有** 约束。
- **p.c\_short.constraint\_mode(1)**: 在禁用所有约束后，单独重新启用了 `c_short`。此时只有 `c_short` 生效，生成的 `length` 会在 1 到 32 之间（短包）。

## 有效性约束

设置多个约束以保证随机激励的正确性是一种很好的随机化技术，它也称为“有效性约束”。例如，总线的“读-修改-写”命令只允许操作长字数据长度。

### 例 6.29 使用有效性约束检查写命令的数据字长

```

class Transaction;
 rand enum {BYTE, WORD, LWRD, QWRD} length;
 rand enum {READ, WRITE, RMW, INTR} opc;

 constraint valid_RMW_LWRD {
 (opc == RMW) -> length == LWRD;
 }
endclass

```

## 内嵌约束

很多测试只会在代码的一个地方随机化对象

SystemVerilog运行使用`randomize()` with来增加额外的约束,这和在该类里增加约束是等效的

这是图片中的 SystemVerilog 代码：

```
// 例 6.30 randomize() with 语句
class Transaction;
 rand bit [31:0] addr, data;
 constraint c1 {addr inside {[0:100], [1000:2000]}};
endclass

Transaction t;

initial begin
 t = new();

 // addr 范围: 50-100, 1000-1500, data < 10
 assert (t.randomize() with {addr >= 50; addr <= 1500;
 data < 10;});

 driveBus(t);

 // 强制 addr 取固定值, data > 10
 assert (t.randomize() with {addr == 2000; data > 10;});

 driveBus(t);
end
```

## 代码解析

这段代码演示了如何使用 ``systemverilog class Transaction; rand enum {BYTE, WORD, LWRD, QWRD} length; rand enum {READ, WRITE, RMW, INTR} opc;

```
constraint valid_RMW_LWRD {
 (opc == RMW) -> length == LWRD;
}
```

endclass

### ### 内嵌约束

很多测试只会在代码的一个地方随机化对象

SystemVerilog运行使用randomize() with来增加额外的约束,这和 in 类里增加约束是等效的

这是图片中的 SystemVerilog 代码:

```
``systemverilog
// 例 6.30 randomize() with 语句
class Transaction;
 rand bit [31:0] addr, data;
 constraint c1 {addr inside {[0:100], [1000:2000]}};
```

```

endclass

Transaction t;

initial begin
 t = new();

 // addr 范围: 50-100, 1000-1500, data < 10
 assert (t.randomize() with {addr >= 50; addr <= 1500;
 data < 10;});

 driveBus(t);

 // 强制 addr 取固定值, data > 10
 assert (t.randomize() with {addr == 2000; data > 10;});

 driveBus(t);
end

```

## 代码解析

这段代码演示了如何使用 `randomize() with` 语法在调用随机化时临时添加或修改约束，而无需修改类定义。

- **constraint c1**: 定义了基础约束，要求 `addr` 必须在 `[0:100]` 或 `[1000:2000]` 范围内。
- **第一次 randomize()**: 使用 `with` 块添加了额外约束。此时求解器需要同时满足 `c1` 和 `with` 块中的条件。因此 `addr` 的实际有效范围被缩小为 `[50:100]` 和 `[1000:1500]`（即基础范围与新增范围的交集）。
- **第二次 randomize()**: 通过 `addr == 2000` 将地址强制锁定为一个特定值（该值同时也满足基础约束 `c1`），并将数据约束改为大于 10。

## 例 6.31 构造浴缸型分布

```

class Bathtub;
 int value; // 浴缸型分布的随机变量
 int WIDTH=50, DEPTH=4, seed=1;

 function void pre_randomize();
 // 计算指数曲线
 value = $dist_exponential(seed, DEPTH);
 if (value > WIDTH) value = WIDTH;

 // 把这一点随机地放在左边或右边的曲线上
 if ($urandom_range(1))
 value = WIDTH - value;
 endfunction
endclass

```

这段代码通过 ``systemverilog class Bathtub; int value; // 浴缸型分布的随机变量 int WIDTH=50, DEPTH=4, seed=1;

```
function void pre_randomize();
 // 计算指数曲线
 value = $dist_exponential(seed, DEPTH);
 if (value > WIDTH) value = WIDTH;

 // 把这这个点随机地放在左边或右边的曲线上
 if ($urandom_range(1))
 value = WIDTH - value;
endfunction
```

endclass

这段代码通过 `pre\_randomize()` 函数在标准随机化过程之前，利用系统函数 `\$dist\_exponential` 生成一个符合特定分布（如指数分布）的值，并通过逻辑判断将其映射为“浴缸型”分布（即两端概率高、中间概率低）。

因为 `pre\_randomize` 和 `post\_randomize` 函数只能调用其他函数，不能调用消耗时间的任务，所以在执行 `randomize()` 函数的期间无法产生一段延时。如果想调试随机化过程中出现的问题，可以调用预先准备好的 void 类型的显示程序来显示中间结果。

### ### 随机数函数

Verilog-1995 中的各种分布函数都可以采用类似的方法使用，另外 SystemVerilog 还提供了一些新的分布函数。关于 dist 函数的更详细的内容，请查阅随机过程的书籍。下面是一些常用的函数：

- (1) `\$random()` — 平均分布，返回 32 位有符号随机数；
- (2) `\$urandom()` — 平均分布，返回 32 位无符号随机数；
- (3) `\$urandom\_range()` — 在指定范围内的平均分布；
- (4) `\$dist\_exponential()` — 指数衰落，如图 6.1 所示；
- (5) `\$dist\_normal()` — 钟型分布；
- (6) `\$dist\_poisson()` — 钟型分布；
- (7) `\$dist\_uniform()` — 平均分布。

`\$urandom\_range()` 函数有两个参数，一个上限参数和一个可选的下限参数。

### ### 例 6.32 使用 \$urandom\_range 函数

```
``systemverilog
a = $urandom_range(3, 10); // 值的范围是 3~10
a = $urandom_range(10, 3); // 值的范围是 3~10
b = $urandom_range(5); // 值的范围是 0~5
```

## 约束的技巧和技术

```
//使用变量的约束
class bounds;
 rand int size;
 int max_size = 100;
 constraint c_size {size <= max_size;}
endclass
```

这段代码展示了 SystemVerilog 中随机约束（Random Constraint）的一个高级用法：**\*\*在 ``systemverilog a = \$urandom\_range(3, 10); // 值的范围是 3~10 a = \$urandom\_range(10, 3); // 值的范围是 3~10 b = \$urandom\_range(5); // 值的范围是 0~5**

```
约束的技巧和技术

``systemverilog
//使用变量的约束
class bounds;
 rand int size;
 int max_size = 100;
 constraint c_size {size <= max_size;}
endclass
```

这段代码展示了 SystemVerilog 中随机约束（Random Constraint）的一个高级用法：**在 `dist` 分布约束中使用变量作为权重**。这允许用户在仿真运行时动态调整随机变量的概率分布，而无需修改代码中的硬编码数值。

## 提取的代码

```
typedef enum {READ8, READ16, READ32} read_e;

class ReadCommands;
 rand read_e read_cmd;
 int read8_wt = 1, read16_wt = 1, read32_wt = 1;

 constraint c_read {
 read_cmd dist {READ8 := read8_wt,
 READ16 := read16_wt,
 READ32 := read32_wt};
 }
endclass
```

## 代码解析

### 枚举类型定义

•

```
typedef enum {READ8, READ16, READ32} read_e;
```

```
class ReadCommands; rand read_e read_cmd; int read8_wt = 1, read16_wt = 1, read32_wt = 1;
```

```
constraint c_read {
 read_cmd dist {READ8 := read8_wt,
 READ16 := read16_wt,
 READ32 := read32_wt};
}
```

```
endclass
```

### ### 代码解析

#### #### 枚举类型定义

- ``typedef enum {READ8, READ16, READ32} read_e;``
- 定义了一个名为 ``read_e`` 的枚举类型，包含三个成员：``READ8``、``READ16`` 和 ``READ32``。这通常用于表示不同的总线命令或操作码。

#### #### 类定义与成员变量

- ``class ReadCommands;``
- 定义了一个名为 ``ReadCommands`` 的类。
- ``rand read_e read_cmd;``
- 声明了一个随机变量 ``read_cmd``，其类型为前面定义的枚举 ``read_e``。这意味着在进行随机化（``randomize()``）时，求解器会尝试给这个变量赋值。
- ``int read8_wt = 1, read16_wt = 1, read32_wt = 1;``
- 声明了三个整型变量，用作权重（weight）。初始值都设为 1，意味着默认情况下三种命令被选中的概率是相等的（各占 1/3）。这些变量\*\*不是\*\*随机的（没有 ``rand`` 关键字），它们通常由测试用例在随机化之前进行赋值。

#### #### 约束块

- ``constraint c_read { ... }``
- 这是核心部分，定义了 ``read_cmd`` 的分布规则。
- ``read_cmd dist { ... };``
- ``dist`` 是 SystemVerilog 的分布操作符，用于指定随机变量取特定值的权重。
- \*\*关键特性：使用变量作为权重\*\*
  - 传统的写法通常是硬编码数字，例如 ``READ8 := 50``。
  - 这里的写法是 ``READ8 := read8_wt``。这意味着权重的值来自于变量 ``read8_wt``。
  - \*\*优势\*\*：你可以在调用 ``randomize()`` 之前，通过外部代码修改 ``read8_wt``、``read16_wt`` 等变量的值，从而动态改变随机化的行为。例如，如果你想让测试主要集中在 8 位读取上，只需将 ``read8_wt`` 设为 90，其他设为 5 即可。

#### #### 语法细节

- `:=` 操作符：表示“范围权重”。对于枚举类型这种单值情况，它等同于 `/`（单值权重）。意思是如果选中该值，其权重就是指定的数值。
- 概率计算公式：某个值被选中的概率 = (该值的权重) / (所有可能值的权重之和)。

### ### 应用场景示例

假设你想模拟一个主要进行 32 位读取的系统，你可以这样编写测试代码：

```
``systemverilog
ReadCommands cmd = new();

// 动态调整权重
cmd.read8_wt = 10;
cmd.read16_wt = 10;
cmd.read32_wt = 80; // 80% 的概率生成 READ32

// 执行随机化
assert(cmd.randomize());

// 此时 cmd.read_cmd 有 80% 的概率是 READ32
```

这种机制极大地提高了验证环境的灵活性和复用性。

## 使用非随机值

如果只有少数几个变量需要修改,可以使用rand\_mode函数把这些变量设置为非随机变量

```
// 例 6.35 用 rand_mode 禁止变量的随机化
// 产生变长负载的包
class Packet;
 rand bit [7:0] length, payload[];
 constraint c_valid {length > 0;
 payload.size == length;}

 function void display(string msg);
 $display("\n% s", msg);
 $write("Packet len=%0d, payload size=%0d, bytes=",
 length, payload.size());
 for (int i=0; (i < 4 && i < payload.size()); i++)
 $write(" %0d", payload[i]);
 $display;
 endfunction
endclass

Packet p;
initial begin
 p = new();

 // 随机化所有的变量
 assert (p.randomize());
 p.display("Simple randomize");
```



```

 p.length.rand_mode(0); // 设置包长为非随机值
 p.length = 42; // 设置包长为常数
 assert (p.randomize()); // 然后再随机化 payload
 p.display("Randomize with rand_mode");
end

```

可以使用`handle.randomize(null)`把所有的变量当作非随机变量,仅仅检查这些变量是否满足约束条件

随机化个别变量

在调用`randomize()`函数时只传递变量的一个子集,这样就只会随机化类里的几个变量

```

class Rising;
 byte low;//非随机变量
 rand byte med,hi;//随机变量
 constraint up
 {
 low<med;med<hi;
 }
endclass

initial begin
 Rising r;
 r= new();
 r.randomize();
 r.randomize(med);//只随机化med\
 r.randomize(low);//只随机化med和hi
end

```

```

// 例 6.38 使用 constraint_mode 打开或关闭约束
class Instruction;
 rand opcode_e opcode;
 // ... (省略部分代码)

 constraint c_no_operands {
 opcode == NOP || opcode == HALT;
 }

 constraint c_one_operand {
 opcode == CLR || opcode == NOT;
 }
endclass

Instruction instr;
initial begin
 instr = new();

 // 产生没有操作数的指令
 instr.constraint_mode(0); // 关闭所有的约束

```

```

instr.c_no_operands.constraint_mode(1); // 仅开启 c_no_operands
assert (instr.randomize());

// 产生只有一个操作数的指令
instr.constraint_mode(0); // 再次关闭所有的约束
instr.c_one_operand.constraint_mode(1); // 仅开启 c_one_operand
assert (instr.randomize());
end

```

函数的函数体可以在函数的外部定义，同样，约束的约束体也可以在类的外部定义，如 5.11 节所示。可以在一个文件里定义一个类，这个类只有一个空的约束，然后在每个不同的测试里定义这个约束的不同版本以产生不同的激励。

### 例 6.39 带有外部约束的类

```

// packet.sv
class Packet;
 rand bit [7:0] length;
 rand bit [7:0] payload[];
 constraint c_valid {length > 0;
 payload.size() == length;}

 constraint c_external;
endclass

```

### 例 6.40 定义外部约束的程序

```

// test.sv
program automatic test;
 include "packet.sv"
 constraint Packet::c_external {length == 1;}

 ...
endprogram

```

外部约束和内嵌约束相比具有很多优点。外部约束可以放在另一个文件里，从而在不同的测试里可以复用外部约束。外部约束对类的所有实例都起作用，而内嵌约束仅仅影响一次 ``systemverilog ReadCommands cmd = new();

// 动态调整权重 cmd.read8\_wt = 10; cmd.read16\_wt = 10; cmd.read32\_wt = 80; // 80% 的概率生成 READ32

// 执行随机化 assert(cmd.randomize());

// 此时 cmd.read\_cmd 有 80% 的概率是 READ32

这种机制极大地提高了验证环境的灵活性和复用性。

### ### 使用非随机值

如果只有少数几个变量需要修改,可以使用`rand_mode`函数把这些变量设置为非随机变量

```
```systemverilog
// 例 6.35 用 rand_mode 禁止变量的随机化
// 产生变长负载的包
class Packet;
    rand bit [7:0] length, payload[];
    constraint c_valid {length > 0;
                        payload.size == length;}

    function void display(string msg);
        $display("\n% s", msg);
        $write("Packet len=%0d, payload size=%0d, bytes=",
              length, payload.size());
        for (int i=0; (i < 4 && i < payload.size()); i++)
            $write(" %0d", payload[i]);
        $display;
    endfunction
endclass

Packet p;
initial begin
    p = new();

    // 随机化所有的变量
    assert (p.randomize());
    p.display("Simple randomize");

    p.length.rand_mode(0);      // 设置包长为非随机值
    p.length = 42;              // 设置包长为常数
    assert (p.randomize());     // 然后再随机化 payload
    p.display("Randomize with rand_mode");
end
```

可以使用`handle.randomize(null)`把所有的变量当作非随机变量,仅仅检查这些变量是否满足约束条件

随机化个别变量

在调用`randomize()`函数时只传递变量的一个子集,这样就只会随机化类里的几个变量

```
class Rising;
    byte low;//非随机变量
    rand byte med,hi;//随机变量
    constraint up
    {
        low<med;med<hi;
```

```

    }
endclass

initial begin
    Rising r;
    r = new();
    r.randomize();
    r.randomize(med); // 只随机化 med
    r.randomize(low); // 只随机化 med 和 hi
end

```

```

// 例 6.38 使用 constraint_mode 打开或关闭约束
class Instruction;
    rand opcode_e opcode;
    // ... (省略部分代码)

    constraint c_no_operands {
        opcode == NOP || opcode == HALT;
    }

    constraint c_one_operand {
        opcode == CLR || opcode == NOT;
    }
endclass

Instruction instr;
initial begin
    instr = new();

    // 产生没有操作数的指令
    instr.constraint_mode(0); // 关闭所有的约束
    instr.c_no_operands.constraint_mode(1); // 仅开启 c_no_operands
    assert (instr.randomize());

    // 产生只有一个操作数的指令
    instr.constraint_mode(0); // 再次关闭所有的约束
    instr.c_one_operand.constraint_mode(1); // 仅开启 c_one_operand
    assert (instr.randomize());
end

```

函数的函数体可以在函数的外部定义，同样，约束的约束体也可以在类的外部定义，如 5.11 节所示。可以在一个文件里定义一个类，这个类只有一个空的约束，然后在每个不同的测试里定义这个约束的不同版本以产生不同的激励。

例 6.39 带有外部约束的类

```

// packet.sv
class Packet;
    rand bit [7:0] length;

```

```

    rand bit [7:0] payload[];
    constraint c_valid {length > 0;
                        payload.size() == length;}

    constraint c_external;
endclass

```

例 6.40 定义外部约束的程序

```

// test.sv
program automatic test;
    include "packet.sv"
    constraint Packet::c_external {length == 1;}

    ...
endprogram

```

外部约束和内嵌约束相比具有很多优点。外部约束可以放在另一个文件里，从而在不同的测试里可以复用外部约束。外部约束对类的所有实例都起作用，而内嵌约束仅仅影响一次 `randomize()` 调用。外部约束提供了一种不需要学习高级的 OOP 技术就可以改变类的方法。但要注意，这种方法只能增加约束，而不能改变已有的约束，而且必须事先在原来的类里定义外部约束的原型。

约束动态数组的大小

```

class dyn_size;
    rand logic [31:0] d[];
    constraint d_size {d.size() inside {[1:10]}};
endclass

```

随机脉冲类

```

class Pulse;
    rand bit strobe[MAX_TRANSFER_LEN];
    constraint c_set_four{
        strobe.sum() == 4'h4;
    }
endclass

initial begin
    StrobePat sp;
    int count = 0; //数据数组的索引
    sp = new();
    assert(sp.randomize());

    foreach(sp.strobe[i]) begin
        @bus.cb;
    end
end

```

```

        bus.cb.strobe = sp.strobe[i];
        if(sp.strobe[i]) begin
            bus.cb.strobe <= data[count++];
        end
    end
end

```

约束数组和队列的每一个元素

```

class goot_sum5;
    rand uint len[];
    constraint c_len{
        foreach(len[i])
            len[i] inside {[1:255]};
        len.sum() < 1024;
        len.size() inside {[1:8]};
    }
endclass

```

使用foreach产生递增的数组元素的值

```

class Ascend;
    rand uint d[10];
    constraint c{
        foreach(d[i])
            if(i>0)
                d[i] > d[i-1];
    }
endclass

```

使用foreach产生唯一的元素值

```

class UniqueSlow;
    rand bit[7:0] ua[64];
    constraint c{
        foreach(ua[i])
            foreach(ua[j])
                if(i!=j)
                    ua[i] != ua[j];
    }
endclass

```

```

// 例 6.59 用 randc 辅助类产生唯一的元素值
class randc8;
    randc bit [7:0] val;
endclass

```

```

class LittleUniqueArray;
    bit [7:0] ua [64];      // 每个元素具有唯一值的数组

    function void pre_randomize;
        randc8 rc8;
        rc8 = new();
        foreach (ua[i]) begin
            assert(rc8.randomize());
            ua[i] = rc8.val;
        end
    endfunction
endclass

```

唯一值发生器

```

class Unique;
    rand bit [7:0] val;
    int max_value;
    function new(int max_value = 10);
        this.max_value = max_value;
    endfunction

    constraint c {
        val < max_value;
    }
endclass

```

```

class UniqueArray;
    int max_array_size, max_value;      // 数组最大长度、元素最大值
    rand bit [7:0] a[];                // 随机化动态数组（元素为8位无符号数）
    constraint c_size {a.size() inside {[1:max_array_size]}}; // 约束：数组大小在
    [1, max_array_size]内

    function new(int max_array_size=2, max_value=2); // 构造函数（带默认参数）
        this.max_array_size = max_array_size;
        if (max_value < max_array_size) // 若“元素最大值”小于“数组最大长度”，则调整
        （避免重复）
            this.max_value = max_array_size;
        else
            this.max_value = max_value;
    endfunction

    // 随机化后处理：填充唯一值
    function void post_randomize;
        RandcRange rr;                // 假设RandcRange是自定义类（含randc变量
+randomize）
        rr = new(max_value);          // 初始化RandcRange（内部维护不重复的随机
        序列）
    endfunction

```

```

        foreach (a[i]) begin // 遍历数组每个元素
            assert (rr.randomize()); // 断言：确保RandcRange能成功随机化（获取
下一个唯一值）
            a[i] = rr.value; // 将RandcRange的唯一值赋给数组元素
        end
    endfunction

    function void display(); // 显示数组信息
        $write("Size: %3d:", a.size());
        foreach (a[i]) $write("%4d", a[i]);
        $display;
    endfunction
endclass

```

使用UniqueArray类

```

program automatic test;
    UniqueArray ua;
    initial begin
        ua = new(50);

        repeat(10) begin
            assert(ua.randomize());
            ua.display();
        end
    end
endprogram

```

随机化句柄数组

```

parameter MAX_SIZE=10;

class RandStuff;
    rand int value;
endclass

class RandArray;
    rand RandStuff array[]; // 不要忘记使用 rand!

    constraint c {array.size() inside {[1:MAX_SIZE]}; }

    function new();
        array=new[MAX_SIZE]; // 按最大的容量分配
        foreach (array[i])
            array[i]=new();
    endfunction;
endclass

RandArray ra;

```



```

initial begin
    ra=new();                // 构造数组和所有的对象
    assert(ra.randomize());   // 随机化，可能会减小数组
    foreach (ra.array[i])
        $display(ra.array[i].value);
end

```

产生原子激励和场景

这段代码展示了 SystemVerilog 中 **序列 (Sequence)** 的定义与使用，特别是利用 ``systemverilog class dyn_size; rand logic [31:0] d[]; constraint d_size {d.size() inside {[1:10]};} endclass

随机脉冲类

```

``systemverilog
class Pulse;
    rand bit strobe[MAX_TRANSFER_LEN];
    constraint c_set_four{
        strobe.sum() == 4'h4;
    }
endclass

initial begin
    StrobePat sp;
    int count = 0; //数据数组的索引
    sp = new();
    assert(sp.randomize());

    foreach(sp.strobe[i]) begin
        @bus.cb;
        bus.cb.strobe = sp.strobe[i];
        if(sp.strobe[i]) begin
            bus.cb.strobe <= data[count++];
        end
    end
end

```

约束数组和队列的每一个元素

```

class goot_sum5;
    rand uint len[];
    constraint c_len{
        foreach(len[i])
            len[i] inside {[1:255]};
        len.sum() < 1024;
        len.size() inside {[1:8]};
    }
endclass

```

```

    }
endclass

```

使用foreach产生递增的数组元素的值

```

class Ascend;
  rand uint d[10];
  constraint c{
    foreach(d[i])
      if(i>0)
        d[i] > d[i-1];
  }
endclass

```

使用foreach产生唯一的元素值

```

class UniqueSlow;
  rand bit[7:0] ua[64];
  constraint c{
    foreach(ua[i])
      foreach(ua[j])
        if(i!=j)
          ua[i] != ua[j];
  }
endclass

```

```

// 例 6.59 用 randc 辅助类产生唯一的元素值
class randc8;
  randc bit [7:0] val;
endclass

class LittleUniqueArray;
  bit [7:0] ua [64];      // 每个元素具有唯一值的数组

  function void pre_randomize;
    randc8 rc8;
    rc8 = new();
    foreach (ua[i]) begin
      assert(rc8.randomize());
      ua[i] = rc8.val;
    end
  endfunction
endclass

```

唯一值发生器

```

class Unique;
  rand bit [7:0] val;
  int max_value;
  function new(int max_value = 10);
    this.max_value = max_value;
  endfunction

  constraint c {
    val < max_value;
  }
endclass

```

```

class UniqueArray;
  int max_array_size, max_value;          // 数组最大长度、元素最大值
  rand bit [7:0] a[];                     // 随机化动态数组（元素为8位无符号数）
  constraint c_size {a.size() inside {[1:max_array_size]}}; // 约束：数组大小在
[1, max_array_size]内

  function new(int max_array_size=2, max_value=2); // 构造函数（带默认参数）
    this.max_array_size = max_array_size;
    if (max_value < max_array_size) // 若“元素最大值”小于“数组最大长度”，则调整
（避免重复）
      this.max_value = max_array_size;
    else
      this.max_value = max_value;
  endfunction

  // 随机化后处理：填充唯一值
  function void post_randomize;
    RandcRange rr;                        // 假设RandcRange是自定义类（含randc变量
+randomize）
    rr = new(max_value);                  // 初始化RandcRange（内部维护不重复的随机
序列）
    foreach (a[i]) begin                  // 遍历数组每个元素
      assert (rr.randomize());            // 断言：确保RandcRange能成功随机化（获取
下一个唯一值）
      a[i] = rr.value;                    // 将RandcRange的唯一值赋给数组元素
    end
  endfunction

  function void display();                // 显示数组信息
    $write("Size: %3d:", a.size());
    foreach (a[i]) $write("%4d", a[i]);
    $display;
  endfunction
endclass

```

使用UniqueArray类

```

program automatic test;
  UniqueArray ua;
  initial begin
    ua = new(50);

    repeat(10) begin
      assert(ua.randomize());
      ua.display();
    end
  end
endprogram

```

随机化句柄数组

```

parameter MAX_SIZE=10;

class RandStuff;
  rand int value;
endclass

class RandArray;
  rand RandStuff array[];          // 不要忘记使用 rand!

  constraint c {array.size() inside {[1:MAX_SIZE]}; }

  function new();
    array=new[MAX_SIZE];          // 按最大的容量分配
    foreach (array[i])
      array[i]=new();
  endfunction;
endclass

RandArray ra;
initial begin
  ra=new();                      // 构造数组和所有的对象
  assert(ra.randomize());        // 随机化，可能会减小数组
  foreach (ra.array[i])
    $display(ra.array[i].value);
end

```

产生原子激励和场景

这段代码展示了 SystemVerilog 中 **序列 (Sequence)** 的定义与使用，特别是利用 **randsequence** 进行随机指令流生成的场景。

以下是提取的代码及其详细解析：

提取的代码

```

initial begin
  for (int i=0; i<15; i++) begin
    randsequence (stream)
      stream : cfg_read :=1 |
              io_read  :=2 |
              mem_read  :=5;

    cfg_read : { cfg_read_task; } |
               { cfg_read_task; } cfg_read;

    mem_read : { mem_read_task; } |
               { mem_read_task; } mem_read;

    io_read  : { io_read_task; } |
               { io_read_task; } io_read;
    endsequence
  end // for
end

task cfg_read_task;
  ...
endtask

```

代码解析

这段代码的主要目的是在一个循环中生成随机的读写操作流，模拟硬件验证中的随机激励。

1. ``systemverilog

```

initial begin for (int i=0; i<15; i++) begin randsequence (stream) stream : cfg_read :=1 | io_read :=2 |
mem_read :=5;

```

```

    cfg_read : { cfg_read_task; } |
               { cfg_read_task; } cfg_read;

    mem_read : { mem_read_task; } |
               { mem_read_task; } mem_read;

    io_read  : { io_read_task; } |
               { io_read_task; } io_read;
    endsequence
  end // for

```

```

end

```

```

task cfg_read_task; ... endtask

```

代码解析

这段代码的主要目的是在一个循环中生成随机的读写操作流，模拟硬件验证中的随机激励。

1. `randsequence` 结构

这是 SystemVerilog 特有的结构，用于定义上下文无关文法（Context-Free Grammar），从而生成符合特定规则的随机序列。

- **起始符（Start Symbol）**：`stream` 是序列的入口点。
- **权重分配（Weights）**：在 `stream` 的定义中使用了 `:=` 符号来指定概率权重。
 - `cfg\_read := 1`：配置读操作的权重为 1。
 - `io\_read := 2`：IO 读操作的权重为 2。
 - `mem\_read := 5`：内存读操作的权重为 5。
- **结果分析**：总权重为 $1+2+5=8$ 。这意味着在生成的序列中，`mem\_read` 出现的概率最高（约 62.5%），其次是 `io\_read`（25%），最后是 `cfg\_read`（12.5%）。这种加权机制常用于模拟真实的业务负载分布。

2. 递归定义（Recursion）

代码中的 `cfg\_read`、`mem\_read` 和 `io\_read` 定义展示了递归特性，用于控制操作的突发长度（Burst Length）。以 `cfg\_read` 为例：

```

`systemverilog
cfg_read : { cfg_read_task; } |           // 选项 A: 执行一次任务后结束
           { cfg_read_task; } cfg_read;    // 选项 B: 执行一次任务后，再次调用
cfg_read

```

- **含义**：这是一个典型的左递归或右递归结构（此处表现为执行任务后可能继续递归）。
- **行为**：当求解器选择 ``systemverilog cfg_read : { cfg_read_task; } | // 选项 A: 执行一次任务后结束 { cfg_read_task; } cfg_read; // 选项 B: 执行一次任务后，再次调用 cfg_read`

- **含义**：这是一个典型的左递归或右递归结构（此处表现为执行任务后可能继续递归）。
- **行为**：当求解器选择 `cfg\_read` 时，它会先执行 `cfg\_read\_task`。然后，它有 50% 的概率直接结束该分支，或者有 50% 的概率再次进入 `cfg\_read` 状态，从而形成连续的配置读操作（Burst）。
- **注意**：由于没有显式的终止条件限制递归深度，理论上这可能产生无限长的序列，但在仿真工具的实际实现中通常会有内部保护机制或依赖于随机种子的自然终止。

3. 循环控制

```

`systemverilog
for (int i=0; i<15; i++) begin ... end

```

外层包裹了一个 ``systemverilog for (int i=0; i<15; i++) begin ... end`

外层包裹了一个 `for` 循环，意味着上述的随机序列生成过程会重复执行 15 次。每次循环都会重新评估 `randsequence`，生成一段新的随机操作流。

4. 任务调用

花括号 `{ ... }` 中的内容（如 `{ cfg\_read\_task; }`）表示在该序列节点匹配成功时执行的具体动作。这里调用了外部定义的 task（如 `cfg\_read\_task`），这些 task 通常包含具体的驱动信号、时序等待等验证逻辑。

总结

该代码片段是一个高效的**随机指令生成器**。它通过加权概率控制了不同操作类型的分布比例，并通过递归结构实现了操作的突发性（Burst），非常适合用于总线协议或存储控制器的压力测试。

随机控制

使用randcase和\$urandom_range的随机控制

```
``systemverilog
initial begin
    int len;
    randcase
    1: len = $urandom_range(0,2);
    2: len = $urandom_range(3,5);
    7: len = $urandom_range(6,8);
    endcase
    $display("Random length: %0d", len);

end
```

等效的约束类

```
class LenDist;
    rand int len;
    constraint c
    {
        len dist {[0:2]:=1, [3:5]:=2, [6:8]:=7}
    }
endclass

LenDist lenD;

initial begin
    lenD = new();
    assert(lenD.randomize());
    $display("Random length: %0d", lenD.len);
end
```

使用randcase建立决策树

```
// 例 6.67 用 randcase 建立决策树
initial begin
    // Level 1
    randcase
        one_write_wt: do_one_write();
        one_read_wt:  do_one_read();
        seq_write_wt: do_seq_write();
        seq_read_wt:  do_seq_read();
    endcase
end

// Level 2
task do_one_write;
    randcase
        mem_write_wt: do_mem_write();
        io_write_wt:  do_io_write();
        cfg_write_wt: do_cfg_write();
    endcase
endtask

task do_one_read;
    randcase
        mem_read_wt: do_mem_read();
        io_read_wt:  do_io_read();
        cfg_read_wt: do_cfg_read();
    endcase
endtask
```

随机数发生器

简单的伪随机数发生器

```
reg [31:0] state = 32'h12345678;
function logic [31:0] my_random;
    logic [63:0] s64;
    s64 = state * state;
    state = (s64 >> 16) + state;
    my_random = state;
endfunction
```

随机器件配置

以太网交换机的配置

```
class eth_cfg;
    rand bit [3:0] in_use; //测试中使用的端口
```



```

rand bit [47:0] mac_addr[4]; //MAC地址
rand bit [3:0] is_100;
rand uint run_for_n_frames; //测试中的帧数

//在unicast 模式时设置某些地址位
constraint local_unicast{
    foreach(mac_addr[i])
        mac_addr[i][47:40] == 4'b00;
}

constraint reasonable{
    run_for_n_frames inside{[1:100]};
}

endclass eth_cfg

```

在Environment类的不同阶段使用了配置类,配置在Environment的构造函数是创建, 但直到gen_cfg阶段才随机化,这就使你在调用randomize()之前打开或关闭约束

```

class Environment;
    eth_cfg cfg;           // 配置对象句柄
    eth_src gen[4];        // 4个发生器 (Generator) 数组
    eth_mii drv[4];        // 4个驱动器 (Driver) 数组

    function new();
        cfg = new();      // 创建配置对象
    endfunction

    // 随机化配置
    function void gen_cfg;
        assert(cfg.randomize());
    endfunction

    // 使用随机配置建立环境
    function void build();
        foreach (gen[i])
            if (cfg.in_use[i]) begin
                gen[i] = new(); // 实例化发生器
                drv[i] = new(); // 实例化驱动器
                if (cfg.is_100[i])
                    drv[i].set_speed(100); // 根据配置设置速度
            end
        endfunction

    task run();
        foreach (gen[i])
            if (cfg.in_use[i]) begin
                // 启动测试平台的事务处理器
                gen[i].run();
                ...
            end
    end

```

```

endtask

task wrap_up();
    // 暂时还没有使用
endtask

endclass Environment

```

```

program test;
    Environment env;
    initial begin
        env = new();
        env.gen_cfg(); // 随机化配置
        env.build(); // 建立环境
        env.run(); // 运行测试
        env.wrap_up(); // 结束测试
    end
endprogram

```

修改随机配置的简单测试

```

program test;
    Environment env;

    initial begin
        env = new();
        env.gen_cfg();
        env.cfg.in_use = 4'b1111; // 修改配置
        env.build();
        env.run();
        env.wrap_up();
    end
endprogram

```

线程及线程间通信

线程的使用

```

// 例 7.1 fork...join 和 begin...end 的相互作用
initial begin
    $display("@%0t: start fork...join example", $time);
    #10 $display("@%0t: sequential after #10", $time);

    fork
        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after #50", $time);
        #10 $display("@%0t: parallel after #10", $time);
    join
end

```

```

        begin
            #30 $display("@%0t: sequential after #30", $time);
            #10 $display("@%0t: sequential after #10", $time); // 注意：此处相对时
间为#10，绝对时间为#40
        end
    join

    $display("@%0t: after join", $time);
    #80 $display("@%0t: finish after #80", $time);
end

```

fork join 直到 #50开头的最后那条语句执行结束后才完成

fork join_none 块在调度其内快语句时,父线程继续执行,除了join被换成join_none以外,其余均相同

```

// 例 7.3 fork...join_none 代码
initial begin
    $display("@%0t: start fork...join_none example", $time);
    #10 $display("@%0t: sequential after # 10", $time);

    fork
        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after # 50", $time);
        #10 $display("@%0t: parallel after # 10", $time);

        begin
            #30 $display("@%0t: sequential after # 30", $time);
            #10 $display("@%0t: sequential after # 10", $time);
        end
    join_none

    $display("@%0t: after join_none", $time);
    #80 $display("@%0t: finish after # 80", $time);
end

```

注意： 这个块相应的框图类似于图 7.3。注意 ``systemverilog initial begin int len; randcase 1: len = \$urandom_range(0,2); 2: len = \$urandom_range(3,5); 7: len = \$urandom_range(6,8); endcase \$display("Random length: %0d", len);

end

等效的约束类

```

``systemverilog
class LenDist;
    rand int len;
    constraint c

```

```

    {
        len dist {[0:2]:=1, [3:5]:=2, [6:8]:=7}
    }
endclass

LenDist lenD;

initial begin
    lenD = new();
    assert(lenD.randomize());
    $display("Random length: %0d", lenD.len);
end

```

使用randcase建立决策树

```

// 例 6.67 用 randcase 建立决策树
initial begin
    // Level 1
    randcase
        one_write_wt: do_one_write();
        one_read_wt:  do_one_read();
        seq_write_wt: do_seq_write();
        seq_read_wt:  do_seq_read();
    endcase
end

// Level 2
task do_one_write;
    randcase
        mem_write_wt: do_mem_write();
        io_write_wt:  do_io_write();
        cfg_write_wt: do_cfg_write();
    endcase
endtask

task do_one_read;
    randcase
        mem_read_wt: do_mem_read();
        io_read_wt:  do_io_read();
        cfg_read_wt: do_cfg_read();
    endcase
endtask

```

随机数发生器

简单的伪随机数发生器

```

reg [31:0] state = 32'h12345678;
function logic [31:0] my_random;

```

```

    logic [63:0] s64;
    s64 = state * state;
    state = (s64 >> 16) + state;
    my_random = state;
endfunction

```

随机器件配置

以太网交换机的配置

```

class eth_cfg;
    rand bit [3:0] in_use; //测试中使用的端口
    rand bit [47:0] mac_addr[4]; //MAC地址
    rand bit [3:0] is_100;
    rand uint run_for_n_frames; //测试中的帧数

    //在unicast 模式时设置某些地址位
    constraint local_unicast{
        foreach(mac_addr[i])
            mac_addr[i][47:40] == 4'b00;
    }

    constraint reasonable{
        run_for_n_frames inside{[1:100]};
    }

endclass eth_cfg

```

在Environment类的不同阶段使用了配置类,配置在Environment的构造函数是创建, 但直到gen_cfg阶段才随机化,这就使你在调用randomize()之前打开或关闭约束

```

class Environment;
    eth_cfg cfg;           // 配置对象句柄
    eth_src gen[4];        // 4个发生器 (Generator) 数组
    eth_mii drv[4];        // 4个驱动器 (Driver) 数组

    function new();
        cfg = new();      // 创建配置对象
    endfunction

    // 随机化配置
    function void gen_cfg;
        assert(cfg.randomize());
    endfunction

    // 使用随机配置建立环境
    function void build();
        foreach (gen[i])
            if (cfg.in_use[i]) begin

```

```

        gen[i] = new(); // 实例化发生器
        drv[i] = new(); // 实例化驱动器
        if (cfg.is_100[i])
            drv[i].set_speed(100); // 根据配置设置速度
    end
endfunction

task run();
    foreach (gen[i])
        if (cfg.in_use[i]) begin
            // 启动测试平台的事务处理器
            gen[i].run();
            ...
        end
    end
endtask

task wrap_up();
    // 暂时还没有使用
endtask

endclass Environment

```

```

program test;
    Environment env;
    initial begin
        env = new();
        env.gen_cfg(); // 随机化配置
        env.build(); // 建立环境
        env.run(); // 运行测试
        env.wrap_up(); // 结束测试
    end
endprogram

```

修改随机配置的简单测试

```

program test;
    Environment env;

    initial begin
        env = new();
        env.gen_cfg;
        env.cfg.in_use = 4'b1111; // 修改配置
        env.build();
        env.run();
        env.wrap_up();
    end
endprogram

```

线程及线程间通信

线程的使用

```
// 例 7.1 fork...join 和 begin...end 的相互作用
initial begin
    $display("@%0t: start fork...join example", $time);
    #10 $display("@%0t: sequential after #10", $time);

    fork

        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after #50", $time);
        #10 $display("@%0t: parallel after #10", $time);

        begin
            #30 $display("@%0t: sequential after #30", $time);
            #10 $display("@%0t: sequential after #10", $time); // 注意：此处相对时
间为#10，绝对时间为#40
        end
    join

    $display("@%0t: after join", $time);
    #80 $display("@%0t: finish after #80", $time);
end
```

fork join 直到 #50开头的最后那条语句执行结束后才完成

fork join_none 块在调度其内快语句时,父线程继续执行,除了join被换成join_none以外,其余均相同

```
// 例 7.3 fork...join_none 代码
initial begin
    $display("@%0t: start fork...join_none example", $time);
    #10 $display("@%0t: sequential after # 10", $time);

    fork

        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after # 50", $time);
        #10 $display("@%0t: parallel after # 10", $time);

        begin
            #30 $display("@%0t: sequential after # 30", $time);
            #10 $display("@%0t: sequential after # 10", $time);
        end
    join_none

    $display("@%0t: after join_none", $time);
    #80 $display("@%0t: finish after # 80", $time);
end
```

注意：这个块相应的框图类似于图 7.3。注意 `join_none` 块后那个语句的执行早于 `fork...join_none` 内的任何语句（指那些带有延时的语句）。

例 7.4 `fork...join_none` 的输出

这是上述代码运行后的仿真日志输出，验证了并行与顺序执行的交错情况。

```
@0: start fork...join_none example
@10: sequential after #10
@10: after join_none
@10: parallel start
@20: parallel after #10
@40: sequential after #30
@50: sequential after #10
@60: parallel after #50
@90: finish after #80
```

`fork join_any`块对块内语句进行调度,当第一个语句完成时,父线程才继续执行,其他停顿的线程也得以继续

```
initial begin
    $display("@%0t: start fork...join_any example", $time);
    #10 $display("@%0t: sequential after #10", $time);

    fork
        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after #50", $time);
        #10 $display("@%0t: parallel after #10", $time);

        begin
            #30 $display("@%0t: sequential after #30", $time);
            #10 $display("@%0t: sequential after #10", $time);
        end
    endjoin_any
    $display("@%0t: after join_any", $time);
    #80 $display("@%0t: finish after #80", $time);
end
```

在类中创建线程

使用 `fork...join_none` 可以开启一个线程,比如随机事务发生器的替代代码

完整的测试平台还包括用于驱动,监测,检验以及其他操作的类,所有这些都带有并发运行的事务处理器

```
// 例 7.7 带有任务 run 的发生器/驱动器类
class Gen_drive;
    // 创建 N 个数据包的事务处理器
    task run(int n);
        Packet p;
```



```
        fork
            repeat (n) begin
                p = new();
                assert(p.randomize());
                transmit(p);
            end
        join_none // 使用 fork-join_none 以使 run() 不发生阻塞
    endtask

    task transmit(input Packet p);
        ...
    endtask
endclass

Gen_drive gen;

initial begin
    gen = new();
    gen.run(10);
    // 启动检验、监测和其他线程
    ...
end
```

动态线程

在Verilog中,线程是可预知的,可以通过统计源代码中initial ,always和fork...join块的数量来确定一个模块中有多少线程,而在System Verilog中,可以动态地创建线程,而不用等到它们都执行完毕

动态线程的创建

```

program automatic test(bus_ifc.TB bus);
    task check_trans(Transaction tr);
        fork
            begin
                wait(bus.cb.addr == tr.addr);
                $display("@%0t: transaction %0d received", $time, tr.id);
            end
        join_none
    endtask

    Transaction tr;
    initial begin
        repeat(10) begin
            tr = new();
            assert(tr.randomize());
            transmit(tr);
            check_trans(tr);
        end
        #100 $display("@%0t: finish", $time);
    end
endprogram

```

当任务check_trans被调用时,它便会产生一个线程用来检测总线以获取匹配的事务地址

线程中的自动变量

在循环中内嵌fork ... join_none的不良代码的执行过程

以下是从图片中提取的 SystemVerilog 代码及其详细解析。这段代码展示了一个在 ``text @0: start fork...join_none example @10: sequential after #10 @10: after join_none @10: parallel start @20: parallel after #10 @40: sequential after #30 @50: sequential after #10 @60: parallel after #50 @90: finish after #80

fork join_any块对块内语句进行调度,当第一个语句完成时,父线程才继续执行,其他停顿的线程也得以继续

```

``systemverilog
initial begin
    $display("@%0t: start fork...join_any example", $time);
    #10 $display("@%0t: sequential after #10", $time);

    fork
        $display("@%0t: parallel start", $time);
        #50 $display("@%0t: parallel after #50", $time);
        #10 $display("@%0t: parallel after #10", $time);

    begin
        #30 $display("@%0t: sequential after #30", $time);
        #10 $display("@%0t: sequential after #10", $time);
    end
end

```

```

    endjoin_any
    $display("@%0t: after join_any", $time);
    #80 $display("@%0t: finish after #80", $time);
end

```

在类中创建线程

使用 fork...join_none 可以开启一个线程,比如随机事务发生器的替代代码

完整的测试平台还包括用于驱动,监测,检验以及其他操作的类,所有这些都带有并发运行的事务处理器

```

// 例 7.7 带有任务 run 的发生器/驱动器类
class Gen_drive;
    // 创建 N 个数据包的事务处理器
    task run(int n);
        Packet p;
        fork
            repeat (n) begin
                p = new();
                assert(p.randomize());
                transmit(p);
            end
        join_none // 使用 fork-join_none 以使 run() 不发生阻塞
    endtask

    task transmit(input Packet p);
        ...
    endtask
endclass

Gen_drive gen;

initial begin
    gen = new();
    gen.run(10);
    // 启动检验、监测和其他线程
    ...
end

```

动态线程

在Verilog中,线程是可预知的,可以通过统计源代码中initial ,always和fork...join块的数量来确定一个模块中有多少线程,而在System Verilog中,可以动态地创建线程,而不用等到它们都执行完毕

动态线程的创建

```

program automatic test(bus_ifc.TB bus);
    task check_trans(Transaction tr);
        fork
            begin

```

```

        wait(bus.cb.addr == tr.addr);
        $display("@%0t: transaction %0d received", $time, tr.id);
    end
    join_none
endtask

Transaction tr;
initial begin
    repeat(10) begin
        tr = new();
        assert(tr.randomize());
        transmit(tr);
        check_trans(tr);
    end
    #100 $display("@%0t: finish", $time);
end
endprogram

```

当任务check_trans被调用时,它便会产生一个线程用来检测总线以获取匹配的事务地址

线程中的自动变量

在循环中内嵌fork ... join_none的不良代码的执行过程

以下是从图片中提取的 SystemVerilog 代码及其详细解析。这段代码展示了一个在 **for** 循环中使用 **fork...join_none** 时非常经典的 **变量作用域陷阱**。

提取的代码

```

// 例 7.9 不良代码：在循环中内嵌 fork...join_none
program no_auto;
    initial begin
        for (int j=0; j<3; j++)
            fork
                $write(j);      // 漏洞——得到的是最终的索引值
            join_none
        #0 $display("\n");
    end
endprogram

```

- **预期：**开发者通常希望打印出 ``systemverilog // 例 7.9 不良代码：在循环中内嵌 fork...join_none program no_auto; initial begin for (int j=0; j<3; j++) fork \$write(j); // 漏洞——得到的是最终的索引值 join_none #0 \$display("\n"); end endprogram

- ****预期**：**开发者通常希望打印出 ``012``，即每次循环迭代时 ``j`` 的值（0, 1, 2）。
- ****实际**：**输出通常是 ``333``。

这个现象是由 SystemVerilog 的 **线程调度机制** 和 **变量生命周期** 共同导致的：

1. **非阻塞特性（`join\_none`）**：

- ``fork...join_none`` 会立即启动一个后台线程，但不会等待该线程执行完毕，主线程（父进程）会立即继续执行下一次循环。
- 这意味着 ``for`` 循环会在极短的时间内（同一个时间片内）迅速跑完，将 ``j`` 从 0 增加到 3。

2. **变量共享与引用**：

- 虽然 ``j`` 是在 ``for`` 循环内部声明的，但在某些仿真器实现或特定的程序块上下文（如 ``program`` 块默认是静态的）中，或者由于线程调度的延迟，后台启动的子线程并没有立即读取 ``j`` 的值。
- 当子线程真正被调度执行 ``$write(j)`` 时，主循环早已结束，此时 ``j`` 的值已经变成了循环终止条件 **3**。
- 所有三个后台线程实际上都引用了同一个变量 ``j``（或者说是该变量在循环结束时的最终状态）。

3. **零延时的作用**：

- 代码最后的 ``#0 $display("\n")`` 是一个零延时语句。它的作用是挂起当前线程，让出控制权，确保之前产生的三个后台线程有机会在当前时间步内被执行。如果没有这行代码，仿真可能会直接结束，导致什么都打印不出来。

为了避免这个问题，必须为每次迭代创建一个 **自动变量（Automatic Variable）** 的副本。通常的做法是在 ``fork`` 块内部使用 ``begin...end`` 并声明一个局部变量：

```
``systemverilog
for (int j=0; j<3; j++) begin
    fork
        automatic int k = j; // 创建 j 的副本
        $write(k);           // 打印副本，值为 0, 1, 2
    join_none
end
```

通过这种方式，每个后台线程都会捕获当前循环迭代中 ```systemverilog for (int j=0; j<3; j++) begin fork automatic int k = j; // 创建 j 的副本 $write(k); // 打印副本，值为 0, 1, 2 join_none end`

通过这种方式，每个后台线程都会捕获当前循环迭代中 ``j`` 的独立副本，从而得到预期的 ``012`` 输出。

另一种写法是在 `fork...join_none` 外部声明自动变量

```
``systemverilog
program automatic bug_free;
    initial begin
        for( int j = 0 ; j < 3 ; j++) begin
            int k = j; // 创建 j 的副本
            fork
                $write(k); // 打印副本，值为 0, 1, 2
            join_none
        end
    end
```

```

        #0 $display("\n");
    end
endprogram

```

等待所有衍生线程

使用wait fork等待所有子线程结束

```

task run_threads;
    fork
        check_trans(tr1) ; //产生第一个线程
        check_trans(tr2) ; //产生第二个线程
        check_trans(tr3) ; //产生第三个线程
    join_none
    //。。。在这里等待上述线程结束
    wait fork;
endtask

```

在线程间共享变量

使用共享程序变量导致的漏洞

以下是从图片中提取的 SystemVerilog 代码，展示了由于在 ``systemverilog program automatic bug_free; initial begin for(int j = 0 ; j < 3 ; j++) begin int k = j; // 创建 j 的副本 fork \$write(k); // 打印副本，值为 0, 1, 2 join_none end #0 \$display("\n"); end endprogram

```

等待所有衍生线程

使用wait fork等待所有子线程结束
``systemverilog
task run_threads;
    fork
        check_trans(tr1) ; //产生第一个线程
        check_trans(tr2) ; //产生第二个线程
        check_trans(tr3) ; //产生第三个线程
    join_none
    //。。。在这里等待上述线程结束
    wait fork;
endtask

```

在线程间共享变量

使用共享程序变量导致的漏洞

以下是从图片中提取的 SystemVerilog 代码，展示了由于在 `program` 块中使用共享变量而导致的经典并发错误。

例 7.15 使用共享程序变量导致的漏洞

这段代码演示了一个常见的陷阱：当循环变量 `i` 被定义为程序级（Program-level）的全局变量时，`fork...join_none` 创建的并行线程会共享同一个 `i`，导致数据竞争和不可预测的结果。

```
// 例 7.15 使用共享程序变量导致的漏洞
program bug;
  class Buggy;
    int data[10];
    task transmit;
      fork
        // i 在这里并没有声明，它引用的是外部的全局变量 i
        for (i=0; i<10; i++)
          send(data[i]);
      join_none
    endtask
  endclass

  int i;                // 共享的程序级变量 i
  Buggy b;
  event receive;

  initial begin
    b = new();
    // i 在这里没有声明，使用的是上面的全局变量 i
    for (i=0; i<10; i++)
      b.data[i] = i;

    b.transmit();

    // i 在这里没有声明，使用的是上面的全局变量 i
    for (i=0; i<10; i++)
      @(receive) $display(b.data[i]);
  end
endprogram
```

在上述代码中，变量 `i` 被定义为程序级（Program-level）的全局变量，导致在 `fork...join_none` 创建的并行线程中共享同一个 `i`，从而引发数据竞争和不可预测的结果。

```
int i;                // 共享的程序级变量 i
Buggy b;
event receive;

initial begin
  b = new();
  // i 在这里没有声明，使用的是上面的全局变量 i
  for (i=0; i<10; i++)
    b.data[i] = i;
```

```

        b.transmit();

        // i 在这里没有声明，使用的是上面的全局变量 i
        for (i=0; i<10; i++)
            @(receive) $display(b.data[i]);
    end

```

endprogram

在上述代码中，变量 `i` 是在 `program` 块层级声明的。这意味着它是 **静态的** 且 **全局共享** 的。

- 当 `b.transmit()` 中的 `fork` 启动时，它创建了一个后台线程来执行 `for` 循环。
- 然而，这个后台线程使用的 `i` 与 `initial` 块中后续循环使用的 `i` 是 **同一个物理变量**。
- 当 `transmit` 任务还在后台尝试遍历数组发送数据时，`initial` 块的主线程可能已经继续执行到了下一个 `for` 循环，修改了 `i` 的值。这会导致索引错乱，甚至数组越界访问。

图片底部的文字给出了标准的修复方案：

> 解决的办法是，在包含所有变量使用的最小范围内声明所有的变量。在例 7.15 中，索引变量的声明应该放在 `for` 循环内部而不是在程序或整个作用域的层级上。更好的做法是，尽可能使用 `foreach` 语句。

为了避免这种竞态条件，应将 `i` 的声明限制在局部作用域内：

```

``systemverilog
// 修正方案 1: 在 for 循环内部声明变量 (SystemVerilog 支持)
task transmit_fixed;
    fork
        // 这里的 i 是局部的，每个迭代或线程上下文独立
        for (int i=0; i<10; i++)
            send(data[i]);
    join_none
endtask

// 修正方案 2: 使用 foreach (推荐)
task transmit_foreach;
    foreach(data[idx]) begin
        // idx 自动作为局部变量处理，且逻辑更清晰
        send(data[idx]);
    end
endtask

```

通过这种方式，每个循环或线程都有自己独立的索引变量副本，从而消除了数据竞争风险。

停止线程

等待回应,或者达到某个最大时延

```
parameter TIME_OUT = 1000;
task check_trans(Transaction tr);
  fork
    begin
      //等待回应
      fork:time_block
        begin
          wait(bus.cb.addr == tr.addr);
          $display("@%0t: transaction %0d received", $time, tr.id);
        end
        #TIME_OUT $display("@%0t: timeout", $time);
      join_any
      disable fork:time_block;
    end
  join_none
end
endtask
```

停止多个线程

```
initial begin
  check_trans(tr0); //线程0
  fork
    begin
      check_trans(tr1);
      fork
        check_trans(tr2);
      join
      #(TIME_OUT/2) disable fork; //停止线程1-4,保留线程0
    end
  join
end
```

```
// 例 7.18 使用带标号的 disable 来停止线程
initial begin
  check_trans(tr0);           // 线程 0
  fork                        // 线程 1
    begin : threads_inner
      check_trans(tr1);       // 线程 2
      check_trans(tr2);       // 线程 3
    end
  // 停止线程 2 和 3,单独保留线程 0
  #(TIME_OUT/2) disable threads_inner;
```

```

    join
end

```

当禁止被多次调用的任务，要小心停止的可能会比预期得多,按照预期,如果你在某个任务内部禁止该任务,就像在任务的返回语句,这也会停止所有由该任务启动的线程,如果该任务已经被多个线程调用,禁止其中的一个将导致它们全部被禁止

```

// 例 7.19 使用 disable 标号来停止一个任务
task wait_for_time_out(int id);
    if (id == 0)
        fork
            begin
                #2;
                $display("@%0t: disable wait_for_time_out", $time);
                disable wait_for_time_out; // 禁用当前任务实例
            end
        join_none

    fork : just_a_little
        begin
            $display("@%0t: % m: %0d entering thread", $time, id);
            #TIME_OUT;
            $display("@%0t: % m: %0d done", $time, id);
        end
    join_none
endtask

initial begin
    wait_for_time_out(0);      // 衍生线程 0
    wait_for_time_out(1);      // 衍生线程 1
    wait_for_time_out(2);      // 衍生线程 2

    #(TIME_OUT * 2) $display("@%0t: All done", $time);
end

```

因为线程0 disabled了 wait_for_time_out,所以线程1和2也会被禁止,因此只有线程0会执行完毕

线程间通信

System Verilog可以使用时间,旗语和信箱来完成IPC

事件

Verilog事件可以实现线程间的同步,就像在打电话时一个人等待另一个人的呼叫

```

// 例 7.20 Verilog 中阻塞在一个事件上
event e1, e2;

initial begin

```

```

    $display("@%0t: 1: before trigger", $time);
    -> e1;
    @e2;
    $display("@%0t: 1: after trigger", $time);
end

initial begin
    $display("@%0t: 2: before trigger", $time);
    -> e2;
    @e1;
    $display("@%0t: 2: after trigger", $time);
end

```

预期输出

根据图片底部及仿真逻辑，该代码的输出如下：

```

@0: 1: before trigger
@0: 2: before trigger
@0: 1: after trigger
@0: 2: after trigger

```

- **定义：**```systemverilog // 修正方案 1：在 for 循环内部声明变量 (SystemVerilog 支持) task transmit_fixed; fork // 这里的 i 是局部的，每个迭代或线程上下文独立 for (int i=0; i<10; i++) send(data[i]); join_none endtask

// 修正方案 2：使用 foreach (推荐) task transmit_foreach; foreach(data[idx]) begin // idx 自动作为局部变量处理，且逻辑更清晰 send(data[idx]); end endtask

通过这种方式，每个循环或线程都有自己独立的索引变量副本，从而消除了数据竞争风险。

停止线程

等待回应,或者达到某个最大时延

```

```systemverilog
parameter TIME_OUT = 1000;
task check_trans(Transaction tr);
 fork
 begin
 //等待回应
 fork:time_block
 begin
 wait(bus.cb.addr == tr.addr);
 $display("@%0t: transaction %0d received", $time, tr.id);
 end
 #TIME_OUT $display("@%0t: timeout", $time);
 end
 end
endtask

```

```

 join_any
 disable fork:time_block;
 end
 join_none
end
endtask

```

## 停止多个线程

```

initial begin
 check_trans(tr0); //线程0
 fork
 begin
 check_trans(tr1);
 fork
 check_trans(tr2);
 join
 #(TIME_OUT/2) disable fork; //停止线程1-4,保留线程0
 end
 join
end

```

```

// 例 7.18 使用带标号的 disable 来停止线程
initial begin
 check_trans(tr0); // 线程 0
 fork // 线程 1
 begin : threads_inner
 check_trans(tr1); // 线程 2
 check_trans(tr2); // 线程 3
 end
 // 停止线程 2 和 3,单独保留线程 0
 #(TIME_OUT/2) disable threads_inner;
 join
end

```

当禁止被多次调用的任务，要小心停止的可能会比预期得多,按照预期,如果你在某个任务内部禁止该任务,就像在任务的返回语句,这也会停止所有由该任务启动的线程,如果该任务已经被多个线程调用,禁止其中的一个将导致它们全部被禁止

```

// 例 7.19 使用 disable 标号来停止一个任务
task wait_for_time_out(int id);
 if (id == 0)
 fork
 begin
 #2;
 $display("@%0t: disable wait_for_time_out", $time);
 disable wait_for_time_out; // 禁用当前任务实例
 end
 join
 end
endtask

```

```

 end
 join_none

 fork : just_a_little
 begin
 $display("@%0t: % m: %0d entering thread", $time, id);
 #TIME_OUT;
 $display("@%0t: % m: %0d done", $time, id);
 end
 join_none
endtask

initial begin
 wait_for_time_out(0); // 衍生线程 0
 wait_for_time_out(1); // 衍生线程 1
 wait_for_time_out(2); // 衍生线程 2

 #(TIME_OUT * 2) $display("@%0t: All done", $time);
end

```

因为线程0 disabled了 wait\_for\_time\_out,所以线程1和2也会被禁止,因此只有线程0会执行完毕

## 线程间通信

System Verilog可以使用时间,旗语和信箱来完成IPC

## 事件

Verilog事件可以实现线程间的同步,就像在打电话时一个人等待另一个人的呼叫

```

// 例 7.20 Verilog 中阻塞在一个事件上
event e1, e2;

initial begin
 $display("@%0t: 1: before trigger", $time);
 -> e1;
 @e2;
 $display("@%0t: 1: after trigger", $time);
end

initial begin
 $display("@%0t: 2: before trigger", $time);
 -> e2;
 @e1;
 $display("@%0t: 2: after trigger", $time);
end

```

## 预期输出

根据图片底部及仿真逻辑,该代码的输出如下:

```
@0: 1: before trigger
@0: 2: before trigger
@0: 1: after trigger
@0: 2: after trigger
```

- **定义：**event e1, e2; 定义了两个同步对象 e1 和 e2。
- **零持续时间：**在 Verilog 中，事件是一个“瞬时”的概念。它没有持续时间，仅仅表示一个时间点上的信号跳变或通知。如果错过了触发时刻，等待者将永远等待下去（除非再次被触发）。

这段代码巧妙地演示了两个 initial 块之间的交互：

### 1. 时间 0 (Active Region):

- 两个 initial 块同时启动。
- **线程 1** 打印 "1: before trigger", 然后立即触发事件 e1 (-> e1)。此时 e1 处于激活状态。
- **线程 2** 打印 "2: before trigger", 然后立即触发事件 e2 (-> e2)。此时 e2 处于激活状态。

### 2. 时间 0 (NBA / Monitor Region):

- **线程 1** 执行到 @e2;。由于刚才线程 2 已经触发了 e2，且在同一时间步内，线程 1 能够捕获到这个事件，因此它**不会阻塞**，继续向下执行。
- **线程 2** 执行到 @e1;。同理，由于线程 1 已经触发了 e1，线程 2 也能捕获到，因此它**也不会阻塞**。

### 3. 结果:

- 两个线程都成功通过了事件等待点。
- 线程 1 打印 "1: after trigger"。
- 线程 2 打印 "2: after trigger"。
- **交叉握手：**这是一个典型的交叉握手模式。线程 1 发 e1 等 e2，线程 2 发 e2 等 e1。只要触发发生在等待之前（或在同一时间步的调度顺序允许范围内），程序就能顺利运行。
- **非阻塞 vs 阻塞：**虽然 @e 语法看起来像阻塞赋值，但在本例中，因为事件已经被触发，所以它表现为“非阻塞”通过。如果删除其中任何一个 -> 语句，对应的另一个线程就会永久挂起（Deadlock）。

等待事件的触发

```
// 例 7.22 等待事件
event e1, e2;

initial begin
 $display("@%0t: 1: before trigger", $time);
 ->e1;
 wait (e2.triggered());
 $display("@%0t: 1: after trigger", $time);
end

initial begin
 $display("@%0t: 2: before trigger", $time);
```

```

 ->e2;
 wait (e1.triggered());
end

```

在循环中使用事件

可以使用事件来实现两个线程的同步

如果在循环中使用wait(handshake.triggered())一定要确保在下次等待之前时间可以向前推进,否则你的代码将会进入一个零时延环,原因是wait会在单个事件触发器上反复执行

等待事件导致零时延循环

```

forever begin
//零时延循环
 wait(handshake.triggered());
 $display("Received next event");
 process_in_zero_time();
end

```

等待事件 的边沿

```

// 例 7.23 等待事件的边沿
forever begin
 @handshake;
 $display("Received next event");
 process_in_zero_time();
end

```

传递事件, SystemVerilog中的事件可以像参数一样传递给子程序

```

class Generator;
 event done;
 function new(event done)
 begin
 this.done = done;
 end
 endfunction

 task run();
 fork
 begin

 -> done; // 生成事件
 end
 join_none
 endtask
endclass

```

```

program automatic test;
 event gen_done;
 Generator gen;
 initial begin
 gen = new(gen_done); //测试程序实例化
 gen.run();
 wait(gen_done.triggered()); // 等待生成事件
 $display("Received event from generator");
 end
endprogram

```

## 等待多个事件

解决办法是创建一个新线程并从中衍生子线程,然后保证每个线程阻塞在每个发生器的一个事件上

```

event done[N_GENERATORS];
initial begin
 foreach (gen[i]) begin
 gen[i] = new();
 gen[i].run(done[i]);
 end

 //通过等待每个事件来等待所有发生器完成
 foreach(gen[i])
 fork
 automatic int k = i;
 wait(done[k].triggered());
 join_none
 wait fork; //等待所有触发器完成
end

```

## 通过对触发事件进行计数来等待多个线程

```

event done[N_GENERATORS];
int done_count;
initial begin
 foreach (gen[i]) begin
 gen[i] = new();
 gen[i].run(done[i]);
 end

 //等待所有发生器完成
 foreach(gen[i])
 fork
 automatic int k = i;
 begin
 wait(done[k].triggered());
 done_count++;
 end
 end
end

```



```

 join_none
 wait(done_count == N_GENERATORS); //等待触发
end

```

使用线程计数来等待多个线程

```

class Generator;
 static int thread_count = 0;
 task run();
 thread_count++;
 fork
 begin
 //这里省略实际工作的代码
 //当工作完成时,对线程数目减计数
 thread_count--;
 end
 join_none
 endtask
endclass

Generator gen[N_GENERATORS];
initial begin
 foreach (gen[i]) begin
 gen[i] = new();
 end
 foreach (gen[i]) begin
 gen[i].run();
 end
 wait(gen[0].thread_count == 0); //等待所有线程完成
end

```

## 旗语

使用旗语可以实现对同一资源的访问控制

旗语的操作

使用new方法可以创建一个带单个或者多个钥匙的旗语,使用get可以获取一个或者多个钥匙,而put则可以返回一个或者多个钥匙

如果试图获取一个旗语而不希望被阻塞,可以使用try\_get()函数,它返回1表示有足够多的钥匙,而返回0表示钥匙不够

```

// 例 7.30 用旗语实现对硬件资源的访问控制
program automatic test (bus_ifc.TB bus);
 semaphore sem; // 创建一个旗语

 initial begin
 sem = new(1); // 分配 1 个钥匙
 end
end

```

```

 fork
 sequencer(); // 产生两个总线事务线程
 sequencer();
 join
 end

 task sequencer;
 repeat ($urandom % 10) // 随机等待 0-9 个周期
 @bus.cb;
 sendTrans(); // 执行总线事务
 endtask

 task sendTrans;
 sem.get(1); // 获取总线钥匙
 @bus.cb; // 把信号驱动到总线上
 bus.cb.addr <= t.addr;
 ...
 sem.put(1); // 处理完成时把钥匙返回
 endtask
endprogram

```

## 信箱

把发生器和驱动器想象成具备自治能力的事务处理器对象,它们通过对信道交换数据,每个对象从它的上游对象中得到事务,进行一些处理,然后把它们传递给下游任务,这里的信道必须允许驱动器和接收器异步操作

创建多个对象的良性发生器

```

task generator_good(int n,mailbox mbx);
 Transaction t;
 repeat(n) begin
 t = new(); //创建一个新的事务
 assert(t.randomize());
 $display("Generated transaction %0d", t.id);
 mbx.put(t); //将事务放入信箱
 end
endtask

```

接收来自信箱事务的良性驱动器

```

task driver(mailbox mbx);
 Transaction t;
 forever begin
 mbx.get(t); //获取来自信箱的事务
 $display("Received transaction %0d", t.id);
 //驱动事务到待测设计中
 end
endtask

```

```
end
endtask
```

## 测试平台里的信箱

### 使用信箱实现对象的交换,Generator类

```
class Generator;
 Transaction tr;
 mailbox mbx;
 function new(mailbox mbx);
 this.mbx = mbx;
 endfunction

 task run(int count);
 repeat(count) begin
 tr = new();
 assert(tr.randomize());
 $display("Generated transaction %0d", tr.id);
 mbx.put(tr);
 end
endtask
endclass
```

### 使用信箱实现对象的交换:Driver类

```
class Driver;
 Transaction tr;
 mailbox mbx;
 function new(mailbox mbx);
 this.mbx = mbx;
 endfunction

 task run();
 forever begin
 mbx.get(tr);
 $display("Received transaction %0d", tr.id);
 //驱动事务到待测设计中
 end
endtask
endclass
```

## 测试平台程序块

```
program automatic mailbox_example(bus_if.TB bus,...);
 'include "transaction.sv"
 'include "generator.sv"
```

```

'include "driver.sv"

mailbox mbx;
Generator gen;
Driver drv;
int count;

initial begin
 count = $urandom_range(50);
 mbx = new(); // 创建一个信箱
 gen = new(mbx); // 创建发生器对象
 drv = new(mbx); // 创建驱动器对象

 fork
 gen.run(count); // 发生器线程
 drv.run(count); // 驱动器线程
 join
end
endprogram

```

## 定容信箱

当往信箱里放入多于设定容量的物品,则put会阻塞,直到你从邮箱里搬走物品腾出空间

```

`timescale 1ns/1ns

program automatic bounded;
 mailbox mbx;

 initial begin
 mbx = new(1); // 容量为 1

 fork
 // 生产方线程
 for (int i=1; i<4; i++) begin
 $display("Producer: before put(%0d)", i);
 mbx.put(i);
 $display("Producer: after put(%0d)", i);
 end

 // 消费方线程
 repeat (4) begin
 int j;
 #1ns mbx.get(j);
 $display("Consumer: after get(%0d)", j);
 end
 join
 end
endprogram

```

使用定容信箱和探视peek来实现线程的同步

以下是从图片中提取的 **SystemVerilog** 代码。这段代码演示了如何使用 **定容邮箱 (Bounded Mailbox)** 结合 ``systemverilog // 例 7.22 等待事件 event e1, e2;

```
initial begin $display("@%0t: 1: before trigger", $time); ->e1; wait (e2.triggered()); $display("@%0t: 1: after trigger", $time); end
```

```
initial begin $display("@%0t: 2: before trigger", $time); ->e2; wait (e1.triggered()); end
```

在循环中使用事件

可以使用事件来实现两个线程的同步

如果在循环中使用`wait(handshake.triggered())`一定要确保在下次等待之前时间可以向前推进,否则你的代码将会进入一个零时延环,原因是`wait`会在单个事件触发器上反复执行

等待事件导致零时延循环

```
``systemverilog
forever begin
//零时延循环
 wait(handshake.triggered());
 $display("Received next event");
 process_in_zero_time();
end
```

等待事件 的边沿

```
// 例 7.23 等待事件的边沿
forever begin
 @handshake;
 $display("Received next event");
 process_in_zero_time();
end
```

传递事件, SystemVerilog中的事件可以像参数一样传递给子程序

```
class Generator;
 event done;
 function new(event done)
 begin
 this.done = done;
 end
endfunction

 task run();
 fork
 begin
```

```


 -> done; // 生成事件
 end
 join_none
endclass

program automatic test;
 event gen_done;
 Generator gen;
 initial begin
 gen = new(gen_done); //测试程序实例化
 gen.run();
 wait(gen_done.triggered()); // 等待生成事件
 $display("Received event from generator");
 end
endprogram

```

## 等待多个事件

解决办法是创建一个新线程并从中衍生子线程,然后保证每个线程阻塞在每个发生器的一个事件上

```

event done[N_GENERATORS];
initial begin
 foreach (gen[i]) begin
 gen[i] = new();
 gen[i].run(done[i]);
 end

 //通过等待每个事件来等待所有发生器完成
 foreach(gen[i])
 fork
 automatic int k = i;
 wait(done[k].triggered());
 join_none
 wait fork; //等待所有触发器完成
end

```

## 通过对触发事件进行计数来等待多个线程

```

event done[N_GENERATORS];
int done_count;
initial begin
 foreach (gen[i]) begin
 gen[i] = new();
 gen[i].run(done[i]);
 end

 //等待所有发生器完成
 foreach(gen[i])
 fork

```

```

 automatic int k = i;
 begin
 wait(done[k].triggered());
 done_count++;
 end
 join_none
 wait(done_count == N_GENERATORS); //等待触发
end

```

使用线程计数来等待多个线程

```

class Generator;
 static int thread_count = 0;
 task run();
 thread_count++;
 fork
 begin
 //这里省略实际工作的代码
 //当工作完成时,对线程数目减计数
 thread_count--;
 end
 join_none
 endtask
endclass

Generator gen[N_GENERATORS];
initial begin
 foreach (gen[i]) begin
 gen[i] = new();
 end
 foreach (gen[i]) begin
 gen[i].run();
 end
 wait(gen[0].thread_count == 0); //等待所有线程完成
end

```

## 旗语

使用旗语可以实现对同一资源的访问控制

旗语的操作

使用new方法可以创建一个带单个或者多个钥匙的旗语,使用get可以获取一个或者多个钥匙,而put则可以返回一个或者多个钥匙

如果试图获取一个旗语而不希望被阻塞,可以使用try\_get()函数,它返回1表示有足够多的钥匙,而返回0表示钥匙不够

```
// 例 7.30 用旗语实现对硬件资源的访问控制
program automatic test (bus_ifc.TB bus);
 semaphore sem; // 创建一个旗语

 initial begin
 sem = new(1); // 分配 1 个钥匙
 fork
 sequencer(); // 产生两个总线事务线程
 sequencer();
 join
 end

 task sequencer;
 repeat ($urandom % 10) // 随机等待 0-9 个周期
 @bus.cb;
 sendTrans(); // 执行总线事务
 endtask

 task sendTrans;
 sem.get(1); // 获取总线钥匙
 @bus.cb; // 把信号驱动到总线上
 bus.cb.addr <= t.addr;
 ...
 sem.put(1); // 处理完成时把钥匙返回
 endtask
endprogram
```

## 信箱

把发生器和驱动器想象成具备自治能力的事务处理器对象,它们通过对信道交换数据,每个对象从它的上游对象中得到事务,进行一些处理,然后把它们传递给下游任务,这里的信道必须允许驱动器和接收器异步操作

创建多个对象的良性发生器

```
task generator_good(int n,mailbox mbx);
 Transaction t;
 repeat(n) begin
 t = new(); //创建一个新的事务
 assert(t.randomize());
 $display("Generated transaction %0d", t.id);
 mbx.put(t); //将事务放入信箱
 end
endtask
```

接收来自信箱事务的良性驱动器

```
task driver(mailbox mbx);
 Transaction t;
```



```
 forever begin
 mbx.get(t); //获取来自信箱的事务
 $display("Received transaction %0d", t.id);
 //驱动事务到待测设计中
 end
endtask
```

## 测试平台里的信箱

### 使用信箱实现对象的交换,Generator类

```
class Generator;
 Transaction tr;
 mailbox mbx;
 function new(mailbox mbx);
 this.mbx = mbx;
 endfunction

 task run(int count);
 repeat(count) begin
 tr = new();
 assert(tr.randomize());
 $display("Generated transaction %0d", tr.id);
 mbx.put(tr);
 end
 endtask
endclass
```

### 使用信箱实现对象的交换:Driver类

```
class Driver;
 Transaction tr;
 mailbox mbx;
 function new(mailbox mbx);
 this.mbx = mbx;
 endfunction

 task run();
 forever begin
 mbx.get(tr);
 $display("Received transaction %0d", tr.id);
 //驱动事务到待测设计中
 end
 endtask
endclass
```

## 测试平台程序块

```

program automatic mailbox_example(bus_if.TB bus,...);
 'include "transaction.sv"
 'include "generator.sv"
 'include "driver.sv"

 mailbox mbx;
 Generator gen;
 Driver drv;
 int count;

 initial begin
 count = $urandom_range(50);
 mbx = new(); // 创建一个信箱
 gen = new(mbx); // 创建发生器对象
 drv = new(mbx); // 创建驱动器对象

 fork
 gen.run(count); // 发生器线程
 drv.run(count); // 驱动器线程
 join
 end
endprogram

```

## 定容信箱

当往信箱里放入多于设定容量的物品,则put会阻塞,直到你从邮箱里搬走物品腾出空间

```

`timescale 1ns/1ns

program automatic bounded;
 mailbox mbx;

 initial begin
 mbx = new(1); // 容量为 1

 fork
 // 生产方线程
 for (int i=1; i<4; i++) begin
 $display("Producer: before put(%0d)", i);
 mbx.put(i);
 $display("Producer: after put(%0d)", i);
 end

 // 消费方线程
 repeat (4) begin
 int j;
 #1ns mbx.get(j);
 $display("Consumer: after get(%0d)", j);
 end
 join
 end
endprogram

```

```
end
endprogram
```

使用定容信箱和探视peek来实现线程的同步

以下是从图片中提取的 **SystemVerilog** 代码。这段代码演示了如何使用 **定容邮箱 (Bounded Mailbox)** 结合 **peek** 方法来实现生产者和消费者之间的同步。

提取的代码

```
// 例 7.41 使用定容信箱实现同步的生产方和消费方
program automatic synch_peek;
 mailbox mbx;

 // 使用 7.39 中的生产方
 class Producer;
 task run();
 for (int i=1; i<4; i++) begin
 $display("Producer: before put(%0d)", i);
 mbx.put(i);
 $display("Producer: after put(%0d)", i);
 end
 endtask
 endclass Producer

 class Consumer;
 task run();
 int i;
 repeat (3) begin
 mbx.peek(i); // 探视 mbx 里的整数
 $display("Consumer: after get(%0d)", i);
 mbx.get(i); // 从 mbx 里移出
 end
 endtask
 endclass Consumer

 Producer p;
 Consumer c;

 initial begin
 // 创建信箱、生产方和消费方
 mbx = new(1); // 定容信箱 — 容量限定为 1!

 p = new();
 c = new();

 // 使生产方和消费方并发运行
 fork
 p.run();
 c.run();
 join
 end
endprogram
```

```
end
endprogram
```

1. 核心机制：定容邮箱 (``systemverilog // 例 7.41 使用定容信箱实现同步的生产方和消费方 program automatic synch\_peek; mailbox mbx;

```
// 使用 7.39 中的生产方 class Producer; task run(); for (int i=1; i<4; i++) begin $display("Producer:
before put(%0d)", i); mbx.put(i); $display("Producer: after put(%0d)", i); end endtask endclass Producer
```

```
class Consumer; task run(); int i; repeat (3) begin mbx.peek(i); // 探视 mbx 里的整数
$display("Consumer: after get(%0d)", i); mbx.get(i); // 从 mbx 里移出 end endtask endclass Consumer
```

```
Producer p; Consumer c;
```

```
initial begin // 创建信箱、生产方和消费方 mbx = new(1); // 定容信箱 —— 容量限定为 1!
```

```
 p = new();
 c = new();

 // 使生产方和消费方并发运行
 fork
 p.run();
 c.run();
 join
```

```
end endprogram
```

1. 核心机制：定容邮箱 (`mbx = new(1)`)

- **容量限制**：这里将邮箱的大小设置为 **1**。这意味着同一时刻，邮箱中最多只能存放一个数据。
- **同步作用**：这种物理上的“满”状态强制实现了同步。当生产者放入一个数据后，邮箱变满，`put()` 操作会阻塞生产者，直到消费者取走数据腾出空间。这避免了无界邮箱可能导致的“生产者跑得太快，数据堆积”的问题。

2. `peek` vs `get`

- **`mbx.peek(i)`**：**偷看/窥视**。它读取邮箱头部的数据并赋值给变量 `i`，但 **不会** 将数据从邮箱中移除。如果邮箱为空，它会阻塞等待。
- **`mbx.get(i)`**：**获取/取出**。它将数据赋值给 `i` 并将其从邮箱中 **彻底删除**，从而腾出空间让生产者可以放入下一个数据。

3. 执行流程分析

由于邮箱容量为 1，执行顺序将被严格锁定为“放一个 -> 看一眼 -> 取走 -> 再放一个...”的交替模式：

1. **初始状态**：邮箱空。

2. **\*\*Producer\*\***: 打印 ``"before put(1)"``, 调用 ``put(1)``。邮箱变为满 (含 1)。
3. **\*\*Producer\*\***: 尝试再次循环或继续执行, 但被阻塞在下一个 ``put`` (如果有的话) 或者因为时间片切换。
4. **\*\*Consumer\*\***: 被唤醒 (或获得时间片), 调用 ``peek(i)``。看到数据 1, 打印 ``"after get(1)"``。注意此时邮箱里仍然有 1。
5. **\*\*Consumer\*\***: 调用 ``get(i)``。数据 1 被移除, 邮箱变空。
6. **\*\*Producer\*\***: 检测到邮箱有空位, 解除阻塞, 开始处理数据 2...

这种模式确保了数据处理是严格有序的, 并且内存占用被控制在最小范围。

以上即为利用定容邮箱实现生产者与消费者同步的代码逻辑分析。

使用事件实现同步的生产方和消费方

```

```systemverilog
// 例 7.43 使用事件实现同步的生产方和消费方
program automatic mbx_evt;
    mailbox mbx;
    event handshake;

    class Producer;
        task run;
            for (int i=1; i<4; i++) begin
                $display("Producer: before put(%0d)", i);
                mbx.put(i);
                @handshake;
                $display("Producer: after put(%0d)", i);
            end
        endtask
    endclass Producer

// 下接例 7.44
// 例 7.44 用事件实现同步的生产方和消费方, 接上例
class Consumer;
    task run;
        int i;
        repeat (3) begin
            mbx.get(i);
            $display("Consumer: after get(%0d)", i);
            -> handshake;
        end
    endtask
endclass Consumer

Producer p;
Consumer c;

initial begin
    mbx = new();
    p = new();
    c = new();
    // 使生产方和消费方并发运行

```

```

        fork
            p.run();
            c.run();
        join
    end
endprogram

```

代码解析

核心机制：邮箱 + 事件

- `***`systemverilog // 例 7.43 使用事件实现同步的生产方和消费方 program automatic mbx_evt; mailbox mbx; event handshake;`

`class Producer; task run; for (int i=1; i<4; i++) begin $display("Producer: before put(%0d)", i); mbx.put(i); @handshake; $display("Producer: after put(%0d)", i); end endtask endclass Producer`

`// 下接例 7.44 // 例 7.44 用事件实现同步的生产方和消费方, 接上例 class Consumer; task run; int i; repeat (3) begin mbx.get(i); $display("Consumer: after get(%0d)", i); -> handshake; end endtask endclass Consumer`

`Producer p; Consumer c;`

`initial begin mbx = new(); p = new(); c = new(); // 使生产方和消费方并发运行 fork p.run(); c.run(); join end endprogram`

代码解析

核心机制：邮箱 + 事件

- `***`mailbox mbx;`**`: 用于传输数据（整数 ``i``）。这里初始化为无界邮箱（``new()``），所以不会因为容量满而阻塞，但为了演示同步逻辑，通常配合有界邮箱使用效果更好。
- `***`event handshake;`**`: 这是一个“握手”信号。它不携带数据，仅用于通知对方“我已经处理完了”或“你可以继续了”。

生产者逻辑（``Producer``）

1. 打印 "before put"。
2. 将数据放入邮箱 ``mbx.put(i)``。
3. ****关键点****: ``@handshake;``。生产者在这里 ****阻塞等待****。它在等待消费者发出信号，表示“我已经取走了数据并处理完毕”。
4. 收到信号后，打印 "after put"，然后进入下一次循环。

消费者逻辑（``Consumer``）

1. 从邮箱获取数据 ``mbx.get(i)``。
2. 打印 "after get"。
3. ****关键点****: ``-> handshake;``。消费者触发事件，通知生产者：“我已经拿到了数据，你可以继续生产下一个了”。

这种模式实现了严格的 **一一对应** 同步：

1. 生产者放入数据 -> 阻塞等待。
2. 消费者取出数据 -> 触发事件。
3. 生产者收到事件 -> 解除阻塞，继续下一轮。

这种方式常用于需要严格控制生产速率，或者需要在消费者处理完数据后才能进行下一步操作的场景。

以上即为该段代码的逻辑分析与同步机制说明。

构筑带线程并可实现线程间通信的测试程序

基本的事务处理器

```
``systemverilog
class Agent;
    mailbox gen2agt,agt2drv;
    Transaction t;

    function new(mailbox gen2agt, mailbox agt2drv);
        this.gen2agt = gen2agt;
        this.agt2drv = agt2drv;
    endfunction

    task run();
        fork
            begin
                while (1) begin
                    gen2agt.get(t);
                    $display("Agent: received transaction %0d", t.id);
                    agt2drv.put(t);
                end
            end
            begin
                while (1) begin
                    agt2drv.get(t);
                    $display("Agent: sent transaction %0d", t.id);
                end
            end
        join
    endtask
endclass
```

从上游模块获取数据,把事务再发送给下游模块

配置类

```
class Config;
    bit [31:0] run_for_n_trans;
    constraint reasonable
```

```
{
    run_for_n_trans inside [1:1000];
}
endclass
```

环境类

```
class Environment;

    Generator gen;
    Agent agt;
    Driver drv;
    Monitor mon;
    Checker chk;
    Scoreboard scb;
    Config cfg;
    mailbox gen2agt, agt2drv, mon2chk;

    extern function new();
    extern function void gen_cfg();
    extern function void build();
    extern task run();
    extern task wrap_up();

endclass

function Environment::new();
    cfg = new();
endfunction

function void Environment::gen_cfg();
    assert (cfg.randomize());
endfunction

function void Environment::build();
    // 初始化信箱
    gen2agt = new();
    agt2drv = new();
    mon2chk = new();

    // 初始化事务处理器
    gen = new(gen2agt);
    agt = new(gen2agt, agt2drv);
    drv = new(agt2drv);
    mon = new(mon2chk);
    chk = new(mon2chk);
    scb = new();
endfunction

task Environment::run();
    fork
```



```

        gen.run(cfg.run_for_n_trans);
        agt.run();
        drv.run();
        mon.run();
        chk.run();
        scb.run(cfg.run_for_n_trans);
    join
endtask

task Environment::wrap_up();
    fork
        gen.wrap_up();
        agt.wrap_up();
        drv.wrap_up();
        mon.wrap_up();
        chk.wrap_up();
        scb.wrap_up();
    join
endtask

```

基本测试程序

```

program automatic test;
    Environment env;
    initial begin
        env = new();
        env.gen_cfg();
        env.build();
        env.run();
        env.wrap_up();
    end
endprogram

```

面向对象编程的高级技巧指南

继承简介

事务基类

```

class Transaction;
    rand bit[31:0] src,dst,data[8]; //随机变量
    bit[31:0] crc; //计算得到的CRC值

    virtual function void calc_crc;
        crc = src ^ dst ^ data.xor;
    endfunction

    virtual function void print();

```

```

        $display("Transaction: src=%0d, dst=%0d, data=%0d, crc=%0d", src, dst,
data, crc);
    endfunction
endclass

```

扩展的Transaction类

```

class BadTr : public Transaction;
    rand bit bad_crc;
    virtual function void calc_crc;
        super.calc_crc(); //计算正确的CRC
        if (bad_crc) crc = ~crc; //故意计算错误的CRC
    endfunction

    virtual function void display(input string prefix = "");
        $write("%sTransaction: src=%0d, dst=%0d, data=%0d, crc=%0d, bad_crc=%0d",
prefix, src, dst, data, crc, bad_crc);
        super.display();
    endfunction
endclass

```

扩展类的构造函数

```

class Base1;
    int var;
    function new(input int var);
        this.var = var;
    endfunction
endclass

class Extended extends Base1;
    function new(input int var);
        super.new(var); //必须是new函数的第一行
    endfunction
endclass

```

驱动类 从发生器接收事务信息,然后将它们输送给DUT

```

class Driver;
    mailbox gen2drv;
    function new(input mailbox gen2drv);
        this.gen2drv = gen2drv;
    endfunction

    task main;
        Transaction tr;
        forever begin

```

```

        gen2drv.get(tr);
        tr.calc_crc();
        @ifc.cb.src = tr.src;
        ...
    end
endtask
endclass

```

简单的发生器类 测试平台的发生器创建一个随机的事务,然后将其放入邮箱传递给驱动器

```

class Generator;
    mailbox gen2drv;
    Transaction tr;
    function new(input mailbox gen2drv);
        this.gen2drv = gen2drv;
    endfunction

    task run;
        forever begin
            tr = new(); //创建事务
            assert(tr.randomize()); //随机化
            gen2drv.put(tr); //放入邮箱
        end
    endtask
endclass

```

蓝图(Blueprint)模式

在这里我们先构建一个对象的蓝图,然后修改它的约束,甚至使用一个扩展对象替换它,然后当你随机化这个蓝图的时候,它就会具有你像赋予的随机值

使用蓝图模式的发生器类

```

class Generator;
    mailbox gen2drv;
    Transaction blueprint;

    function new(input mailbox gen2drv);
        this.gen2drv = gen2drv;
        blueprint = new();
    endfunction

    task run;
        Transaction tr;
        forever begin
            assert(blueprint.randomize);
            tr = blueprint.copy();
            gen2drv.put(tr); //发送到驱动器
        end
    endtask
endclass

```

```
endtask
endclass
```

environment类

执行的三个阶段:创建build,运行run和收尾wrap-up

```
class Environment;
  Generator gen;
  Driver drv;
  mailbox gen2drv;

  function void build(); //通过构建邮箱,发生器和驱动器来创建环境
    gen2drv = new();
    gen = new(gen2drv);
    drv = new(gen2drv);
  endfunction

  task run();
    fork
      gen.run();
      drv.run();
    join none;
  endtask

  task wrap_up();
    //暂时为空,调用计分板(scoreboard)生成报告
  endtask
endclass
```

一个简单的测试平台

基本的测试仅仅使envrionment类按默认方式运行

```
program automatic test;
  Environment env;
  initial begin
    env = new();
    env.build(); //创建测试平台对象
    env.run(); //运行测试
    env.wrap_up(); //清理
  end
endprogram
```

使用扩展的Transaction类

```

program automatic test;
  Environment env;
  initial begin
    env = new();
    env.build();
    begin
      BadTr bad = new();
      env.gen.blueprint = bad;
    end

    env.run(); //运行带BadTr的测试
    env.wrap_up(); //清理内存
  end
endprogram

```

使用扩展类改变随机约束

使用继承来增加一个约束

```

class Nearby extends Transaction;
  constraint c_nearby{
    dst_inside{[src-100:src+100]};
  }
endclass

program automatic test;
  Environment env;
  initial begin
    env = new();
    env.build(); //创建发生器等

    begin
      Nearby nb = new(); //创建一个新蓝图
      env.gen.blueprint = nb; //替换蓝图
    end

    env.run(); //运行带Nearby的测试
    env.wrap_up(); //清理内存
  end
endprogram

```

类型向下转换和虚方法

使用\$cast作类型向下转换

```

// 例 8.11 基类和派生类
class Transaction;
  rand bit [31:0] src;

```

```

        virtual function void display(input string prefix="");
            $display("%sTransaction:src=%0d", prefix, src);
        endfunction
    endclass

    class BadTr extends Transaction;
        bit bad_crc;

        virtual function void display(input string prefix="");
            $display("%sBadTr:bad_crc=%b", prefix, bad_crc);
            super.display(prefix);
        endfunction
    endclass

    Transaction tr;
    BadTr bad, bad2;

    Transaction tr;
    BadTr bad;
    bad = new();
    tr = bad; // 类型向下转换
    $cast(bad2, tr); // 类型向下转换
    tr.display(); // 调用派生类的display方法

```

```

bad = new();           // 构建 BadTr 扩展对象
tr = bad;              // 基类句柄指向扩展对象

// 检查对象类型并且拷贝。如果类型失配则在仿真时报错
// 如果成功，bad2 就指向 tr 所引用的对象
$cast(bad2, tr);

// 检查类型失配，如果类型失配，在仿真时也不会输出错误信息
if (!$cast(bad2, tr))
    $display("cannot assign tr to bad2");

$display(bad2.bad_crc); // 原始对象中存在 bad_crc 成员

```

当你将 ``systemverilog class Agent; mailbox gen2agt,agt2drv; Transaction t;

```

function new(mailbox gen2agt, mailbox agt2drv);
    this.gen2agt = gen2agt;
    this.agt2drv = agt2drv;
endfunction

task run();
    fork
        begin
            while (1) begin

```

```

        gen2agt.get(t);
        $display("Agent: received transaction %0d", t.id);
        agt2drv.put(t);
    end
end
begin
    while (1) begin
        agt2drv.get(t);
        $display("Agent: sent transaction %0d", t.id);
    end
end
join
endtask
endclass

```

从上游模块获取数据,把事务再发送给下游模块

配置类

```

``systemverilog
class Config;
    bit [31:0] run_for_n_trans;
    constraint reasonable
    {
        run_for_n_trans inside [1:1000];
    }
endclass

```

环境类

```

class Environment;

    Generator gen;
    Agent agt;
    Driver drv;
    Monitor mon;
    Checker chk;
    Scoreboard scb;
    Config cfg;
    mailbox gen2agt, agt2drv, mon2chk;

    extern function new();
    extern function void gen_cfg();
    extern function void build();
    extern task run();
    extern task wrap_up();

endclass

```

```
function Environment::new();
    cfg = new();
endfunction

function void Environment::gen_cfg();
    assert (cfg.randomize());
endfunction

function void Environment::build();
    // 初始化信箱
    gen2agt = new();
    agt2drv = new();
    mon2chk = new();

    // 初始化事务处理器
    gen = new(gen2agt);
    agt = new(gen2agt, agt2drv);
    drv = new(agt2drv);
    mon = new(mon2chk);
    chk = new(mon2chk);
    scb = new();
endfunction

task Environment::run();
    fork
        gen.run(cfg.run_for_n_trans);
        agt.run();
        drv.run();
        mon.run();
        chk.run();
        scb.run(cfg.run_for_n_trans);
    join
endtask

task Environment::wrap_up();
    fork
        gen.wrap_up();
        agt.wrap_up();
        drv.wrap_up();
        mon.wrap_up();
        chk.wrap_up();
        scb.wrap_up();
    join
endtask
```

基本测试程序

```
program automatic test;
    Environment env;
    initial begin
```



```

        env = new();
        env.gen_cfg();
        env.build();
        env.run();
        env.wrap_up();
    end
endprogram

```

面向对象编程的高级技巧指南

继承简介

事务基类

```

class Transaction;
    rand bit[31:0] src,dst,data[8]; //随机变量
    bit[31:0] crc; //计算得到的CRC值

    virtual function void calc_crc;
        crc = src ^ dst ^ data.xor;
    endfunction

    virtual function void print();
        $display("Transaction: src=%0d, dst=%0d, data=%0d, crc=%0d", src, dst,
data, crc);
    endfunction
endclass

```

扩展的Transaction类

```

class BadTr : public Transaction;
    rand bit bad_crc;
    virtual function void calc_crc;
        super.calc_crc(); //计算正确的CRC
        if (bad_crc) crc = ~crc; //故意计算错误的CRC
    endfunction

    virtual function void display(input string prefix = "");
        $write("%sTransaction: src=%0d, dst=%0d, data=%0d, crc=%0d, bad_crc=%0d",
prefix, src, dst, data, crc, bad_crc);
        super.display();
    endfunction
endclass

```

扩展类的构造函数

```

class Base1;
  int var;
  function new(input int var);
    this.var = var;
  endfunction
endclass

class Extended extends Base1;
  function new(input int var);
    super.new(var); //必须是new函数的第一行
  endfunction
endclass

```

驱动类 从发生器接收事务信息,然后将它们输送给DUT

```

class Driver;
  mailbox gen2drv;
  function new(input mailbox gen2drv);
    this.gen2drv = gen2drv;
  endfunction

  task main;
    Transaction tr;
    forever begin
      gen2drv.get(tr);
      tr.calc_crc();
      @ifc.cb.src = tr.src;
      ...
    end
  endtask
endclass

```

简单的发生器类 测试平台的发生器创建一个随机的事务,然后将其放入邮箱传递给驱动器

```

class Generator;
  mailbox gen2drv;
  Transaction tr;
  function new(iput mailbox gen2drv);
    this.gen2drv = gen2drv;
  endfunction

  task run;
    forever begin
      tr = new(); //创建事务
      assert(tr.randomize()); //随机化
      gen2drv.put(tr); //放入邮箱
    end
  end
endclass

```

```
endtask
endclass
```

蓝图(Blueprint)模式

在这里我们先构建一个对象的蓝图,然后修改它的约束,甚至使用一个扩展对象替换它,然后当你随机化这个蓝图的时候,它就会具有你像赋予的随机值

使用蓝图模式的发生器类

```
class Generator;
  mailbox gen2drv;
  Transaction blueprint;

  function new(input mailbox gen2drv);
    this.gen2drv = gen2drv;
    blueprint = new();
  endfunction

  task run;
    Transaction tr;
    forever begin
      assert(blueprint.randomize());
      tr = blueprint.copy();
      gen2drv.put(tr); //发送到驱动器
    end
  endtask
endclass
```

environment类

执行的三个阶段:创建build,运行run和收尾wrap-up

```
class Environment;
  Generator gen;
  Driver drv;
  mailbox gen2drv;

  function void build(); //通过构建邮箱,发生器和驱动器来创建环境
    gen2drv = new();
    gen = new(gen2drv);
    drv = new(gen2drv);
  endfunction

  task run();
    fork
      gen.run();
      drv.run();
    join none;
  endtask
endclass
```

```

    endtask

    task wrap_up();
        //暂时为空,调用计分板(scoreboard)生成报告
    endtask
endclass

```

一个简单的测试平台

基本的测试仅仅使envrionment类按默认方式运行

```

program automatic test;
    Environment env;
    initial begin
        env = new();
        env.build(); //创建测试平台对象
        env.run(); //运行测试
        env.wrap_up(); //清理
    end
endprogram

```

使用扩展的Transaction类

```

program automatic test;
    Environment env;
    initial begin
        env = new();
        env.build();
        begin
            BadTr bad = new();
            env.gen.blueprint = bad;
        end

        env.run(); //运行带BadTr的测试
        env.wrap_up(); //清理内存
    end
endprogram

```

使用扩展类改变随机约束

使用继承来增加一个约束

```

class Nearby extends Transaction;
    constraint c_nearby{
        dst_inside{[src-100:src+100]};
    }
endclass

```

```

program automatic test;
  Environment env;
  initial begin
    env = new();
    env.build(); //创建发生器等

    begin
      Nearby nb = new(); //创建一个新蓝图
      env.gen.blueprint = nb; //替换蓝图
    end

    env.run(); //运行带Nearby的测试
    env.wrap_up(); //清理内存
  end
endprogram

```

类型向下转换和虚方法

使用\$cast作类型向下转换

```

// 例 8.11 基类和派生类
class Transaction;
  rand bit [31:0] src;

  virtual function void display(input string prefix="");
    $display("%sTransaction:src=%0d", prefix, src);
  endfunction
endclass

class BadTr extends Transaction;
  bit bad_crc;

  virtual function void display(input string prefix="");
    $display("%sBadTr:bad_crc=%b", prefix, bad_crc);
    super.display(prefix);
  endfunction
endclass

Transaction tr;
BadTr bad, bad2;

Transaction tr;
BadTr bad;
bad = new();
tr = bad; // 类型向下转换
$cast(bad2, tr); // 类型向下转换
tr.display(); // 调用派生类的display方法

```

```

bad = new();           // 构建 BadTr 扩展对象
tr = bad;             // 基类句柄指向扩展对象

// 检查对象类型并且拷贝。如果类型失配则在仿真时报错
// 如果成功，bad2 就指向 tr 所引用的对象
$cast(bad2, tr);

// 检查类型失配，如果类型失配，在仿真时也不会输出错误信息
if (!$cast(bad2, tr))
    $display("cannot assign tr to bad2");

$display(bad2.bad_crc); // 原始对象中存在 bad_crc 成员

```

当你将 `$cast` 作为一个任务来使用的时候，SystemVerilog 会在运行时检查源对象类型，如果跟目的对象类型不匹配则给出一个错误报告。当将 `$cast` 作为函数使用时，SystemVerilog 仍然做类型检查，但是在失败时不再输出错误信息。如果类型不兼容，`$cast` 函数返回 0，如果类型兼容则返回非零值。

虚方法

```

class Transaction;
    rand bit [31:0] src, dst, data[8]; // 变量
    bit [31:0] crc;

    virtual function void calc_crc(); // 异或所有的域
        crc = src ^ dst ^ data.xor;
    endfunction
endclass

class BadTr extends Transaction;
    rand bit bad_crc;

    virtual function void calc_crc();
        super.calc_crc(); // 计算正确的 CRC
        if (bad_crc) crc = ~crc; // 产生错误的 CRC 位
    endfunction
endclass

Transaction tr;
BadTr bad;

initial begin
    tr = new();
    tr.calc_crc(); // 调用 Transaction::calc_crc

    bad = new();
    bad.calc_crc(); // 调用 BadTr::calc_crc

    tr = bad;

```

```
tr.calc_crc();          // 调用 BadTr::calc_crc
end
```

签名

使用虚方法:一旦你定义了一个虚拟的子程序,所有带有该虚拟子程序的扩展类就必须使用相同的签名,例如相同类型和个数的参数,在扩展类的虚拟子程序中不能增加或者删除参数

问 题	继 承	合 成
1. 你是否需要将多个子类组合到一起 (SystemVerilog 不支持多继承)	否	是
2. 较高级别的类是否代表了具有相近抽象级别的对象	是	否
3. 较低级别的信息是否总会出现或者一定需要出现	是	否
4. 现有代码在处理原始类的时候,是否可以处理附加的额外数据	是	否

可以创建单一不分层的类,包含所有的变量和子程序,这种方法会使得类变得很大,可以判别变量来决定哪个变量有效

创建一个不分层的以太帧类

```
class eth_mac_frame;
    typedef enum {II, IEEE} kind_e;
    rand kind_e kind;
    rand bit [47:0] da, sa;
    rand bit [15:0] len, vlan;
    // ... (省略部分代码)

    constraint eth_mac_frame_II {
        if (kind == II) {
            data.size() inside {[46:1500]};
            len == data.size();
        }
    }

    constraint eth_mac_frame_ieee {
        if (kind == IEEE) {
            data.size() inside {[46:1500]};
            len < 1522;
        }
    }
endclass
```

对象的复制

```
class Transaction;
    rand bit [31:0] src, dst, data[8]; // 变量
    bit [31:0] crc;
```

```

    virtual function Transaction copy();
        copy = new();
        copy.src = src;                // 复制数据域
        copy.dst = dst;
        copy.data = data;
        copy.crc = crc;
    endfunction
endclass

class BadTr extends Transaction;
    rand bit bad_crc;

    virtual function Transaction copy();
        BadTr bad;
        bad = new();
        bad.src = src;                // 复制数据域
        bad.dst = dst;
        bad.data = data;
        bad.crc = crc;
        bad.bad_crc = bad_crc;
        return bad;
    endfunction
endclass

```

copy_data方法

```

class Transaction;
    rand bit [31:0] src, dst, data[8]; // 变量
    bit [31:0] crc;

    virtual function void copy_data(input Transaction tr);
        // 注意：原图中此处写为 copy.src=src, 但根据上下文逻辑应为 tr.src=src
        // 这里保留原图文字, 但在实际工程中应修正为参数名 tr
        copy.src = src;                // 复制数据域
        copy.dst = dst;
        copy.data = data;
        copy.crc = crc;
    endfunction

    virtual function Transaction copy();
        copy = new();
        copy_data(copy);
    endfunction
endclass

```

```

class BadTr extends Transaction;
    rand bit bad_crc;

```



```

    virtual function void copy_data(input Transaction tr);
        BadTr bad;
        super.copy_data(tr);           // 复制基类数据
        $cast(bad, tr);                 // 基类句柄类型转换成扩展类
        bad.bad_crc = bad_crc;          // 复制派生类数据
    endfunction

    virtual function Transaction copy();
        BadTr bad;
        bad = new();                    // 创建 BadTr 对象

        copy_data(bad);                 // 复制数据域
        return bad;
    endfunction
endclass

```

指定复制的目标

使用copy函数的事务基类

```

class Transaction;
    virtual function Transaction copy(Transaction to=null);
        if(to == null) copy = new();
        else copy = to;
        copy_data(copy);
    endfunction
endclass

```

含有新copy函数的扩展事务类

```

class BadTr;
    virtual function Transaction copy(Transaction to=null);
        BadTr bad;
        if(to == null) bad = new();
        else assert($cast(bad, to));
        copy_data(bad);
        return bad;
    endfunction
endclass

```

抽象类和纯虚方法

```

virtual class BaseTr;
    static int count;
    int id;

    function new();

```

```

        id = count++;
    endfunction

    pure virtual function bit compare(input BaseTr to);
    pure virtual function BaseTr copy(input BaseTr to = null);
    pure virtual function void display(input string prefix="");
endclass:BaseTr

```

```

function bit Transaction::compare(input BaseTr to);
    Transaction tr;
    assert ($cast(tr, to));           // 检查 to 是否为正确类型
    return ((this.src==tr.src) &&
            (this.dst==tr.dst) &&
            (this.crc==tr.crc) &&
            (this.data==tr.data));
endfunction : compare

function BaseTr Transaction::copy(input BaseTr to=null);
    Transaction cp;
    if (to == null) cp = new();
    else
        $cast(cp, to);
    copy_data(cp);
    return cp;
endfunction

function void Transaction::copy_data(Transaction copy);
    copy.src = src;                   // 复制数据域
    copy.dst = dst;
    copy.data = data;
    copy.crc = crc;
endfunction

function void Transaction::display(string prefix="");
    $display("%sTransaction@0d src=%h,dst=%h,crc=%x",
            prefix, id, src, dst, crc);
endfunction : display

function Transaction::new();
    super.new();
endfunction : new

```

回调

一个回调任务应该在顶层测试中创建,在环境中的最低级即驱动器中调用,驱动器无须知道关于测试的任何信息——它只需要使用一个可以在测试中扩展的通用类,驱动器使用一个队列来保存回调对象,这样就可以增加多个对象,回调基类是一个抽象类,使用前必须进行扩展

```

virtual class Driver cbs; //驱动器回调
    virtual task pre_tx(ref Transaction tr,ref bit drop);
    //默认情况下回调不做任何动作
    endtask

    virtual task_post_tx(ref Transaction tr);
    //默认情况下回调不做任何动作
    endtask
endclass

```

```

class Driver;
    Driver_cbs cbs[$];

    task run();
        bit drop;
        Transaction tr;

        forever begin
            drop = 0;
            agt2drv.get(tr);
            foreach (cbs[i]) cbs[i].pre_tx(tr, drop);
            if (!drop) continue;

            transmit(tr);

            foreach (cbs[i]) cbs[i].post_tx(tr);
        end
    endtask
endclass

```

使用回调来注入干扰

这段代码展示了如何在 SystemVerilog 验证环境中使用回调机制（Callback）来实现错误注入。通过扩展基类

```

`systemverilog class Transaction; rand bit [31:0] src, dst, data[8]; // 变量 bit [31:0] crc;

```

```

virtual function void calc_crc(); // 异或所有的域
    crc = src ^ dst ^ data.xor;
endfunction

```

```

endclass

```

```

class BadTr extends Transaction; rand bit bad_crc;

```

```

virtual function void calc_crc();
    super.calc_crc(); // 计算正确的 CRC

```

```

    if (bad_crc) crc = ~crc; // 产生错误的 CRC 位
endfunction

```

```

endclass

```

```

Transaction tr; BadTr bad;

```

```

initial begin tr = new(); tr.calc_crc(); // 调用 Transaction::calc_crc

```

```

bad = new();
bad.calc_crc();           // 调用 BadTr::calc_crc

tr = bad;
tr.calc_crc();           // 调用 BadTr::calc_crc

```

```

end

```

签名

使用虚方法:一旦你定义了一个虚拟的子程序,所有带有该虚拟子程序的扩展类就必须使用相同的签名,例如相同类型和个数的参数,在扩展类的虚拟子程序中不能增加或者删除参数

问 题	继 承	合 成	
:---	:---:	:---:	
1. 你是否需要将多个子类组合到一起 (SystemVerilog 不支持多继承)	否	是	
2. 较高级别的类是否代表了具有相近抽象级别的对象	是	否	
3. 较低级别的信息是否总会出现或者一定需要出现	是	否	
4. 现有代码在处理原始类的时候,是否可以处理附加的额外数据	是	否	

可以创建单一不分层的类,包含所有的变量和子程序,这种方法会使得类变得很大,可以判别变量来决定哪个变量有效

创建一个不分层的以太网帧类

```

```systemverilog
class eth_mac_frame;
 typedef enum {II, IEEE} kind_e;
 rand kind_e kind;
 rand bit [47:0] da, sa;
 rand bit [15:0] len, vlan;
 // ... (省略部分代码)

 constraint eth_mac_frame_II {
 if (kind == II) {
 data.size() inside {[46:1500]};
 len == data.size();
 }
 }
}

```

```

 constraint eth_mac_frame_ieee {
 if (kind == IEEE) {
 data.size() inside {[46:1500]};
 len < 1522;
 }
 }
endclass

```

## 对象的复制

```

class Transaction;
 rand bit [31:0] src, dst, data[8]; // 变量
 bit [31:0] crc;

 virtual function Transaction copy();
 copy = new();
 copy.src = src; // 复制数据域
 copy.dst = dst;
 copy.data = data;
 copy.crc = crc;
 endfunction
endclass

class BadTr extends Transaction;
 rand bit bad_crc;

 virtual function Transaction copy();
 BadTr bad;
 bad = new();
 bad.src = src; // 复制数据域
 bad.dst = dst;
 bad.data = data;
 bad.crc = crc;
 bad.bad_crc = bad_crc;
 return bad;
 endfunction
endclass

```

## copy\_data方法

```

class Transaction;
 rand bit [31:0] src, dst, data[8]; // 变量
 bit [31:0] crc;

 virtual function void copy_data(input Transaction tr);
 // 注意：原图中此处写为 copy.src=src, 但根据上下文逻辑应为 tr.src=src
 // 这里保留原图文字, 但在实际工程中应修正为参数名 tr
 copy.src = src; // 复制数据域
 copy.dst = dst;
 endfunction
endclass

```

```

 copy.data = data;
 copy.crc = crc;
 endfunction

 virtual function Transaction copy();
 copy = new();
 copy_data(copy);
 endfunction
endclass

```

```

class BadTr extends Transaction;
 rand bit bad_crc;

 virtual function void copy_data(input Transaction tr);
 BadTr bad;
 super.copy_data(tr); // 复制基类数据
 $cast(bad, tr); // 基类句柄类型转换成扩展类
 bad.bad_crc = bad_crc; // 复制派生类数据
 endfunction

 virtual function Transaction copy();
 BadTr bad;
 bad = new(); // 创建 BadTr 对象

 copy_data(bad); // 复制数据域
 return bad;
 endfunction
endclass

```

指定复制的目标

使用copy函数的事务基类

```

class Transaction;
 virtual function Transaction copy(Transaction to=null);
 if(to == null) copy = new();
 else copy = to;
 copy_data(copy);
 endfunction
endclass

```

含有新copy函数的扩展事务类

```

class BadTr;
 virtual function Transaction copy(Transaction to=null);
 BadTr bad;
 if(to == null) bad = new();

```

```

 else assert($cast(bad, to));
 copy_data(bad);
 return bad;
 endfunction
endclass

```

## 抽象类和纯虚方法

```

virtual class BaseTr;
 static int count;
 int id;

 function new();
 id = count++;
 endfunction

 pure virtual function bit compare(input BaseTr to);
 pure virtual function BaseTr copy(input BaseTr to = null);
 pure virtual function void display(input string prefix="");
endclass:BaseTr

```

```

function bit Transaction::compare(input BaseTr to);
 Transaction tr;
 assert ($cast(tr, to)); // 检查 to 是否为正确类型
 return ((this.src==tr.src) &&
 (this.dst==tr.dst) &&
 (this.crc==tr.crc) &&
 (this.data==tr.data));
endfunction : compare

function BaseTr Transaction::copy(input BaseTr to=null);
 Transaction cp;
 if (to == null) cp = new();
 else
 $cast(cp, to);
 copy_data(cp);
 return cp;
endfunction

function void Transaction::copy_data(Transaction copy);
 copy.src = src; // 复制数据域
 copy.dst = dst;
 copy.data = data;
 copy.crc = crc;
endfunction

function void Transaction::display(string prefix="");
 $display("%sTransaction@0d src=%h,dst=%h,crc=%x",
 prefix, id, src, dst, crc);

```

```

endfunction : display

function Transaction::new();
 super.new();
endfunction : new

```

## 回调

一个回调任务应该在顶层测试中创建,在环境中的最低级即驱动器中调用,驱动器无须知道关于测试的任何信息——它只需要使用一个可以在测试中扩展的通用类,驱动器使用一个队列来保存回调对象,这样就可以增加多个对象,回调基类是一个抽象类,使用前必须进行扩展

```

virtual class Driver cbs; //驱动器回调
 virtual task pre_tx(ref Transaction tr,ref bit drop);
 //默认情况下回调不做任何动作
 endtask

 virtual task_post_tx(ref Transaction tr);
 //默认情况下回调不做任何动作
 endtask
endclass

```

```

class Driver;
 Driver_cbs cbs[$];

 task run();
 bit drop;
 Transaction tr;

 forever begin
 drop = 0;
 agt2drv.get(tr);
 foreach (cbs[i]) cbs[i].pre_tx(tr, drop);
 if (!drop) continue;

 transmit(tr);

 foreach (cbs[i]) cbs[i].post_tx(tr);
 end
 endtask
endclass

```

## 使用回调来注入干扰

这段代码展示了如何在 SystemVerilog 验证环境中使用回调机制 (Callback) 来实现错误注入。通过扩展基类 `Driver_cbs`,可以在不修改原有驱动代码的情况下,动态地改变其行为 (例如随机丢弃事务)。



```

class Driver_cbs_drop extends Driver_cbs;
 virtual task pre_tx(ref Transaction tr, ref bit drop);
 // 每 100 个事务中随机丢弃 1 个
 drop = ($urandom_range(0, 99) == 0);
 endtask
endclass

program automatic test;
 Environment env;
 initial begin
 env = new();
 env.gen_cfg();
 env.build();

 begin // 创建错误注入的回调任务
 Driver_cbs_drop dcd = new();
 env.drv.cbs.push_back(dcd); // 放入驱动器队列
 end

 env.run();
 env.wrapup();
 end
endprogram

```

- `***`systemverilog class Driver_cbs_drop extends Driver_cbs; virtual task pre_tx(ref Transaction tr, ref bit drop); // 每 100 个事务中随机丢弃 1 个 drop = ($urandom_range(0, 99) == 0); endtask endclass`

```

program automatic test; Environment env; initial begin env = new(); env.gen_cfg(); env.build();

```

```

 begin // 创建错误注入的回调任务
 Driver_cbs_drop dcd = new();
 env.drv.cbs.push_back(dcd); // 放入驱动器队列
 end

 env.run();
 env.wrapup();
end

```

```

endprogram

```

- **\*\*\*`Driver\_cbs\_drop` 类\*\***: 这是具体的回调实现类。
- 它重写了 ``pre_tx`` 虚任务。
- **\*\*逻辑分析\*\***: ``$urandom_range(0, 99)`` 生成 0 到 99 之间的随机数。当结果为 0 时（概率为 1%），表达式返回 1（true），将 ``drop`` 置位。结合之前的 ``Driver`` 代码逻辑（``if (!drop) continue``），这意味着当 ``drop`` 为 1 时，事务会被正常发送；当 ``drop`` 为 0 时，事务被跳过。
- **\*注意\***: 这里的变量命名 ``drop`` 和注释“丢弃”可能存在逻辑上的反直觉设计，或者依赖于特定

的 `Driver` 实现逻辑（即 `drop=1` 代表“允许通过/不丢弃”，或者原书意图是 `drop` 为真时执行某种操作，但在前文 `Driver` 代码中 `!drop` 才导致跳过）。按照代码字面意思，这里实现了约 1% 的概率触发特定行为。

- `**program automatic test**`: 这是测试程序的入口。
- `**环境搭建**`: 实例化环境 `env`，生成配置 `gen\_cfg` 并构建 `build`。
- `**注册回调**`: 实例化自定义的回调对象 `dcd`，并通过 `push\_back` 将其添加到驱动程序 `env.drv` 的回调队列 `cbs` 中。这样，当驱动程序运行时，就会自动调用这个自定义的 `pre\_tx` 逻辑。
- `**运行与清理**`: 启动仿真 `run()` 并进行后续处理 `wrapup()`。

以上即为该段代码的详细解析。

## 计分板

计分板设计取决于你的待测设计(DUT) 对于一个处理原子事务的DUT，例如处理包信息的DUT，其计分板就需要包含一个将输入事务转换成期望值的传输函数,用来保存这些值的内存空间以及一个进行比较的子程序

一个处理器的设计需要一个参考模型来预测期望输出,而对期望值和实际值的比较可能会在仿真的末尾进行

用于原子事务的简单计分板

```

`systemverilog
class Scoreboard;
 Transaction scb[$] //保存期望的事务的队列

 function void save_expect(input Transaction tr);
 scb.push_back(tr);
 endfunction

 function void compare_actual(input Transaction tr);
 int q[$]

 q = scb.find_index(x) with (x.src == tr.src);
 case(q.size())
 0: $display("Transaction not found");
 1: $display("Transaction found");
 default : $display("Multiple transactions found");
 endcase
 endfunction:compare_actual
endclass

```

这段代码展示了 SystemVerilog 验证环境中一种高级的解耦设计模式：**利用回调（Callback）机制将记分板（Scoreboard）与驱动器（Driver）连接起来。**

这种设计允许记分板在事务被发送之前“截获”数据，从而建立预期结果（Expected Result），而无需修改 Driver 的源代码。

以下是提取的代码及详细解析：

```

class Driver_cbs_scoreboard extends Driver_cbs;
 Scoreboard scb;

 virtual task pre_tx(ref Transaction tr, ref bit drop);
 // 将事务放入记分板
 Scb.save_expected(tr);
 endtask

 function new(input Scoreboard scb);
 this.scb = scb;
 endfunction
endclass

program automatic test;
 Environment env;

 initial begin
 env = new();
 env.gen_cfg();
 env.build();

 begin // 创建 scb 回调
 Driver_cbs_scoreboard dcs = new(env.scb);
 env.drv.cbs.push_back(dcs); // 放入驱动器队列
 end

 env.run();
 env.wrapup();
 end
endprogram

```

- **继承关系：**该类继承自 ``systemverilog class Scoreboard; Transaction scb[\$] //保存期望的事务的队列

```
function void save_expect(input Transaction tr); scb.push_back(tr); endfunction
```

```
function void compare_actual(input Transaction tr); int q[$]
```

```

 q = scb.find_index(x) with (x.src == tr.src);
 case(q.size())
 0: $display("Transaction not found");
 1: $display("Transaction found");
 default : $display("Multiple transactions found");
 endcase
endfunction:compare_actual

```

```
endclass
```

这段代码展示了 SystemVerilog 验证环境中一种高级的解耦设计模式：**\*\*利用回调（Callback）机制将记分板（Scoreboard）与驱动器（Driver）连接起来\*\***。

这种设计允许记分板在事务被发送之前“截获”数据，从而建立预期结果（Expected Result），而无需修改 Driver 的源代码。

以下是提取的代码及详细解析：

```

```systemverilog
class Driver_cbs_scoreboard extends Driver_cbs;
    Scoreboard scb;

    virtual task pre_tx(ref Transaction tr, ref bit drop);
        // 将事务放入记分板
        Scb.save_expected(tr);
    endtask

    function new(input Scoreboard scb);
        this.scb = scb;
    endfunction
endclass

program automatic test;
    Environment env;

    initial begin
        env = new();
        env.gen_cfg();
        env.build();

        begin // 创建 scb 回调
            Driver_cbs_scoreboard dcs = new(env.scb);
            env.drv.cbs.push_back(dcs); // 放入驱动器队列
        end

        env.run();
        env.wrapup();
    end
endprogram

```

- **继承关系**：该类继承自 `Driver_cbs`（驱动器的回调基类）。这意味着它拥有访问驱动器内部流程的能力。
- **成员变量**：`Scoreboard scb`；持有一个记分板的句柄。这是为了在回调函数执行时，能够调用记分板的方法。
- **构造函数**：`function new(input Scoreboard scb)` 通过参数传递的方式，将环境中的记分板实例注入到回调对象中。这是一种依赖注入的思想。

- **pre_tx 方法**：这是核心逻辑所在。
 - 当驱动器准备发送事务时，会调用此方法。
 - `Scb.save_expected(tr)`：此时，事务 `tr` 还是原始数据（Golden Data）。回调将其传递给记分板保存下来，作为后续比对的“标准答案”。
- **环境搭建**：标准的 UVM/OOP 验证流程，依次执行 `new()`、`gen_cfg()`、`build()`。
- **注册回调**：
 - `Driver_cbs_scoreboard dcs = new(env.scb)`：实例化回调对象，并传入当前环境中的记分板 `env.scb`。
 - `env.drv.cbs.push_back(dcs)`：将这个回调对象添加到驱动器的回调队列中。
- **运行**：启动仿真 `env.run()`。

这种设计的最大优势在于**解耦**。

- **Driver** 不需要知道 Scoreboard 的存在，也不需要包含任何 Scoreboard 的头文件或句柄。它只负责遍历回调队列并执行 `pre_tx`。
- **Scoreboard** 不需要主动去“拉取”数据，而是被动地通过回调接收数据。
- **灵活性**：如果未来需要更换记分板逻辑，或者增加多个记分板，只需修改回调类的实现或注册不同的回调对象即可，完全不影响 Driver 的稳定性。

通过上述分析可以看出，该设计模式有效提升了验证环境的可维护性和扩展性。

使用回调来调试事务处理器

通过增加一个显示事务内容的回调来着手这件事情

如果该事务处理器存在多个实例,可以使用`$display("%m")`来显示类的层次化路径 然后将调试diamagnetic放置到其他回调的前面和后面来定位引起问题的回调

参数化的类

一个简单的堆栈

这段内容展示了 SystemVerilog 中从普通类向参数化类（Parameterized Class）演进的过程。参数化类类似于 C++ 中的模板，允许在实例化时指定数据类型，从而极大地提高了代码的复用性。

以下是提取的代码及解析：

例 8.34 使用整数的堆栈

这是一个基础版本的堆栈类，硬编码了 `int` 类型。

```
class IntStack;
    local int stack[100];      // 保存数据值
    local int top;

    function void push(input int i);  // 从顶端压栈
        stack[++top] = i;
```

```

endfunction : push

function int pop();           // 从顶端出栈
    return stack[top--];
endfunction
endclass IntStack

```

这是改进后的版本，使用了 ``systemverilog class IntStack; local int stack[100]; // 保存数据值 local int top;

```

function void push(input int i); // 从顶端压栈
    stack[++top] = i;
endfunction : push

function int pop();           // 从顶端出栈
    return stack[top--];
endfunction

```

endclass IntStack

这是改进后的版本，使用了 `#(type T=int)` 语法。这使得同一个类可以用于 `int`、`real`、`string` 甚至自定义类对象。

```

``systemverilog
class Stack #(type T=int);
    local T stack[100];           // 保存数据值
    local int top;

    function void push(input T i); // 从顶部压栈
        stack[++top] = i;
    endfunction : push

    function T pop();             // 从顶部出栈
        return stack[top--];
    endfunction
endclass Stack

```

- **类型局限性：**在例 8.34 中，``systemverilog class Stack #(type T=int); local T stack[100]; // 保存数据值 local int top;

```
function void push(input T i); // 从顶部压栈 stack[++top] = i; endfunction : push
```

```
function T pop(); // 从顶部出栈 return stack[top--]; endfunction endclass Stack
```

- **类型局限性**：在例 8.34 中，`IntStack` 只能存储整数。如果需要存储实数（`real`），必须复制整个类并将所有 `int` 改为 `real`，这会导致代码冗余和维护困难。
- **参数化语法**：例 8.35 引入了 `#(type T=int)`。
 - `type T`：定义了一个类型参数 `T`。
 - `=int`：设置了默认类型为 `int`。这意味着如果实例化时不指定类型（如 `Stack s = new();`），它表现得和例 8.34 一样。
- **泛型应用**：在类内部，所有的变量声明（`T stack[100]`）和函数参数/返回值（`input T i`，`function T pop()`）都使用了占位符 `T`。
- **实例化示例**（虽然图中未显示，但通常是接下来的用法）：

```

`systemverilog
// 使用默认类型 (int)
Stack int_stack = new();

// 指定为实数类型
Stack #(real) real_stack = new();

// 指定为字符串类型
Stack #(string) str_stack = new();

```

这种设计模式是 SystemVerilog 验证方法学（如 UVM）的基础，UVM 中的许多核心类（如 `systemverilog // 使用默认类型 (int) Stack int_stack = new();`

`// 指定为实数类型 Stack #(real) real_stack = new();`

`// 指定为字符串类型 Stack #(string) str_stack = new();`

这种设计模式是 SystemVerilog 验证方法学（如 UVM）的基础，UVM 中的许多核心类（如 `uvm_config_db`、`uvm_analysis_port`）都是基于这种参数化机制构建的。

使用参数化的堆栈类

```

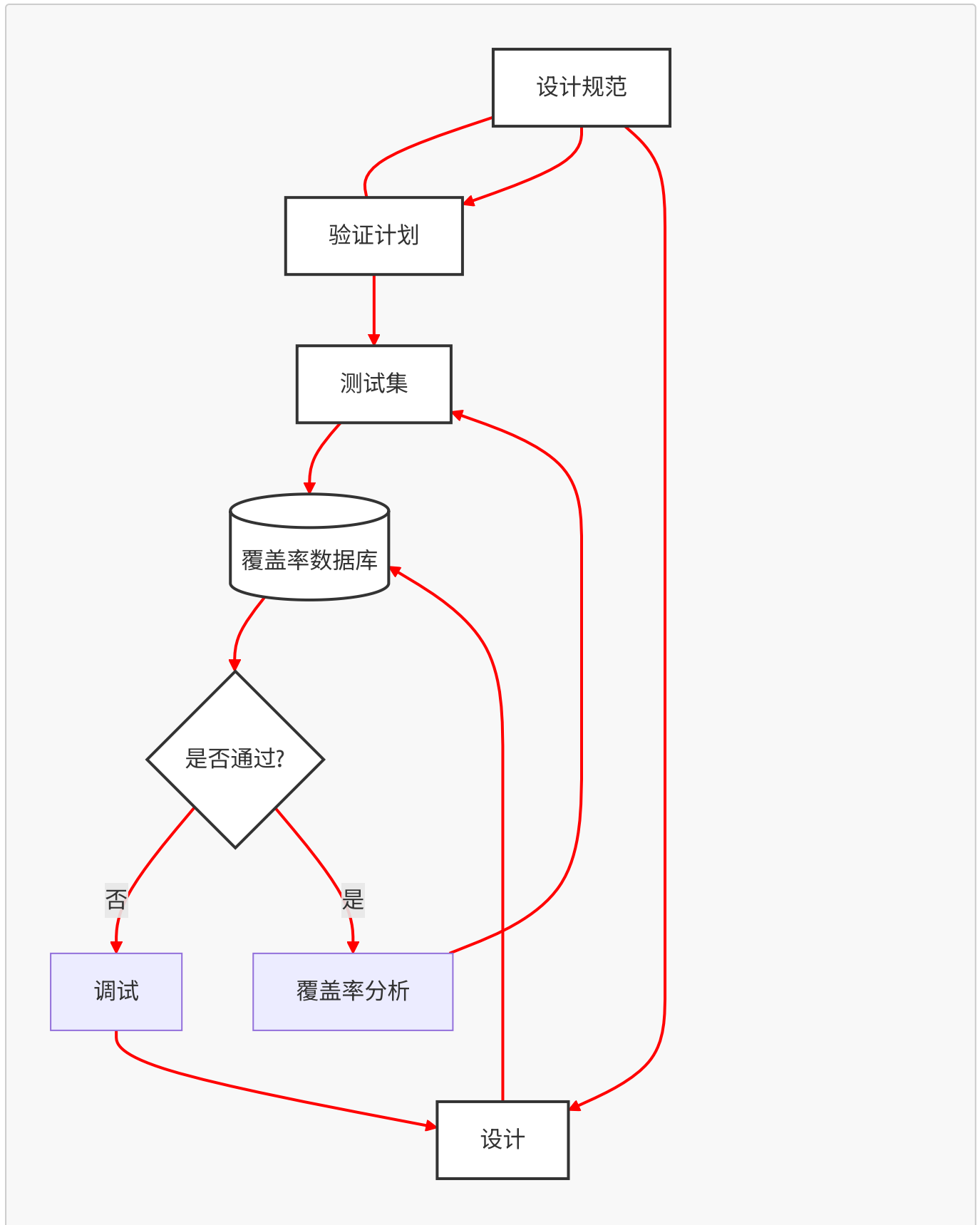
`systemverilog
initial begin
    Stack #(real) rStack; // 创建一个用于实数类型的堆栈
    rStack = new();
    for (int i = 0; i < 10; i++) begin
        rStack.push(i); // 压入实数
    end
    for (int i = 0; i < 10; i++) begin
        $display("Popped: %f", rStack.pop()); // 弹出并显示实数
    end
end

```

功能覆盖率

功能覆盖率用来衡量哪些设计已经被测试程序测试过的一个指标

比如如果设计与总线相连,就需要对设计和总线之间全部可能的交互方式进行测试



断言覆盖率

断言用于一次性或在一段时间内核对两个设计信号之间关系的声明性代码

断言可以用于局部变量,并且可以进行简单的数据检查

断言用于查找错误,例如两个信号是否应该互斥或者请求是否被许可等

功能覆盖率简单例子

```
program automatic test(busifc.TB ifc);
    class Transaction;
        rand bit[31:0] data;
        rand bit[2:0] port; //八种端口port数据
    endclass

    covergroup CovPort;
        coverpoint tr.port; //测量覆盖率
    endgroup

    initial begin
        Transaction tr;
        CovPort ck;
        ck = new(); //实例化组
        tr = new();
        repeat(32) begin // 运行几个周期
            assert(tr.randomize()); //创建一个事务
            ifc.cb.port<=tr.port; //并发送到接口上
            ifc.cb.data<=tr.data;
            ck.sample(); //更新覆盖率
            @ifc.cb; //等待一个周期
        end
    end
endprogram
```

一个简单对象的覆盖率报告

Coverpoint Coverage report CoverageGroup: CovPort Coverpoint: tr.port

- Coverage: 87.50
- Goal: 100
- Number of Expected auto- bins: 8
- Number of User Defined Bins: 0
- Number of Automatically Generated Bins: 7
- Number of User Defined Transitions: 0

Automatically Generated Bins

Bin	# hits	at least
auto[1]	7	1
auto[2]	7	1
auto[3]	1	1
auto[4]	5	1
auto[5]	4	1

Bin	# hits	at least
auto[6]	2	1
auto[7]	6	1

覆盖组详解

覆盖组与类相似——一次定义后便可以进行多次实例化 它含有覆盖点,选项,形式参数和可选触发,一个覆盖组包含了一个或多个数据点,全都在同一时间采集

覆盖组应该带有明白无误的名字,用来表明要测量的对象,并且尽可能与验证计划关联

覆盖组可以定义在类里,也可以定义在程序或模块层次上

类里的功能覆盖率

```
class Transaction;
  Transaction tr;
  mailbox mbx_in;
  covergroup CovPort;
    coverpoint tr.port;
  endgroup

  function new(mailbox mbx_in);
    CovPort = new(); //实例化覆盖组
    this.mbx_in = mbx_in;
  endfunction

  task main;
    forever begin
      tr = mbx_in.get; //获取下一个事务
      ifc.cb.port<= tr.port; //发送到待测设计中
      ifc.cb.data<= tr.data;
      CovPort.sample(); //收集覆盖率
    end
  endtask
endclass
```

覆盖组的触发

功能覆盖率的两个主要部分是采集的数据和数据被采样的时刻 当这些新数据都准备好了以后,测试平台便会触发覆盖组

这个过程可以通过直接使用sample函数来完成

或者在covergroup的定义中采样阻塞表达式,阻塞表达式可以使用wait或者@来实现在信号或事件上的阻塞

使用回调函数进行采样

```

program automatic test;
  Environment env;

  initial begin
    Driver_cbs_coverage dcc;
    env = new();
    env.gen_cfg();
    env.build();
    //创建并登记覆盖率回调函数
    dcc = new();
    env.drv.cbs.push_back(dcc); //放进驱动器的队列里
    env.run();
    env.wrap_up();
  end
endprogram

//用于测量功能覆盖率的回调函数
class Driver_cbs_coverage extends Driver_cbs;
  covergroup CovPort;
  .....
endgroup
  virtual task post_tx(Transaction tr);
    CovPort.sample(); //采样覆盖率数值
  endtask
endclass

```

使用事件触发的覆盖组

代码展示了 **SystemVerilog 中断言（SVA）与功能覆盖率（Functional Coverage）结合使用** 的典型场景，分为两个模块：``systemverilog initial begin Stack #(real) rStack; //创建一个用于实数类型的堆栈 rStack = new(); for (int i = 0; i < 10; i++) begin rStack.push(i); //压入实数 end for (int i = 0; i < 10; i++) begin \$display("Popped: %f", rStack.pop()); //弹出并显示实数 end end

功能覆盖率

功能覆盖率用来衡量哪些设计已经被测试程序测试过的一个指标

比如如果设计与总线相连,就需要对设计和总线之间全部可能的交互方式进行测试‘

```

``mermaid
flowchart TD
  %% 定义节点样式
  classDef docShape fill:#fff,stroke:#333,stroke-width:2px;
  classDef processShape fill:#fff,stroke:#333,stroke-width:2px;
  classDef dbShape fill:#fff,stroke:#333,stroke-width:2px;
  classDef decisionShape fill:#fff,stroke:#333,stroke-width:2px;

  %% 节点定义
  Spec[设计规范]:::docShape

```

```

Plan[验证计划]:::docShape
Design[设计]:::processShape
TestSet[测试集]:::processShape
DB[(覆盖率数据库)]:::dbShape
Decision{是否通过?}:::decisionShape

%% 布局控制：使用不可见连线强制左右布局
Spec ~~~ Plan

%% 主要流程连线
Spec ==> Plan
Spec ==> Design
Plan ==> TestSet

Design ==> DB
TestSet ==> DB

DB ==> Decision

%% 循环回路（否 -> 调试 -> 设计）
Decision -- 否 --> DebugLoop[调试]
DebugLoop ==> Design

%% 循环回路（是 -> 覆盖率分析 -> 测试集）
Decision -- 是 --> AnalysisLoop[覆盖率分析]
AnalysisLoop ==> TestSet

%% 调整连线样式以接近原图（红色箭头）
linkStyle default stroke:red,stroke-width:2px;

```

断言覆盖率

断言用于一次性或在一段时间内核对两个设计信号之间关系的声明性代码

断言可以用于局部变量,并且可以进行简单的数据检查

断言用于查找错误,例如两个信号是否应该互斥或者请求是否被许可等

功能覆盖率简单例子

```

program automatic test(busifc.TB ifc);
  class Transaction;
    rand bit[31:0] data;
    rand bit[2:0] port; //八种端口port数据
  endclass

  covergroup CovPort;
    coverpoint tr.port; //测量覆盖率
  endgroup

  initial begin
    Transaction tr;

```

```
CovPort ck;
ck = new(); //实例化组
tr = new();
repeat(32) begin // 运行几个周期
    assert(tr.randomize); //创建一个事务
    ifc.cb.port<=tr.port; //并发送到接口上
    ifc.cb.data<=tr.data;
    ck.sample(); //更新覆盖率
    @ifc.cb;      //等待一个周期
end
end
endprogram
```

一个简单对象的覆盖率报告

Coverpoint Coverage report CoverageGroup: CovPort Coverpoint: tr.port

- Coverage: 87.50
- Goal: 100
- Number of Expected auto- bins: 8
- Number of User Defined Bins: 0
- Number of Automatically Generated Bins: 7
- Number of User Defined Transitions: 0

Automatically Generated Bins

Bin	# hits	at least
auto[1]	7	1
auto[2]	7	1
auto[3]	1	1
auto[4]	5	1
auto[5]	4	1
auto[6]	2	1
auto[7]	6	1

覆盖组详解

覆盖组与类相似——一次定义后便可以进行多次实例化 它含有覆盖点,选项,形式参数和可选触发,一个覆盖组包含了一个或多个数据点,全都在同一时间采集

覆盖组应该带有明白无误的名字,用来表明要测量的对象,并且尽可能与验证计划关联

覆盖组可以定义在类里,也可以定义在程序或模块层次上

类里的功能覆盖率

```

class Transaction;
    Transaction tr;
    mailbox mbx_in;
    covergroup CovPort;
        coverpoint tr.port;
    endgroup

    function new(mailbox mbx_in);
        CovPort = new();    //实例化覆盖组
        this.mbx_in = mbx_in;
    endfunction

    task main;
        forever begin
            tr = mbx_in.get;    //获取下一个事务
            ifc.cb.port<= tr.port; //发送到待测设计中
            ifc.cb.data<= tr.data;
            CovPort.sample(); //收集覆盖率
        end
    endtask
endclass

```

覆盖组的触发

功能覆盖率的两个主要部分是采集的数据和数据被采样的时刻 当这些新数据都准备好了以后,测试平台便会触发覆盖组

这个过程可以通过直接使用sample函数来完成

或者在covergroup的定义中采样阻塞表达式,阻塞表达式可以使用wait或者@来实现在信号或事件上的阻塞

使用回调函数进行采样

```

program automatic test;
    Environment env;

    initial begin
        Driver_cbs_coverage dcc;
        env = new();
        env.gen_cfg();
        env.build();
        //创建并登记覆盖率回调函数
        dcc = new();
        env.drv.cbs.push_back(dcc); //放进驱动器的队列里
        env.run();
        env.wrap_up();
    end
endprogram

//用于测量功能覆盖率的回调函数

```

```

class Driver_cbs_coverage extends Driver_cbs;
    covergroup CovPort;
    .....
endgroup
virtual task post_tx(Transaction tr);
    CovPort.sample(); //采样覆盖率数值
endtask
endclass

```

使用事件触发的覆盖组

代码展示了 **SystemVerilog 中断言（SVA）与功能覆盖率（Functional Coverage）结合使用** 的典型场景，分为两个模块：**mem**（设计/验证模块）和 **test**（测试程序块）。以下是逐部分解析：

一、例9.9：带 SystemVerilog 断言的模块 **mem**

```

module mem(simple_bus sb);
    bit[7:0] data, addr;    // 8位数据信号和地址信号
    event write_event;      // 事件类型变量，用于跨模块同步触发

    cover property          // 覆盖属性（Cover Property），属于SVA的“覆盖率”特性
        (@(posedge sb.clock) sb.write_ena==1) -> write_event;
endmodule

```

关键语法与作用：

1. **接口实例化**：``systemverilog module mem(simple_bus sb); bit[7:0] data, addr; // 8位数据信号和地址信号 event write_event; // 事件类型变量，用于跨模块同步触发

cover property // 覆盖属性（Cover Property），属于SVA的“覆盖率”特性 (@(posedge sb.clock) sb.write_ena==1) -> write_event; endmodule

关键语法与作用：

1. ****接口实例化****：`simple\_bus sb` 是接口（Interface）类型的端口，用于连接模块与外部信号（如时钟、写使能等）。
2. ****信号声明****：`bit[7:0] data, addr` 定义8位宽的数据和地址信号；`event write\_event` 是**事件（Event）**，用于在不同模块间传递“触发信号”。
3. ****覆盖属性（Cover Property）****：
 - 格式：`cover property (条件) -> 动作;`
 - 语义：**当 `(posedge sb.clock)`（时钟上升沿）时，若 `sb.write\_ena==1`（写使能有效），则触发 `write\_event` 事件**。
 - 作用：通过 SVA 的“覆盖属性”，将“写操作发生”这一行为转化为**事件触发**，为后续覆盖率收集提供“采样时机”。

二、例9.10：使用 SVA 触发覆盖组的测试程序 `test`

```

``systemverilog
program automatic test(simple_bus sb);

```

```

    covergroup Write_cg @($root.top.m1.write_event); // 覆盖组，由 write_event 触发采样
        coverpoint $root.top.m1.data; // 覆盖点：监测 m1 模块的 data 信号
        coverpoint $root.top.m1.addr; // 覆盖点：监测 m1 模块的 addr 信号
    endgroup

    Write_cg wcg; // 覆盖组实例化

    initial begin
        wcg = new(); // 动态创建覆盖组实例
        // 在此处添加激励（如驱动 sb 的信号，模拟读写操作）
        sb.write_ena <= 1; // 示例：拉高写使能（实际需更完整的激励序列）
        ... // 其他激励逻辑
        #10000 $finish; // 仿真10000时间单位后结束
    end
endprogram

```

关键语法与作用：

1. **Program Block**: ``systemverilog program automatic test(simple_bus sb); covergroup Write_cg @(\$root.top.m1.write_event); // 覆盖组，由 write_event 触发采样 coverpoint \$root.top.m1.data; // 覆盖点：监测 m1 模块的 data 信号 coverpoint \$root.top.m1.addr; // 覆盖点：监测 m1 模块的 addr 信号 endgroup

Write_cg wcg; // 覆盖组实例化

initial begin wcg = new(); // 动态创建覆盖组实例 // 在此处添加激励（如驱动 sb 的信号，模拟读写操作） sb.write_ena <= 1; // 示例：拉高写使能（实际需更完整的激励序列） ... // 其他激励逻辑 #10000 \$finish; // 仿真10000时间单位后结束 end endprogram

关键语法与作用：

1. **Program Block**: ``program automatic test(...)` 是 SystemVerilog 专为**测试平台（Testbench）**设计的结构，避免与设计模块的竞争冒险（Race Condition）。
2. **覆盖组（Covergroup）**:
 - 定义: ``covergroup Write_cg @(触发事件);` 表示\*\*当 `write_event` 触发时，自动采样所有 `coverpoint`**。
 - 覆盖点（Coverpoint）: ``coverpoint \$root.top.m1.data` 和 ``coverpoint \$root.top.m1.addr` 分别监测 `m1` 模块的 `data` 和 `addr` 信号的取值分布。
 - 层次引用 ``\$root.top.m1.xxx`: ``\$root` 指向仿真顶层，`top` 是顶层模块名，`m1` 是 `mem` 模块的实例名（假设顶层中 `mem` 被实例化为 `m1`）。
3. **覆盖组实例化与初始化**: ``Write\_cg wcg;` 声明覆盖组变量，``wcg = new();` 在 `initial` 块中动态创建实例（确保覆盖率统计生效）。
4. **激励与仿真控制**: ``initial` 块中先初始化覆盖组，再添加激励（如驱动 `sb.write\_ena`），最后用 ``#10000 \$finish` 控制仿真时长。

整体逻辑：“断言触发 → 覆盖率采样”的协作流程

1. **mem 模块的行为**: 当时钟上升沿且写使能有效时，通过 `cover property` 触发 `write\_event` 事件。
2. **test 程序的响应**: `Write\_cg` 覆盖组监听 `write\_event`，一旦事件触发，立即采样

`m1.data` 和 `m1.addr` 的当前值，并记录到覆盖率数据库。

3. ****核心目的****：利用 SVA 的“事件触发”能力，精准控制****功能覆盖率的采样时机****——仅在“写操作发生”时收集 `data` 和 `addr` 的覆盖信息，避免无效采样，提升覆盖率分析的针对性。

- ****接口（Interface）****：`simple\_bus` 是自定义接口，通常封装时钟、写使能、数据、地址等信号，简化模块间连接。
- ****事件（Event）****：`event` 是 SystemVerilog 的同步原语，类似“信号量”，用于跨模块传递“某事已发生”的通知。
- ****覆盖属性 vs 普通断言****：`cover property` 不用于“检查错误”，而是****统计“某行为发生的次数”****，属于覆盖率驱动验证手段。

这段代码的核心价值是演示 ****“如何用 SVA 的事件机制，联动功能覆盖率的采样逻辑”****，是复杂验证环境中“精准覆盖率收集”的典型实践。

这部分涉及不少 SVA 概念，需要我帮你整理一份‘常用断言与覆盖率语法速查表’方便以后查阅吗？

当在覆盖点指定一个变量或者表达式时，SystemVerilog 会创建很多仓来记录每个数值被捕捉到的次数。这些仓是衡量功能覆盖率的基本单位，如果采样一个单比特变量，最多会有两个仓被创建。

在每次访问的末尾，所有带标的仓会被汇聚到一个新创建的数据库里。

个体仓和总体覆盖率

为了计算一个点上的覆盖率，首先必须确定所有可能数值的个数，这也被称为域。

一个仓中可能有一个或者多个值，覆盖率就是采样值的数目除以域中仓的数目。

自动创建仓：对于一个位宽为 N 的表达式，有 2^N 个可能的值，因此会自动创建 2^N 个仓。

限制自动创建仓的数目

覆盖组选项 `auto_bin_max` 指明了自动创建仓的最大数目，缺省值是 64。如果覆盖点变量或表达式的值域超过指定的最大值，SystemVerilog 会把值域范围平均分配给 `auto_bin_max` 个仓，例如一个 16 比特变量有 65536 个可能值，所以 64 个 bin 中的每一个都覆盖了 1024 个值。

```
```systemverilog
covergroup CovPort;
 coverpoint tr.port
 {
 options.auto_bin_max = 2; //分成2个仓
 }
endgroup
```

在所有覆盖点中使用 `auto_bin_max`

```
covergroup CovPort;
 options.auto_bin_max = 2; //影响port和data
 coverpoint tr.port;
 coverpoint tr.data;
endgroup
```

## 对表达式进行采样

```
class Transaction;
 rand bit[2:0] hdr_len; //范围: 0:7
 rand bit[3:0] payload_len; //范围: 0:15
 rand bit[3:0] kind; //范围0:15
endclass

Transaction tr;

covergroup CovLen;
 len16: coverpoint(tr.hdr_len + tr.payload_len);
 len32: coverpoint(tr.hdr_len + tr.payload_len + 5'b0);
endgroup
```

## 使用用户自定义的仓发现漏洞

```
covergroup CovLen;
 len:coverpoint(tr.hdr_len + tr.payload_len + 5'b0)
 {bins len[] = {[0:23]}};
endgroup
```

## 命名覆盖点的仓

### 指定仓名:

```
covergroup CovKind;
 coverpoint tr.kind {
 bins zero = {0}; // 1个仓代表 kind==0
 bins lo = {[1:3], 5}; // 1个仓代表 1:3 和 5 的值
 bins hi[] = {[8:$]}; // 8个独立的仓: 8...15
 bins misc = default; // 1个仓代表剩余的所有值
 // 没有分号
 }
endgroup // CoverKind
```

## 条件覆盖率

### 使用关键字iff给覆盖点添加条件 条件覆盖:复位期间禁止

```
covergroup CoverPort;
// 当reset == 1 时不收集覆盖率数据
 coverpoint port iff(!bus_if.reset);
endgroup
```

## 使用start和stop函数

```
initial begin
 CovPort ck = new(); //实例化覆盖组
 // 复位期间停止收集覆盖率数据
 # 1ns ck.stop();
 bus_if.reset = 1;
 # 100ns bus_if.reset = 0; //复位结束
 ck.start();

end
```

## 翻转覆盖率

查询port有没有从0变成1, 2或3

```
covergroup CoverPort;
 coverpoint port{
 bins t1 = (0 => 1, 0 => 2, 0 => 3);
 }
endgroup
```

## 用在覆盖点仓中的通配符

```
bit [2:0] port;
covergroup CoverPort;
 coverpoint port{
 wildcard bins even = {3'b?? 0}///??代表通配前两位置
 wildcard bins odd = {3'b?? 1}
 }
endgroup
```

## 忽略数值

使用ignore\_bins覆盖点

```
bit[2:0] low_ports_0_5; //只使用数值0-5
covergroup CoverPort;
 coverpoint low_ports_0_5{
```

```

 ignore_bins hi={[6:7]}; //忽略最后两个仓
 }
endgroup

bit[2:0] low_ports_0_5; //只使用数值0-5
covergroup CoverPort;
 coverpoint low_ports_0_5{
 options.auto_bin_max = 4; //分成2个仓 0:1., 2:3, 4:5,6:7
 ignore_bins hi={[6:7]}; //忽略最后两个仓
 }
endgroup

```

最后一个仓被忽略所以只有前三个仓

不合法的仓

```

bit [2:0] low_ports_0_5; //只使用数值0-5 //只使用数值0-5
covergroup CoverPort;
 coverpoint low_ports_0_5{
 illegal_bins hi={[6:7]}; //如果出现就报错
 }
endgroup

```

交叉覆盖率

```

class Transaction;
 rand bit[3:0] kind;
 rand bit[2:0] port;
endclass

Transaction tr;

covergroup CovPort;
 kind:coverpoint tr.kind; //创建覆盖点kind
 port:coverpoint tr.port; //创建覆盖点port
 cross kind,port; //把kind和port交叉
endgroup

```

记录一个组里两个或两个以上覆盖点的组合值

对交叉覆盖仓进行标号

```

covergroup CovPortKind;
 port:coverpoint tr.port
 {
 bins port[] = {[0:$]};
 }

```

```

 kind:coverpoint tr.kind
 {
 bins zero = {0};
 bins lo = {[1:3], 5};
 bins hi[] = {[8:$]};
 bins misc = default;
 }
 cross kind,port
endgroup

```

### 排除掉部分交叉覆盖仓

```

covergroup Covport;
 port:coverpoint tr.port
 {
 bins port[] = {[0:$]};
 }
 kind:coverpoint tr.kind
 {
 bins zero = {0};
 bins lo = {[1:3], 5};
 bins hi[] = {[8:$]};
 bins misc = default;
 }
 cross kind,port
 {
 ignore_bins hi=binsof(port) intersect{7};
 ignore_bins md = binsof(port) intersect{0} &&
 binsof(kind) intersect{[9:11]};
 ignore_bins lo = binsof(kind.lo);
 }
endgroup

```

上面代码忽略port为7,和任意kind值组合的仓 因为kind是一个4比特数值,这个语句排除掉了16个仓 第二个 ignore\_bins语句排除了port为0,kind值为9-11的仓

一个组的覆盖率是基于所有简单覆盖点和交叉覆盖率的,如果你只希望对一个coverpoint上的变量或表达式进行采样,而这个coverpoint将会被用到cross语句中,那么你应该把它的权重设置为0,这样就不会对

### 指定交叉覆盖率的权重

这段代码展示了 SystemVerilog 功能覆盖率（Functional Coverage）中一个非常关键的技巧：**通过调整权重（Weight）来定制覆盖率的计算逻辑。**

它的核心目的是：**让某些特定的组合（Cross）成为考核重点，而将单独的信号采样仅作为辅助数据，不计入总分“分母”中。**

以下是详细解析：

```

covergroup CovPort;
 // 覆盖点 1: tr.kind
 kind: coverpoint tr.kind {
 bins zero = {0};
 bins lo = {[1:3]};
 bins hi[] = {[8:$]};
 bins misc = default;
 option.weight = 5; // 关键点 A
 }

 // 覆盖点 2: tr.port
 port: coverpoint tr.port {
 bins port[] = {[0:$]};
 option.weight = 0; // 关键点 B
 }

 // 交叉覆盖: kind 和 port 的组合
 cross kind, port {
 option.weight = 10; // 关键点 C
 }
endgroup

```

在 SystemVerilog 中，一个 Covergroup 的总覆盖率计算公式大致如下：

$$\text{Total Coverage} = \frac{\sum(\text{每个点的命中仓数})}{\sum(\text{每个点的总仓数} \times \text{该点的权重})}$$

简单来说，**权重（Weight）** 决定了这个覆盖点在总分里占多大的比重。

- **代码含义：** 定义了 ``systemverilog covergroup CovPort; coverpoint tr.port { options.auto\_bin\_max = 2; //分成2个仓 } endgroup

在所有覆盖点中使用auto\_bin\_max

```

``systemverilog
covergroup CovPort;
 options.auto_bin_max = 2; //影响port和data
 coverpoint tr.port;
 coverpoint tr.data;
endgroup

```

对表达式进行采样

```

class Transaction;
 rand bit[2:0] hdr_len; //范围: 0:7
 rand bit[3:0] payload_len; //范围: 0:15
 rand bit[3:0] kind; //范围0:15

```

```

endclass

Transaction tr;

covergroup CovLen;
 len16: coverpoint(tr.hdr_len + tr.payload_len);
 len32: coverpoint(tr.hdr_len + tr.payload_len + 5'b0);
endgroup

```

使用用户自定义的仓发现漏洞

```

covergroup CovLen;
 len:coverpoint(tr.hdr_len + tr.payload_len + 5'b0)
 {bins len[] = {[0:23]}};
endgroup

```

命名覆盖点的仓

指定仓名:

```

covergroup CovKind;
 coverpoint tr.kind {
 bins zero = {0}; // 1个仓代表 kind==0
 bins lo = {[1:3], 5}; // 1个仓代表 1:3 和 5 的值
 bins hi[] = {[8:$]}; // 8个独立的仓: 8...15
 bins misc = default; // 1个仓代表剩余的所有值
 // 没有分号
 }
endgroup // CoverKind

```

条件覆盖率

使用关键字iff给覆盖点添加条件 条件覆盖:复位期间禁止

```

covergroup CoverPort;
// 当reset == 1 时不收集覆盖率数据
 coverpoint port iff(!bus_if.reset);
endgroup

```

使用start和stop函数

```

initial begin
 CovPort ck = new(); //实例化覆盖组
 // 复位期间停止收集覆盖率数据
 # 1ns ck.stop();

```

```

 bus_if.reset = 1;
 # 100ns bus_if.reset = 0; //复位结束
 ck.start();

end

```

## 翻转覆盖率

查询port有没有从0变成1, 2或3

```

covergroup CoverPort;
 coverpoint port{
 bins t1 = (0 => 1, 0 => 2, 0 => 3);
 }
endgroup

```

## 用在覆盖点仓中的通配符

```

bit [2:0] port;
covergroup CoverPort;
 coverpoint port{
 wildcard bins even = {3'b?? 0}///??代表通配前两位置
 wildcard bins odd = {3'b?? 1}
 }
endgroup

```

## 忽略数值

使用ignore\_bins覆盖点

```

bit[2:0] low_ports_0_5; //只使用数值0-5
covergroup CoverPort;
 coverpoint low_ports_0_5{
 ignore_bins hi={[6:7]}; //忽略最后两个仓
 }
endgroup

bit[2:0] low_ports_0_5; //只使用数值0-5
covergroup CoverPort;
 coverpoint low_ports_0_5{
 options.auto_bin_max = 4; //分成2个仓 0:1., 2:3, 4:5,6:7
 ignore_bins hi={[6:7]}; //忽略最后两个仓
 }
endgroup

```

最后一个仓被忽略所以只有前三个仓



## 不合法的仓

```
bit [2:0] low_ports_0_5; //只使用数值0-5 //只使用数值0-5
covergroup CoverPort;
 coverpoint low_ports_0_5{
 illegal_bins hi={[6:7]}; //如果出现就报错
 }
endgroup
```

## 交叉覆盖率

```
class Transaction;
 rand bit[3:0] kind;
 rand bit[2:0] port;
endclass

Transaction tr;

covergroup CovPort;
 kind:coverpoint tr.kind; //创建覆盖点kind
 port:coverpoint tr.port; //创建覆盖点port
 cross kind,port; //把kind和port交叉
endgroup
```

记录一个组里两个或两个以上覆盖点的组合值

对交叉覆盖仓进行标号

```
covergroup CovPortKind;
 port:coverpoint tr.port
 {
 bins port[] = {[0:$]};
 }
 kind:coverpoint tr.kind
 {
 bins zero = {0};
 bins lo = {[1:3], 5};
 bins hi[] = {[8:$]};
 bins misc = default;
 }
 cross kind,port
endgroup
```

排除掉部分交叉覆盖仓

```

covergroup Covport;
 port:coverpoint tr.port
 {
 bins port[] = {[0:$]};
 }
 kind:coverpoint tr.kind
 {
 bins zero = {0};
 bins lo = {[1:3], 5};
 bins hi[] = {[8:$]};
 bins misc = default;
 }
 cross kind,port
 {
 ignore_bins hi=binsof(port) intersect{7};
 ignore_bins md = binsof(port) intersect{0} &&
 binsof(kind) intersect{[9:11]};
 ignore_bins lo = binsof(kind.lo);
 }
endgroup

```

上面代码忽略port为7,和任意kind值组合的仓 因为kind是一个4比特数值,这个语句排除掉了16个仓 第二个 ignore\_bins语句排除了port为0,kind值为9-11的仓

一个组的覆盖率是基于所有简单覆盖点和交叉覆盖率的,如果你只希望对一个coverpoint上的变量或表达式进行采样,而这个coverpoint将会被用到cross语句中,那么你应该把它的权重设置为0,这样就不会对

指定交叉覆盖率的权重

这段代码展示了 SystemVerilog 功能覆盖率（Functional Coverage）中一个非常关键的技巧：**通过调整权重（Weight）来定制覆盖率的计算逻辑。**

它的核心目的是：**让某些特定的组合（Cross）成为考核重点，而将单独的信号采样仅作为辅助数据，不计入总分“分母”中。**

以下是详细解析：

```

covergroup CovPort;
 // 覆盖点 1: tr.kind
 kind: coverpoint tr.kind {
 bins zero = {0};
 bins lo = {[1:3]};
 bins hi[] = {[8:$]};
 bins misc = default;
 option.weight = 5; // 关键点 A
 }

 // 覆盖点 2: tr.port
 port: coverpoint tr.port {
 bins port[] = {[0:$]};
 }
endgroup

```

```

 option.weight = 0; // 关键点 B
 }

 // 交叉覆盖: kind 和 port 的组合
 cross kind, port {
 option.weight = 10; // 关键点 C
 }
endgroup

```

在 SystemVerilog 中，一个 Covergroup 的总覆盖率计算公式大致如下：

$$\text{Total Coverage} = \frac{\sum(\text{每个点的命中仓数})}{\sum(\text{每个点的总仓数} \times \text{该点的权重})}$$

简单来说，**权重 (Weight)** 决定了这个覆盖点在总分里占多大的比重。

- **代码含义：**定义了 `tr.kind` 的几个区间 (`zero`, `lo`, `hi`, `misc`)。
- **权重设置：**`option.weight = 5;`
- **理解：**这意味着 `kind` 这个信号本身的采样情况是有价值的，但它不是最重要的。它在计算总分时，其贡献度是基准值的 5 倍（或者相对于 `weight=1` 的点来说更重要）。
- **代码含义：**定义了 `tr.port` 的所有可能值 (`[0:$]` 表示全范围)。
- **权重设置：**`option.weight = 0;`
- **为什么要设为 0?**
  - 如果 `port` 是一个 32 位的地址或数据总线，它的可能取值有  $2^{32}$  个（40多亿个）。
  - 如果你把它的权重设为默认值（1），那么分母会变得巨大无比。哪怕你测了很久，只要没跑遍所有地址，覆盖率都会显示为 **0.00%**，这会掩盖其他重要信号的测试进度。
  - **设为 0 的效果：**告诉仿真器，“我不关心 `port` 单独出现了多少次，也不要把它算进总分里”。它在这里存在的唯一目的，是为了配合下面的 `cross` 语句使用。
- **代码含义：**检查 `kind` 和 `port` 是否同时出现过各种组合（例如：当 `kind` 为 `zero` 时，`port` 是否为 `5` ? ）。
- **权重设置：**`option.weight = 10;`
- **理解：**这是验证人员最关心的场景——“某种类型的操作（`kind`）是否在各种端口（`port`）上都发生过？”
- 因为这是验证的重点，所以给了最高的权重（10），比单独的 `kind`（权重5）还要高。

这段代码构建了一个\*\*“重组合，轻单点”\*\*的考核体系：

1. **忽略单点干扰：**通过 `port` 的 `weight=0`，排除了大位宽信号对整体覆盖率数据的“稀释”作用。
2. **聚焦业务逻辑：**通过 `cross` 的高权重，强迫验证团队关注信号之间的**关联性**（即：不仅要看有没有发数据，还要看是不是在正确的模式下发了数据）。
3. **保留基础监控：**`kind` 保留了非零权重，确保基本的模式切换也被记录在案。

从多个值域中合并数据

## 使用仓名来收集交叉覆盖率数据

```

class Transaction;
 rand bit a,b;
endclass

covergroup CrossBinNames;
 a: coverpoint a;
 {
 bins a0 = {0};
 bins a1 = {1};
 option.weight = 0;
 } // 不计算此点的覆盖率

 b: coverpoint b;
 {
 bins b0 = {0};
 bins b1 = {1};
 option.weight = 0;
 } // 不计算此点的覆盖率

 ab: cross a,b
 {
 bins a0b0 = binsof{a.a0} && binsof{b.b0};
 bins a1b0 = binsof{a.a1} && binsof{b.b0};
 bins b1 = binsof(b.b1);
 }
endgroup

```

```

// 例 9.36 使用 binsof 的交叉覆盖率
class Transaction;
 rand bit a, b;
endclass

covergroup CrossBinsofIntersect;
 // 覆盖点 a: 权重设为0, 表示不单独计算该点的覆盖率, 仅作为交叉覆盖的源
 a: coverpoint tr.a {
 option.weight = 0;
 }

 // 覆盖点 b: 权重设为0, 同上
 b: coverpoint tr.b {
 option.weight = 0;
 }

 // 交叉覆盖 ab: 自定义具体的组合仓
 ab: cross a, b {
 // 定义仓 a0b0: 要求 a 为 0 且 b 为 0
 bins a0b0 = binsof(a) intersect {0} &&
 binsof(b) intersect {0};
 }
endgroup

```

```

// 定义仓 a1b0: 要求 a 为 1 且 b 为 0
bins a1b0 = binsof(a) intersect {1} &&
 binsof(b) intersect {0};

// 定义仓 b1: 只要 b 为 1 (无论 a 是什么)
bins b1 = binsof(b) intersect {1};
}
endgroup

```

### 使用串联值替代交叉覆盖

```

covergroup CrossManual;
 ab:coverpoint{tr.a,tr.b}
 {bins a0b0 = {0,0};
 bins a1b0 = {1,0};}
 wildcard bins b1 = {2'b ? 1};
endgroup

```

### 使用串联值来替代交叉覆盖

```

covergroup CrossManual;
 ab:coverpoint{tr.a,tr.b}
 {bins a0b0 = {2'b 00};
 bins a1b0 = {2'b 10};
 wildcard bins b1 = {2'b 1?};
 }
endgroup

```

## 通用的覆盖组

### 通过数值传递覆盖组参数

```

bit [2:0] port;
covergroup CoverPort(int mid);
 coverpoint port
 {
 bins lo = {[0:mid-1]};
 bins hi = {[mid:$]};
 }
endgroup

CoverPort cp;
initial
 cp = new(5); //lo = 0:4, hi = 5:7

```

## 通过引用传递覆盖参数

```
// 例 9.39 通过引用传递
bit [2:0] port_a, port_b;

// 定义带参数的 covergroup
// ref bit[2:0] port : 引用传递信号, 用于监控
// input int mid : 值传递整数, 用于定义分界点
covergroup CoverPort (ref bit [2:0] port, input int mid);
 coverpoint port {
 bins lo = {[0:mid-1]}; // 低位范围: 0 到 mid-1
 bins hi = {[mid:$]}; // 高位范围: mid 到最大值($)
 }
endgroup

CoverPort cpa, cpb; // 声明两个 covergroup 实例

initial begin
 // 实例化 cpa: 监控 port_a, 分界点为 4
 cpa = new(port_a, 4); // lo=0:3, hi=4:7

 // 实例化 cpb: 监控 port_b, 分界点为 2
 cpb = new(port_b, 2); // lo=0:1, hi=2:7
end
..
```

## 覆盖选项

### 单个实例的覆盖率

### 指定单个实例的覆盖率

```
covergroup CoverLength;
 coverpoint tr.length;
 option.per_instance = 1;
 //在注释中使用层次化路径
 option.comment = $sprintf("%m");
endgroup
```

关键设置：启用“按实例统计”模式。默认情况下，如果同一个 covergroup 被多次实例化（如在多个对象中创建），所有实例的覆盖率会合并计算。设置为 1 后，每个实例的覆盖率将独立统计，便于定位具体哪个实例未覆盖到某些值。例如：若你有 10 个 Transaction 对象，每个都创建了 CoverLength 实例，那么你会看到 10 个独立的覆盖率报告，而不是一个汇总报告。

### 覆盖组的注释

```
covergroup CoverPort;
 type_option.comment = "Section 9.40";
```

```
 coverpoint port;
endgroup
```

为单个覆盖组实例指定注释

```
Covergroup CoverPort(int lo,hi,string comment);
 option.comment = comment;
 option.per_instance = 1;
 coverpoint port{
 bins range = {[lo:hi]};
 }
endgroup

...

CoverPort cp_lo = new(0,3,"low port numbers");
CoverPort cp_hi = new(4,7,"high port numbers");
```

**覆盖阈值** 如果认为一个仓被命中8次以上,其所有组合就都能被测试到,那么可以把 option.at\_least设置为8或者更高

**option.at\_least** 如果定义在覆盖组里,那么它就会作用于所有的覆盖点

**打印空仓**

**报告所有的仓,包括空仓**

```
covergroup CovPort;
 kind:coverpoint tr.kind;
 port:coverpoint tr.port
 cross kind,port;
 option.cross_num_print_missing = 1_000;
endgroup
```

**覆盖率目标**

一个覆盖组或覆盖点的目标是达到该点被认为已经完全覆盖的水平,缺省情况是百分百的覆盖,但可以设置为低于100%

```
covergroup CovPort;
 coverpoint port;
 option.goal = 90; // 目标是90%覆盖率
endpoint
```

**覆盖率数据的分析**

如果覆盖点只有一个采样值甚至没有,那么你的约束可能根本就没有定位在预期的区域上

```
class Transaction;
 rand bit [2:0] hdr_len;
 rand bit [3:0] payload_len;
 rand bit [4:0] len;
 constraint length {len == hdr_len + payload_len;}
endclass
```

对事务长度使用solve...before约束

```
constraint length{len == hdr_len + payload_len;
 solve hdr_len before hdr_len,payload_len;}
```

solve...before约束通常的替代选择是dist约束 但是,这并不起作用,因为len同时也受两个长度之和的约束

在仿真过程中进行覆盖率统计

- \*\*全局层面: ```systemverilog class Transaction; rand bit a,b; endclass

covergroup CrossBinNames; a: coverpoint a; { bins a0 = {0}; bins a1 = {1}; option.weight = 0; }//不计算此点的覆盖率

```
b: coverpoint b;
{
 bins b0 = {0};
 bins b1 = {1};
 option.weight = 0;
} //不计算此点的覆盖率

ab:cross a,b
{
 bins a0b0 = binsof{a.a0} && binsof{b.b0};
 bins a1b0 = binsof{a.a1} && binsof{b.b0};
 bins b1 = binsof(b.b1);
}
```

endgroup

```
```systemverilog
// 例 9.36 使用 binsof 的交叉覆盖率
class Transaction;
  rand bit a, b;
endclass
```



```

covergroup CrossBinsofIntersect;
  // 覆盖点 a: 权重设为0, 表示不单独计算该点的覆盖率, 仅作为交叉覆盖的源
  a: coverpoint tr.a {
    option.weight = 0;
  }

  // 覆盖点 b: 权重设为0, 同上
  b: coverpoint tr.b {
    option.weight = 0;
  }

  // 交叉覆盖 ab: 自定义具体的组合仓
  ab: cross a, b {
    // 定义仓 a0b0: 要求 a 为 0 且 b 为 0
    bins a0b0 = binsof(a) intersect {0} &&
               binsof(b) intersect {0};

    // 定义仓 a1b0: 要求 a 为 1 且 b 为 0
    bins a1b0 = binsof(a) intersect {1} &&
               binsof(b) intersect {0};

    // 定义仓 b1: 只要 b 为 1 (无论 a 是什么)
    bins b1   = binsof(b) intersect {1};
  }
endgroup

```

使用串联值替代交叉覆盖

```

covergroup CrossManual;
  ab:coverpoint{tr.a,tr.b}
  {bins a0b0 = {0,0};
   bins a1b0 = {1,0};}
  wildcard bins b1 = {2'b ? 1};
endgroup

```

使用串联值来替代交叉覆盖

```

covergroup CrossManual;
  ab:coverpoint{tr.a,tr.b}
  {bins a0b0 = {2'b 00};
   bins a1b0 = {2'b 10};
   wildcard bins b1 = {2'b 1?};
  }
endgroup

```

通用的覆盖组

通过数值传递覆盖组参数

```

bit [2:0] port;
covergroup CoverPort(int mid);
    coverpoint port
    {
        bins lo = {[0:mid-1]};
        bins hi = {[mid:$]};
    }
endgroup

CoverPort cp;
initial
    cp = new(5); //lo = 0:4, hi = 5:7

```

通过引用传递覆盖参数

```

// 例 9.39 通过引用传递
bit [2:0] port_a, port_b;

// 定义带参数的 covergroup
// ref bit[2:0] port : 引用传递信号, 用于监控
// input int mid      : 值传递整数, 用于定义分界点
covergroup CoverPort (ref bit [2:0] port, input int mid);
    coverpoint port {
        bins lo = {[0:mid-1]}; // 低位范围: 0 到 mid-1
        bins hi = {[mid:$]};   // 高位范围: mid 到最大值($)
    }
endgroup

CoverPort cpa, cpb; // 声明两个 covergroup 实例

initial begin
    // 实例化 cpa: 监控 port_a, 分界点为 4
    cpa = new(port_a, 4); // lo=0:3, hi=4:7

    // 实例化 cpb: 监控 port_b, 分界点为 2
    cpb = new(port_b, 2); // lo=0:1, hi=2:7
end
..

```

覆盖选项

单个实例的覆盖率

指定单个实例的覆盖率

```

covergroup CoverLength;
    coverpoint tr.length;
    option.per_instance = 1;

```

```
//在注释中使用层次化路径
option.comment = $psprintf("%m");
endgroup
```

关键设置：启用“按实例统计”模式。默认情况下，如果同一个 covergroup 被多次实例化（如在多个对象中创建），所有实例的覆盖率会合并计算。设置为 1 后，每个实例的覆盖率将独立统计，便于定位具体哪个实例未覆盖到某些值。例如：若你有 10 个 Transaction 对象，每个都创建了 CoverLength 实例，那么你会看到 10 个独立的覆盖率报告，而不是一个汇总报告。

覆盖组的注释

```
covergroup CoverPort;
    type_option.comment = "Section 9.40";
    coverpoint port;
endgroup
```

为单个覆盖组实例指定注释

```
Covergroup CoverPort(int lo,hi,string comment);
    option.comment = comment;
    option.per_instance = 1;
    coverpoint port{
        bins range = {[lo:hi]};
    }
endgroup

...

CoverPort cp_lo = new(0,3,"low port numbers");
CoverPort cp_hi = new(4,7,"high port numbers");
```

覆盖阈值 如果认为一个仓被命中8次以上,其所有组合就都能被测试到,那么可以把 option.at_least设置为8或者更高

option.at_least 如果定义在覆盖组里,那么它就会作用于所有的覆盖点

打印空仓

报告所有的仓,包括空仓

```
covergroup CovPort;
    kind:coverpoint tr.kind;
    port:coverpoint tr.port
    cross kind,port;
    option.cross_num_print_missing = 1_000;
endgroup
```

覆盖率目标

一个覆盖组或覆盖点的目标是达到该点被认为已经完全覆盖的水平,缺省情况是百分百的覆盖,但可以设置为低于100%

```
covergroup CovPort;
  coverpoint port;
  option.goal = 90; // 目标是90%覆盖率
endpoint
```

覆盖率数据的分析

如果覆盖点只有一个采样值甚至没有,那么你的约束可能根本就没有定位在预期的区域上

```
class Transaction;
  rand bit [2:0] hdr_len;
  rand bit [3:0] payload_len;
  rand bit [4:0] len;
  constraint length {len == hdr_len + payload_len;}
endclass
```

对事务长度使用solve...before约束

```
constraint length{len == hdr_len + payload_len;
  solve hdr_len before hdr_len,payload_len;}
```

solve...before约束通常的替代选择是dist约束 但是,这并不起作用,因为len同时也受两个长度之和的约束

在仿真过程中进行覆盖率统计

- **全局层面：** `$get_coverage`
 - **作用：** 获取整个仿真环境中所有 Covergroup 的总覆盖率。
 - **场景：** 宏观把控整体进度。
- **类型层面：** `CoverGroup::get_coverage()`
 - **作用：** 获取某个特定 Covergroup 定义下所有实例的平均或综合覆盖率。
 - **用法：** 通过类名调用。
- **实例层面：** `cgInst.get_inst_coverage()`
 - **作用：** 获取某一个具体实例 (Instance) 的覆盖率。
 - **关键点：** 如果要精确统计单个实例的数据，通常需要在定义 Covergroup 时设置 `option.per_instance = 1`。

高级接口

ATM路由器的虚接口

只含有物理接口的测试平台

```
//带有modport和时钟块的Rx接口
interface Rx_if(input logic clk);
    logic [7:0] data;
    logic soc,en,clav,rclk;

    clocking cb@(posedge clk);
        output data,soc,clav; //方向是相对于测试平台的
        input en;
    endclocking:cb

    modport TB(clocking cb);

    modport DUT(output en,rclk,input data,soc,clav);
endinterface: Rx_if
```

```
//带有时钟块的TX接口
interface Tx_if(input logic clk);
    logic [7:0] data;
    logic soc,en,clav,tclk;

    clocking cb@(posedge clk);
        input data,soc,en;
        output clav;
    endclocking:cb

    modport TB(clocking cb);
    modport DUT(output data,soc,en,tclk, input clav);
endinterface: Tx_if
```

modport 的核心作用 定义信号方向：在接口内部，通过 input、output 关键字明确指定哪些信号对当前模块是输入，哪些是输出。这解决了“同一个信号，对 TB 是输出，对 DUT 是输入”的矛盾。封装与复用：将一组相关的信号打包成一个逻辑单元，方便在不同模块间传递和复用，避免重复声明和连接错误。工具检查：帮助仿真工具和综合工具自动检查信号连接的合法性，防止驱动冲突或未连接信号。

这段代码展示了一个 **SystemVerilog ````systemverilog 带有modport和时钟块的Rx接口 interface Rx_if(input logic clk); logic [7:0] data; logic soc,en,clav,rclk;

```
clocking cb@(posedge clk);
    output data,soc,clav; //方向是相对于测试平台的
    input en;
endclocking:cb

modport TB(clocking cb);
```

```
modport DUT(output en,rclk,input data,soc,clav);
```

endinterface: Rx_if

```
```systemverilog
//带有时钟块的TX接口
interface Tx_if(input logic clk);
 logic [7:0] data;
 logic soc,en,clav,tclk;

 clocking cb@(posedge clk);
 input data,soc,en;
 output clav;
 endclocking:cb

 modport TB(clocking cb);
 modport DUT(output data,soc,en,tclk, input clav);
endinterface: Tx_if
```

**modport** 的核心作用 定义信号方向：在接口内部，通过 **input**、**output** 关键字明确指定哪些信号对当前模块是输入，哪些是输出。这解决了“同一个信号，对 TB 是输出，对 DUT 是输入”的矛盾。封装与复用：将一组相关的信号打包成一个逻辑单元，方便在不同模块间传递和复用，避免重复声明和连接错误。工具检查：帮助仿真工具和综合工具自动检查信号连接的合法性，防止驱动冲突或未连接信号。

这段代码展示了一个 **SystemVerilog program 块**，它是验证环境的核心执行体。它通过实例化之前定义的接口（特别是 **Tx\_if**），并利用 **clocking block** 来驱动被测设计（DUT）。

结合你提供的上下文，这段代码完美演示了 **Tx\_if（发送接口）** 是如何被测试平台使用的。以下是详细解析：

#### 1. 接口的实例化与连接 在 **program** 的声明部分：

```
program automatic test(Rx_if.TB Rx0, Rx1, Rx2, Rx3,
 Tx_if.TB Tx0, Tx1, Tx2, Tx3, ...);
```

- ````systemverilog program automatic test(Rx_if.TB Rx0, Rx1, Rx2, Rx3, Tx_if.TB Tx0, Tx1, Tx2, Tx3, ...);`

- ````Rx_if.TB` / `Tx_if.TB````：这里使用了我们在上一段代码中定义的 **Modport**。
- ``.TB`` 后缀表明这些接口是以“测试平台视角”连接的。这意味着在 ``program`` 内部，我们可以直接驱动那些在 **Modport** 中定义为 ``output`` 的信号（例如 ``Tx0.cb.clav``）。
- **多路复用**：代码实例化了 4 个接收接口（``Rx0-Rx3``）和 4 个发送接口（``Tx0-Tx3``），模拟了一个具有多个端口的 ATM 路由器环境。

#### 1. 核心逻辑：`receive\_cell0` 任务

这个任务是代码的灵魂，它展示了如何使用 **Clocking Block（`cb`）** 来进行精确的时序控制。注意看它的操作对象是 ``Tx0``（发送接口 0）。

虽然任务名叫 `receive\_cell0`（可能是指从 DUT 的角度看是接收，或者是指测试平台的第 0 号接收逻辑），但在代码实现上，它是在 **\*\*驱动 `Tx0` 接口向 DUT 发送数据\*\***。

#### A. 同步机制

```
```systemverilog
@(Tx0.cb); // 等待时钟块的一个周期
```

- 这是使用 Clocking Block 的标准起手式。它确保后续的所有信号操作都与 ``systemverilog @(Tx0.cb); // 等待时钟块的一个周期

- 这是使用 Clocking Block 的标准起手式。它确保后续的所有信号操作都与 `Tx0` 的时钟沿（`posedge clk`）严格同步，避免了竞争冒险。

B. 握手协议（Handshake）

代码模拟了一个典型的 ****Valid/Ready**** 或 ****Start/Data**** 握手流程：

1. **发起请求**：

```
```systemverilog
Tx0.cb.clav <= 1; // 驱动 clav 信号为高（Cell Available?）
```
```

- 这里使用了非阻塞赋值 `<=`, 这是 Clocking Block 输出的推荐写法，信号会在时钟沿后的特定 skew 时间生效。

2. **等待响应**：

```
```systemverilog
wait(Tx0.cb.soc == 1); // 阻塞等待 soc (Start of Cell) 变高
```
```

- `wait` 语句会暂停当前线程，直到条件满足。这模拟了发送方等待接收方（DUT）准备好接收数据的时刻。

C. 数据传输循环

```
```systemverilog
for(int i=0; i < `ATM_SIZE; i++) begin
 wait(Tx0.cb.en == 0); // 1. 等待使能信号无效（可能是流控或间隔）
 @(Tx0.cb); // 2. 对齐到下一个时钟沿
 bytes[i] = Tx0.cb.data; // 3. 采样数据（如果是接收）或驱动数据（如果是发送）
 // *注：根据上下文，这里看起来像是在读取 Tx0 上的数据，
 // 或者如果 Tx0 是双向的，这里可能是在校验回环数据。
 // 但通常 Tx 接口是 DUT 的输入，这里更像是模拟外部源发送数据。
 @(Tx0.cb); // 4. 等待一个周期
end
```

- **流控逻辑**： ``systemverilog for(int i=0; i < `ATM\_SIZE; i++) begin wait(Tx0.cb.en == 0); // 1. 等待使能信号无效（可能是流控或间隔） @(Tx0.cb); // 2. 对齐到下一个时钟沿 bytes[i] = Tx0.cb.data; // 3. 采样数据（如果是接收）或驱动数据（如果是发送） // \*注：根据上下文，这里看起来像是在读取 Tx0 上的数据， // 或者如果 Tx0 是双向的，这里可能是在校验回环数据。 // 但通常 Tx 接口是 DUT 的输入，这里更像是模拟外部源发送数据。 @(Tx0.cb); // 4. 等待一个周期 end

- **\*\*流控逻辑\*\***: ``wait(Tx0.cb.en == 0)`` 展示了如何处理复杂的时序依赖。在发送下一个字节前, 必须确保 ``en`` 信号处于特定状态。
- **\*\*数据存取\*\***: ``Tx0.cb.data`` 直接访问时钟块内的数据总线, 保证了读写的时序安全性。

### 1. 复位与初始化

在 ``initial`` 块中:

```
```systemverilog
rst <= 1;           // 全局复位
Rx0.cb.data <= 0;   // 初始化接收接口数据
```

- 这里展示了如何同时操作不同的接口实例。先复位整个系统, 然后单独清理 ````systemverilog rst <= 1; // 全局复位 Rx0.cb.data <= 0; // 初始化接收接口数据`

- 这里展示了如何同时操作不同的接口实例。先复位整个系统, 然后单独清理 ``Rx0`` 的状态, 准备开始测试。

总结: 接口是如何被“用”起来的?

这段代码回答了“接口定义好后怎么用”的问题:

1. ****作为参数传递****: 接口被传递给 ``program``, 实现了物理连接。
2. ****通过 Modport 访问****: 使用 ``.TB`` 视图, 确保只有测试平台该驱动的信号才能被驱动。
3. ****通过 Clocking Block 操作****: 所有的信号读写都带上了 ``.cb`` 后缀 (如 ``Tx0.cb.clav``)。这不仅是为了同步, 更是为了利用 SystemVerilog 的 ****输入/输出 Skew**** 特性, 自动处理建立时间和保持时间, 让测试代码写起来像 RTL 一样简单, 但跑起来却非常稳健。

含有接口数组的顶层模块

```
```systemverilog
module top;
 logic clk,rst;

 Rx_if Rx[4](clk);
 Tx_if Tx[4](clk);

 test t1(Rx,Tx,rst);
 atm_router a1(Rx[0],Rx[1],Rx[2],Rx[3],
 Tx[0],Tx[1],Tx[2],Tx[3],clk,rst);

 initial begin
 clk = 0;
 forever #20 clk = ~clk;
 end
endmodule:top
```

### 使用虚拟接口的驱动类

```
class Driver;
 int stream_id;
```



```

bit done = 0;
mailbox exp_mbx;
virtual Rx_if.TB Rx;

function new(input mailbox exp_mbx, input int stream_id,input virtual Rx_if.TB
Rx);
 this.exp_mbx = exp_mbx;
 this.stream_id = stream_id;
 this.Rx = Rx;
endfunction

task run(input int ncells ,input event driver_done);
 ATM_Cell ac;

 fork //将此任务派生为一个独立的线程
 begin
 //初始化输出信号
 Rx.cb.clav<= 0;
 Rx.cb.soc<= 0;
 @Rx.cb;

 //驱动信元,直到发送最后一个信元
 repeat(ncells) begin
 ac=new
 assert(ac.randomize);
 if(ac.eot_cell) break;
 drive_cell(ac);
 end

 $display("@%0t Driver %0d done", $time, stream_id);
 ->driver_done;
 end
 join_none
endtask:run

task drive_cell(input ATM_Cell ac);
 bit [7:0] bytes[];

 #ac.delay;
 $display("@%0t Driver %0d sending cell %0d", $time, stream_id,
ac.cell_id);

 //等待下一个时钟周期

 @Rx.cb; // 等待时钟沿同步
 Rx.cb.clav <= 1; // 置位 xfr (Cell Available), 通知 DUT 有数据可用

 do
 @Rx.cb; // 等待下一个时钟周期
 while(Rx.cb.en != 0); // 等待使能信号变低 (Wait for enable signal to
go low)

 // *注：此处逻辑取决于具体协议，通常是在等待 DUT
 准备好接收或完成当前传输*

```

```

Rx.cb.soc <= 1; // 信元的开始 (Start of Cell)
Rx.cb.data <= bytes[0]; // 驱动第一个字节
@Rx.cb; // 等待时钟沿
Rx.cb.soc <= 0; // 信元传送完毕 (Start of Cell 结束)
Rx.cb.data <= bytes[1]; // 驱动第二个字节 (注释误写为“第一个”，实为第二个)

for(int i=2; i < 'ATM_SIZE; i++) begin
 @Rx.cb; // 每个字节传输都需要等待时钟沿
 Rx.cb.data <= bytes[i]; // 依次驱动后续字节
end

@Rx.cb; // 最后一个字节传输后的等待
Rx.cb.soc <= 1'bz; // 结束时 SOC 置高阻态
Rx.cb.clav <= 0; // 撤销 Cell Available 信号
Rx.cb.data <= 8'bz; // 清除数据行，置高阻态

$display("@%0d:Driver::drive_cell(%0d) finish",
 $time, stream_id); // 打印调试信息，显示完成时间和流 ID

// 将信元送至记分板 (Scoreboard) 进行比对
exp_mbx.put(ac); // 把预期数据放入 mailbox

endtask: drive_cell_t
endclass: Driver

```

将测试平台链接到端口列表中的接口

```

module top;
 bus_ifc bus(); //例化接口
 test t1(bus); //通过端口列表传递给测试程序
 dut d1(bus); //通过端口列表传递给被测模块
endmodule:top

```

```

program automatic test(bus_ifc bus);
 initial $display(bus.data);
endprogram

```

端口列表中含有第二个接口的顶层模块

```

module top;
 bus_ifc bus(); //例化接口
 new_ifc newb(); //再例化一个接口
 test t1(bus,newb); //使用两个接口的测试程序
 dut d1(bus,newb); //使用两个接口的被测模块
endmodule

```

## 使用XMR(跨模块引用)连接接口和测试程序

```
//- 使用虚接口和XMR的测试程序
program automatic test();
 virtual bus_ifc bus= top.bus; //跨模块引用
 initial $display(bus.data); //使用接口信号
endprogram
```

## 连接到多个不同的设计配置

### 8位计数器的接口

```
interface X_if(input logic clk);
 logic[7:0] din,dout;
 logic reset_l,load;

 clocking cb @(posedge clk);
 output din,load;
 input dout;
 endclocking

 always @ cb
 $strobe("@%0t %0d", $time, din);

 modport DUT(input clk,din,reset_l,load,output dout);
endinterface
```

### 使用X\_if接口的计数器模型

```
module dut(X_if.DUT xi);
 logic[7:0] count;
 assign xi.dout = count;

 always @(posedge xi.clk or negedge xi.reset_l)
 begin
 if (xi.reset_l)
 count <= 0;
 else if (xi.load)
 count <= xi.din;
 else
 count <= count + 1;
 end
endmodule
```

### 使用虚拟接口数组的测试平台

```

parameter NUM_XI = 2; // 设计实例的个数
parameter NUM_XI = 2;

module top;
 //时钟发生器
 bit clk;

 initial begin
 clk <= '0;
 forever #20 clk <= ~clk;
 end

 //例化N个接口
 X_if xi[NUM_XI] (clk);

 //例化测试平台
 test tb();

 //产生N个DUT实例
 generate
 for(genvar i=0; i < NUM_XI; i++)
 begin:dut_blk
 dut dut_i(xi[i]);
 end
 endgenerate
endmodule

```

### 使用虚拟接口的计数器测试平台

```

// 例 10.19 使用虚接口的计数器测试平台
program automatic test;

 // 1. 声明虚接口数组和 Driver 动态数组
 virtual X_if.TB vxi[NUM_XI]; // 虚接口数组
 Driver driver[];

 initial begin
 // 2. 将顶层物理接口连接到局部虚接口
 vxi = top.xi;

 // 3. 创建 N 个驱动器对象 (Driver)
 driver = new[NUM_XI];
 foreach(driver[i])
 driver[i] = new(vxi[i], i);

 // 4. 并发执行所有驱动器的任务
 foreach(driver[i]) begin
 int j = i; // 关键步骤: 复制循环变量
 fork
 begin

```

```

 driver[j].reset();
 driver[j].load_op();
 end
 join_none
 end

 // 5. 等待一段时间让任务运行
 repeat(10) @(vxi[0].cb);
end

endprogram

```

## 使用虚接口的Driver类

```

// 例 10.20 使用虚接口的 Driver 类
class Driver;
 virtual X_if xi; // 声明虚接口句柄
 int id; // 驱动器 ID, 用于区分不同实例

 // 构造函数: 接收虚接口和 ID
 function new(input virtual X_if.TB xi, input int id);
 this.xi = xi;
 this.id = id;
 endfunction

 // 复位任务
 task reset();
 $display("@%0t:%m:Start reset[%0d]", $time, id);

 // 设备复位逻辑
 xi.reset_l <= 1; // 拉高复位信号 (假设低电平有效, 此处可能是释放复位或特定
 协议)

 xi.cb.load <= 0; // 拉低 load 信号
 xi.cb.din <= 0; // 数据输入置 0
 @(xi.cb) xi.reset_l <= 0; // 等待时钟沿, 拉低复位信号
 @(xi.cb) xi.reset_l <= 1; // 再等一个时钟沿, 拉高复位信号 (完成复位脉冲)

 $display("@%0t:%m:End reset[%0d]", $time, id);
 endtask : reset

 // 加载操作任务
 task load_op();
 $display("@%0t:%m:Start load[%0d]", $time, id);

 #1 xi.cb.load <= 1; // 延时 1 个时间单位后, 拉高 load 信号
 xi.cb.din <= id + 10; // 设置数据输入为 id + 10

 #1 xi.cb.load <= 0; // 延时 1 个时间单位后, 拉低 load 信号
 repeat(5) @(xi.cb); // 等待 5 个时钟周期

 $display("@%0t:%m:End load[%0d]", $time, id);
 endtask : load_op

```

```
endclass Driver
```

## 对虚接口使用typedef

```
typedef virtual x_if.TB vx_if;

program automatic test;
 vx_if vxi[NUM_XI]; //虚接口数组
 Driver driver[];

 ...
endprogram
```

## 使用端口传递虚接口的测试平台

```
parameter NUM_XI = 2; //实例个数

module top;
 //例化N个接口
 X_if xi[NUM_XI] (clk);

 //例化测试平台
 test tb(xi);
endmodule
```

## 使用端口传递虚接口测试平台

```
// 例 10.21 使用端口传递虚接口的测试平台
Driver driver[]; // 声明 Driver 动态数组
virtual X_if vxi[NUM_XI]; // 声明虚接口数组

initial begin
 // 将局部虚接口连到顶层
 if (NUM_XI <= 0) $finish; // 安全检查: 如果通道数为0则结束仿真

 driver = new[NUM_XI]; // 为 Driver 数组分配内存空间
 vxi = xi; // 把顶层物理接口数组赋值给本地虚接口数组

 for (int i=0; i < NUM_XI; i++) begin
 driver[i] = new(vxi[i], i); // 实例化第 i 个 Driver, 传入对应的虚接口和 ID
 driver[i].reset(); // 调用该 Driver 的复位任务
 end
 ...
end
```

```
end
endprogram
```

## 接口中的过程代码

### 并行协议接口

```
interface simple_if(input logic clk);
 // 1. 信号定义
 logic [7:0] addr;
 logic [7:0] data;
 bus_cmd_e cmd;

 // 2. 目标端口 (Target) - 面向从机或接收端
 modport TARGET (
 input addr, cmd, data,
 import task targetRcv(output bus_cmd_e c, logic[7:0] a, d)
);

 // 3. 发起者端口 (Initiator) - 面向主机或发送端
 modport INITIATOR (
 output addr, cmd, data,
 import task initiatorSend(input bus_cmd_e c, logic[7:0] a, d)
);

 // 4. 并行发送任务 (由 Initiator 使用)
 task initiatorSend(input bus_cmd_e c, logic[7:0] a, d);
 @(posedge clk); // 等待时钟上升沿
 cmd <= c; // 非阻塞赋值驱动命令
 addr <= a; // 非阻塞赋值驱动地址
 data <= d; // 非阻塞赋值驱动数据
 endtask

 // 5. 并行接收任务 (由 Target 使用)
 task targetRcv(output bus_cmd_e c, logic[7:0] a, d);
 @(posedge clk); // 等待时钟上升沿采样
 a = addr; // 阻塞赋值采样地址
 d = data; // 阻塞赋值采样数据
 c = cmd; // 阻塞赋值采样命令
 endtask

endinterface simple_if
```

### 串行接口代码

```
// 例 10.26 含有使用串行协议任务的接口
interface simple_if(input logic clk);
 logic addr;
 logic data;
```

```

logic start = 0;
bus_cmd_e cmd;

// 定义 Target（从机/接收端）端口视图
modport TARGET (
 input addr, cmd, data,
 import task targetRcv(output bus_cmd_e c, logic [7:0] a, d)
);

// 定义 Initiator（主机/发送端）端口视图
modport INITIATOR (
 output addr, cmd, data,
 import task initiatorSend(input bus_cmd_e c, logic [7:0] a, d)
);

// --- 串行发送任务 ---
task initiatorSend(input bus_cmd_e c, logic [7:0] a, d);
 @(posedge clk); // 等待时钟上升沿
 start <= 1; // 拉高 Start 信号，开始传输
 cmd <= c; // 发送命令

 // 循环遍历数组进行串行发送
 foreach(a[i]) begin
 addr <= a[i]; // 发送地址位
 data <= d[i]; // 发送数据位
 @(posedge clk); // 等待下一个时钟沿（串行传输的核心）
 end

 start <= 0; // 拉低 Start 信号，结束传输
 cmd <= IDLE; // 命令恢复空闲状态
endtask

// --- 串行接收任务 ---
task targetRcv(output bus_cmd_e c, logic [7:0] a, d);
 @(posedge start); // 等待 Start 信号变高（检测传输开始）
 c = cmd; // 采样命令

 // 循环遍历数组进行串行接收
 foreach(a[i]) begin
 @(posedge clk); // 等待时钟沿以同步采样
 a[i] = addr; // 采样地址位
 d[i] = data; // 采样数据位
 end
endtask

endinterface simple_if

```

发送端 (initiatorSend)：使用了 foreach(a[i]) 循环。这表明传入的参数 a 和 d 是数组。在循环内部有一个 @(posedge clk)。这是串行化的关键：每过一个时钟周期，才发送下一位数据。整个过程被包裹在 start 信号的高电平期间。接收端 (targetRcv)：首先阻塞在 @(posedge start)，这是一种典型的握手等待机制。只有当发送方发起传输时，接收方才被唤醒。随后进入与发送方完全同步的 foreach 循环，利用 @(posedge clk) 确保在正确的时间点采样数据。在 Task 内部：发送端使用非阻塞赋值 (<=) 来驱动信号，符合硬件仿真规范，避免竞



争冒险。接收端使用阻塞赋值（=）来将接口信号的值赋给局部变量（参数），这是为了立即获取当前时刻的信号值用于后续处理。

## 完整的SystemVerilog测试平台

这段代码展示了 SystemVerilog 验证环境中的\*\*顶层模块（Top Module）\*\*设计。它定义了一个名为

```
```systemverilog module top; logic clk,rst;
```

```
Rx_if Rx[4](clk);
Tx_if Tx[4](clk);

test t1(Rx,Tx,rst);
atm_router a1(Rx[0],Rx[1],Rx[2],Rx[3],
Tx[0],Tx[1],Tx[2],Tx[3],clk,rst);

initial begin
    clk = 0;
    forever #20 clk = ~clk;
end
endmodule:top
```

使用虚拟接口的驱动类

```
```systemverilog
class Driver;
 int stream_id;
 bit done = 0;
 mailbox exp_mbx;
 virtual Rx_if.TB Rx;

 function new(input mailbox exp_mbx, input int stream_id,input virtual Rx_if.TB
Rx);
 this.exp_mbx = exp_mbx;
 this.stream_id = stream_id;
 this.Rx = Rx;
 endfunction

 task run(input int ncells ,input event driver_done);
 ATM_Cell ac;

 fork //将此任务派生为一个独立的线程
 begin
 //初始化输出信号
 Rx.cb.clav<= 0;
 Rx.cb.soc<= 0;
 @Rx.cb;

 //驱动信元,直到发送最后一个信元
 repeat(ncells) begin
 ac=new
```

```

 assert(ac.randomize);
 if(ac.eot_cell) break;
 drive_cell(ac);
 end

 $display("@%0t Driver %0d done", $time, stream_id);
 ->driver_done;
end
join_none
endtask:run

task drive_cell(input ATM_Cell ac);
 bit [7:0] bytes[];

 #ac.delay;
 $display("@%0t Driver %0d sending cell %0d", $time, stream_id,
ac.cell_id);

 //等待下一个时钟周期

 @Rx.cb; // 等待时钟沿同步
 Rx.cb.clav <= 1; // 置位 xfr (Cell Available), 通知 DUT 有数据可
用
 do
 @Rx.cb; // 等待下一个时钟周期
 while(Rx.cb.en != 0); // 等待使能信号变低 (Wait for enable signal to
go low)
 // *注：此处逻辑取决于具体协议，通常是在等待 DUT
准备好接收或完成当前传输*

 Rx.cb.soc <= 1; // 信元的开始 (Start of Cell)
 Rx.cb.data <= bytes[0]; // 驱动第一个字节
 @Rx.cb; // 等待时钟沿
 Rx.cb.soc <= 0; // 信元传送完毕 (Start of Cell 结束)
 Rx.cb.data <= bytes[1]; // 驱动第二个字节 (注释误写为“第一个”，实为第二个)

 for(int i=2; i < 'ATM_SIZE; i++) begin
 @Rx.cb; // 每个字节传输都需要等待时钟沿
 Rx.cb.data <= bytes[i]; // 依次驱动后续字节
 end

 @Rx.cb; // 最后一个字节传输后的等待
 Rx.cb.soc <= 1'bz; // 结束时 SOC 置高阻态
 Rx.cb.clav <= 0; // 撤销 Cell Available 信号
 Rx.cb.data <= 8'bz; // 清除数据行，置高阻态

 $display("@%0d:Driver::drive_cell(%0d) finish",
 $time, stream_id); // 打印调试信息，显示完成时间和流 ID

 // 将信元送至记分板 (Scoreboard) 进行比对
 exp_mbx.put(ac); // 把预期数据放入 mailbox

```

```
endtask: drive_cell_t
endclass: Driver
```

将测试平台链接到端口列表中的接口

```
module top;
 bus_ifc bus(); //例化接口
 test t1(bus); //通过端口列表传递给测试程序
 dut d1(bus); //通过端口列表传递给被测模块
endmodule:top
```

```
program automatic test(bus_ifc bus);
 initial $display(bus.data);
endprogram
```

端口列表中含有第二个接口的顶层模块

```
module top;
 bus_ifc bus(); //例化接口
 new_ifc newb(); //再例化一个接口
 test t1(bus,newb); //使用两个接口的测试程序
 dut d1(bus,newb); //使用两个接口的被测模块
endmodule
```

使用XMR(跨模块引用)连接接口和测试程序

```
//- 使用虚接口和XMR的测试程序
program automatic test();
 virtual bus_ifc bus= top.bus; //跨模块引用
 initial $display(bus.data); //使用接口信号
endprogram
```

连接到多个不同的设计配置

8位计数器的接口

```
interface X_if(input logic clk);
 logic[7:0] din,dout;
 logic reset_l,load;

 clocking cb @(posedge clk);
 output din,load;
```

```

 input dout;
 endclocking

 always @ cb
 $strobe("@%0t %0d", $time, din);

 modport DUT(input clk,din,reset_l,load,output dout);
endinterface

```

### 使用X\_if接口的计数器模型

```

module dut(X_if.DUT xi);
 logic[7:0] count;
 assign xi.dout = count;

 always @(posedge xi.clk or negedge xi.reset_l)
 begin
 if (xi.reset_l)
 count <= 0;
 else if (xi.load)
 count <= xi.din;
 else
 count <= count + 1;
 end
endmodule

```

### 使用虚拟接口数组的测试平台

```

parameter NUM_XI = 2; // 设计实例的个数
parameter NUM_XI = 2;

module top;
 //时钟发生器
 bit clk;

 initial begin
 clk <= '0;
 forever #20 clk <= ~clk;
 end

 //例化N个接口
 X_if xi[NUM_XI] (clk);

 //例化测试平台
 test tb();

 //产生N个DUT实例
 generate
 for(genvar i=0; i < NUM_XI; i++)

```

```

 begin:dut_blk
 dut dut_i(xi[i]);
 end
 endgenerate
endmodule

```

## 使用虚拟接口的计数器测试平台

```

// 例 10.19 使用虚接口的计数器测试平台
program automatic test;

 // 1. 声明虚接口数组和 Driver 动态数组
 virtual X_if.TB vxi[NUM_XI]; // 虚接口数组
 Driver driver[];

 initial begin
 // 2. 将顶层物理接口连接到局部虚接口
 vxi = top.xi;

 // 3. 创建 N 个驱动器对象 (Driver)
 driver = new[NUM_XI];
 foreach(driver[i])
 driver[i] = new(vxi[i], i);

 // 4. 并发执行所有驱动器的任务
 foreach(driver[i]) begin
 int j = i; // 关键步骤: 复制循环变量
 fork
 begin
 driver[j].reset();
 driver[j].load_op();
 end
 join_none
 end

 // 5. 等待一段时间让任务运行
 repeat(10) @(vxi[0].cb);
 end

endprogram

```

## 使用虚接口的Driver类

```

// 例 10.20 使用虚接口的 Driver 类
class Driver;
 virtual X_if xi; // 声明虚接口句柄
 int id; // 驱动器 ID, 用于区分不同实例

 // 构造函数: 接收虚接口和 ID

```

```

function new(input virtual X_if.TB xi, input int id);
 this.xi = xi;
 this.id = id;
endfunction

// 复位任务
task reset();
 $display("@%0t:%m:Start reset[%0d]", $time, id);

 // 设备复位逻辑
 xi.reset_l <= 1; // 拉高复位信号（假设低电平有效，此处可能是释放复位或特定
协议）
 xi.cb.load <= 0; // 拉低 load 信号
 xi.cb.din <= 0; // 数据输入置 0
 @(xi.cb) xi.reset_l <= 0; // 等待时钟沿，拉低复位信号
 @(xi.cb) xi.reset_l <= 1; // 再等一个时钟沿，拉高复位信号（完成复位脉冲）

 $display("@%0t:%m:End reset[%0d]", $time, id);
endtask : reset

// 加载操作任务
task load_op();
 $display("@%0t:%m:Start load[%0d]", $time, id);

 #1 xi.cb.load <= 1; // 延时 1 个时间单位后，拉高 load 信号
 xi.cb.din <= id + 10; // 设置数据输入为 id + 10

 #1 xi.cb.load <= 0; // 延时 1 个时间单位后，拉低 load 信号
 repeat(5) @(xi.cb); // 等待 5 个时钟周期

 $display("@%0t:%m:End load[%0d]", $time, id);
endtask : load_op

endclass Driver

```

## 对虚接口使用typedef

```

typedef virtual x_if.TB vx_if;

program automatic test;
 vx_if vxi[NUM_XI]; //虚接口数组
 Driver driver[];

 ...
endprogram

```

## 使用端口传递虚接口的测试平台

```

parameter NUM_XI = 2; //实例个数

module top;
 //例化N个接口
 X_if xi[NUM_XI] (clk);

 //例化测试平台
 test tb(xi);
endmodule

```

## 使用端口传递虚接口测试平台

```

// 例 10.21 使用端口传递虚接口的测试平台
Driver driver[]; // 声明 Driver 动态数组
virtual X_if vxi[NUM_XI]; // 声明虚接口数组

initial begin
 // 将局部虚接口连到顶层
 if (NUM_XI <= 0) $finish; // 安全检查: 如果通道数为0则结束仿真

 driver = new[NUM_XI]; // 为 Driver 数组分配内存空间
 vxi = xi; // 把顶层物理接口数组赋值给本地虚接口数组

 for (int i=0; i < NUM_XI; i++) begin
 driver[i] = new(vxi[i], i); // 实例化第 i 个 Driver, 传入对应的虚接口和 ID
 driver[i].reset(); // 调用该 Driver 的复位任务
 end
 ...
end
endprogram

```

## 接口中的过程代码

### 并行协议接口

```

interface simple_if(input logic clk);
 // 1. 信号定义
 logic [7:0] addr;
 logic [7:0] data;
 bus_cmd_e cmd;

 // 2. 目标端口 (Target) - 面向从机或接收端
 modport TARGET (
 input addr, cmd, data,
 import task targetRcv(output bus_cmd_e c, logic[7:0] a, d)
);

```

```
// 3. 发起者端口 (Initiator) - 面向主机或发送端
modport INITIATOR (
 output addr, cmd, data,
 import task initiatorSend(input bus_cmd_e c, logic[7:0] a, d)
);

// 4. 并行发送任务 (由 Initiator 使用)
task initiatorSend(input bus_cmd_e c, logic[7:0] a, d);
 @(posedge clk); // 等待时钟上升沿
 cmd <= c; // 非阻塞赋值驱动命令
 addr <= a; // 非阻塞赋值驱动地址
 data <= d; // 非阻塞赋值驱动数据
endtask

// 5. 并行接收任务 (由 Target 使用)
task targetRcv(output bus_cmd_e c, logic[7:0] a, d);
 @(posedge clk); // 等待时钟上升沿采样
 a = addr; // 阻塞赋值采样地址
 d = data; // 阻塞赋值采样数据
 c = cmd; // 阻塞赋值采样命令
endtask

endinterface simple_if
```

## 串行接口代码

```
// 例 10.26 含有使用串行协议任务的接口
interface simple_if(input logic clk);
 logic addr;
 logic data;
 logic start = 0;
 bus_cmd_e cmd;

 // 定义 Target (从机/接收端) 端口视图
 modport TARGET (
 input addr, cmd, data,
 import task targetRcv(output bus_cmd_e c, logic [7:0] a, d)
);

 // 定义 Initiator (主机/发送端) 端口视图
 modport INITIATOR (
 output addr, cmd, data,
 import task initiatorSend(input bus_cmd_e c, logic [7:0] a, d)
);

 // --- 串行发送任务 ---
 task initiatorSend(input bus_cmd_e c, logic [7:0] a, d);
 @(posedge clk); // 等待时钟上升沿
 start <= 1; // 拉高 Start 信号, 开始传输
 cmd <= c; // 发送命令

 // 循环遍历数组进行串行发送
 endtask
endinterface
```



```

 foreach(a[i]) begin
 addr <= a[i]; // 发送地址位
 data <= d[i]; // 发送数据位
 @(posedge clk); // 等待下一个时钟沿（串行传输的核心）
 end

 start <= 0; // 拉低 Start 信号，结束传输
 cmd <= IDLE; // 命令恢复空闲状态
 endtask

 // --- 串行接收任务 ---
 task targetRcv(output bus_cmd_e c, logic [7:0] a, d);
 @(posedge start); // 等待 Start 信号变高（检测传输开始）
 c = cmd; // 采样命令

 // 循环遍历数组进行串行接收
 foreach(a[i]) begin
 @(posedge clk); // 等待时钟沿以同步采样
 a[i] = addr; // 采样地址位
 d[i] = data; // 采样数据位
 end
 endtask

endinterface simple_if

```

发送端 (initiatorSend): 使用了 `foreach(a[i])` 循环。这表明传入的参数 `a` 和 `d` 是数组。在循环内部有一个 `@(posedge clk)`。这是串行化的关键：每过一个时钟周期，才发送下一位数据。整个过程被包裹在 `start` 信号的高电平期间。接收端 (targetRcv): 首先阻塞在 `@(posedge start)`，这是一种典型的握手等待机制。只有当发送方发起传输时，接收方才被唤醒。随后进入与发送方完全同步的 `foreach` 循环，利用 `@(posedge clk)` 确保在正确的时间点采样数据。在 Task 内部：发送端使用非阻塞赋值 (`<=`) 来驱动信号，符合硬件仿真规范，避免竞争冒险。接收端使用阻塞赋值 (`=`) 来将接口信号的值赋给局部变量（参数），这是为了立即获取当前时刻的信号值用于后续处理。

## 完整的SystemVerilog测试平台

这段代码展示了 SystemVerilog 验证环境中的\*\*顶层模块（Top Module）\*\*设计。它定义了一个名为 `squat` 的 DUT（被测设计），并为其配置了时钟、复位以及多个 Utopia 接口实例，最后将 DUT 和测试平台（Testbench）连接在一起。

```

// 例 11.1 顶层模块
`timescale 1ns/1ns
`define TxPorts 4 // 发送端口的个数
`define RxPorts 4 // 接收端口的个数

module top;
 parameter int NumRx = `RxPorts;
 parameter int NumTx = `TxPorts;

 logic rst, clk;

```

```

// 系统时钟和复位
initial begin
 rst = 0; clk = 0;
 # 5ns rst = 1;
 # 5ns clk = 1;
 # 5ns rst = 0; clk = 0;
 forever
 # 5ns clk = ~clk;
end

// 接口实例化
Utopia Rx[0:NumRx-1] (); // NumRx 个 Level 1 Utopia Rx 接口
Utopia Tx[0:NumTx-1] (); // NumTx 个 Level 1 Utopia Tx 接口
cpu_ifc mif(); // Utopia management interface

// DUT 实例化
squat #(NumRx, NumTx) squat(Rx, Tx, mif, rst, clk);

// Testbench 实例化
test #(NumRx, NumTx) t1(Rx, Tx, mif, rst, clk);

endmodule top

```

## 测试平台程序

```

// 例 11.2 测试平台的程序
program automatic test
 #(parameter int NumRx = 4, parameter int NumTx = 4)
 (Utopia.TB_Rx Rx[0:NumRx-1],
 Utopia.TB_Tx Tx[0:NumTx-1],
 cpu_ifc.Test mif,
 input logic rst, clk);

 `include "environment.sv"
 Environment env;

 initial begin
 // 1. 创建环境对象
 env = new(Rx, Tx, NumRx, NumTx, mif);

 // 2. 标准验证流程
 env.gen_cfg(); // 生成配置
 env.build(); // 构建组件（如 Driver, Monitor 等）
 env.run(); // 运行仿真
 env.wrap_up(); // 结束处理与报告
 end

endprogram // test

```

测试平台通过管理接口来装载控制信息

## CPU管理接口

```

interface cpu_ifc;

 logic BusMode,Rd_Ds,Wr_Rw,Rdy_Dtack;
 logic [11:0] Addr;
 CellCfgType DataIn,DataOut;

 modport Peripheral(input BusMode,Addr,Sel,DataIn,Rd_DS,Wr_RW,output
DataOut,Rdy_Dtack);

 modport Test(output BusMode,Addr,Sel,DataIn,Bd_Ds,Wr_RW,input
DataOut,Rdy_Dtack);

 endinterface cpu_ifc
 typedef virtual cpu_ifc.Test vCPU_T;

```

## Utopia接口与待测设计squat进行通信，发送和接收ATM信元

```

interface Utopia;
 parameter int IfWidth = 8;

 logic [IfWidth-1:0] data;
 bit clk_in, clk_out;
 bit soc, en, clav, valid, ready, reset, selected;

 ATMCellType ATMCell; // ATM信元结构的联合

 // 顶层接收端口视图
 modport TopReceive (
 input data, soc, clav,
 output clk_in, reset, ready, clk_out, en, ATMCell, valid
);

 // 顶层发送端口视图
 modport TopTransmit (
 input clav,
 inout selected,
 output clk_in, clk_out, ATMCell, data, soc, en, valid,
 reset, ready
);

 // 核心接收端口视图
 modport CoreReceive (
 input clk_in, data, soc, clav, ready, reset,
 output clk_out, en, ATMCell, valid
);

 // 核心发送端口视图

```

```

modport CoreTransmit (
 input clk_in, clav, ATMCell, valid, reset,
 output clk_out, data, soc, en, ready
);

// 接收端测试平台时钟块
clocking cbr @ (negedge clk_out);
 input clk_in, clk_out, ATMCell, valid, reset, en, ready;
 output data, soc, clav;
endclocking : cbr
modport TB_Rx (clocking cbr);

// 发送端测试平台时钟块
clocking cbt @ (negedge clk_out);
 input clk_out, clk_in, ATMCell, soc, en, valid,
 reset, data, ready;
 output clav;
endclocking : cbt

modport TB_Tx (clocking cbt);

endinterface

// 虚拟接口类型定义
typedef virtual Utopia vUtopia;
typedef virtual Utopia.TB_Rx vUtopiaRx;
typedef virtual Utopia.TB_Tx vUtopiaTx;

```

## 测试平台的模块environment

```

// 例 11.5 Environment 类的首部
class Environment;

 // --- 组件声明 ---
 UNI_generator gen[]; // 激励生成器数组
 mailbox gen2drv[]; // 生成器到驱动器的信箱（用于传输事务）
 event drv2gen[]; // 驱动器到生成器的事件（用于握手/同步）
 Driver drv[]; // 驱动器数组
 Monitor mon[]; // 监视器数组
 Config cfg; // 配置对象
 Scoreboard scb; // 记分板（用于数据比对）
 Coverage cov; // 覆盖率收集组件

 // --- 虚拟接口声明 ---
 virtual Utopia.TB_Rx Rx[]; // 接收端口的虚拟接口数组
 virtual Utopia.TB_Tx Tx[]; // 发送端口的虚拟接口数组

 // --- 内部变量 ---
 int numRx, numTx; // 记录端口数量
 vCPU_T mif; // CPU 管理接口的虚拟接口句柄
 CPU_driver cpu; // CPU 驱动器

```

```
// --- 方法声明 (extern 表示定义在类外部) ---

// 构造函数
extern function new(
 input vUtopiaRx Rx[],
 input vUtopiaTx Tx[],
 input int numRx, numTx,
 input vCPU_T mif
);

// 核心生命周期方法
extern virtual function void gen_cfg(); // 生成配置
extern virtual function void build(); // 构建环境 (实例化组件)
extern virtual task run(); // 运行仿真 (启动各组件)
extern virtual task wrap_up(); // 结束处理 (检查结果、打印报告)

endclass Environment
```

## Environment类的方法

### //构造Environment类的方法

```
// -----
// 构造 environment 实例
function Environment::new(input vUtopiaRx Rx[],
 input vUtopiaTx Tx[],
 input int numRx, numTx,
 input vCPU_T mif);

// 动态数组的内存分配与赋值
this.Rx = new[Rx.size()];
foreach (Rx[i]) this.Rx[i] = Rx[i];

this.Tx = new[Tx.size()];
foreach (Tx[i]) this.Tx[i] = Tx[i];

// 标量参数赋值
this.numRx = numRx;
this.numTx = numTx;
this.mif = mif;

// 创建配置对象
cfg = new(numRx, numTx);

// 处理随机种子 (Random Seed)
if ($test$plusargs("ntb_random_seed")) begin
 int seed;
 $value$plusargs("ntb_random_seed=%d", seed);
 $display("Simulation run with random seed=%0d", seed);
end
else
```

```

 $display("Simulation run with default random seed");

endfunction : new

// -----
// 随机化配置描述符
function void Environment::gen_cfg();
 assert(cfg.randomize());
 cfg.display();
endfunction : gen_cfg

// 这样可以避免空句柄错误。
function void Environment::build();
 // 1. 实例化基础组件
 cpu = new(mif, cfg);
 gen = new[numRx];
 drv = new[numRx];
 gen2drv = new[numRx];
 drv2gen = new[numRx];
 scb = new(cfg);
 cov = new();

 // 建立发生器 (Generators)
 foreach (gen[i]) begin
 gen2drv[i] = new();
 gen[i] = new(gen2drv[i], drv2gen[i],
 cfg.cells_per_chan[i], i);
 drv[i] = new(gen2drv[i], drv2gen[i], Rx[i], i);
 end

 // 建立监视器 (Monitors)
 mon = new[numTx];
 foreach (mon[i])
 mon[i] = new(Tx[i], i);

 // 通过回调函数连接记分板到驱动器和监视器
 begin
 Scb_Driver_cbs sdc = new(scb);
 Scb_Monitor_cbs smc = new(scb);
 foreach (drv[i]) drv[i].cbsq.push_back(sdc);
 foreach (mon[i]) mon[i].cbsq.push_back(smc);
 end

 // 通过回调函数连接覆盖率程序到监视器
 begin
 Cov_Monitor_cbs smc = new(cov);
 foreach (mon[i])
 mon[i].cbsq.push_back(smc);
 end
endfunction : build

// -----
// 启动事务:发生器、驱动器、监视器
// 不会启动没有使用的通道

```

```

task Environment::run();
 int num_gen_running;

 // CPU接口必须最先初始化
 cpu.run();

 num_gen_running = numRx;

 // 为每个 Rx 接收通道启动发生器和驱动器
 foreach (gen[i]) begin
 int j = i; // 在交换出的线程里,自动变量保持了索引值
 fork
 begin
 if (cfg.in_use_Rx[j])
 gen[j].run(); // 等待发生器结束
 num_gen_running--; // 减少驱动器的个数
 end
 if (cfg.in_use_Rx[j]) drv[j].run();
 join_none
 end

 // 为每个 Tx 输出通道启动监视器
 foreach (mon[i]) begin
 int j=i; // 在交换出的线程里,自动变量保持了索引值
 fork
 mon[j].run();
 join_none
 end

 // 等待所有的发生器结束或超时
 fork : timeout_block
 wait (num_gen_running == 0);
 begin
 repeat (1_000_000) @(Rx[0].cbr);
 $display("%0t: %m ERROR: Generator timeout ", $time);
 cfg.nErrors++;
 end
 join_any
 disable timeout_block;

 // 等待数据送到监视器和记分板
 repeat (1_000) @(Rx[0].cbr);
 endtask : run

 // -----
 // 运行结束后的清除/报告工作
 function void Environment::wrap_up();
 $display("%0t: End of sim, %0d errors, %0d warnings",
 $time, cfg.nErrors, cfg.nWarnings);

 scb.wrap_up;
 endfunction : wrap_up

```

这段代码展示了 SystemVerilog 验证环境中 ``systemverilog // 例 11.1 顶层模块 timescale 1ns/1ns define TxPorts 4 // 发送端口的个数 `define RxPorts 4 // 接收端口的个数

```
module top; parameter int NumRx = RxPorts; parameter int NumTx = TxPorts;
```

```
 logic rst, clk;

 // 系统时钟和复位
 initial begin
 rst = 0; clk = 0;
 # 5ns rst = 1;
 # 5ns clk = 1;
 # 5ns rst = 0; clk = 0;
 forever
 # 5ns clk = ~clk;
 end

 // 接口实例化
 Utopia Rx[0:NumRx-1] (); // NumRx 个 Level 1 Utopia Rx 接口
 Utopia Tx[0:NumTx-1] (); // NumTx 个 Level 1 Utopia Tx 接口
 cpu_ifc mif(); // Utopia management interface

 // DUT 实例化
 squat #(NumRx, NumTx) squat(Rx, Tx, mif, rst, clk);

 // Testbench 实例化
 test #(NumRx, NumTx) t1(Rx, Tx, mif, rst, clk);
```

```
endmodule top
```

测试平台程序

```
``systemverilog
// 例 11.2 测试平台的程序
program automatic test
 #(parameter int NumRx = 4, parameter int NumTx = 4)
 (Utopia.TB_Rx Rx[0:NumRx-1],
 Utopia.TB_Tx Tx[0:NumTx-1],
 cpu_ifc.Test mif,
 input logic rst, clk);

 `include "environment.sv"
 Environment env;

 initial begin
 // 1. 创建环境对象
 env = new(Rx, Tx, NumRx, NumTx, mif);
```



```

// 2. 标准验证流程
env.gen_cfg(); // 生成配置
env.build(); // 构建组件（如 Driver, Monitor 等）
env.run(); // 运行仿真
env.wrap_up(); // 结束处理与报告

end
endprogram // test

```

测试平台通过管理接口来装载控制信息

CPU管理接口

```

interface cpu_ifc;

 logic BusMode,Rd_Ds,Wr_Rw,Rdy_Dtack;
 logic [11:0] Addr;
 CellCfgType DataIn,DataOut;

 modport Peripheral(input BusMode,Addr,Sel,DataIn,Rd_DS,Wr_RW,output
DataOut,Rdy_Dtack);

 modport Test(output BusMode,Addr,Sel,DataIn,Bd_Ds,Wr_RW,input
DataOut,Rdy_Dtack);

 endinterface cpu_ifc
 typedef virtual cpu_ifc.Test vCPU_T;

```

Utopia接口与待测设计squat进行通信，发送和接收ATM信元

```

interface Utopia;
 parameter int IfWidth = 8;

 logic [IfWidth-1:0] data;
 bit clk_in, clk_out;
 bit soc, en, clav, valid, ready, reset, selected;

 ATMCellType ATMCell; // ATM信元结构的联合

 // 顶层接收端口视图
 modport TopReceive (
 input data, soc, clav,
 output clk_in, reset, ready, clk_out, en, ATMCell, valid
);

 // 顶层发送端口视图
 modport TopTransmit (
 input clav,
 inout selected,

```

```

 output clk_in, clk_out, ATMCell, data, soc, en, valid,
 reset, ready
);

 // 核心接收端口视图
 modport CoreReceive (
 input clk_in, data, soc, clav, ready, reset,
 output clk_out, en, ATMCell, valid
);

 // 核心发送端口视图
 modport CoreTransmit (
 input clk_in, clav, ATMCell, valid, reset,
 output clk_out, data, soc, en, ready
);

 // 接收端测试平台时钟块
 clocking cbr @ (negedge clk_out);
 input clk_in, clk_out, ATMCell, valid, reset, en, ready;
 output data, soc, clav;
 endclocking : cbr
 modport TB_Rx (clocking cbr);

 // 发送端测试平台时钟块
 clocking cbt @ (negedge clk_out);
 input clk_out, clk_in, ATMCell, soc, en, valid,
 reset, data, ready;
 output clav;
 endclocking : cbt

 modport TB_Tx (clocking cbt);

endinterface

// 虚拟接口类型定义
typedef virtual Utopia vUtopia;
typedef virtual Utopia.TB_Rx vUtopiaRx;
typedef virtual Utopia.TB_Tx vUtopiaTx;

```

## 测试平台的模块environment

```

// 例 11.5 Environment 类的首部
class Environment;

 // --- 组件声明 ---
 UNI_generator gen[]; // 激励生成器数组
 mailbox gen2drv[]; // 生成器到驱动器的信箱（用于传输事务）
 event drv2gen[]; // 驱动器到生成器的事件（用于握手/同步）
 Driver drv[]; // 驱动器数组
 Monitor mon[]; // 监视器数组
 Config cfg; // 配置对象
 Scoreboard scb; // 记分板（用于数据比对）

```

```

Coverage cov; // 覆盖率收集组件

// --- 虚拟接口声明 ---
virtual Utopia.TB_Rx Rx[]; // 接收端口的虚拟接口数组
virtual Utopia.TB_Tx Tx[]; // 发送端口的虚拟接口数组

// --- 内部变量 ---
int numRx, numTx; // 记录端口数量
vCPU_T mif; // CPU 管理接口的虚拟接口句柄
CPU_driver cpu; // CPU 驱动器

// --- 方法声明 (extern 表示定义在类外部) ---

// 构造函数
extern function new(
 input vUtopiaRx Rx[],
 input vUtopiaTx Tx[],
 input int numRx, numTx,
 input vCPU_T mif
);

// 核心生命周期方法
extern virtual function void gen_cfg(); // 生成配置
extern virtual function void build(); // 构建环境（实例化组件）
extern virtual task run(); // 运行仿真（启动各组件）
extern virtual task wrap_up(); // 结束处理（检查结果、打印报告）

endclass Environment

```

## Environment类的方法

### //构造Environment类的方法

```

// -----
// 构造 environment 实例
function Environment::new(input vUtopiaRx Rx[],
 input vUtopiaTx Tx[],
 input int numRx, numTx,
 input vCPU_T mif);

// 动态数组的内存分配与赋值
this.Rx = new[Rx.size()];
foreach (Rx[i]) this.Rx[i] = Rx[i];

this.Tx = new[Tx.size()];
foreach (Tx[i]) this.Tx[i] = Tx[i];

// 标量参数赋值
this.numRx = numRx;
this.numTx = numTx;
this.mif = mif;

```

```

// 创建配置对象
cfg = new(numRx, numTx);

// 处理随机种子 (Random Seed)
if ($test$plusargs("ntb_random_seed")) begin
 int seed;
 $value$plusargs("ntb_random_seed=%d", seed);
 $display("Simulation run with random seed=%0d", seed);
end
else
 $display("Simulation run with default random seed");

endfunction : new

// -----
// 随机化配置描述符
function void Environment::gen_cfg();
 assert(cfg.randomize());
 cfg.display();
endfunction : gen_cfg

// 这样可以避免空句柄错误。
function void Environment::build();
 // 1. 实例化基础组件
 cpu = new(mif, cfg);
 gen = new[numRx];
 drv = new[numRx];
 gen2drv = new[numRx];
 drv2gen = new[numRx];
 scb = new(cfg);
 cov = new();

 // 建立发生器 (Generators)
 foreach (gen[i]) begin
 gen2drv[i] = new();
 gen[i] = new(gen2drv[i], drv2gen[i],
 cfg.cells_per_chan[i], i);
 drv[i] = new(gen2drv[i], drv2gen[i], Rx[i], i);
 end

 // 建立监视器 (Monitors)
 mon = new[numTx];
 foreach (mon[i])
 mon[i] = new(Tx[i], i);

 // 通过回调函数连接记分板到驱动器和监视器
 begin
 Scb_Driver_cbs sdc = new(scb);
 Scb_Monitor_cbs smc = new(scb);
 foreach (drv[i]) drv[i].cbsq.push_back(sdc);
 foreach (mon[i]) mon[i].cbsq.push_back(smc);
 end
end

```

```

// 通过回调函数连接覆盖率程序到监视器
begin
 Cov_Monitor_cbs smc = new(cov);
 foreach (mon[i])
 mon[i].cbsq.push_back(smc);
end
endfunction : build

// -----
// 启动事务:发生器、驱动器、监视器
// 不会启动没有使用的通道
task Environment::run();
 int num_gen_running;

 // CPU接口必须最先初始化
 cpu.run();

 num_gen_running = numRx;

 // 为每个 Rx 接收通道启动发生器和驱动器
 foreach (gen[i]) begin
 int j = i; // 在交换出的线程里,自动变量保持了索引值
 fork
 begin
 if (cfg.in_use_Rx[j])
 gen[j].run(); // 等待发生器结束
 num_gen_running--; // 减少驱动器的个数
 end
 if (cfg.in_use_Rx[j]) drv[j].run();
 join_none
 end

 // 为每个 Tx 输出通道启动监视器
 foreach (mon[i]) begin
 int j=i; // 在交换出的线程里,自动变量保持了索引值
 fork
 mon[j].run();
 join_none
 end

 // 等待所有的发生器结束或超时
 fork : timeout_block
 wait (num_gen_running == 0);
 begin
 repeat (1_000_000) @(Rx[0].cbr);
 $display("%0t: %m ERROR: Generator timeout ", $time);
 cfg.nErrors++;
 end
 join_any
 disable timeout_block;

 // 等待数据送到监视器和记分板
 repeat (1_000) @(Rx[0].cbr);
 endtask : run

```

```
// -----
// 运行结束后的清除/报告工作
function void Environment::wrap_up();
 $display("%0t: End of sim, %0d errors, %0d warnings",
 $time, cfg.nErrors, cfg.nWarnings);

 scb.wrap_up;
endfunction : wrap_up
```

这段代码展示了 SystemVerilog 验证环境中 `Environment` 类的核心运行逻辑（`run`）和收尾工作（`wrap_up`）。它通过并发控制、超时机制和计数器来管理整个仿真流程，确保测试有序进行并在异常时能够自动终止。

```
// -----
// 启动事务:发生器、驱动器、监视器
// 不会启动没有使用的通道
task Environment::run();
 int num_gen_running;

 // CPU接口必须最先初始化
 cpu.run();

 num_gen_running = numRx;

 // 为每个 Rx 接收通道启动发生器和驱动器
 foreach (gen[i]) begin
 int j = i; // 在交换出的线程里,自动变量保持了索引值
 fork
 begin
 if (cfg.in_use_Rx[j])
 gen[j].run(); // 等待发生器结束
 num_gen_running--; // 减少驱动器的个数
 end
 if (cfg.in_use_Rx[j]) drv[j].run();
 join_none
 end

 // 为每个 Tx 输出通道启动监视器
 foreach (mon[i]) begin
 int j=i; // 在交换出的线程里,自动变量保持了索引值
 fork
 mon[j].run();
 join_none
 end

 // 等待所有的发生器结束或超时
 fork : timeout_block
 wait (num_gen_running == 0);
 begin
 repeat (1_000_000) @(Rx[0].cbr);
 end
 endfork
 end
endtask
```

```

 $display("%0t: %m ERROR: Generator timeout ", $time);
 cfg.nErrors++;
 end
 join_any
 disable timeout_block;

 // 等待数据送到监视器和记分板
 repeat (1_000) @(Rx[0].cbr);
endtask : run

// -----
// 运行结束后的清除/报告工作
function void Environment::wrap_up();
 $display("%0t: End of sim, %0d errors, %0d warnings",
 $time, cfg.nErrors, cfg.nWarnings);

 scb.wrap_up;
endfunction : wrap_up

```

代码详细解析 任务 ``systemverilog // ----- // 启动事务:发生器、驱动器、监视器 // 不会启动没有使用的通道 task Environment::run(); int num\_gen\_running;

```

// CPU接口必须最先初始化
cpu.run();

num_gen_running = numRx;

// 为每个 Rx 接收通道启动发生器和驱动器
foreach (gen[i]) begin
 int j = i; // 在交换出的线程里,自动变量保持了索引值
 fork
 begin
 if (cfg.in_use_Rx[j])
 gen[j].run(); // 等待发生器结束
 num_gen_running--; // 减少驱动器的个数
 end
 if (cfg.in_use_Rx[j]) drv[j].run();
 join_none
 end

// 为每个 Tx 输出通道启动监视器
foreach (mon[i]) begin
 int j=i; // 在交换出的线程里,自动变量保持了索引值
 fork
 mon[j].run();
 join_none
end

// 等待所有的发生器结束或超时
fork : timeout_block

```

```

 wait (num_gen_running == 0);
 begin
 repeat (1_000_000) @(Rx[0].cbr);
 $display("%0t: %m ERROR: Generator timeout ", $time);
 cfg.nErrors++;
 end
join_any
disable timeout_block;

// 等待数据送到监视器和记分板
repeat (1_000) @(Rx[0].cbr);

```

endtask : run

```

// ----- // 运行结束后的清除/报告工作 function void
Environment::wrap_up(); $display("%0t: End of sim, %0d errors, %0d warnings", $time, cfg.nErrors,
cfg.nWarnings);

```

```
scb.wrap_up;
```

endfunction : wrap\_up

---

### 代码详细解析

任务 `run()` — 仿真主控流程

这个任务是验证环境的“引擎”，负责按顺序启动各个组件并管理并发执行。

- **CPU 初始化**：首先调用 `cpu.run()`，通常用于配置 DUT 的寄存器或设置初始状态，这是后续数据传输的前提。
- **动态数组与 `fork...join\_none`**：
  - 使用 `foreach` 遍历所有通道。
  - **关键技巧 `int j = i;`**：在 `fork` 块内部，循环变量 `i` 是共享的。为了防止并发线程在执行时 `i` 的值已经改变（导致所有线程操作同一个错误的索引），代码定义了一个局部变量 `j` 来捕获当前的 `i` 值。这是 SystemVerilog 并发编程的标准做法。
  - **join\_none**：表示父进程不需要等待子进程（Generator 或 Monitor）结束即可继续执行。这使得所有的 Generator 和 Monitor 能够同时并行运行。
- **条件启动**：通过 `if (cfg.in\_use\_Rx[j])` 检查配置，只启动被启用的通道，避免资源浪费。
- **超时保护机制 (`timeout\_block`)**：
  - 这是一个非常健壮的设计。它创建了一个名为 `timeout\_block` 的命名块。
  - 使用 `fork...join\_any`：只要以下两个条件中\*\*任意一个\*\*满足，仿真就会继续往下走：
    1. `wait (num\_gen\_running == 0)`：所有的 Generator 都正常跑完了（计数器归零）。
    2. `repeat (1\_000\_000) ...`：如果 Generator 卡死或陷入死循环，经过 100 万个时钟周期后，强制触发超时错误。
  - **disable timeout\_block**：一旦 `join\_any` 触发（无论是正常结束还是超时），立即禁用该块，防止超时的分支在后台继续运行报错。



- **\*\*收尾延迟\*\***: 最后的 ``repeat (1_000)`` 是为了给 `Monitor` 和 `Scoreboard` 留出时间处理最后的一批数据，防止仿真结束时数据还没比对完。
- 函数 ``wrap_up()`` — 报告与清理
- **\*\*打印摘要\*\***: 在仿真结束时，打印总耗时、错误数和警告数，让工程师一眼看出测试结果。
- **\*\*Scoreboard 检查\*\***: 调用 ``scb.wrap_up``，通常用于检查 `Scoreboard` 内部是否还有未匹配的数据（即“漏网之鱼”），确保数据完整性。

回调类连接了驱动器和计分板

```
```systemverilog
class Scb_Driver_cbs extends Driver_cbs;
    Scoreboard scb;

    function new(input Scoreboard scb);
        this.scb = scb;
    endfunction : new

    //把收集到的信元发送到计分板
    virtual task post_tx(input Driver drv,
        input UNI_cell cell);
        scb.save_expected(cell);
    endtask : post_tx
endclass Scb_Driver_cbs
```

回调类连接了监视器和计分板

```
class Scb_Monitor_cbs extends Monitor_cbs;

    Scoreboard scb;
    function new(input Scoreboard scb);
        this.scb = scb;
    endfunction : new

    //把收到的信元发送到计分板
    virtual task post_rx(input Monitor mon,input NNI_cell cell);
        scb.check_actual(cell,mon,PortID);
    endtask : post_rx
endclass Scb_Monitor_cbs
```

`environment`通过`Cov_Monitor_cbs`回调类连接了监视器和覆盖率类

```
class Cov_Monitor_cbs extends Monitor_cbs;
    Coverage cov;

    function new(input Coverage cov);
```

```

        this.cov = cov;
    endfunction : new

    // 把收到的信元发送到覆盖率类
    virtual task post_rx( input Monitor mon,
                        input NNI_cell cell);

        CellCfgType CellCfg = top.squat.lut.read(cell.VPI);
        cov.sample(mon.PortID, CellCfg.FWD);
    endtask : post_rx

endclass Cov_Monitor_cbs

```

```

class Config;
    // 错误和警告的个数
    int nErrors, nWarnings;
    // 把参数复制一份
    bit [31:0] numRx, numTx;

    // 信元的总数 (随机变量)
    rand bit [31:0] nCells;

    // 约束: 信元数必须大于0
    constraint c_nCells_valid {
        nCells > 0;
    }

    // 约束: 限制参与测试的信元个数小于 1000 个
    constraint c_nCells_reasonable {
        nCells < 1000;
    }

    // 允许使用的输入/输出通道 (随机动态数组)
    rand bit in_use_Rx[];

    // 约束: 至少需要一个 RX 通道被使用
    constraint c_in_use_valid {
        in_use_Rx.sum() > 0;
    }

    // 每个通道的信元数 (随机动态数组)
    rand bit [31:0] cells_per_chan[];

    // 约束: 把信元分配到各个通道 (各通道信元总数等于 nCells)
    constraint c_sum_ncells_sum {
        cells_per_chan.sum() == nCells;
    }

    // 约束: 把未使用的通道的信元个数设为 0
    constraint zero_unused_channels {
        foreach (cells_per_chan[i]) {

```

```

// 在均匀分布时，先求解 in_use_Rx[]
solve in_use_Rx[i] before cells_per_chan[i];

if (in_use_Rx[i])
    cells_per_chan[i] inside {[1:nCells]};
else
    cells_per_chan[i] == 0;
}
}

extern function new(input bit [31:0] numRx, numTx);
extern virtual function void display(input string prefix="");
endclass Config

```

总量控制 (nCells) 首先确定这次仿真总共要发送多少个信元 (ATM Cell)。约束 c_nCells_valid 和 c_nCells_reasonable 保证数量在 1 到 999 之间，避免仿真时间过长或过短。通道激活 (in_use_Rx) 这是一个位宽为 1 的动态数组。如果 in_use_Rx[0] 为 1，表示第 0 号端口被激活；为 0 则表示关闭。约束 c_in_use_valid 确保不会发生所有端口都关闭的情况（即总和必须大于 0）。流量分配 (cells_per_chan) 这是最复杂的部分。它定义了一个数组，存储每个端口具体要发多少数据。全局守恒：cells_per_chan.sum() == nCells 保证了所有端口发出的数据加起来正好等于总任务量。局部逻辑：zero_unused_channels 约束配合 solve...before 语句起到了关键作用：它告诉求解器：先决定哪些端口是开着的 (in_use_Rx)，再决定每个端口发多少数据 (cells_per_chan)。如果端口没开 (in_use_Rx[i] == 0)，则该端口的信元数强制为 0。如果端口开了，则信元数在 1 到总数之间随机

信元配置类型

```

typedef struct packed{
    bit ['TxPorts-1:0] FWD;
    bit [11:0] VPI;
}CellCfgType;

function Config::new(input bit [31:0] numRx, numTx);
    this.numRx = numRx;
    this.numTx = numTx;
    int_use_Rx = new[numRx];
    cells_per_chan = new[numRx];
endfunction : new

function void Config::display(input string prefix);
    $write("%sConfig:numRx = %0d, numTx = %0d, nCells = %0d\n{",
        prefix, numRx, numTx, nCells);

    foreach(cells_per_chan[i])
        $write("%0d", cells_per_chan[i]);
    $write("),enabled Rx:", prefix);
    foreach(in_use_Rx[i]) if(in_use_Rx[i]) $write("%0d", i);
    $display;
endfunction : display

```

ATM交换机接收UNI格式的信元,发送NNI格式的信元

这些信元经过基于OOP的测试平台和结构化的设计,所以使用typedef定义

UNI信元格式

```
typedef struct packed{
    bit    [3:0]    GFC;
    bit    [7:0]    VPI;
    bit    [15:0]   VCI;
    bit          CLP;
    bit    [2:0]    PT;
    bit    [7:0]    HEC;
    bit    [0:47] [7:0] payload;
}uniType;
```

NNI信元格式

```
typedef struct packed{
    bit [11:0] VPI;
    bit [15:0] VCI;
    bit          CLP;
    bit [2:0]    PT;
    bit [7:0]    HEC;
    bit [0:47] [7:0] payload;
}nniType;
```

ATM信元类型

```
typedef struct packed{
    uniType uni;
    nniType nni;
    bit [0:52][7:0] Mem;
}ATM_cellType;
```

UNI_cell扩展定义的BaseTr类

```
class UNI_cell extends BaseTr;
    // 物理域
    rand bit    [3:0]    GFC;
    rand bit    [7:0]    VPI;
    rand bit    [15:0]   VCI;
    rand bit          CLP;
    rand bit    [2:0]    PT;
```

```

        bit      [7:0]   HEC;
    rand bit [0:47] [7:0]   Payload;

    // 数据域
    static bit [7:0] syndrome[0:255];
    static bit syndrome_not_generated=1;

    extern function new();
    extern function void post_randomize();
    extern virtual function bit compare(input BaseTr to);
    extern virtual function void display(input string prefix="");
    extern virtual function void copy_data(input UNI_cell copy);
    extern virtual function BaseTr copy(input BaseTr to=null);
    extern virtual function void pack(output ATMCellType to);
    extern virtual function void unpack(input ATMCellType from);
    extern function NNI_cell to_NNI();
    extern function void generate_syndrome();

    extern function bit [7:0] hec (bit [31:0] hdr);
endclass UNI_cell

function UNI_cell::new();
    if(syndrome_not_generated)
        generate_syndrome();
endfunction : new

//在所有其他数据都确定后计算HEC
function void UNI_cell::post_randomize();
    HEC = hec(GFC, VPI, VCI, CLP, PT, Payload);
endfunction : post_randomize

//和其他信元进行比较
function bit UNI_cell::compare(input BaseTr to);
    UNI_cell cell;
    $cast(cell,to);
    if(GFC != cell.GFC) return 0;
    if(VPI != cell.VPI) return 0;
    if(VCI != cell.VCI) return 0;
    if(CLP != cell.CLP) return 0;
    if(PT != cell.PT) return 0;
    if(HEC != cell.HEC) return 0;
    if(Payload != cell.Payload) return 0;
    return 1;
endfunction : compare

//复制信元的详细内容
function void UNI_cell::copy_data(input string prefix);
    ATMCellType p;
    $display("%sUNI_cell:GFC = %0d, VPI = %0d, VCI = %0d, CLP = %0d, PT = %0d, HEC
= %0d, Payload = %0d",
        prefix, GFC, VPI, VCI, CLP, PT, HEC, Payload);
    foreach(p.Mem)
        $display
endfunction : display

```

```
//复制信元的数据域
function void UNI_cell::copy_data(input UNI_cell copy);
    GFC = copy.GFC;
    VPI = copy.VPI;
    VCI = copy.VCI;
    CLP = copy.CLP;
    PT = copy.PT;
    HEC = copy.HEC;
    Payload = copy.Payload;
endfunction : copy_data

//复制对象
function BaseTr UNI_cell::copy(input BaseTr to=null);
    UNI_cell cell;
    if(to == null) to = new UNI_cell();
    $cast(cell,to);
    cell.copy_data(this);
    return to;
endfunction : copy

//把对象打包到一个字节数组
function void UNI_cell::pack(output ATMCellType to);
    to.uni.GFC = GFC;
    to.uni.VPI = VPI;
    to.uni.VCI = VCI;
    to.uni.CLP = CLP;
    to.uni.PT = PT;
    to.uni.HEC = HEC;
    to.uni.Payload = Payload;
endfunction : pack

//把字节数组的内容按域展开到this对象
function void UNI_cell::unpack(input ATMCellType from);
    this.GFC = from.uni.GFC;
    this.VPI = from.uni.VPI;
    this.VCI = from.uni.VCI;
    this.CLP = from.uni.CLP;
    this.PT = from.uni.PT;
    this.HEC = from.uni.HEC;
    this.Payload = from.uni.Payload;
endfunction : unpack

//根据UNI信元产生NNI信元
function NNI_cell UNI_cell::to_NNI();
    NNI_cell copy;
    $cast(copy,this);
    copy = new();
    copy.VPI = this.VPI;
    copy.VCI = this.VCI;
    copy.CLP = this.CLP;
    copy.PT = this.PT;
    copy.HEC = this.HEC;
    copy.Payload = this.Payload;
```

```

    return copy;
endfunction : to_NNI

//产生用于计算HEC的syndrome数组
function void UNI_cell::generate_syndrome();
    bit [7:0] sndrm;
    for(int i = 0; i < 256; i++) begin
        sndrm = i;
        repeat(8) begin
            if(sndrm[7] == 1'b1)
                sndrm = (sndrm << 1) ^ 8'h07;
            else
                sndrm = sndrm << 1;
            end
        syndrome_not_generated = 0;
    endfunction : generate_syndrome

//计算对象的HEC
function bit [7:0] hec (bit [31:0] hdr);
    bit [7:0] hec;
    hec = 0;
    repeat(4) begin
        hec = (hec << 1) ^ (hdr[7:0]);
        hdr = hdr >> 1;
    end
    return hec;
endfunction : hec

```

UNI信元的随机发生器,产生一个UNI类型的blueprint随机信元,然后把它的副本发送给driver

```

class UNI_cell_gen extends Generator;
    UNI_cell blueprint; //BluePrint信元
    mailbox gen2drv; //driver的Mailbox
    event drv2gen; //driver完成时的事件
    int nCells; //要产生 信元个数
    int PortID; //产生哪个Rx端口的信元

    function new(input mailbox gen2drv,
        input event drv2gen,
        input int nCells,PortID);
        this.gen2drv = gen2drv;
        this.drv2gen = drv2gen;
        this.nCells = nCells;
        this.PortID = PortID;
        blueprint = new();
    endfunction : new

    task run();
        UNI_cell cell;
        repeat(nCells) begin
            assert(blueprint.randomize());

```

```

        $cast(cell,blueprint.copy());
        cell.display(%psprintf("UNI_cell_gen:UNI_cell %0d",nCells));
        gen2drv.put(cell);
        @drv2gen; //等待driver完成
    end
endtask : run
endclass UNI_generator

```

Driver类把UNI信元发送到ATM交换机，Driver类使用了回调任务

```

typedef class Driver_cbs;

class Driver;
    mailbox gen2drv; //用于存储器发送的信元
    event drv2gen; //通知发生器已经处理完毕
    vUtopiaRx Rx; //发送信元的虚IFC
    Driver_cbs cbsq[$]; //回调对象的队列
    int PortID;

    extern function new(input mailbox gen2drv,
                        input event drv2gen,
                        input vUtopiaRx Rx,
                        input int PortID);

    extern task run();
    extern task send(input UNI_cell cell);

endclass Driver

//new() 构造driver对象
//获取发生器的实物,发送给DUT
task Driver::run();
    UNI_cell cell;
    bit drop = 0;
    //初始化端口
    Rx.cbr.data<= 0;
    Rx.cbr.soc <= 0;
    Rx.cbr.clav<=0;

    forever begin
        //从mailbox队列中读取一个信元
        gen2drv.peek(cell);
        begin : Tx
            //发送前的回调
            foreach(cbsq[i]) begin
                cbsq[i].pre_tx(this,cell);
            end

            gen2drv.get(cell); //从mailbox中删除该信元
            ->drv2gen; //通知发生器该信元处理完毕
        end
        //发送后的回调
    endtask:run

```



```

//send():把信元发送给DUT
task Driver::send(input UNI_cell cell);
    ATMCellType pkt;
    cell.pack(Pkt);
    $write("Sending cell:");
    foreach(Pkt.Mem[i])
        $write("%0d",Pkt.Mem[i]);
    //遍历整个信元
    @(Rx.cbr);
    Rx.cbr.clav<=1;
    for(int i = 0; i <= 52; i++) begin

        //置位信元开始信号,使能信号,发送字节0
        Rx.cbr.soc <= (i == 0);
        Rx.cbr.data<= Pkt.Mem[i];
        @(Rx.cbr);
    end
    Rx.cbr.soc <= 'z;
    Rx.cbr.data <= 8'bx;

    Rx.cbr.clav<=0;
endtask

```

Driver回调类分别在信元发送前和发送后调用回调任务

```

typedef class Driver;

class Driver_cbs;
    virtual task pre_tx(input Driver drv,
                        input UNI_cell cell,
                        inout bit drop);

    endtask : pre_tx

    virtual task post_tx(input Driver drv,
                        input UNI_cell cell);

    endtask : post_tx
endclass Driver_cbs

```

Monitor回调类

```

typedef class Monitor_cbs;

class Monitor;
    vUtopiaTx Tx; //连接DUT输出的虚拟接口
    Monitor_cbs cbsq[$]; //回调对象的队列
    int PortID;

    extern function new(input vUtopiaTx Tx,input int PortID);

```

```

    extern task run();
    extern task receive(output NNI_cell cell);
endclass

//new 构造对象
function Monitor::new(input vUtopiaTx Tx,input int PortID);
    this.Tx = Tx;
    this.PortID = PortID;
endfunction: new

//run(): 运行monitor
task Monitor::run();
    NNI_cell cell;

    forever begin
        receive(cell);
        foreach(cbsq[i])
            cbsq[i].post_rx(this,cell); //接收信元后的回调
        end
    endtask

//receive(): 从DUT读取信号,打包成NNI各式的信元
task Monitor::receive(output NNI_cell cell);
    ATMCellType pkt;
    Tx.cbr.clav <= 1;
    while(Tx.cbr.soc !== 1'b1 && Tx.cbt.en !== 1'b0)
        @(Tx.cbr);
    for(int i = 0; i <= 52; i++) begin
        //如果没有使能,循环等待
        while(Tx.cbt.en !== 1'b0) @(Tx.cbr);

        Pkt.Mem[i] = Tx.cbt.data;
        @Tx.cbt;
    end

    Tx.cbt.clav <= 0;

    cell = new();
    cell.unpack(Pkt);
    cell.display(%psprintf("Monitor:NNI_cell %0d",PortID));
endtask:receive

```

计分板scoreboard通过save_expected函数从Driver获得期望的信元,信元实际上从监视器monitor的check_actual函数接收,save_expected函数回调任务Scb_Driver——cbs

计分板scoreboard类

```

class Expect_cells;
    NNI_cell q[$];
    int iexpect,iactual;
endclass Expect_cells

```

```

class Scoreboard;
  Config cfg;
  Expect_cells expect_cells[];
  NNI_cell cellg[$];
  int iexpect,iactual;

  extern function new(Config cfg);
  extern virtual function void wrap_up();
  extern function void save_expected(UNI_cell cell);
  extern function void check_actual(input NNI_cell cell,input int portn);
  extern function void display(input string prefix="");

endclass Scoreboard

function Scoreboard::new(Config cfg);
  this.cfg = cfg;
  expect_cells = new[NumTx];
  foreach(expect_cells[i])
    expect_cells[i] = new();
endfunction : Scoreboard

function void Scoreboard::save_expected(UNI_cell cell);
  NNI_cell ncell = ucell.to_NNI;
  CellCfgType CellCfg = top.squat.lut.read(ncell.VPI);

  $display("@%0d Scoreboard:UNI_cell %0d, VPI = %0d, VCI = %0d, CLP = %0d, PT =
%0d, HEC = %0d, Payload = %0d",
    $time, iexpect, cell.VPI, cell.VCI, cell.CLP, cell.PT, cell.HEC,
cell.Payload);
  $time,ncell,VPI,CellCfg.FWD;
  ncell.display("psprintf("@ 0t:Scb save :", $time));

  //寻找信元将要转发的Tx端口
  for(int i = 0 ; i < NumTx ; i++)
    if(CellCfg.FWD[i].q.push_back(ncell)); //把信元保存到q
    expect_cells[i].q.push_back(ncell); //把信元保存到q
    expect_cells[i].iexpect++;
    iexpect++;
  end
endfunction : save_expected

function void Scoreboard::check_actual(input NNI_cell cell,input int portn);
  NNI_cell match;
  int match_idx;

  cell.display($psprintf("@%0t:Scb check:", $time));
  if(expect_cells[portn].q.size() == 0) begin
    $display("@%0t:Scb check:No expected cell for port %0d", $time,
portn);
    return;
  end

  expect_cells[portn].iactual++;
  iactual++;

```

```

        foreach(expect_cells[portn].q[i])begin
            if(expect_cells[portn].q[i].compare(cell)) begin
                $display("@%0t:Scb check:Matched expected cell %0d", $time, i);
                expect_cells[portn].q.delete(i);
                return;
            end
        end
        $display("@%0t:Scb check:No match for expected cell %0d", $time, portn);
        cell.display("Not Found");
        cfg.nErrors++;
    endfunction : check_actual

//输出仿真结束的报告
function void Scoreboard::wrap_up();
    $display("Scoreboard:Total expected cells = %0d, Total actual cells = %0d", iexpect, iactual);
    //寻找剩余的信元
    foreach (expect_cells[i]) begin
        if(expect_cells[i].q.size() != 0) begin
            $display("@%0t:Scb wrap up:Port %0d has %0d unmatched expected cells", $time, i, expect_cells[i].q.size())
            this.display("Unclaimed:");
            cfg.nErrors++;
        end
    end
endfunction : wrap_up

//输出计分板的内容,主要用于调试

function void Scoreboard::display(string prefix);
    $display("@%0t:Scb display: %s", $time, prefix);
    foreach(expect_cells[i]) begin
        $display("@%0t:Scb display: Port %0d", $time, i);
        foreach(expect_cells[i].q[j]) begin
            expect_cells[i].q[j].display("$psprintf("%sScoreboard: Tx%0d:",prefix,i));
        end
    end
endfunction : display

```

功能覆盖类

```

class Coverage;
    bit [1:0] src;
    bit [NumTx-1:0] fwd;

    covergroup CG_Forward;
        coverpoint src{
            bins src[] = {[0:3]};
            option.weight =0;
        }
    end
endclass

```

```

        coverpoint fwd
        {bins fwd[] = {[1:15]}}; //忽略fwd == 0
        option.weight =0;
    cross src,fwd;
endgroup : CG_Forward

function new;
    CG_Forward = new;
endfunction : new

//采样输入数据
function void sample(input bit [1:0] src, input bit [NumTx-1:0] fwd);

$display("Coverage:Src = %0d, Fwd = %0d", src, fwd);
this.src = src;
this.fwd = fwd;
CG_Forward.sample();
endfunction : sample
endclass Coverage

```

cpu driver类

```

class CPU_driver;
    vCPU_T mif;
    CellCfgType lookup[255:0]; //复制一份查找表
    Config cfg;
    bit [NumTx-1:0] fwd;

    extern function new(vCPU_T mif,Config cfg);
    extern task Initialize_Host();
    extern task HostWrite(int a,CellCfgType d); //配置
    extern task HostRead(int a,output CellCfgType d);
    extern task run();
endclass CPU_driver

function CPU_driver::Initialize_Host();

    mif.BusMode <= 1;
    mif.Addr <= 0;
    mif.DataIn <= 0;
    mif.Sel <= 1;
    mif.Rd_DS <= 1;
    mif.Wr_RW <=1;
endtask : Initialize_Host

task CPU_driver::HostWrite (int a, CellCfgType d); // 配置
    #10 mif.Addr<=a; mif.DataIn<=d; mif.Sel<=0;
    #10 mif.Wr_RW<=0;
    while (mif.Rdy_Dtack!=0) #10;
    #10 mif.Wr_RW<=1; mif.Sel<=1;

    while (mif.Rdy_Dtack==0) #10;

```

```

endtask : HostWrite

task CPU_driver::HostRead (int a, output CellCfgType d);
    #10 mif.Addr<=a; mif.Sel<=0;
    #10 mif.Rd_DS<=0;
    while (mif.Rdy_Dtack!=0) #10;
    #10 d=mif.DataOut; mif.Rd_DS<=1; mif.Sel<=1;

    // Alternate Tests 377 (注: 此处为书页眉/页脚文字, 非代码)
    while (mif.Rdy_Dtack==0) #10;
endtask : HostRead

task CPU_driver::run();
    CellCfgType CellFwd;
    Initialize_Host();

    //通过主机接口配置
    repeat(10) @(negedge clk);
    $write("Memory: loading...");
    for(int i = 0; i<= 255;i++) begin
        CellFwd.FWD = $urandom();
        `ifdef FWDALL
            CellFwd.FWD = 1;
        `endif
        CellFwd.VPI = i;
        HostWrite(i,CellFwd);
        lookup[i] = CellFwd;
    end

    //验证存储器
    $write("Memory: verifying...");
    for(int i = 0; i<= 255;i++) begin
        CellFwd.FWD = $urandom();
        `ifdef FWDALL
            CellFwd.FWD = 1;
        `endif
        CellFwd.VPI = i;
        HostRead(i,CellFwd);
        if(lookup[i].FWD != CellFwd.FWD) begin
            $display("Error: Memory verification failed for VPI = %0d", i);
            $finish;
        end
    end

    $display("verified");
endtask:run

```

修改测试

在验证期间,根据要测试的功能建立很多测试集,每个测试集使用不同的种子来运行

只有一个信元的测试

```

program automatic test
  #(parameter int NumRx = 4,parameter int NumTx = 4)
  (Utopia.TB_Rx Rx[0:NumRx-1],
  Utopia.TB_Tx Tx[0:NumTx-1],
  cpu_ifc.Test mif,
  input logic rst,clk);

  `include "environment.sv"
  Environment env;

  class Config_1_cell extends Config;
    constraint one_cells {nCells == 1;}
    function new(input int NumRx,NumTx);
      super.new(NumRx,NumTx);
    endfunction : new
  endclass Config_1_cells

  initial begin
    env = new(Rx,Tx,NumRx,NumTx,mif);

    begin //仅仿真一个信元
      Config_1_cell cfg1 = new(NumRx,NumTx);
      env.cfg = c1;
    end

    env.gen_cfg(); //配置成只有一个信元
    env.build();
    env.run();
    env.wrap_up();
  end
endprogram

```

随机丢弃信元

```

program automatic test
  #(parameter int NumRx = 4,parameter int NumTx = 4)
  (Utopia.TB_Rx Rx[0:NumRx-1],
  Utopia.TB_Tx Tx[0:NumTx-1],
  cpu_ifc.Test mif,
  input logic rst,clk);

  `include "environment.sv"
  Environment env;

  class Driver_cbs_drop extends Driver_cbs;
    virtual task pre_tx(input ATM_cell cell,ref bit drop);
      //在每100个事务中随机地丢弃一个
      drop= ($urandom() % 100 == 0);
    endtask : pre_tx
  endclass

```

```

initial begin
    env = new(Rx,Tx,NumRx,NumTx,mif);
    env.gen_cfg();
    env.build();

    begin //故障注入
        Driver_cbs_drop dcd = new();
        env.drv.cbs.push_back(dcd); //放入driver的队列
    end

    env.run();
    env.wrap_up();
end
endprogram

```

SystemVerilog与C语言的接口

SystemVerilog代码调用C语言子程序factorial

```

import "DPI-C" function int factorial(input int i);

program automatic test;
    initial begin
        for(int i = 0; i <= 10; i++)
            $display("Factorial(%0d) = %0d", i, factorial(i));
        end
    endprogram

```

import 声明定义了C任务和函数的原型,但使用的是SystemVerilog的数据类型

带有返回值的C函数会被映射成一个SystemVerilog函数

void 类型的C函数会被映射成一个SystemVerilog任务或者void函数

```

program automatic test;

    // 改变 C 函数名"test"为"my_test"
    import "DPI-C" test = function void my_test();

    initial my_test();

    // C 函数与关键字同名, 需要修改函数名
    // 注意: 原图中此处使用了转义符 \expect=, 但在标准语法中通常直接使用 expect=
    import "DPI-C" expect = function int fexpect();

    ...
    if (actual != fexpect()) $display("ERROR");
    ...

```



```
endprogram
```

参数方向

```
import "DPI-C" function int addmul(input int a,b, output int sum);
import "DPI-C" function void stop_model();
```

SystemVerilog	C(输入)	C(输出)
byte	char	char*
shortint	short int	short int*
int	int	int*
longint	long long int	long int*
shortreal	float	float*
real	double	double*
string	const char*	char
string[N]	const char	char
bit	svBit or unsigned char	svBit* or unsigned char
logic, reg	svLogic or unsigned char	svLogic* or unsigned char*
bit[N:0] reg[N:0] logic[N:0]	const svBitVecVal* const svLogicVecVal*	svBitVecVal* svLogicVecVal*
open array[]	const svOpenArrayHandle	svOpenArrayHandle
chandle	const void*	void*

chandle 数据类型

该数据类型允许你在SystemVerilog代码中存储一个C/C++ 指针,一个chandle变量的宽度足够在其被编译的机器上保存一个指针变量

```
#include <svdpi.h>
#include <malloc.h>
#include <veriusers.h>

typedef struct{
    unsigned char cnt;
}c7;
```

```
//创建一个计数器结构
void * counter7_new(){
    c7 * c = (c7 *)malloc(sizeof(c7));
    c->cnt = 0;
    return c;
}

//计数器运行一个周期
void counter7(c7 * inst,svBitVecVal * count,const svBitVecVal * i,const svBit
reset, const svBit load){
    if(reset) inst->cnt = 0; //复位
    else if(load) inst->cnt = *i //加载计数值
    else inst->cnt++; //计数器加1
    inst->cnt &= 0x7f; //最高位置0

    *count = inst->cnt ; //赋值给输出变量
    io_printf("C:count = %d\n", inst->cnt);
}
```

```
import "DPI-C" function chandle counter7_new();
import "DPI-C" function void counter7
(input chandle inst,
 output bit [6:0] out,
 input bit [6:0] in,
 input bit reset, load);

program automatic test;

// 测试计数器的两个实例
initial begin
    bit [6:0] o1, o2, i1, i2;
    bit reset, load, clk1;
    chandle inst1, inst2; // 指向 C 中的存储空间

    inst1 = counter7_new();
    inst2 = counter7_new();
    fork
        forever #10 clk1 = ~clk1;
        forever @(posedge clk1) begin
            counter7(inst1, o1, i1, reset, load);
            counter7(inst2, o2, i2, reset, load);
        end
    join_none

    reset = 0;
    load = 0;
    i1 = 120;
    i2 = 10;
```

```

        @(negedge clk1);
        load = 1;

        @(negedge clk1);
        load = 0;
        ...
    end
endprogram

```

verilog代码的数值是4状态所以用2位表示，c中高位aval，低位bval

```

// 例 12.12 检查 X 和 Z 值的计数器程序
// 将例 12.9 中的 counter7 替换为四态变量

void counter7(c7 * inst,
               svLogicVecVal* count,
               const svLogicVecVal* i,
               const svLogic reset,
               const svLogic load) {

    // 仅检查标量的 bval 位 (bval=1 表示 X 或 Z)
    if (reset & 0x2) {
        io_printf("Error: Z or X detected on reset\n\n");
        return;
    }

    // 仅检查标量的 bval 位
    if (load & 0x2) {
        io_printf("Error: Z or X detected on load\n\n");
        return;
    }

    // 仅检查 7 比特向量的 bval 位
    if (i->bval) {
        io_printf("Error: Z or X detected on i\n\n");
        return;
    }

    // 核心计数逻辑
    if (reset)
        inst->cnt = 0; // 复位
    else if (load)
        inst->cnt = i->aval; // 加载数值
    else
        inst->cnt++; // 计数

    inst->cnt &= 0x7F; // 最高位清 0 (保持 7-bit)

    // 赋值给输出变量
    count->aval = inst->cnt;
}

```

```
count->bval = 0; // 确保输出为确定的双态值 (0 或 1)
}
```

// 例 12.11 检查 Z 和 X 值的计数器测试平台

```
import "DPI-C" function chandle counter7_new();
import "DPI-C" function void counter7
    (input chandle inst,
     output logic [6:0] out,
     input logic [6:0] in,
     input logic reset, load);
```

调用c++程序

```
// 例 12.13 计数器类
class Counter7 {
public:
    Counter7();
    void counter7_signal(svBitVecVal* count,
                        const svBitVecVal* i,
                        const svBit reset,
                        const svBit load);

private:
    unsigned char cnt;
};

Counter7::Counter7() {
    cnt = 0; // 计数器初始化
}

void Counter7::counter7_signal(svBitVecVal* count,
                              const svBitVecVal* i,
                              const svBit reset,
                              const svBit load) {

    if (reset) cnt = 0; // 复位
    else if (load) cnt = *i; // 加载数值
    else cnt++; // 计数

    cnt &= 0x7F; // 最高位清 0 (保持7位宽度)
    *count = cnt;
}
```

DPI只能调用静态的C或者CPP代码,即在链接时已经存在的方法,这样一来,SystemVerilog代码就不能调用对象中的C++方法,因为被调用的对象在目标代码连接器运行时还不存在

解决方法是创建可以与CPP动态对象和方法通信的静态方法

使用方法通信的cpp计数器

```
class Counter7 {
public:
    Counter7();
    void count();
    void load(const svBitVecVal* i);
    void reset();
    int get();

private:
    unsigned char cnt;
};

// 计数器初始化
Counter7::Counter7() {
    cnt = 0;
}

// 计数器递增
void Counter7::count() {
    cnt = cnt + 1;
    cnt &= 0x7F;          // 最高位清 0 (保持7位宽度)
}

void Counter7::load(const svBitVecVal* i) {
    cnt = *i;
    cnt &= 0x7F;          // 最高位清 0
}

void Counter7::reset() {
    cnt = 0;
}

// 从 svBitVecVal 指针中获取计数器值
int Counter7::get() {
    return cnt;
}
```

再将其封装到事务级静态函数中

```
// 例 12.16 C++ 事务级计数器的静态封装 (wrapper)

#ifdef __cplusplus
extern "C" {
#endif

void* counter7_new() {
    return new Counter7;
}
```

```

}

void counter7_count(void* inst) {
    Counter7 * c7 = (Counter7 *) inst;
    c7->count();
}

void counter7_load(void* inst, const svBitVecVal* i) {
    Counter7 * c7 = (Counter7 *) inst;
    c7->load(i);
}

void counter7_reset(void* inst) {
    Counter7 * c7 = (Counter7 *) inst;
    c7->reset();
}

int counter7_get(void* inst) {
    Counter7 * c7 = (Counter7 *) inst;
    return c7->get();
}

#ifdef __cplusplus
}
#endif

```

使用cpp 模型方法的测试平台

```

// 例 12.17 使用方法的 C++ 模型的测试平台

import "DPI-C" function chandle counter7_new();
import "DPI-C" function void counter7_count(input chandle inst);
import "DPI-C" function void counter7_load(input chandle inst,
                                           input bit [6:0] i);
import "DPI-C" function void counter7_reset(input chandle inst);
import "DPI-C" function int counter7_get(input chandle inst);

// 用类封装计数器接口以隐藏 C++ 实例的句柄
class Counter7;
    chandle inst;

    function new;
        inst = counter7_new();
    endfunction

    function void count();
        counter7_count(inst);
    endfunction

    function void load(bit [6:0] val);
        counter7_load(inst, val);
    endfunction

```

```

function void reset();
    counter7_reset(inst);
endfunction

function bit [6:0] get();
    return counter7_get(inst);
endfunction
endclass Counter7

```

使用cpp模型的测试平台

```

program automatic counter;
    Counter7 c1;
    initial begin
        c1 = new;

        c1.reset();

        $display("SV : Post reset : counter1 = %d",c1.get());

        c1.load(126);

        if(c1.get()!=126) $display("Error in load");

        c1.count();
        c1.count();

        if(c1.get()!= 0) $display("Error in rollover");
    end
endprogram

```

一维数组：双状态

```

void fib(svBitVecVal data[20]){
    int i;
    data[0] = 1;
    data[1] = 1;
    for(i=2;i<20;i++){
        data[i] = data[i-1] + data[i-2];
    }
}

```

```

import "DPI-C" function void fib(svBitVecVal data[20]);

program automatic test;
    bit [31:0] data[20];

```

```

    initial begin
        fib(data);
        foreach(data[i]) $display(i,data[i]);
    end
endprogram

```

一维数组4状态

```

void fib(svLogicVecVal data[20]){
    int i;
    data[0].aval = 1;
    data[0].bval = 0;
    data[1].aval = 1;
    data[1].bval = 0;
    for(i=2;i<20;i++){
        data[i].aval = data[i-1].aval + data[i-2].aval;
        data[i].bval = 0; //别忘了将bval归零
    }
}

```

带4状态 数组的斐波那契C子程序的测试平台

```

import "DPI-C" function void fib(output logic [31:0] data[20]);

program automatic test;
    logic [31:0] data[20];

    initial begin
        fib(data);
        foreach(data[i]) $display(i,data[i]);
    end
endprogram

```

开放数组

```

import "DPI-C" function void open_array(output bit [31:0] data[]);

program automatic test;
    bit [31:0] data[20],r;

    initial begin
        fib_oa(data);
        foreach(data[i])
            $display(i,data[i]);
    end
endprogram

```



```

end
endprogram

```

使用基本开放数组的C代码

```

void fib_oa(const svOpenArrayHandle data_oa){
    int i, *data;
    data = (int *) svGetArrayPtr(data_oa);
    data[0] = 1;
    data[1] = 1;
    for(i = 2; i < 20; i++)
        data[i] = data[i-1] + data[i-2];
}

```

函数

描述

```

```systemverilog

```

```

class Scb_Driver_cbs extends Driver_cbs;

```

```

 Scoreboard scb;

 function new(input Scoreboard scb);
 this.scb = scb;
 endfunction : new

 //把收集到的信元发送到计分板
 virtual task post_tx(input Driver drv,
 input UNI_cell cell);
 scb.save_expected(cell);
 endtask : post_tx

```

```

endclass Scb_Driver_cbs

```

回调类连接了监视器和计分板

```

```systemverilog
class Scb_Monitor_cbs extends Monitor_cbs;

    Scoreboard scb;
    function new(input Scoreboard scb);
        this.scb = scb;
    endfunction : new

    //把收到的信元发送到计分板
    virtual task post_rx(input Monitor mon,input NNI_cell cell);

```

```

        scb.check_actual(cell,mon,PortID);
    endtask : post_rx
endclass Scb_Monitor_cbs

```

environment通过Cov_Monitor_cbs回调类连接了监视器和覆盖率类

```

class Cov_Monitor_cbs extends Monitor_cbs;
    Coverage cov;

    function new(input Coverage cov);
        this.cov = cov;
    endfunction : new

    // 把收到的信元发送到覆盖率类
    virtual task post_rx( input Monitor mon,
                        input NNI_cell cell);

        CellCfgType CellCfg = top.squat.lut.read(cell.VPI);
        cov.sample(mon.PortID, CellCfg.FWD);
    endtask : post_rx

endclass Cov_Monitor_cbs

```

```

class Config;
    // 错误和警告的个数
    int nErrors, nWarnings;
    // 把参数复制一份
    bit [31:0] numRx, numTx;

    // 信元的总数 (随机变量)
    rand bit [31:0] nCells;

    // 约束: 信元数必须大于0
    constraint c_nCells_valid {
        nCells > 0;
    }

    // 约束: 限制参与测试的信元个数小于 1000 个
    constraint c_nCells_reasonable {
        nCells < 1000;
    }

    // 允许使用的输入/输出通道 (随机动态数组)
    rand bit in_use_Rx[];

    // 约束: 至少需要一个 RX 通道被使用
    constraint c_in_use_valid {

```

```

        in_use_Rx.sum() > 0;
    }

    // 每个通道的信元数 (随机动态数组)
    rand bit [31:0] cells_per_chan[];

    // 约束: 把信元分配到各个通道 (各通道信元总数等于 nCells)
    constraint c_sum_ncells_sum {
        cells_per_chan.sum() == nCells;
    }

    // 约束: 把未使用的通道的信元个数设为 0
    constraint zero_unused_channels {
        foreach (cells_per_chan[i]) {
            // 在均匀分布时, 先求解 in_use_Rx[]
            solve in_use_Rx[i] before cells_per_chan[i];

            if (in_use_Rx[i])
                cells_per_chan[i] inside {[1:nCells]};
            else
                cells_per_chan[i] == 0;
        }
    }

    extern function new(input bit [31:0] numRx, numTx);
    extern virtual function void display(input string prefix="");
endclass Config

```

总量控制 (nCells) 首先确定这次仿真总共要发送多少个信元 (ATM Cell)。约束 c_nCells_valid 和 c_nCells_reasonable 保证数量在 1 到 999 之间, 避免仿真时间过长或过短。通道激活 (in_use_Rx) 这是一个位宽为 1 的动态数组。如果 in_use_Rx[0] 为 1, 表示第 0 号端口被激活; 为 0 则表示关闭。约束 c_in_use_valid 确保不会发生所有端口都关闭的情况 (即总和必须大于 0)。流量分配 (cells_per_chan) 这是最复杂的部分。它定义了一个数组, 存储每个端口具体要发多少数据。全局守恒: cells_per_chan.sum() == nCells 保证了所有端口发出的数据加起来正好等于总任务量。局部逻辑: zero_unused_channels 约束配合 solve...before 语句起到了关键作用: 它告诉求解器: 先决定哪些端口是开着的 (in_use_Rx), 再决定每个端口发多少数据 (cells_per_chan)。如果端口没开 (in_use_Rx[i] == 0), 则该端口的信元数强制为 0。如果端口开了, 则信元数在 1 到总数之间随机

信元配置类型

```

typedef struct packed{
    bit ['TxPorts-1:0] FWD;
    bit [11:0] VPI;
}CellCfgType;

function Config::new(input bit [31:0] numRx, numTx);
    this.numRx = numRx;
    this.numTx = numTx;
    in_use_Rx = new[numRx];
    cells_per_chan = new[numRx];

```

```

endfunction : new

function void Config::display(input string prefix);
    $write("%sConfig:numRx = %0d, numTx = %0d, nCells = %0d\n{",
        prefix, numRx, numTx, nCells);

    foreach(cells_per_chan[i])
        $write("%0d",cells_per_chan[i]);
    $write("),enabled Rx:",prefix);
    foreach(in_use_Rx[i]) if(in_use_Rx[i]) $write("%0d",i);
    $display;
endfunction : display

```

ATM交换机接收UNI格式的信元,发送NNI格式的信元

这些信元经过基于OOP的测试平台和结构化的设计,所以使用typedef定义

UNI信元格式

```

typedef struct packed{
    bit    [3:0]    GFC;
    bit    [7:0]    VPI;
    bit    [15:0]   VCI;
    bit                    CLP;
    bit    [2:0]    PT;
    bit    [7:0]    HEC;
    bit    [0:47] [7:0] payload;
}uniType;

```

NNI信元格式

```

typedef struct packed{
    bit [11:0] VPI;
    bit [15:0] VCI;
    bit      CLP;
    bit [2:0] PT;
    bit [7:0] HEC;
    bit [0:47] [7:0] payload;
}nniType;

```

ATM信元类型

```

typedef struct packed{
    uniType uni;
    nniType nni;
}

```

```

    bit [0:52][7:0] Mem;
}ATM_cellType;

```

UNI_cell扩展定义的BaseTr类

```

class UNI_cell extends BaseTr;
    // 物理域
    rand bit          [3:0]    GFC;
    rand bit          [7:0]    VPI;
    rand bit          [15:0]   VCI;
    rand bit          [0:0]    CLP;
    rand bit          [2:0]    PT;
    rand bit          [7:0]    HEC;
    rand bit [0:47] [7:0]    Payload;

    // 数据域
    static bit [7:0] syndrome[0:255];
    static bit syndrome_not_generated=1;

    extern function new();
    extern function void post_randomize();
    extern virtual function bit compare(input BaseTr to);
    extern virtual function void display(input string prefix="");
    extern virtual function void copy_data(input UNI_cell copy);
    extern virtual function BaseTr copy(input BaseTr to=null);
    extern virtual function void pack(output ATMCellType to);
    extern virtual function void unpack(input ATMCellType from);
    extern function NNI_cell to_NNI();
    extern function void generate_syndrome();

    extern function bit [7:0] hec (bit [31:0] hdr);
endclass UNI_cell

function UNI_cell::new();
    if(syndrome_not_generated)
        generate_syndrome();
endfunction : new

//在所有其他数据都确定后计算HEC
function void UNI_cell::post_randomize();
    HEC = hec(GFC, VPI, VCI, CLP, PT, Payload);
endfunction : post_randomize

//和其他信元进行比较
function bit UNI_cell::compare(input BaseTr to);
    UNI_cell cell;
    $cast(cell,to);
    if(GFC != cell.GFC) return 0;
    if(VPI != cell.VPI) return 0;
    if(VCI != cell.VCI) return 0;
    if(CLP != cell.CLP) return 0;
    if(PT != cell.PT) return 0;

```

```

    if(HEC != cell.HEC) return 0;
    if(Payload != cell.Payload) return 0;
    return 1;
endfunction : compare

//复制信元的详细内容
function void UNI_cell::copy_data(input string prefix);
    ATMCellType p;
    $display("%sUNI_cell:GFC = %0d, VPI = %0d, VCI = %0d, CLP = %0d, PT = %0d, HEC
= %0d, Payload = %0d",
        prefix, GFC, VPI, VCI, CLP, PT, HEC, Payload);
    foreach(p.Mem)
        $display
endfunction : display

//复制信元的数据域
function void UNI_cell::copy_data(input UNI_cell copy);
    GFC = copy.GFC;
    VPI = copy.VPI;
    VCI = copy.VCI;
    CLP = copy.CLP;
    PT = copy.PT;
    HEC = copy.HEC;
    Payload = copy.Payload;
endfunction : copy_data

//复制对象
function BaseTr UNI_cell::copy(input BaseTr to=null);
    UNI_cell cell;
    if(to == null) to = new UNI_cell();
    $cast(cell,to);
    cell.copy_data(this);
    return to;
endfunction : copy

//把对象打包到一个字节数组
function void UNI_cell::pack(output ATMCellType to);
    to.uni.GFC = GFC;
    to.uni.VPI = VPI;
    to.uni.VCI = VCI;
    to.uni.CLP = CLP;
    to.uni.PT = PT;
    to.uni.HEC = HEC;
    to.uni.Payload = Payload;
endfunction : pack

//把字节数组的内容按域展开到this对象
function void UNI_cell::unpack(input ATMCellType from);
    this.GFC = from.uni.GFC;
    this.VPI = from.uni.VPI;
    this.VCI = from.uni.VCI;
    this.CLP = from.uni.CLP;
    this.PT = from.uni.PT;
    this.HEC = from.uni.HEC;

```

```

        this.Payload = from.uni.Payload;
    endfunction : unpack

//根据UNI信元产生NNI信元
function NNI_cell UNI_cell::to_NNI();
    NNI_cell copy;
    $cast(copy,this);
    copy = new();
    copy.VPI = this.VPI;
    copy.VCI = this.VCI;
    copy.CLP = this.CLP;
    copy.PT = this.PT;
    copy.HEC = this.HEC;
    copy.Payload = this.Payload;
    return copy;
endfunction : to_NNI

//产生用于计算HEC的syndrome数组
function void UNI_cell::generate_syndrome();
    bit [7:0] sndrm;
    for(int i = 0; i < 256; i++) begin
        sndrm = i;
        repeat(8) begin
            if(sndrm[7] == 1'b1)
                sndrm = (sndrm << 1) ^ 8'h07;
            else
                sndrm = sndrm << 1;
            end
        syndrome_not_generated = 0;
    endfunction : generate_syndrome

//计算对象的HEC
function bit [7:0] hec (bit [31:0] hdr);
    bit [7:0] hec;
    hec = 0;
    repeat(4) begin
        hec = (hec << 1) ^ (hdr[7:0]);
        hdr = hdr >> 1;
    end
    return hec;
endfunction : hec

```

UNI信元的随机发生器,产生一个UNI类型的blueprint随机信元,然后把它的副本发送给driver

```

class UNI_cell_gen extends Generator;
    UNI_cell blueprint; //BluePrint信元
    mailbox gen2drv; //driver的Mailbox
    event drv2gen; //driver完成时的事件
    int nCells; //要产生 信元个数
    int PortID; //产生哪个Rx端口的信元

```

```

function new(input mailbox gen2drv,
             input event drv2gen,
             input int nCells,PortID);
    this.gen2drv = gen2drv;
    this.drv2gen = drv2gen;
    this.nCells = nCells;
    this.PortID = PortID;
    blueprint = new();
endfunction : new

task run();
    UNI_cell cell;
    repeat(nCells) begin
        assert(blueprint.randomize());
        $cast(cell,blueprint.copy());
        cell.display(%psprintf("UNI_cell_gen:UNI_cell %0d",nCells));
        gen2drv.put(cell);
        @drv2gen; //等待driver完成
    end
endtask : run
endclass UNI_generator

```

Driver类把UNI信元发送到ATM交换机，Driver类使用了回调任务

```

typedef class Driver_cbs;

class Driver;
    mailbox gen2drv; //用于存储器发送的信元
    event drv2gen; //通知发生器已经处理完毕
    vUtopiaRx Rx; //发送信元的虚IFC
    Driver_cbs cbsq[$]; //回调对象的队列
    int PortID;

    extern function new(input mailbox gen2drv,
                        input event drv2gen,
                        input vUtopiaRx Rx,
                        input int PortID);

    extern task run();
    extern task send(input UNI_cell cell);

endclass Driver

//new() 构造driver对象
//获取发生器的实物,发送给DUT
task Driver::run();
    UNI_cell cell;
    bit drop = 0;
    //初始化端口
    Rx.cbr.data<= 0;
    Rx.cbr.soc <= 0;
    Rx.cbr.clav<=0;

```



```

forever begin
    //从mailbox队列中读取一个信元
    gen2drv.peek(cell);
    begin : Tx
        //发送前的回调
        foreach(cbsg[i]) begin
            cbsq[i].pre_tx(this,cell);
        end

        gen2drv.get(cell); //从mailbox中删除该信元
        ->drv2gen; //通知发生器该信元处理完毕
    end
    //发送后的回调
endtask:run

//send():把信元发送给DUT
task Driver::send(input UNI_cell cell);
    ATMCellType pkt;
    cell.pack(Pkt);
    $write("Sending cell:");
    foreach(Pkt.Mem[i])
        $write("%0d",Pkt.Mem[i]);
    //遍历整个信元
    @(Rx.cbr);
    Rx.cbr.clav<=1;
    for(int i = 0; i <= 52; i++) begin

        //置位信元开始信号,使能信号,发送字节0
        Rx.cbr.soc <= (i == 0);
        Rx.cbr.data<= Pkt.Mem[i];
        @(Rx.cbr);
    end
    Rx.cbr.soc <= 'z;
    Rx.cbr.data <= 8'bx;

    Rx.cbr.clav<=0;
endtask

```

Driver回调类分别在信元发送前和发送后调用回调任务

```

typedef class Driver;

class Driver_cbs;
    virtual task pre_tx(input Driver drv,
                        input UNI_cell cell,
                        inout bit drop);

    endtask : pre_tx

    virtual task post_tx(input Driver drv,
                        input UNI_cell cell);

```

```

    endtask : post_tx
endclass Driver_cbs

```

Monitor回调类

```

typedef class Monitor_cbs;

class Monitor;
    vUtopiaTx Tx; //连接DUT输出的虚拟接口
    Monitor_cbs cbsq[$]; //回调对象的队列
    int PortID;

    extern function new(input vUtopiaTx Tx,input int PortID);
    extern task run();
    extern task receive(output NNI_cell cell);
endclass

//new 构造对象
function Monitor::new(input vUtopiaTx Tx,input int PortID);
    this.Tx = Tx;
    this.PortID = PortID;
endfunction: new

//run(): 运行monitor
task Monitor::run();
    NNI_cell cell;

    forever begin
        receive(cell);
        foreach(cbsq[i])
            cbsq[i].post_rx(this,cell); //接收信元后的回调
        end
    endtask

//receive(): 从DUT读取信号,打包成NNI各式的信元
task Monitor::receive(output NNI_cell cell);
    ATMCellType pkt;
    Tx.cbr.clav <= 1;
    while(Tx.cbr.soc !== 1'b1 && Tx.cbt.en !== 1'b0)
        @(Tx.cbr);
    for(int i = 0; i <= 52; i++) begin
        //如果没有使能,循环等待
        while(Tx.cbt.en !== 1'b0) @(Tx.cbr);

        Pkt.Mem[i] = Tx.cbt.data;
        @Tx.cbt;
    end

    Tx.cbt.clav <= 0;

    cell = new();
    cell.unpack(Pkt);

```

```

        cell.display(%psprintf("Monitor:NNI_cell %0d",PortID));
    endtask:receive

```

计分板scoreboard通过save_expected函数从Driver获得期望的信元,信元实际上从监视器monitor的check_actual函数接收,save_expected函数回调任务Scb_Driver——cbs

计分板scoreboard类

```

class Expect_cells;
    NNI_cell q[$];
    int iexpect,iactual;
endclass Expect_cells

class Scoreboard;
    Config cfg;
    Expect_cells expect_cells[];
    NNI_cell cellg[$];
    int iexpect,iactual;

    extern function new(Config cfg);
    extern virtual function void wrap_up();
    extern function void save_expected(UNI_cell cell);
    extern function void check_actual(input NNI_cell cell,input int portn);
    extern function void display(input string prefix="");

endclass Scoreboard

function Scoreboard::new(Config cfg);
    this.cfg = cfg;
    expect_cells = new[NumTx];
    foreach(expect_cells[i])
        expect_cells[i] = new();
endfunction : Scoreboard

function void Scoreboard::save_expected(UNI_cell cell);
    NNI_cell ncell = ucell.to_NNI;
    CellCfgType CellCfg = top.squat.lut.read(ncell.VPI);

    $display("@%0d Scoreboard:UNI_cell %0d, VPI = %0d, VCI = %0d, CLP = %0d, PT = %0d, HEC = %0d, Payload = %0d",
        $time, iexpect, cell.VPI, cell.VCI, cell.CLP, cell.PT, cell.HEC,
        cell.Payload);
    $time,ncell,VPI,CellCfg.FWD;
    ncell.display("psprintf("@ 0t:Scb save :", $time));

    //寻找信元将要转发的Tx端口
    for(int i = 0 ; i < NumTx ; i++)
        if(CellCfg.FWD[i].q.push_back(ncell)); //把信元保存到q
        expect_cells[i].q.push_back(ncell); //把信元保存到q
        expect_cells[i].iexpect++;
        iexpect++;
    end

```

```

endfunction : save_expected

function void Scoreboard::check_actual(input NNI_cell cell,input int portn);
    NNI_cell match;
    int match_idx;

    cell.display($psprintf("@%0t:Scb check:", $time));
    if(expect_cells[portn].q.size() == 0) begin
        $display("@%0t:Scb check:No expected cell for port %0d", $time,
portn);
        return;
    end

    expect_cells[portn].iactual++;
    iactual++;

    foreach(expect_cells[portn].q[i])begin
        if(expect_cells[portn].q[i].compare(cell)) begin
            $display("@%0t:Scb check:Matched expected cell %0d", $time, i);
            expect_cells[portn].q.delete(i);
            return;
        end
    end
    $display("@%0t:Scb check:No match for expected cell %0d", $time, portn);
    cell.display("Not Found");
    cfg.nErrors++;
endfunction : check_actual

//输出仿真结束的报告
function void Scoreboard::wrap_up();
    $display("Scoreboard:Total expected cells = %0d, Total actual cells =
%0d", iexpect, iactual);
    //寻找剩余的信元
    foreach (expect_cells[i]) begin
        if(expect_cells[i].q.size() != 0) begin
            $display("@%0t:Scb wrap up:Port %0d has %0d unmatched expected
cells", $time, i, expect_cells[i].q.size())
            this.display("Unclaimed:");
            cfg.nErrors++;
        end
    end
endfunction : wrap_up

//输出计分板的内容,主要用于调试

function void Scoreboard::display(string prefix);
    $display("@%0t:Scb display: %s", $time, prefix);
    foreach(expect_cells[i]) begin
        $display("@%0t:Scb display: Port %0d", $time, i);
        foreach(expect_cells[i].q[j]) begin
            expect_cells[i].q[j].display("$psprintf("%sScoreboard: Tx%
0d:",prefix,i));

```

```

        end
    endfunction : display

```

功能覆盖类

```

class Coverage;
    bit [1:0] src;
    bit [NumTx-1:0] fwd;

    covergroup CG_Forward;
        coverpoint src{
            bins src[] = {[0:3]};
            option.weight =0;
        }

        coverpoint fwd
        {bins fwd[] = {[1:15]}}; //忽略fwd == 0
        option.weight =0;
    cross src,fwd;
endgroup : CG_Forward

function new;
    CG_Forward = new;
endfunction : new

//采样输入数据
function void sample(input bit [1:0] src, input bit [NumTx-1:0] fwd);

$display("Coverage:Src = %0d, Fwd = %0d", src, fwd);
this.src = src;
this.fwd = fwd;
CG_Forward.sample();
endfunction : sample
endclass Coverage

```

cpu driver类

```

class CPU_driver;
    vCPU_T mif;
    CellCfgType lookup[255:0]; //复制一份查找表
    Config cfg;
    bit [NumTx-1:0] fwd;

    extern function new(vCPU_T mif,Config cfg);
    extern task Initialize_Host();
    extern task HostWrite(int a,CellCfgType d); //配置
    extern task HostRead(int a,output CellCfgType d);
    extern task run();
endclass CPU_driver

```

```

function CPU_driver::Initialize_Host();

    mif.BusMode <= 1;
    mif.Addr <= 0;
    mif.DataIn <= 0;
    mif.Sel <= 1;
    mif.Rd_DS <= 1;
    mif.Wr_RW <= 1;
endtask : Initialize_Host

task CPU_driver::HostWrite (int a, CellCfgType d); // 配置
    #10 mif.Addr<=a; mif.DataIn<=d; mif.Sel<=0;
    #10 mif.Wr_RW<=0;
    while (mif.Rdy_Dtack!=0) #10;
    #10 mif.Wr_RW<=1; mif.Sel<=1;

    while (mif.Rdy_Dtack==0) #10;
endtask : HostWrite

task CPU_driver::HostRead (int a, output CellCfgType d);
    #10 mif.Addr<=a; mif.Sel<=0;
    #10 mif.Rd_DS<=0;
    while (mif.Rdy_Dtack!=0) #10;
    #10 d=mif.DataOut; mif.Rd_DS<=1; mif.Sel<=1;

    // Alternate Tests 377 (注: 此处为书页眉/页脚文字, 非代码)
    while (mif.Rdy_Dtack==0) #10;
endtask : HostRead

task CPU_driver::run();
    CellCfgType CellFwd;
    Initialize_Host();

    //通过主机接口配置
    repeat(10) @(negedge clk);
    $write("Memory: loading...");
    for(int i = 0; i<= 255;i++) begin
        CellFwd.FWD = $urandom();
        `ifdef FWDALL
            CellFwd.FWD = 1;
        `endif
        CellFwd.VPI = i;
        HostWrite(i,CellFwd);
        lookup[i] = CellFwd;
    end

    //验证存储器
    $write("Memory: verifying...");
    for(int i = 0; i<= 255;i++) begin
        CellFwd.FWD = $urandom();
        `ifdef FWDALL
            CellFwd.FWD = 1;
        `endif
    end

```

```

        CellFwd.VPI = i;
        HostRead(i,CellFwd);
        if(lookup[i].FWD != CellFwd.FWD) begin
            $display("Error: Memory verification failed for VPI = %0d", i);
            $finish;
        end
    end
end

$display("verified");
endtask:run

```

修改测试

在验证期间,根据要测试的功能建立很多测试集,每个测试集使用不同的种子来运行

只有一个信元的测试

```

program automatic test
    #(parameter int NumRx = 4,parameter int NumTx = 4)
    (Utopia.TB_Rx Rx[0:NumRx-1],
    Utopia.TB_Tx Tx[0:NumTx-1],
    cpu_ifc.Test mif,
    input logic rst,clk);

    `include "environment.sv"
    Environment env;

    class Config_1_cell extends Config;
        constraint one_cells {nCells == 1;}
        function new(input int NumRx,NumTx);
            super.new(NumRx,NumTx);
        endfunction : new
    endclass Config_1_cells

    initial begin
        env = new(Rx,Tx,NumRx,NumTx,mif);

        begin //仅仿真一个信元
            Config_1_cell cfg1 = new(NumRx,NumTx);
            env.cfg = c1;
        end

        env.gen_cfg(); //配置成只有一个信元
        env.build();
        env.run();
        env.wrap_up();
    end
endprogram

```

随机丢弃信元

```

program automatic test
  #(parameter int NumRx = 4,parameter int NumTx = 4)
  (Utopia.TB_Rx Rx[0:NumRx-1],
  Utopia.TB_Tx Tx[0:NumTx-1],
  cpu_ifc.Test mif,
  input logic rst,clk);

  `include "environment.sv"
  Environment env;

  class Driver_cbs_drop extends Driver_cbs;
    virtual task pre_tx(input ATM_cell cell,ref bit drop);
      //在每100个事务中随机地丢弃一个
      drop= ($urandom() % 100 == 0);
    endtask : pre_tx
  endclass

  initial begin
    env = new(Rx,Tx,NumRx,NumTx,mif);
    env.gen_cfg();
    env.build();

    begin //故障注入
      Driver_cbs_drop dcd = new();
      env.drv.cbs.push_back(dcd); //放入driver的队列
    end

    env.run();
    env.wrap_up();
    end
  endprogram

```

SystemVerilog与C语言的接口

SystemVerilog代码调用C语言子程序factorial

```

import "DPI-C" function int factorial(input int i);

program automatic test;
  initial begin
    for(int i = 0; i <= 10; i++)
      $display("Factorial(%0d) = %0d", i, factorial(i));
    end
  endprogram

```

import 声明定义了C任务和函数的原型,但使用的是SystemVerilog的数据类型

带有返回值的C函数会被映射成一个SystemVerilog函数

void 类型的C函数会被映射成一个SystemVerilog任务或者void函数

```
program automatic test;

    // 改变 C 函数名"test"为"my_test"
    import "DPI-C" test = function void my_test();

    initial my_test();

    // C 函数与关键字同名，需要修改函数名
    // 注意：原图中此处使用了转义符 \expect=，但在标准语法中通常直接使用 expect=
    import "DPI-C" expect = function int fexpect();

    ...
    if (actual != fexpect()) $display("ERROR");
    ...

endprogram
```

参数方向

```
import "DPI-C" function int addmul(input int a,b, output int sum);
import "DPI-C" function void stop_model();
```

SystemVerilog	C(输入)	C(输出)
byte	char	char*
shortint	short int	short int*
int	int	int*
longint	long long int	long int*
shortreal	float	float*
real	double	double*
string	const char*	char
string[N]	const char	char
bit	svBit or unsigned char	svBit* or unsigned char
logic, reg	svLogic or unsigned char	svLogic* or unsigned char*
bit[N:0] reg[N:0] logic[N:0]	const svBitVecVal* const svLogicVecVal*	svBitVecVal* svLogicVecVal*

SystemVerilog	C(输入)	C(输出)
open array[]	const svOpenArrayHandle	svOpenArrayHandle
chandle	const void*	void*

chandle 数据类型

该数据类型允许你在SystemVerilog代码中存储一个C/C++指针,一个chandle变量的宽度足够在其被编译的机器上保存一个指针变量

```
#include <svdpi.h>
#include <malloc.h>
#include <veriusers.h>

typedef struct{
    unsigned char cnt;
}c7;

//创建一个计数器结构
void * counter7_new(){
    c7 * c = (c7 *)malloc(sizeof(c7));
    c->cnt = 0;
    return c;
}

//计数器运行一个周期
void counter7(c7 * inst,svBitVecVal * count,const svBitVecVal * i,const svBit
reset, const svBit load){
    if(reset) inst->cnt = 0; //复位
    else if(load) inst->cnt = *i //加载计数值
    else inst->cnt++; //计数器加1
    inst->cnt &= 0x7f; //最高位置0

    *count = inst->cnt ; //赋值给输出变量
    io_printf("C:count = %d\n", inst->cnt);
}
```

```
import "DPI-C" function chandle counter7_new();
import "DPI-C" function void counter7
(input chandle inst,
 output bit [6:0] out,
 input bit [6:0] in,
 input bit reset, load);

program automatic test;

    // 测试计数器的两个实例
    initial begin
```

```

    bit [6:0] o1, o2, i1, i2;
    bit      reset, load, clk1;
   chandle  inst1, inst2;      // 指向 C 中的存储空间

    inst1 = counter7_new();
    inst2 = counter7_new();
    fork
        forever #10 clk1 = ~clk1;
        forever @(posedge clk1) begin
            counter7(inst1, o1, i1, reset, load);
            counter7(inst2, o2, i2, reset, load);
        end
    join_none

    reset = 0;
    load = 0;
    i1 = 120;
    i2 = 10;

    @(negedge clk1);
    load = 1;

    @(negedge clk1);
    load = 0;
    ...
end
endprogram

```

verilog代码的数值是4状态所以用2位表示，c中高位aval，低位bval

```

// 例 12.12 检查 X 和 Z 值的计数器程序
// 将例 12.9 中的 counter7 替换为四态变量

void counter7(c7 * inst,
              svLogicVecVal* count,
              const svLogicVecVal* i,
              const svLogic reset,
              const svLogic load) {

    // 仅检查标量的 bval 位 (bval=1 表示 X 或 Z)
    if (reset & 0x2) {
        io_printf("Error: Z or X detected on reset\n\n");
        return;
    }

    // 仅检查标量的 bval 位
    if (load & 0x2) {
        io_printf("Error: Z or X detected on load\n\n");
        return;
    }

    // 仅检查 7 比特向量的 bval 位

```

```

if (i->bval) {
    io_printf("Error: Z or X detected on i\n\n");
    return;
}

// 核心计数逻辑
if (reset)
    inst->cnt = 0;                // 复位
else if (load)
    inst->cnt = i->aval;          // 加载数值
else
    inst->cnt++;                  // 计数

inst->cnt &= 0x7F;                // 最高位清 0 (保持 7-bit)

// 赋值给输出变量
count->aval = inst->cnt;
count->bval = 0;                  // 确保输出为确定的双态值 (0 或 1)
}

```

// 例 12.11 检查 Z 和 X 值的计数器测试平台

```

import "DPI-C" function chandle counter7_new();
import "DPI-C" function void counter7
    (input  chandle inst,
     output logic [6:0] out,
     input  logic [6:0] in,
     input  logic reset, load);

```

调用c++程序

```

// 例 12.13 计数器类
class Counter7 {
public:
    Counter7();
    void counter7_signal(svBitVecVal* count,
                        const svBitVecVal* i,
                        const svBit reset,
                        const svBit load);

private:
    unsigned char cnt;
};

Counter7::Counter7() {
    cnt = 0;                // 计数器初始化
}

void Counter7::counter7_signal(svBitVecVal* count,

```

```

                                const svBitVecVal* i,
                                const svBit reset,
                                const svBit load) {

    if (reset)      cnt = 0;      // 复位
    else if (load)  cnt = *i;     // 加载数值
    else            cnt++;        // 计数

    cnt &= 0x7F;        // 最高位清 0 (保持7位宽度)
    *count = cnt;
}

```

DPI只能调用静态的C或者CPP代码,即在链接时已经存在的方法,这样一来,SystemVerilog代码就不能调用对象中的Cpp方法,因为被调用的对象在目标代码连接器运行时还不存在

解决方法是创建可以与CPP动态对象和方法通信的静态方法

使用方法通信的cpp计数器

```

class Counter7 {
public:
    Counter7();
    void count();
    void load(const svBitVecVal* i);
    void reset();
    int get();

private:
    unsigned char cnt;
};

// 计数器初始化
Counter7::Counter7() {
    cnt = 0;
}

// 计数器递增
void Counter7::count() {
    cnt = cnt + 1;
    cnt &= 0x7F;        // 最高位清 0 (保持7位宽度)
}

void Counter7::load(const svBitVecVal* i) {
    cnt = *i;
    cnt &= 0x7F;        // 最高位清 0
}

void Counter7::reset() {
    cnt = 0;
}

```

```
// 从 svBitVecVal 指针中获取计数器值
int Counter7::get() {
    return cnt;
}
```

再将其封装到事务级静态函数中

```
// 例 12.16 C++ 事务级计数器的静态封装 (wrapper)

#ifdef __cplusplus
extern "C" {
#endif

void* counter7_new() {
    return new Counter7;
}

void counter7_count(void* inst) {
    Counter7 * c7 = (Counter7 *) inst;
    c7->count();
}

void counter7_load(void* inst, const svBitVecVal* i) {
    Counter7 * c7 = (Counter7 *) inst;
    c7->load(i);
}

void counter7_reset(void* inst) {
    Counter7 * c7 = (Counter7 *) inst;
    c7->reset();
}

int counter7_get(void* inst) {
    Counter7 * c7 = (Counter7 *) inst;
    return c7->get();
}

#ifdef __cplusplus
}
#endif
```

使用cpp 模型方法的测试平台

```
// 例 12.17 使用方法的 C++ 模型的测试平台

import "DPI-C" function chandle counter7_new();
import "DPI-C" function void counter7_count(input chandle inst);
import "DPI-C" function void counter7_load(input chandle inst,
                                           input bit [6:0] i);
```

```

import "DPI-C" function void counter7_reset(input chandle inst);
import "DPI-C" function int counter7_get(input chandle inst);

// 用类封装计数器接口以隐藏 C++ 实例的句柄
class Counter7;
    chandle inst;

    function new;
        inst = counter7_new();
    endfunction

    function void count();
        counter7_count(inst);
    endfunction

    function void load(bit [6:0] val);
        counter7_load(inst, val);
    endfunction

    function void reset();
        counter7_reset(inst);
    endfunction

    function bit [6:0] get();
        return counter7_get(inst);
    endfunction
endclass Counter7

```

使用cpp模型的测试平台

```

program automatic counter;
    Counter7 c1;
    initial begin
        c1 = new;

        c1.reset();

        $display("SV : Post reset : counter1 = %d",c1.get());

        c1.load(126);

        if(c1.get()!=126) $display("Error in load");

        c1.count();
        c1.count();

        if(c1.get()!= 0) $display("Error in rollover");
    end
endprogram

```

一维数组：双状态

```
void fib(svBitVecVal data[20]){
    int i;
    data[0] = 1;
    data[1] = 1;
    for(i=2;i<20;i++){
        data[i] = data[i-1] + data[i-2];
    }
}
```

```
import "DPI-C" function void fib(svBitVecVal data[20]);

program automatic test;
    bit [31:0] data[20];

    initial begin
        fib(data);
        foreach(data[i]) $display(i,data[i]);
    end
endprogram
```

一维数组4状态

```
void fib(svLogicVecVal data[20]){
    int i;
    data[0].aval = 1;
    data[0].bval = 0;
    data[1].aval = 1;
    data[1].bval = 0;
    for(i=2;i<20;i++){
        data[i].aval = data[i-1].aval + data[i-2].aval;
        data[i].bval = 0; //别忘了将bval归零
    }
}
```

带4状态 数组的斐波那契C子程序的测试平台

```
import "DPI-C" function void fib(output logic [31:0] data[20]);

program automatic test;
    logic [31:0] data[20];

    initial begin
        fib(data);
        foreach(data[i]) $display(i,data[i]);
    end
endprogram
```



```
end
endprogram
```

开放数组

```
import "DPI-C" function void open_array(output bit [31:0] data[]);

program automatic test;
    bit [31:0] data[20],r;

    initial begin
        fib_oa(data);
        foreach(data[i])
            $display(i,data[i]);
    end
endprogram
```

使用基本开放数组的C代码

```
void fib_oa(const svOpenArrayHandle data_oa){
    int i, *data;
    data = (int *) svGetArrayPtr(data_oa);
    data[0] = 1;
    data[1] = 1;
    for(i = 2; i < 20; i++)
        data[i] = data[i-1] + data[i-2];
}
```

函数	描述
int svLeft(h, d)	维数 d 的左边界
int svRight(h, d)	维数 d 的右边界
int svLow(h, d)	维数 d 的下界
int svHigh(h, d)	维数 d 的上界
int svIncrement(h, d)	如果左边界大于等于右边界则返回 1， 如果左边界小于右边界则返回 -1
int svSize(h, d)	维数 d 的元素总数目；svHigh - svLow + 1
int svDimension(h)	开放数组的维数
int svSizeOfArray(h)	以字节计量的数组大小
函数	返回指针类型
void* svGetArrayPtr(h)	整个数组的存储位置

函数	返回指针类型
<code>void* svGetArrElemPtr(h, i1, ...)</code>	数组中的一个元素
<code>void* svGetArrElemPtr1(h, i1)</code>	一维数组中的一个元素
<code>void* svGetArrElemPtr2(h, i1, i2)</code>	二维数组中的一个元素
<code>void* svGetArrElemPtr3(h, i1, i2, i3)</code>	三维数组中的一个元素

传递大小未定义的开放数组

```
// 例 12.25 调用参数为多维开放数组的 C 函数的测试平台
import "DPI-C" function void mydisplay(inout int h[][]);

program automatic test;
  // 注意此处索引范围由高到低
  int a[6:1][8:3];

  initial begin
    // 初始化数组: 每个元素的值等于两个索引之和
    foreach (a[i,j]) a[i][j] = i + j;

    // 调用 C 函数处理数组
    mydisplay(a);

    // 打印处理后的结果
    foreach (a[i,j]) $display("V: a[%0d][%0d]=%0d", i, j, a[i][j]);
  end
endprogram
```

```
// 例 12.26 参数为多维开放数组的 C 代码
#include <stdio.h>
#include "svdpi.h" // 包含 svOpenArrayHandle 等定义

void mydisplay(const svOpenArrayHandle h) {
  int i, j;

  // 获取第一维和第二维的上下界
  int lo1 = svLow(h, 1);
  int hi1 = svHigh(h, 1);
  int lo2 = svLow(h, 2);
  int hi2 = svHigh(h, 2);

  // 遍历数组
  for (i = lo1; i <= hi1; i++) {
    for (j = lo2; j <= hi2; j++) {
      // 获取指向当前元素 [i][j] 的指针
      int * a = (int*) svGetArrElemPtr2(h, i, j);

      // 打印当前值
    }
  }
}
```

```

        io_printf("C: a[%d][%d]=%d\n", i, j, * a);

        // 修改数组元素的值：改为两个索引之积
        * a = i * j;
    }
}
}

```

DPI中压缩数组的测试平台

在dpi中一个开放数组被视为拥有一个压缩维度和一个或多个非压缩的维度

压缩开放数组的测试平台

在 DPI 中，一个开放数组被视为拥有一个压缩的维度和一个或多个非压缩的维度。你可以传递多维的压缩数组，只要该压缩数组的元素大小跟形式参数的元素大小相同即可。例如，如果在 ``systemverilog // 例 12.25 调用参数为多维开放数组的 C 函数的测试平台 import "DPI-C" function void mydisplay(inout int h[][]);

program automatic test; // 注意此处索引范围由高到低 int a[6:1][8:3];

```

initial begin
    // 初始化数组：每个元素的值等于两个索引之和
    foreach (a[i,j]) a[i][j] = i + j;

    // 调用 C 函数处理数组
    mydisplay(a);

    // 打印处理后的结果
    foreach (a[i,j]) $display("V: a[%0d][%0d]=%0d", i, j, a[i][j]);
end

```

endprogram

```

``c
// 例 12.26 参数为多维开放数组的 C 代码
#include <stdio.h>
#include "svdpi.h" // 包含 svOpenArrayHandle 等定义

void mydisplay(const svOpenArrayHandle h) {
    int i, j;

    // 获取第一维和第二维的上下界
    int lo1 = svLow(h, 1);
    int hi1 = svHigh(h, 1);
    int lo2 = svLow(h, 2);
    int hi2 = svHigh(h, 2);

    // 遍历数组
    for (i = lo1; i <= hi1; i++) {

```

```

    for (j = lo2; j <= hi2; j++) {
        // 获取指向当前元素 [i][j] 的指针
        int * a = (int*) svGetArrElemPtr2(h, i, j);

        // 打印当前值
        io_printf("C: a[%d][%d]=%d\n", i, j, * a);

        // 修改数组元素的值：改为两个索引之积
        * a = i * j;
    }
}

```

DPI中压缩数组的测试平台

在dpi中一个开放数组被视为拥有一个压缩维度和一个或多个非压缩的维度

压缩开放数组的测试平台

在 DPI 中，一个开放数组被视为拥有一个压缩的维度和一个或多个非压缩的维度。你可以传递多维的压缩数组，只要该压缩数组的元素大小跟形式参数的元素大小相同即可。例如，如果在 `import` 语句中定义了形式参数 `bit [63:0] b64[]`，就可以将实际参数 `bit[1:0][0:3][6:-1] bpack[9:1]` 传递进去。

例 12.27 压缩开放数组的测试平台

```

import "DPI-C" function void view_pack(input bit [63:0] b64[]);

program automatic test;
    bit [1:0][0:3][6:-1] bpack[9:1];

    initial begin
        foreach (bpack[i]) bpack[i] = i;
        bpack[2] = 64'h12345678_90abcdef;

        $display("SV: bpack[2]=%h", bpack[2]);           // 64 位
        $display("SV: bpack[2][0]=%h", bpack[2][0]);    // 32 位
        $display("SV: bpack[2][0][0]=%h", bpack[2][0][0]); // 8 位

        view_pack(bpack);
    end
endprogram

```

例 12.28 使用压缩开放数组的 C 代码

```

void view_pack(const svOpenArrayHandle h) {
    int i;

    for (i = svLow(h, 1); i < svHigh(h, 1); i++)
        io_printf("C: b64[%d]=%11x\n",

```

```

        i, * (long long int *)svGetArrElemPtr1(h, i));
    }

```

注意，例 12.28 中的 C 代码按照 ``systemverilog import "DPI-C" function void view_pack(input bit [63:0] b64[]);

program automatic test; bit [1:0][0:3][6:-1] bpack[9:1];

```

initial begin
    foreach (bpack[i]) bpack[i] = i;
    bpack[2] = 64'h12345678_90abcdef;

    $display("SV: bpack[2]=%h", bpack[2]);           // 64 位
    $display("SV: bpack[2][0]=%h", bpack[2][0]);     // 32 位
    $display("SV: bpack[2][0][0]=%h", bpack[2][0][0]); // 8 位

    view_pack(bpack);
end

```

endprogram

例 12.28 使用压缩开放数组的 C 代码

```

``c
void view_pack(const svOpenArrayHandle h) {
    int i;

    for (i = svLow(h, 1); i < svHigh(h, 1); i++)
        io_printf("C: b64[%d]=%llx\n",
            i, * (long long int *)svGetArrElemPtr1(h, i));
}

```

注意，例 12.28 中的 C 代码按照 %llx 的格式输出一个 64 位数值，然后将结果从 svGetArrayElemPtr1 类型强制转换成了 long long int 类型。

这个例子展示了如何传递“打包”数组，需要我帮你写个对应的 SystemVerilog 接口来调用它吗？

共享符合类型

下面是一个共享用于表示像素的简单结构变量的例子，该结构由三个字节组成，被压缩成一个字。例 12.29 给出了 C 结构的定义。注意，C 将 char 视为有符号变量，这可能会带来不可预测的结果，所以该结构对 char 类型加上 unsigned 限制。这些字节的顺序跟 SystemVerilog 相比刚好相反，因为这段代码运行在小尾序的 Intel X86 处理器上，即低权重字节存储在低位地址上。Sun SPARC 处理器是大尾序的，所以其字节存储顺序跟 SystemVerilog 相同：r, g, b。

例 12.29 共享结构的 C 代码

```
typedef struct {
    unsigned char b, g, r;      // x86 小尾序
    //unsigned char r, g, b;    // SPARC 格式
} * p_rgb;

void invert(p_rgb rgb) {
    rgb->r = ~rgb->r;           // 色彩值取反
    rgb->g = ~rgb->g;
    rgb->b = ~rgb->b;
    io_printf("C: Invert rgb=%02x,%02x,%02x\n",
              rgb->r, rgb->g, rgb->b);
}
```

SystemVerilog 测试平台使用压缩结构来保存一个简单的像素，使用类来封装对像素的操作。由于结构 RGB_T 是压缩的，所以 SystemVerilog 会以连续的方式来保存其字节。如果没有加上 ``c typedef struct { unsigned char b, g, r; // x86 小尾序 //unsigned char r, g, b; // SPARC 格式 } * p_rgb;

```
void invert(p_rgb rgb) { rgb->r = ~rgb->r; // 色彩值取反 rgb->g = ~rgb->g; rgb->b = ~rgb->b; io_printf("C:
Invert rgb=%02x,%02x,%02x\n", rgb->r, rgb->g, rgb->b); }
```

SystemVerilog 测试平台使用压缩结构来保存一个简单的像素，使用类来封装对像素的操作。由于结构 RGB_T 是压缩的，所以 SystemVerilog 会以连续的方式来保存其字节。如果没有加上 `packed` 修饰符，每一个 8 位值都会被保存成一个单字。

```
``systemverilog
// 定义打包结构体类型
typedef struct packed { bit [7:0] r, g, b; } RGB_T;

// 导入 C 函数，参数为 inout 模式的结构体
import "DPI-C" function void invert(inout RGB_T pstruct);

program automatic test;

    class RGB;
        rand bit [7:0] r, g, b;

        function void display(string prefix="");
            $display("%sRGB=%x,%x,%x", prefix, r, g, b);
        endfunction : display

        // 将类成员压缩到一个结构中
        function RGB_T pack();
            pack.r = r;
            pack.g = g;
            pack.b = b;
        endfunction : pack

        // 将结构解压后赋值给类成员
        function void unpack(RGB_T pstruct);
```

```

        r = pstruct.r;
        g = pstruct.g;
        b = pstruct.b;
    endfunction : unpack
endclass RGB

initial begin
    RGB pixel;
    RGB_T pstruct;

    pixel = new;

    repeat (5) begin
        assert(pixel.randomize());           // 创建随机像素
        pixel.display("\nSV: before ");      // 打印像素值
        pstruct = pixel.pack();              // 转换为结构
        invert(pstruct);                     // 调用 C 函数将位取反
        pixel.unpack(pstruct);               // 把结构解压后赋值给类
        pixel.display("SV: after ");         // 打印
    end
end

endprogram

```

在SystemVerilog 和 c之间传递字符串

```

char * print(p_rgb rgb){
    static char s[12];
    sprintf(s,"C: rgb=%02x,%02x,%02x",rgb->r,rgb->g,rgb->b);
    return s;
}

```

使用静态变量的风险在于多个函数开发调用时会引起内存共享问题

下面的代码将字符串保存在一个堆中支持并发调用

```

#define PRINT_SIZE 12
#define MAX_CALLS 16
#define HEAP_SIZE PRINT_SIZE * MAX_CALLS

char * print(p_rgb rgb){
    static char print_heap[HEAP_SIZE+ PRINT_SIZE];
    char * s;
    static int heap_idx = 0;
    int nchara;

    s = &print_heap[heap_idx];
    nchars = sprintf(s,"C: rgb=%02x,%02x,%02x",rgb->r,rgb->g,rgb->b);

    heap_idx+ = nchars + 1;
}

```

```

    if(heap_idx > HEAP_SIZE)
        heap_idx = 0;
    return s;
}

```

纯导入方法和关联导入方法

导入方法分为纯（pure）导入、关联（context）导入和通用（generic）导入。一个纯函数将严格根据其输入来计算输出，跟外部环境没有任何其他交互。具体来说就是一个纯函数不会访问任何全局或者静态变量，不会进行文件操作，不会跟函数体以外的事务如操作系统、进程、共享内存和套接字等有交互。如果没有使用纯函数的输出，那么 SystemVerilog 编译器会优化掉对该函数的调用，对于输入参数相同的两次调用，编译器会将第二次调用直接用第一次的输出结果替换。例 12.5 中的 ``systemverilog // 定义打包结构体类型 typedef struct packed { bit [7:0] r, g, b; } RGB_T;

// 导入 C 函数，参数为 inout 模式的结构体 import "DPI-C" function void invert(inout RGB_T pstruct);

program automatic test;

```

class RGB;
    rand bit [7:0] r, g, b;

    function void display(string prefix="");
        $display("%sRGB=%x,%x,%x", prefix, r, g, b);
    endfunction : display

    // 将类成员压缩到一个结构中
    function RGB_T pack();
        pack.r = r;
        pack.g = g;
        pack.b = b;
    endfunction : pack

    // 将结构解压后赋值给类成员
    function void unpack(RGB_T pstruct);
        r = pstruct.r;
        g = pstruct.g;
        b = pstruct.b;
    endfunction : unpack
endclass RGB

initial begin
    RGB pixel;
    RGB_T pstruct;

    pixel = new;

    repeat (5) begin
        assert(pixel.randomize()); // 创建随机像素
        pixel.display("\nSV: before "); // 打印像素值
        pstruct = pixel.pack(); // 转换为结构
        invert(pstruct); // 调用 C 函数将位取反
    end
end

```



```

        pixel.unpack(pstruct);          // 把结构解压后赋值给类
        pixel.display("SV: after ");    // 打印
    end
end

```

endprogram

在SystemVerilog 和 c之间传递字符串

```

```systemverilog
char * print(p_rgb rgb){
 static char s[12];
 sprintf(s,"C: rgb=%02x,%02x,%02x",rgb->r,rgb->g,rgb->b);
 return s;
}

```

使用静态变量的风险在于多个函数开发调用时会引起内存共享问题

下面的代码将字符串保存在一个堆中支持并发调用

```

#define PRINT_SIZE 12
#define MAX_CALLS 16
#define HEAP_SIZE PRINT_SIZE * MAX_CALLS

char * print(p_rgb rgb){
 static char print_heap[HEAP_SIZE+ PRINT_SIZE];
 char * s;
 static int heap_idx = 0;
 int nchara;

 s = &print_heap[heap_idx];
 nchars = sprintf(s,"C: rgb=%02x,%02x,%02x",rgb->r,rgb->g,rgb->b);

 heap_idx+ = nchars + 1;
 if(heap_idx > HEAP_SIZE)
 heap_idx = 0;
 return s;
}

```

## 纯导入方法和关联导入方法

导入方法分为纯（pure）导入、关联（context）导入和通用（generic）导入。一个纯函数将严格根据其输入来计算输出，跟外部环境没有任何其他交互。具体来说就是一个纯函数不会访问任何全局或者静态变量，不会进行文件操作，不会跟函数体以外的事务如操作系统、进程、共享内存和套接字等有交互。如果没有使用纯函数的输出，那么 SystemVerilog 编译器会优化掉对该函数的调用，对于输入参数相同的两次调用，编译器会将第

二次调用直接用第一次的输出结果替换。例 12.5 中的 `factorial` 函数和例 12.6 中的 `sin` 函数都是纯函数，因为它们的计算结果仅依赖于它们的输入。

```
import "DPI-C" pure function int factorial(input int i);
import "DPI-C" pure function real sin(input real in);
```

导入方法可能需要知道它被调用的环境的上下文信息以决定调用 PLI TF、ACC 还是 VPI 方法或者导出的 SystemVerilog 任务。对这些导入方法需要使用 ```systemverilog import "DPI-C" pure function int factorial(input int i); import "DPI-C" pure function real sin(input real in);`

导入方法可能需要知道它被调用的环境的上下文信息以决定调用 PLI TF、ACC 还是 VPI 方法或者导出的 SystemVerilog 任务。对这些导入方法需要使用 ``context`` 限定词。

```
``systemverilog
import "DPI-C" context task call_sv(bit [31:0] data);
```

如果导入方法使用全局变量，那它就不再是纯方法了，但是它可能没有调用任何 PLI，所以其实它也不需要承受 ```systemverilog import "DPI-C" context task call_sv(bit [31:0] data);`

如果导入方法使用全局变量，那它就不再是纯方法了，但是它可能没有调用任何 PLI，所以其实它也不需要承受 ``context`` 方法所带来的额外开销。Sutherland (2004) 将这样的函数定义为“generic（通用的）”，因为 SystemVerilog 语言参考手册中对此并没有定义专门的名字。缺省情况下导入函数是通用类型，这也是本章中的很多例子使用的类型。

调用关联导入方法时需要记录调用的上下文环境，这会给仿真器带来额外的开支。所以除非确实需要，否则不要将方法定义为关联方法。另一方面，如果一个通用导入方法调用了导出任务或者一个访问 SystemVerilog 数据对象的 PLI 函数，会导致仿真器崩溃。

一个关联感知（context-aware）的 PLI 函数指的是这个函数知道自己身在何处被调用，并且能够依据调用地址来访问跟该位置相关联的信息。

---

在c中与SystemVerilog通信

```
``systemverilog
module block;
 import "DPI-C" context function void c_display();
 export "DPI-C" function sv_display; // 没有类型定义或者参数

 initial c_display();

 function void sv_display();
 $display("SV: block");
```

```
endfunction
endmodule block
```

## 导出一个SystemVerilog任务

```
extern void sv_display();

void c_display() {
 io_printf("C: c_display\n");
 sv_display();
}
```

## 简单内存模型

```
module memory;
 import "DPI-C" function read_file(string fname);
 export "DPI-C" function mem_build; // 没有类型定义或者参数

 initial
 read_file("mem.dat");

 int mem[];

 function mem_build(input int size);
 mem = new[size]; // 分配动态内存元素
 endfunction
endmodule memory
```

```
extern void mem_build(int);

int read_file(char * fname) {
 int cmd;
 FILE * file;

 file = fopen(fname, "r");
 while (!feof(file)) {
 cmd = fgetc(file);
 switch (cmd)
 {
 case 'M': {
 int hi;
 fscanf(file, "%d %d ", &hi);
 mem_build(hi);
 break;
 }
 }
 }
}
```

```
 fclose(file);
}
```

## 调用SystemVerilog任务的C任务

```
module memory;
 import "DPI-C" context task read_file(string fname);
 export "DPI-C" task mem_read;
 export "DPI-C" task mem_write;
 export "DPI-C" function mem_build;

 initial read_file("mem.dat");

 int mem[];

 function mem_build(input int size);
 mem = new[size];
 endfunction

 task mem_read(input int addr, output int data);
 #20 data = mem[addr];
 endtask

 task mem_write(input int addr, input int data);
 #10 mem[addr] = data;
 endtask
endmodule memory
```

```
extern void mem_read(int, int*);
extern void mem_write(int, int);
extern void mem_build(int);

int read_file(char * fname) {
 int cmd;
 FILE * file;

 file = fopen(fname, "r");
 while (!feof(file)) {

 cmd = fgetc(file);
 switch (cmd) {
 case 'M': {
 int hi;
 fscanf(file, "%d %d ", &hi);
 mem_build(hi);
 break;
 }

 case 'R': {
```

```

 int addr, data, exp;
 fscanf(file, "%c %d %d ", &cmd, &addr, &data);
 mem_read(addr, &exp);
 if (data != exp)
 io_printf("C: Data=%d, exp=%d\n", data, exp);
 break;
 }

 case 'W': {
 int addr, data;
 fscanf(file, "%c %d %d ", &cmd, &addr, &data);
 mem_write(addr, data);
 break;
 }
}

fclose(file);
}

```

## 创建对象中的方法

```

module memory;
 import "DPI-C" context task read_file(string fname);
 export "DPI-C" task mem_read;
 export "DPI-C" task mem_write;
 export "DPI-C" function mem_build;

 initial read_file("mem.dat"); // 调用 C 代码读取文件

 class Memory;
 int mem[];

 function new(input int size);
 mem = new[size];
 endfunction

 task mem_read(input int addr, output int data);
 #20 data = mem[addr];
 endtask

 task mem_write(input int addr, input int data);
 #10 mem[addr] = data;
 endtask : mem_write
 endclass Memory

 Memory memq[$]; // 内存对象队列

 // 创建一个新的内存实例并将其压入队列
 function int mem_build(input int size);
 Memory m;
 m = new(size);
 endfunction

```

```
 memq.push_back(m);
 return memq.size() - 1;
 endfunction

 task mem_read(input int idx addr, output int data);
 memq[idx].mem_read(addr, data);
 endtask

 task mem_write(input int idx, addr, input int data);
 memq[idx].mem_write(addr, data);
 endtask

endmodule memory
```

### c中识别导出的SystemVerilog任务作用域

```
extern void sv_display();
svScope my_scope;

void save_my_scope() {
 my_scope = svGetScope();
}

void c_display() {
 // 打印当前作用域
 io_printf("\nC: c_display called from scope %s\n",
 svGetNameFromScope(svGetScope()));

 // 设置新的作用域
 svSetScope(my_scope);
 io_printf("C: calling %s.sv_display\n",
 svGetNameFromScope(svGetScope()));
 sv_display();
}
```